



OpenAgenet (OAN)

Technical Manual v1.0

Author: Jinliang Xu

jlxufly@gmail.com

September 13, 2026

<https://www.openagenet.xyz>

<https://github.com/OpenAgenet>

Contents

Chapter 1 Document Scope and Status	1
1.1 Revision History	2
1.2 Purpose and Scope	2
1.3 Relationship to the OAN White Paper and Yellow Paper	3
1.4 Relationship to the <code>did: oan</code> Method Specification	4
1.5 Relationship to Implementation Repositories and Deployment Materials	4
1.6 Normative Language and Conformance Terms	5
1.7 References	6
Chapter 2 System Goals and Design Principles	7
2.1 OAN and the Internet of Agents	7
2.2 Problems Addressed by OAN	8
2.3 Resource-First Interconnection Model	9
2.4 Trusted Registration, Publication, Discovery, and Invocation	10
2.4.1 Resource Registration and Control Proofs	10
2.4.2 Root Verification and Trusted Publication	11
2.4.3 Content Distribution and Discovery-Node Indexing	11
2.4.4 Pre-Call Verification and Integration	11
2.5 Separation of Resource Control, Node Authorization, and Governance	12
2.6 Federation, Interoperability, and Incremental Adoption	13
2.7 Control Plane, Data Plane, and Semantic Plane	13
2.8 Public Visibility, Discoverability, and Invocation Authorization	14
2.9 References	15
Chapter 3 Terms, Roles, and Trust Boundaries	16
3.1 Core Terms and Abbreviations	17
3.1.1 Unified Terminology Table	17
3.2 Resource Controller and Resource Provider	20
3.3 Subject Identity and Local Identity	21
3.3.1 Subject Identity and Resource Control Identity	21
3.3.2 Local Key Generation, Import, and Backup	21
3.3.3 Private-Key Custody and Node Visibility	22
3.4 Root Platform	22
3.4.1 Trust-Authorization Hub	22

3.4.2	Data-Distribution Hub	23
3.4.3	Semantic-Governance Hub	23
3.4.4	Root Platform Responsibility Boundary	24
3.5	Registrar Service Node	24
3.6	Discovery Service Node	24
3.7	Content Distribution Platform	25
3.8	Off-Chain Trust Indexer	25
3.9	Caller, User, Application, and Agent	26
3.10	Governance Authority and Authorization Domain	26
3.11	Trust Boundaries and Responsibility Matrix	27
3.11.1	Resource Controller and Resource Provider	27
3.11.2	Registrar Node, Root Platform, and Discovery Node	27
3.11.3	Caller and Verifier	28
3.11.4	Responsibility for Key Actions	28
3.12	References	29
Chapter 4	Overall Architecture and Network Topology	31
4.1	Logical Architecture	32
4.2	Root Platform as the Trust and Authorization Hub	33
4.3	Root Platform as the Data Distribution Hub	34
4.4	Root Platform as the Semantic Governance Hub	35
4.5	Publication Path from a Registrar Node to the Root Platform	35
4.5.1	Access Validation by a Registrar Node	36
4.5.2	Upstream Submission and Identity Authentication	36
4.5.3	Root-Platform Verification and Publication Confirmation	37
4.5.4	Failure, Rejection, and Retry	37
4.6	Distribution Path from the Root Platform to Discovery Nodes	37
4.6.1	Distribution Content and Publication Checkpoints	37
4.6.2	Discovery-Node Synchronization and Indexing	38
4.6.3	Sequence Numbers, Freshness, and Convergence	38
4.6.4	Distribution Failure and Recovery	39
4.7	Multiple Registrar Nodes and Discovery Nodes	39
4.7.1	Access by Multiple Registrar Nodes	39
4.7.2	Synchronization by Multiple Discovery Nodes	40
4.7.3	Data Consistency Between Nodes	40
4.7.4	Node Failure and Isolation	40
4.8	Third-Party Node Onboarding Model	40
4.8.1	Pre-Onboarding Authorization	41
4.8.2	Node Capabilities and Authorization Scope	41
4.8.3	Protocol Conformance Checks	41
4.8.4	Runtime Status Management	42
4.9	Public, Private, and Consortium Network Deployments	42
4.10	Availability, Isolation, and Failure Boundaries	43

4.11	Interaction of the Control, Data, and Semantic Planes	43
4.12	Sources of Facts and Derived Read Models	46
4.13	Network Topology and Sequence Consistency	47
4.14	References	48
Chapter 5	OAN Resource Model	50
5.1	Resource Model Overview	50
5.2	Agent Services	51
5.3	Skill Packages	52
5.4	MCP Services	53
5.5	Tool APIs	54
5.6	Resource Relationships and Composition	55
5.6.1	Dependencies Between Resources	56
5.6.2	Composition of Agent Services and Skill Packages	56
5.6.3	Invocation Relationships Between MCP Services and Tool APIs	56
5.6.4	Composition Integrity and Version Constraints	57
5.7	Resources as Independently Identifiable Objects	57
5.8	Resource Metadata and Capability Descriptions	58
5.8.1	Basic Identity and Description Fields	58
5.8.2	Capability Tags, Capability Descriptions, and Use Cases	59
5.8.3	Protocol, Endpoint, and Authentication Information	59
5.8.4	State, Version, and Governance Information	59
5.9	Use Cases, Protocol Bindings, Endpoints, and Authentication Declarations	59
5.9.1	Use Case Descriptions	60
5.9.2	Protocol Bindings	60
5.9.3	Service Endpoints	60
5.9.4	Authentication and Invocation Requirements	60
5.10	Resource Packages and Publication Artifacts	61
5.10.1	ResourcePackage Contents	63
5.10.2	Metadata and Resource Content	63
5.10.3	Hashes and Integrity Proofs	63
5.10.4	Publication, Download, and Verification Artifacts	63
5.11	Resource Lifecycle and Visibility State	63
5.11.1	Registration State	64
5.11.2	Publication and Indexing State	64
5.11.3	Suspension, Revocation, and Recovery State	65
5.11.4	Publicly Visible and Trustfully Discoverable State	65
5.12	Resource Ownership, Control, and Delegation Relationships	65
5.13	Resource Dependencies and Composition Integrity	66
5.14	References	67

Chapter 6	did:oon Method and Resource Identity	68
6.1	Method Purpose and Identifier Scope	68
6.2	DID Syntax and Method-Specific Identifier	69
6.3	Relationship Between Resource DID and Subject/Controller	70
6.3.1	Subject DID	70
6.3.2	Resource DID	70
6.3.3	Control and Signature Relationships	70
6.3.4	Separation of Subject, Resource, and Provider	71
6.4	DID Document Structure	71
6.4.1	DID Identifier and Method Metadata	72
6.4.2	verificationMethod	73
6.4.3	authentication and assertionMethod	73
6.4.4	service Endpoint	73
6.4.5	Resource Metadata and Extension Fields	73
6.5	Verification Methods and Public Keys	74
6.6	Service Endpoints and Discovery References	74
6.7	Resource Metadata Binding	75
6.8	Control Proofs and DID Document Proofs	76
6.9	DID Generation and Local Key Handling	77
6.9.1	DID Generation	78
6.9.2	Key Generation and Storage	78
6.9.3	Local Identity Backup	78
6.9.4	Import, Recovery, and Key-Use Boundaries	78
6.10	DID Resolution Requirements	78
6.11	DID Update, Version Management, Suspension, Recovery, and Deactivation	79
6.11.1	Update with the Same DID	80
6.11.2	Version Selection	80
6.11.3	Suspension and Recovery	80
6.11.4	Deactivation and Historical State	80
6.12	DID Method Security and Privacy Considerations	81
6.13	DID Method Conformance Requirements and Test Vectors	81
6.14	DID Method Interoperability and Resolver Behavior	82
6.15	References	83
Chapter 7	Resource Metadata and Resource Package Types	85
7.1	Resource Metadata Schema	85
7.2	Required and Optional Fields	87
7.3	Resource Type and Subject Type Constraints	88
7.4	Name, Description, Tags, and Use Cases	88
7.5	Semantic Tags and Protocol Bindings	89
7.6	Service Endpoints and Invocation Requirements	89
7.7	Resource Versions and Compatibility Fields	90
7.8	Package Hash and Metadata Hash	90

7.9	Canonical Serialization and Deterministic Hashing	91
7.10	ResourcePackage Construction Rules	92
7.10.1	Input Materials	93
7.10.2	Field Canonicalization	93
7.10.3	Hash Computation	93
7.10.4	Signature and Proof Generation	93
7.10.5	Post-construction Validation	94
7.11	Structure of a Root-Platform-Verified ResourcePackage	94
7.11.1	Resource Identity Section	94
7.11.2	Resource Metadata Section	95
7.11.3	Root Platform Verification Proof	95
7.11.4	Fields Required for Distribution and Indexing	95
7.12	ResourcePackage Integrity and Provenance Verification	95
7.13	ResourcePackage Provenance Chain and Attribution	96
7.13.1	Resource Controller Provenance	96
7.13.2	Registration Service Node Provenance	97
7.13.3	Root Platform Publication Provenance	97
7.13.4	Provenance Chain Verification	97
7.14	Package Size, Content Limits, and Validation Strategy	97
7.15	Schema Evolution and Backward Compatibility	98
7.16	References	98
Chapter 8	Identity, Keys, Signatures, VCs, and VPs	99
8.1	Subject Identity and Resource Identity	99
8.2	Key Generation, Custody, Backup, and Recovery	100
8.3	Resource Control Signatures	101
8.4	Infrastructure Node Identity Keys and Node Signing Keys	102
8.5	Root Platform Authorization VC for Infrastructure Nodes	103
8.6	Registration VC Issued to the Resource Controller	104
8.7	Governance and Authorization Evidence	105
8.8	VC and VP Structural Requirements	106
8.8.1	Basic VC Structure	106
8.8.2	Basic VP Structure	106
8.8.3	Issuer, Holder, and Subject	107
8.8.4	Credential Status and Verification	107
8.9	Relationship among Issuer, Holder, Controller, and Credential Subject	108
8.9.1	Issuer	108
8.9.2	Holder	108
8.9.3	Resource Controller	108
8.9.4	Credential Subject	108
8.10	Signature Verification and Status Verification	108
8.11	Credential Privacy and Disclosure Boundaries	110
8.12	Key Rotation and Compromise Response	110

8.13	Registration VC and Resource Publication Evidence	111
8.14	Node Authorization VC and Infrastructure Permissions	111
8.15	Credential Revocation, Suspension, and Status Propagation	111
8.15.1	Credential Status Changes	112
8.15.2	Node Authorization VC Status	112
8.15.3	Resource Registration VC Status	112
8.15.4	Status Propagation and Cache Updates	112
8.16	Signature Formats, Normalization, and Verification Test Vectors	112
8.17	Reference Sources	113
Chapter 9	Registration Protocol	114
9.1	Registration Targets and States	115
9.2	Registration Request Structure	116
9.3	Local Identity Initialization and Import	119
9.3.1	Create a New Local Identity	120
9.3.2	Import a Local Identity	120
9.3.3	Private Keys and Subject Configuration	120
9.3.4	Identity Backup and Recovery	120
9.4	DID Document Draft Generation	120
9.5	Pre-submission Local Validation	121
9.6	Resource Control Proof Verification	121
9.7	Registrar Node Authorization and Scope Validation	122
9.8	Resource Type, Metadata, Endpoint, and ResourcePackage Validation	122
9.9	Duplicate Registration and Idempotency	123
9.9.1	Repeated Registration of the Same DID	123
9.9.2	Basis for Idempotency Decisions	123
9.9.3	VC Reissuance	123
9.9.4	Duplicate Publication Control at the Root Platform	123
9.10	Resource Updates and Same-DID Version Updates	124
9.10.1	Update Authorization Verification	124
9.10.2	DID Document Updates	124
9.10.3	Resource Metadata and Endpoint Updates	124
9.10.4	New Version Publication and Old-Version Handling	124
9.11	Registration VC Issuance and Retrieval	124
9.11.1	Conditions for VC Issuance	125
9.11.2	VC Delivery	125
9.11.3	Handling a Missing or Empty VC	125
9.11.4	Local VC Storage	125
9.12	Registration Responses and Error Handling	125
9.13	Registration Audit Records	127
9.14	Resource Update Authorization and Same-DID Replacement	127
9.15	Submission Retries, Idempotency Keys, and Duplicate Handling	128
9.16	Registration Rate Limiting and Abuse Control	128

9.17 References	129
Chapter 10 Root Platform Verification and Trusted Publication	130
10.1 Root Platform Publication Responsibilities	130
10.2 Upstream Submission by a Registrar Service Node	131
10.3 Resource DID and Control-Proof Verification	132
10.4 Registrar Service Node Authorization Verification	132
10.5 ResourcePackage and Hash Verification	132
10.6 Metadata and Endpoint Policy Checks	133
10.7 Issuance of the Root Platform Trusted Publication Proof	133
10.7.1 Issuance Preconditions	133
10.7.2 Proof Contents	133
10.7.3 Proof Signature	134
10.7.4 Proof Retrieval and Verification	134
10.8 Publication Sequence and Version Selection	134
10.8.1 Publication Sequence	134
10.8.2 Version Selection for the Same DID	134
10.8.3 Duplicate Submission Handling	135
10.8.4 Conflicting-Version Handling	135
10.9 Publication Status and Rejection Reasons	135
10.10 Content Distribution Preparation	136
10.11 Root Platform Publication Audit Trail	136
10.12 Republish, Rollback, and Recovery	137
10.13 Same-DID Update Authorization and Version Acceptance	137
10.14 Publication Rejection, Isolation, and Reprocessing	137
10.15 Root Platform Publication State Machine	138
10.15.1 Pending Verification	138
10.15.2 Verification Passed	138
10.15.3 Published	138
10.15.4 Rejected, Isolated, and Recovered	138
10.16 References	138
Chapter 11 Content Distribution and Network Propagation	140
11.1 Distribution Model and Content Categories	140
11.2 Content Distribution Service Responsibilities	140
11.3 Distribution Manifest and Publication Checkpoints	141
11.4 Distribution of Resource Packages, Metadata, Proofs, and Index Data	141
11.4.1 Resource Package Distribution	141
11.4.2 DID Document and Metadata Distribution	142
11.4.3 VC and Root Proof Distribution	142
11.4.4 Index Data Distribution	142
11.5 Distribution-Boundary Integrity Validation	142
11.6 Version Freshness and Synchronization Cursors	142

11.6.1 Synchronization Cursor	143
11.6.2 Version Freshness	143
11.6.3 Expired Data Determination	143
11.6.4 Gap Compensation	143
11.7 Initial Synchronization and Incremental Synchronization	144
11.7.1 Initial Full Synchronization	144
11.7.2 Incremental Synchronization	144
11.7.3 Synchronization Confirmation	144
11.7.4 Recovery After Interruption	144
11.8 Retry, Backoff, and Interruption Recovery	144
11.9 Cache Behavior and Expired-Data Handling	145
11.9.1 Browser or Client Cache	145
11.9.2 Node-Side Cache	145
11.9.3 Preferential Display of Old Snapshots	145
11.9.4 Background Refresh and Failure Degradation	145
11.10 Propagation Observability and Audit Records	146
11.11 Publication Ordering and Conflict Resolution	146
11.12 Distribution Consistency and Freshness Guarantees	146
11.13 Partial Failure and Recovery Semantics	147
11.14 References	147
Chapter 12 Discovery Protocol and Query Semantics	148
12.1 Discovery Objectives and Query Types	148
12.1.1 Exact DID Discovery	149
12.1.2 Structured-Condition Discovery	149
12.1.3 Capability-Tag Discovery	149
12.1.4 Text and Semantic Discovery	149
12.1.5 Combined-Condition Discovery	149
12.2 Exact DID Queries	150
12.3 Structured Resource Queries	151
12.4 Capability-Tag Queries	151
12.5 Text and Semantic Queries	152
12.6 Combined Semantic and Structured-Condition Queries	152
12.7 Authorization-Scope Filtering	153
12.8 Latest Valid Version Selection	153
12.9 Search Ranking and Deterministic Result Ordering	154
12.9.1 Matching Conditions	154
12.9.2 Trust and Governance State	154
12.9.3 Version and Freshness	154
12.9.4 Ranking Stability	155
12.9.5 Duplicate-Result Handling	155
12.10 Search Explanations and Evidence Display	155
12.10.1 Match Reasons	155

12.10.2 Resource Provenance	155
12.10.3 DID, VC, and Root Platform Proof	155
12.10.4 Governance State and Version Information	156
12.11 Discovery Response Model	156
12.12 Discovery-Result Freshness and State	158
12.13 Discovery Errors and Degraded Responses	160
12.14 Public Visibility and Trusted Discoverability	161
12.14.1 DID-Level Deduplication	161
12.14.2 Version-Level Deduplication	161
12.14.3 Duplicate Records from Registration Nodes	161
12.14.4 Handling Identical Content with Different DIDs	161
12.15 Result Deduplication and Resource-Identity Matching	162
12.16 Query Limits, Pagination, and Result Stability	163
12.17 Discovery Privacy and Authorization-Scope Visibility	165
12.18 References	166
Chapter 13 Semantic Governance and Semantic Discovery	167
13.1 Semantic Model and Capability Vocabulary	167
13.2 Root Platform-Managed Capability Tag Tree	167
13.2.1 Tag Tree Organization	168
13.2.2 Tag Identifiers and Hierarchical Relationships	168
13.2.3 Tag Recommendation and Discovery Use	169
13.2.4 Tag Tree Version Management	169
13.3 Tag Hierarchies, Identifiers, Aliases, and Localization	169
13.4 Tag Selection and Recommendation During Registration	170
13.4.1 Resource Provider Selection	170
13.4.2 Root Platform Recommendation	170
13.4.3 Custom Tag Supplement	170
13.4.4 Tag Validation and Persistence	170
13.5 Custom Tags and Provider Descriptions	170
13.6 Semantic Governance Change Process	171
13.7 Semantic Index Construction by Discovery Nodes	171
13.7.1 Index Inputs	171
13.7.2 Tag Index	171
13.7.3 Text and Semantic Indexes	172
13.7.4 Index Update and Deletion	172
13.8 Matching Tags, Text, Protocols, and Resource Types	172
13.9 Query Explanation and Matching Details	172
13.9.1 Query Intent Parsing	172
13.9.2 Tag and Text Matching	173
13.9.3 Structured Condition Matching	173
13.9.4 Result Explanation	173
13.10 Multilingual Semantic Discovery	173

13.10.1 Chinese and English Resource Descriptions	173
13.10.2 Multilingual Tags	174
13.10.3 Query Language Identification	174
13.10.4 Cross-Language Matching	174
13.11 Extensibility for Vector and Hybrid Retrieval	174
13.12 Semantic Governance Compatibility and Migration	175
13.13 Semantic Vocabulary Versioning and Change Impact	175
13.14 Semantic Governance Auditability	175
13.15 Mapping Natural-Language Queries to Structured Conditions	176
13.16 References	176
Chapter 14 Node Authorization and On-Chain Governance	178
14.1 Governance Objectives and the On-Chain/Off-Chain Boundary	178
14.1.1 On-Chain Governance Facts	178
14.1.2 Root Platform Runtime State	179
14.1.3 Registration and Publication Status	179
14.1.4 Discovery Node Read Model	180
14.2 Infrastructure Node Authorization Model	180
14.3 Authorization Domains and Capability Scopes	181
14.4 Governance Event Model	181
14.4.1 Node Authorization Events	184
14.4.2 Node Suspension and Resumption Events	184
14.4.3 Node Revocation Events	185
14.4.4 Event Time, Sequence, and Source	185
14.5 Node Authorization, Suspension, Resumption, and Revocation	185
14.5.1 Authorization	186
14.5.2 Suspension	186
14.5.3 Resumption	186
14.5.4 Revocation	186
14.5.5 Effects of Status on Registration, Distribution, and Discovery	186
14.6 Governance Notifications and Authorization State	187
14.7 The Off-Chain Trust Indexer as a Governance Read-Side Projection	187
14.7.1 Reading On-Chain Events	187
14.7.2 State Projection	188
14.7.3 Synchronization Cursor and Lag	188
14.7.4 Query Interfaces	188
14.7.5 Synchronization Failure and Recovery	189
14.8 Enforcing Governance State at Root Platform Runtime	189
14.9 Governance Filtering on the Discovery Side	189
14.10 Governance Event History and Public Observability	190
14.11 Chain Endpoints, Packages, Objects, and Deployment Configuration	190
14.12 Governance Upgrade and Migration Rules	191
14.13 Governance Event Ordering and Finality	191

14.14 Consistency Among On-Chain Facts, Off-Chain Publication State, and Read Models	192
14.14.1 Final On-Chain Facts	192
14.14.2 Root Platform Publication State	192
14.14.3 Discovery Node Index State	193
14.14.4 State Inconsistency and Convergence	193
14.15 Governance Authority Key Management and Emergency Procedures	193
14.16 References	194
Chapter 15 Network Node Protocols and Interoperability	195
15.1 Common Protocol Envelope and Request Headers	195
15.2 Inter-Node Authentication	196
15.3 Signing Upstream Requests	196
15.4 Node Authorization VC Exchange	197
15.5 Registrar node to Root platform Protocol	198
15.6 Root platform to content distribution platform Protocol	198
15.7 content distribution platform to Discovery node Protocol	199
15.8 Discovery Response Signatures and Source Proofs	199
15.9 Sequence Numbers, Request Identifiers, and Replay Protection	200
15.10 Pagination, Filtering, and Compatibility Rules	200
15.11 Timeouts, Retries, and Idempotent Operations	201
15.12 Protocol Version Negotiation	201
15.13 Message Size, Compression, and Transport Limits	202
15.14 Correlation, Traceability, and Audit Identifiers	202
15.15 Protocol Compatibility and Deprecation Rules	203
15.16 References	203
Chapter 16 Public HTTP API	204
16.1 API Conventions and Base URLs	204
16.1.1 Service Base Addresses	204
16.1.2 Path and Version Conventions	205
16.1.3 Request Authentication	205
16.1.4 Response Format	205
16.2 Health, Readiness, and Service Status Endpoints	205
16.3 Registrar Node Registration Endpoints	206
16.4 Root Platform Publication and Status Endpoints	206
16.5 Content Distribution Endpoints	207
16.6 Discovery Query and Resource Detail Endpoints	208
16.7 DID Resolution Endpoints	209
16.8 VC and Proof Retrieval Endpoints	209
16.9 Governance Status and Event Endpoints	209
16.10 Metrics and Observability Endpoints	210
16.11 Request and Response Models	210
16.11.1 Registration Requests and Responses	211

16.11.2 Publication Requests and Responses	212
16.11.3 Discovery Requests and Responses	212
16.11.4 Status and Error Responses	212
16.12 HTTP Status Codes and Common Errors	213
16.13 Rate Limits, Caching, and Security Headers	213
16.14 CORS, Browser Clients, and Cross-Origin Policy	214
16.15 Boundary Between Public Endpoints and Operations-Only Endpoints	215
16.15.1 Public Query Interfaces	215
16.15.2 Node-to-Node Interfaces	215
16.15.3 Operations Interfaces	215
16.15.4 Access Control and Exposure Boundaries	216
16.16 API Examples and Machine-Readable Interface Descriptions	216
16.17 References	217
Chapter 17 Client SDK, Community Skill, and Developer Experience	218
17.1 TypeScript SDK Scope	218
17.2 DID Identity and Key Operations in the SDK	219
17.3 Registration Flow Example	220
17.4 Exact DID Discovery Example	220
17.5 Semantic Discovery Example	221
17.6 Verification and Trust Evidence Handling	222
17.7 Community Skill Package Responsibilities	222
17.8 Resource Authoring and Registration Assistance	223
17.9 Node Operators and Deployment Assistance	223
17.10 Package Distribution and Compatibility Management	224
17.11 Example Resources and Integration Templates	225
17.12 Client Security and Secret Handling	225
17.13 Browser Registration and Local Identity Backup Behavior	226
17.13.1 Browser-Side Key Generation	226
17.13.2 Local Identity Import	226
17.13.3 Local Backup Download	226
17.13.4 Registration-Result Package Download	227
17.14 DID, VC, and ResourcePackage Download Formats	227
17.15 References	229
Chapter 18 Security, Privacy, and Abuse Prevention	230
18.1 Threat Model and Trust Assumptions	230
18.1.1 Resource-Control Threats	231
18.1.2 Node-Trust Threats	231
18.1.3 Data-Poisoning and Abuse Threats	231
18.1.4 Availability Threats	232
18.1.5 Invocation-Endpoint Security Threats	232
18.2 Resource Impersonation and DID Control Protection	232

18.3 Protection Against Forged Registration and Unauthorized Updates	233
18.4 Malicious or Compromised Registrar Nodes	233
18.5 Malicious or Compromised Discovery Nodes	233
18.6 Root-Platform Compromise and Key Protection	233
18.7 Spam Registration and Resource-Catalog Poisoning	234
18.7.1 Fake Resource Registration	234
18.7.2 Duplicate Registration and Version Poisoning	234
18.7.3 Semantic-Tag Poisoning	234
18.7.4 Rate Limiting, Review, and Cleanup	235
18.8 Denial of Service and Resource Exhaustion	235
18.8.1 Registration-Interface Attacks	236
18.8.2 Discovery-Query Attacks	236
18.8.3 Inter-Node Synchronization Pressure	237
18.8.4 Database and Index Resource Exhaustion	237
18.9 Replay, Duplicate Submission, and Expired Publication	237
18.10 Endpoint Abuse and Unsafe Resource Invocation	238
18.11 Sensitive Metadata and Credential Exposure	239
18.12 Logging, Auditing, and Incident Response	239
18.13 Security Requirements for Third-Party Nodes	240
18.14 Endpoint Validation, Callback Security, and SSRF Considerations	240
18.14.1 Endpoint Format and Protocol Validation	240
18.14.2 Internal and Loopback Address Restrictions	241
18.14.3 Callback Request Security	241
18.14.4 Redirect and DNS-Resolution Risks	241
18.15 Package and Dependency Supply-Chain Integrity	241
18.16 Availability Objectives and Graceful Degradation	241
18.17 References	242
Chapter 19 Resource Lifecycle, Governance, and Version Management	243
19.1 Resource Lifecycle State Machine	243
19.2 Initial Registration	244
19.3 Updating a Resource with the Same DID	245
19.4 DID Document Version Management	245
19.5 Resource Package Version Management	246
19.6 Metadata and Endpoint Changes	246
19.7 Registration VC Lifecycle	247
19.8 Resource Suspension and Recovery	247
19.9 Resource Revocation and Deactivation	247
19.10 Node Suspension and Resource Visibility	248
19.11 Version Selection by a Discovery Node	248
19.12 Historical Versions and Auditability	248
19.13 Resource VC and Publication Evidence Lifecycle	249
19.14 Update Propagation and Stale Index Convergence	249

19.15 Resource Ownership Transfer Strategy	250
19.16 References	250
Chapter 20 Observability, Metrics, and Operations	252
20.1 Service Health and Readiness Model	252
20.2 Registration Metrics	253
20.3 Publication and Propagation Metrics	254
20.4 Discovery Query Metrics	255
20.5 Index Freshness and Synchronization Metrics	255
20.6 Governance and Authorization Metrics	256
20.7 Network-Level and Node-Level Statistics	256
20.7.1 Network-Level Metrics	257
20.7.2 Node-Level Metrics	257
20.7.3 Registration, Publication, and Discovery Metrics	257
20.7.4 Metric Time Windows	257
20.7.5 Missing Data and Degraded Display	257
20.8 Visibility of On-Chain Governance Events	257
20.9 Logs, Traces, and Correlation Identifiers	258
20.10 Snapshot, Cache, and Degraded-Mode Behavior	258
20.10.1 Loading an Old Snapshot	259
20.10.2 Background Data Refresh	259
20.10.3 New Data Replacement	259
20.10.4 Long-Term Failure and Alerts	259
20.10.5 First Load Without a Cache	260
20.11 Backup, Recovery, and Disaster Recovery	260
20.12 Deployment Smoke Tests and Functional Verification	260
20.12.1 Registration Function Smoke Test	261
20.12.2 Discovery Function Smoke Test	261
20.12.3 Node Health Check	261
20.12.4 Database and Indexer Check	261
20.12.5 Deployment Handling after Failure	262
20.13 Data Retention, Deletion, and Privacy Operations	262
20.14 Operational Alerts and Long-Term Failure Detection	262
20.15 Public Service Status and Maintenance Communication	263
20.16 References	263
Chapter 21 Deployment and Conformance Requirements	264
21.1 Reference Deployment Topology	264
21.2 Configuration and Secret Management	265
21.3 Build and Release Artifacts	265
21.4 Database and Storage Requirements	266
21.5 TLS, Domains, Reverse Proxy, and Network Exposure	267
21.6 Deployment Order and Dependency Checks	267

21.6.1	Code Synchronization and Build	267
21.6.2	Database Preparation	267
21.6.3	Node Startup Order	267
21.6.4	Service Health Checks	268
21.6.5	Post-deployment Functional Verification	268
21.7	Registrar Node Functional Smoke Tests	268
21.8	Discovery Node Functional Smoke Tests	269
21.9	Root Platform, Distribution, and Indexer Verification	269
21.10	Governance Synchronization Verification	269
21.11	Upgrade, Rollback, and Compatibility Checks	269
21.11.1	Pre-upgrade Checks	270
21.11.2	Data and Configuration Compatibility	270
21.11.3	Rollback Conditions	270
21.11.4	Post-rollback Verification	270
21.12	Third-party Node Onboarding and Conformance Testing	270
21.13	Reproducible Builds and Release Provenance	271
21.14	Environment-specific Configuration Files	271
21.15	Post-deployment Functional and Security Verification	271
21.16	References	271
Chapter 22	External Protocols and Ecosystem Mapping	273
22.1	W3C DID Core and <code>did:oon</code>	274
22.1.1	DID Core Data Structure Correspondence	274
22.1.2	<code>did:oon</code> Method Extensions	274
22.1.3	Resolution Behavior	275
22.1.4	Interoperability Boundaries	275
22.2	A2A Agent Card Mapping	275
22.2.1	Capability Description Mapping	276
22.2.2	Use-Case and Skill Mapping	276
22.2.3	Service Endpoint Mapping	277
22.2.4	Identity, Signature, and Trusted-Evidence Mapping	277
22.3	MCP Resource and Server Metadata Mapping	278
22.3.1	MCP Server Mapping	278
22.3.2	Tool Definition Mapping	278
22.3.3	Input and Output Constraint Mapping	278
22.3.4	Invocation and Authentication Information Mapping	278
22.4	Tool and API Description Mapping	279
22.5	VC and VP Ecosystem Interoperability	279
22.6	External Organization and Platform Identity Mapping	279
22.7	External Credential and Trust-Evidence Mapping	280
22.8	Field Source, Version, and Hash Requirements	280
22.9	Compatibility Profiles and Migration Paths	280
22.10	External Protocol Capability Negotiation	280

22.11 External Signature and Evidence Retention	281
22.12 References	281
Chapter 23 End-to-End Reference Flows	283
23.1 Create a Local Identity	284
23.2 Register an Agent Service	284
23.3 Register a Skill Package	285
23.4 Register an MCP Service	285
23.5 Register a Tool API	286
23.6 Publish and Distribute Resources	286
23.6.1 Registration Result	287
23.6.2 Root Platform Publication	287
23.6.3 Content Distribution Platform	287
23.6.4 Discovery Node Indexing	288
23.7 Discover by DID	288
23.8 Discover by Structured Conditions	288
23.9 Discover by Natural-Language Task Description	289
23.10 Verify a Discovered Resource Before Invocation	289
23.10.1 DID Resolution	290
23.10.2 Control-Rights Verification	290
23.10.3 Registration VC Verification	291
23.10.4 Root Proof Verification	291
23.10.5 Governance Status Check	292
23.11 Update a Resource While Retaining Its DID	292
23.12 Suspend, Revoke, and Recover a Resource	292
23.13 Connect a Third-Party Registrar Node	293
23.14 Connect a Third-Party Discovery Node	294
23.15 Observe Governance Events Through the Network	294
23.16 Resolve and Verify a Resource DID	294
23.17 Retrieve Historical Resource Versions	295
23.18 Handle an Unavailable or Stale Discovery Node	295
23.18.1 Discovery Node Unavailable	295
23.18.2 Return an Old Snapshot	295
23.18.3 Determine Data Freshness	296
23.18.4 Switch to Another Discovery Node	296
23.18.5 Re-synchronize After Recovery	296
23.19 References	297
Chapter 24 Conformance Profiles and Acceptance Criteria	298
24.1 Resource Provider Conformance Profile	299
24.2 Registrar Node Conformance Profile	299
24.3 Root Platform Conformance Profile	300
24.4 Content Distribution Service Conformance Profile	301

24.5	Discovery Node Conformance Profile	302
24.6	Off-Chain Trust Indexer Conformance Profile	302
24.7	Parser and Client Conformance Profile	303
24.8	SDK Conformance Profile	304
24.9	Security and Key Custody Acceptance Criteria	304
24.10	Interoperability and Compatibility Acceptance Criteria	305
24.11	Performance and Availability Acceptance Criteria	306
24.12	Evidence Required for Conformance Implementation	307
24.12.1	Protocol and Data-Structure Evidence	307
24.12.2	Cryptographic and Identity Evidence	308
24.12.3	Node-Behavior Evidence	308
24.12.4	Deployment and Smoke-Test Evidence	308
24.12.5	Version and Provenance Evidence	308
24.13	Test Vectors and Interoperability Fixtures	308
24.14	Negative Tests and Rejection Behavior	309
24.15	Conformance Evidence Retention and Review	309
24.16	References	310
Chapter 25	Versioning, Compatibility, and Change Management	311
25.1	Specification Version Management	312
25.2	Protocol Version Management	312
25.3	Resource Schema Version Management	313
25.4	DID Method Compatibility	314
25.5	Backward-Compatible and Breaking Changes	314
25.5.1	Compatible Changes	315
25.5.2	Breaking Changes	315
25.5.3	Version Identifiers	316
25.5.4	Migration and Transition Period	316
25.6	Specification Changes for Governance Controls	316
25.6.1	Change Proposal	317
25.6.2	Governance Approval	317
25.6.3	Release and Effectiveness	317
25.6.4	Rollback and Emergency Changes	318
25.7	Deprecation and Migration Strategy	318
25.8	Change Records and Release Notes	318
25.8.1	Specification Change Records	319
25.8.2	Protocol Change Records	319
25.8.3	Implementation Change Records	320
25.8.4	Migration Notices and Known Impacts	320
25.9	References	320

Appendix A Normative Data Schemas	322
A.1 DID Document Schema	322
A.2 Resource Metadata Schema	323
A.3 ResourcePackage Schema	325
A.4 Root proof Schema	326
A.5 Registration VC Schema	327
A.6 Node Authorization VC Schema	328
A.7 Governance Event Schema	328
A.8 Registration Request and Response Schema	329
A.9 Discovery Request and Response Schema	330
A.10 Error Response Schema	331
A.11 Node Authorization and Governance Status Schema	331
A.12 Distribution Manifest and Synchronization Cursor Schema	332
A.13 Search Explanation and Evidence Schema	334
A.14 References	335
Appendix B Protocol and API Reference	336
B.1 Endpoint Catalog	336
B.2 Request Header Fields and Signatures	338
B.3 Response Header Fields and Cache Directives	339
B.4 Status Codes and Error Codes	339
B.5 Pagination and Cursor Rules	340
B.6 Version and Compatibility Header Fields	341
B.7 Health, Readiness, and Liveness Response Formats	341
B.8 Public, Authenticated, and Operations-Only API Matrix	342
B.9 Reference Sources	343
Appendix C Cryptography and Encoding	344
C.1 Supported Algorithms	344
C.2 Canonicalized JSON and Hashing	345
C.3 JWS and Detached Signatures	346
C.4 DID and Key Encoding	347
C.5 VC and VP Encoding	348
C.6 Key Backup and Recovery Format	349
C.7 DID and Resource Identifier Test Vectors	350
C.8 Hash and Signature Verification Fixtures	351
C.9 References	351
Appendix D Conformance Test Matrix	352
D.1 Resource Registration Tests	353
D.2 Root Platform Publication Tests	355
D.3 Distribution and Synchronization Tests	356
D.4 Discovery and Semantic Search Tests	357
D.5 Governance Authorization Tests	358

D.6	Version Update and Lifecycle Tests	359
D.7	Security and Abuse-Prevention Tests	359
D.8	Third-Party Node Admission Tests	360
D.9	DID Resolution and Method Conformance Tests	360
D.10	API Compatibility and Error-Contract Tests	361
D.11	Freshness, Convergence, and Recovery Tests	362
D.12	References	363
Appendix E Reference Deployments and Examples		364
E.1	Official Network Deployment	365
E.2	Private and Alliance Network Deployment	366
E.3	Minimal Registrar Node Deployment	367
E.4	Minimal Discovery Node Deployment	368
E.5	SDK Registration Example	369
E.6	DID and Semantic Discovery Examples	370
E.7	Resource Verification Example	371
E.8	Operations Smoke-Test Example	372
E.9	Same-DID Update Example	373
E.10	Resource Suspension and Recovery Example	373
E.11	Cross-Protocol A2A and MCP Mapping Example	374
E.12	References	375
Appendix F Repository and Implementation Mapping		376
F.1	oan-protocol-common	377
F.2	oan-root-services	379
F.3	oan-registrar-node	380
F.4	oan-discovery-node	381
F.5	oan-trust-indexer	383
F.6	oan-sdk-ts	383
F.7	oan-community-skill	385
F.8	Official Website and Public Developer Entry Points	386
F.9	Specification Chapter and Implementation Module Mapping	387
F.10	References	389

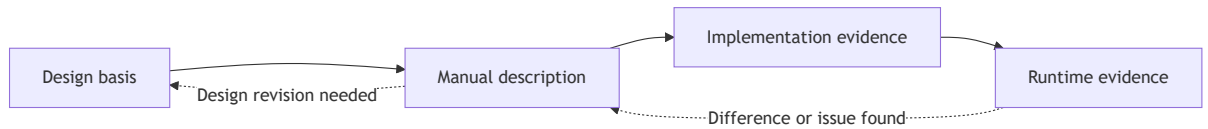
Chapter 1 Document Scope and Status

This chapter is the entry point for reading this technical manual. It is intended for resource providers, node operators, client and SDK developers, protocol implementers, integration developers, and technical personnel evaluating OpenAget (OAN). The manual covers resources such as Agent Services, Skills, MCP Servers, and Tool APIs, as well as supporting mechanisms including subject identities, resource DIDs, DID Documents, registration VCs, Root-verified ResourcePackages, content distribution, discovery indexing, and pre-call verification. Readers can use it as a guide from concepts to implementation: first read Chapters 2 through 5 to understand the goals, roles, architecture, and resource model, and then consult the identity, registration, publication, discovery, operations, and conformance chapters as needed.

The manual distinguishes four content statuses: **design basis**, which explains why a design was made; **current system fact**, which describes behavior demonstrated by existing code, interfaces, or deployments; **reference implementation**, which describes one implementation approach used by the OAN code repositories; and **runtime evidence**, which describes build, deployment, or smoke-test results from a particular environment at a particular time. Normative requirements are used only where interoperability, security, or data integrity must be preserved. A website display, example configuration, or result from one deployment must not be generalized into a mandatory capability for every node. When later chapters cite interfaces, code, or runtime data, they should state the evidence type and scope of applicability.

```
{
  "manual": "technical manual",
  "scope": ["resource identity", "registration", "trusted publication", "discovery", "governance"],
  "evidence_types": ["design", "current-fact", "reference-implementation", "runtime-evidence"],
  "status_rule": "Every conclusion should state its source, scope of applicability, and verification time"
}
```

This manual classifies its materials as design basis, normative rules, implementation evidence, and runtime evidence. Design basis explains goals and trade-offs; normative rules state interoperability baselines; implementation evidence explains how code realizes the rules; and runtime evidence states whether a particular environment works at a particular time. These four types of material may refer to one another, but one type must not substitute for another. During publication or review, record the manual version, referenced material versions, implementation commit, configuration version, test time, and applicable environment.



1.1 Revision History

Date	Change	Author
2026-09-14	Release v1.0	Jinliang Xu

1.2 Purpose and Scope

The manual covers the complete path from local preparation of a resource to its discovery and verification: the resource provider prepares the resource and local identity; the client submits the materials to a Registrar node; the Registrar node performs admission checks; the Root platform verifies and publishes the trusted ResourcePackage; the content distribution platform hosts retrievable materials; a Discovery node synchronizes and builds indexes; and the caller checks the DID, credentials, version, and governance status before connecting. The corresponding chapters explain the resource model, `did: oan`, DID Documents, ResourcePackages, VC/VP, signed messages, node authorization, semantic tags, public HTTP APIs, SDKs, the community Skill, lifecycle, operations, and conformance.

Task for the reader	Suggested reading path	Main result
Understand OAN's positioning and components	Chapters 1, 2, 3, and 4	Understand the goals, roles, network topology, and resource types
Register or publish a resource	Chapters 5, 6, 7, 8, 9, and 10	Understand identity, ResourcePackages, registration, trusted publication, and distribution
Integrate discovery and semantic queries	Chapters 11, 12, 14, 15, and 16	Understand queries, semantic governance, node interoperability, and SDK usage
Deploy, troubleshoot, and evaluate	Chapters 17, 19, 20, 23, and 24 plus the appendices	Obtain security, operations, deployment, testing, and version references

The manual does not prescribe which model or reasoning algorithm an agent must use internally, does not define all business logic of a business system, and does not replace external protocols such as A2A or MCP. Each chapter should state its applicable roles, inputs and outputs, responsible party, verifying party, state changes, exception handling, and retainable evidence. When a conclusion applies only to a reference implementation or a current deployment, its version, environment, and boundary should be stated.

1.3 Relationship to the OAN White Paper and Yellow Paper

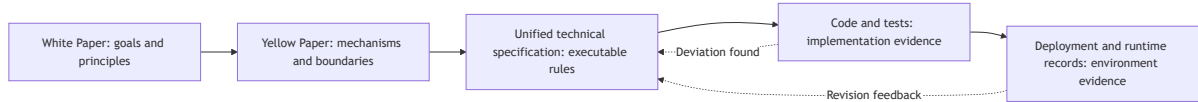
The White Paper is the conceptual entry point for the project. It mainly explains why infrastructure for resource identity, trusted publication, and semantic discovery is needed for the Internet of Agents (IoA). The Yellow Paper focuses more on how the mechanisms work, explaining roles, data objects, trust boundaries, publication paths, authorization domains, state machines, security invariants, and reference verification methods. This manual organizes those materials into technical guidance that readers can consult and use, while adding implementation mappings, examples, operational boundaries, and verification paths.

Material	What readers mainly obtain	Boundary of use
White Paper	Background, vision, overall positioning, and design principles	Not directly equivalent to interface fields or deployment commitments
Yellow Paper	Roles, mechanisms, trust boundaries, and engineering derivations	Reference algorithms and derivations do not automatically define a fixed format for every implementation
This technical manual	Conceptual explanations, processes, implementation mappings, examples, operations, and verification entry points	Normative requirements, reference implementation behavior, and current runtime evidence must still be distinguished

Readers can start with the White Paper to understand “why,” use the Yellow Paper to understand “how the mechanisms are organized,” and then return to this manual to find “how to understand, integrate, implement, and verify” them. When different materials differ only in abstraction level, the manual should explain their relationship. When an actual conflict exists in fields, processes, or security boundaries, it should record the conflict location, impact, applicable version, and revision status rather than conceal it with vague wording.

Material or evidence	Main question answered	What it can directly demonstrate	What it cannot demonstrate alone
White Paper	Why OAN is being built and what its overall vision is	Background, goals, value, and principles	Specific interface fields or a deployed implementation
Yellow Paper	How the mechanisms are organized and where the security boundaries are	Roles, processes, states, and mechanism derivations	That every implementation uses the same code
Unified technical specification	How different implementations interoperate	Fields, behavior, errors, and conformance requirements	Long-term availability of a particular deployment
Code, tests, and configuration	How a reference implementation realizes the rules	Module behavior, test results, and configuration constraints	That the design vision has already been implemented

Material or evidence	Main question answered	What it can directly demonstrate	What it cannot demonstrate alone
Deployment and runtime records	Whether a particular environment currently works	Build, startup, smoke-test, and runtime status	The protocol's inherent requirements



During publication or review, record at least the specification version, referenced material versions, implementation commit, configuration version, test time, and applicable environment. When materials differ, first determine whether the difference is caused by abstraction level or is an actual conflict; only the latter should enter the conflict record and revision process.

1.4 Relationship to the did: oan Method Specification

The `did: oan` method materials define the method-specific DID syntax, method-specific identifiers, DID Document structure, verification methods, public-key representation, service endpoints, resolution behavior, and boundaries for control proofs. The related chapters of this manual do not reinvent those method rules. Instead, they explain how OAN uses them: how clients generate and store local keys, how resource controllers prove control, how Registrar nodes validate DID Documents, how the Root platform associates resource versions with trusted publication records, how Discovery nodes build verifiable indexes, and how callers resolve and verify signatures before connecting.

A minimal status decision can be represented as follows:

```

DID Document valid
-> Resource control proof valid
-> Accepted by Registrar node
-> Published by Root platform
-> Indexed by Discovery node
-> Caller completes version, governance, and local permission checks
  
```

The steps above are not different names for one “usable” state. A DID Document that is valid in syntax, structure, and signature does not mean that the resource has been accepted by a Registrar node, published by the Root platform, indexed by a Discovery node, or granted permission for use. A Root publication fact cannot replace the caller's checks of the DID Document, VC, endpoint, version, governance status, and local business permissions. Later chapters should explain how these states connect through fields, processes, and status transitions.

1.5 Relationship to Implementation Repositories and Deployment Materials

Implementation mappings in this manual help readers answer “where is this capability implemented, how can it be verified, and what are its current boundaries”; they are not themselves

conformance proofs. Shared data structures, serialization, hashing, and cryptographic behavior should first be checked against `oan-protocol-common`; Root platform publication, governance status, and publication proofs against `oan-root-services`; registration admission and registration credentials against `oan-registrar-node`; synchronization, indexing, semantic retrieval, and discovery responses against `oan-discovery-node`; projection of on-chain governance events into runtime authorization read models against `oan-trust-indexer`; and client integration against `oan-sdk-ts` and the community Skill. README files, schemas, route handlers, tests, configuration examples, and deployment scripts provide different kinds of evidence, and their purpose should be stated when cited.

```
mapping:
  manual_section: "Chapter 8 Registration Protocol"
  repository: "oan-registrar-node"
  evidence: "Route handlers, request validation, integration tests"
  status: "Implemented with test evidence"
  checked_at: "YYYY-MM-DD"
  boundary: "The target environment's exact commit and deployment configuration prevail"
```

Each mapping should record at least the manual section, repository and module or file, corresponding version, implementation status, verification method, and known deviation. Implementation status should distinguish implemented with test evidence, implemented with insufficient evidence, partially implemented, and design-only. Deployment materials should also record environment variables, external services, databases, key boundaries, build artifacts, startup order, and post-deployment smoke tests. Addresses, node counts, and runtime metrics from the official environment are runtime evidence and should not be written as mandatory conditions for every environment.

1.6 Normative Language and Conformance Terms

This manual uses RFC-style keywords to indicate degrees of obligation: MUST corresponds to “must,” MUST NOT to “must not,” SHOULD to “should,” SHOULD NOT to “normally should not,” and MAY to “may.” These words are used only for confirmed interoperability, security, data-integrity, or conformance requirements. When describing current code behavior, deployment status, or operational advice, use more precise wording such as “current implementation,” “normally,” “recommended,” or “may.”

Conformance judgments should target a specific role and configuration profile rather than making a broad judgment about OAN as a whole. The capability boundaries of resource providers, Registrar nodes, the Root platform, the content distribution platform, Discovery nodes, the off-chain trust indexer, resolvers, and SDKs are not identical. Each test or runtime check should state its fixed input, prerequisite configuration, execution steps, expected output, failure impact, and evidence-retention method. Passing tests in a reference implementation does not mean that every third-party implementation is inherently conformant, and normal operation of the official deployment does not mean that the protocol covers every environment.

- This item states whether it is a normative requirement, reference implementation behavior, current deployment fact, or a capability not yet implemented.

- ☐ A verifiable source, version, or commit is provided.
- ☐ Evidence for resource registration, Root publication, discovery indexing, and pre-call verification is distinguished.
- ☐ The impact and handling of a deviation or implementation failure are stated.

1.7 References

Source	Type	Link
OAN White Paper	arXiv White Paper: project vision and positioning	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv Yellow Paper: technical architecture and trust model	https://arxiv.org/abs/2606.03163
did: oan DID Method Specification	Standard/method specification: did: oan identifiers and method rules	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Resource Identity and Discovery	IETF Internet-Draft: resource identity and discovery	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Agentic Overlay Network Architecture	IETF Internet-Draft: agentic overlay network architecture	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
Efficient Agent Discovery Profile	IETF Internet-Draft: efficient agent discovery profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Chapter 2 System Goals and Design Principles

This chapter explains why OpenAgenet (OAN) needs trusted resource infrastructure for the Internet of Agents (IoA), and how the system goals are realized through design principles covering identity, registration, trusted publication, content distribution, discovery, governance, and pre-call verification. The focus is not on listing isolated product features, but on explaining how these mechanisms work together to address unidentifiable resources, unverifiable origins, untraceable versions, undiscoverable capabilities, and insufficient information for decisions before invocation. It also clarifies the relationship among open interoperability, minimum trust, traceable evidence, and separation of responsibilities.

2.1 OAN and the Internet of Agents

OpenAgenet (OAN) provides resource interconnection infrastructure for the Internet of Agents (IoA). It is not primarily concerned with the conversational capability of a particular agent. Instead, it enables Agent Services, Skills, MCP Servers, and Tool APIs to have stable identities, verifiable control relationships, describable capabilities, traceable publication status, and reusable discovery entry points. OAN treats these objects as network resources, allowing them to participate in cross-node interconnection through a unified `did: oan`, resource metadata, registration protocol, and discovery protocol rather than through a manually maintained directory on a single platform.

The core result of OAN is to connect “what capability exists” with “whether that capability can be trusted.” The resource provider prepares a DID Document and resource description locally. After admission by a Registrar node, the Root platform verifies the Registrar node's authorization, the resource control proof, metadata, and version information. The content distribution platform then makes the materials available to authorized Discovery nodes. After receiving search results, the caller still performs pre-call verification using the DID, VC, Root proof, governance status, and endpoint information. This organization preserves differences among resource types and business protocols while providing common foundations for resource discovery, composition, and subsequent governance.

From the user's perspective, the goal of OAN can be summarized as four consecutive outcomes: the resource can be identified, its origin can be verified, it can be discovered within an authorized scope, and the caller can obtain sufficient grounds for judgment before connecting. These outcomes are jointly supported by identity materials, registration and publication evidence, discovery indexes, and caller-side verification. None of them can be replaced by a single page, directory, or search score.

Design goal	Main supporting mechanisms	Observable result for readers
Resources are identifiable	did:oan , DID Documents, resource metadata	Distinguish the resource subject, type, version, and service references
Origins are verifiable	Control proofs, registration VCs, Root-verified ResourcePackages	Check the control relationship, publication origin, and material hashes
Resources are discoverable	Authorization domains, capability tags, structured queries, and semantic retrieval	Find candidate resources by DID or task requirements
Decisions can be made before invocation	Version, lifecycle, governance status, endpoints, and local policy	Decide whether to proceed to a business invocation

2.2 Problems Addressed by OAN

When Agent Services, Skills, MCP Servers, and Tool APIs are distributed across different teams, code repositories, or runtime environments, resource providers often have to describe their locations and capabilities through webpages, documentation, configuration files, or manual communication. Such approaches make it difficult to express the controller, version, endpoint, dependencies, applicable protocols, and usage conditions in a stable way. They also make it difficult to determine whether a search result represents the current version, a historical version, or a resource that has already been suspended. As the number of resources grows, directory maintenance, duplicate registration, version updates, and cross-organization collaboration create continuing manual costs.

For callers, name or text matching can answer only “possibly relevant.” It cannot by itself answer whether the resource is genuine, whether it was published by an authorized node, whether it remains valid, or whether it can be used within the current authorization domain. If registration, publication, discovery, and invocation do not have clear responsibility boundaries, an intermediate directory may introduce unverified resources into results or continue displaying resources that have been revoked, replaced, or whose origins are unknown. OAN addresses these problems separately through DID control proofs, node authorization, Root publication, ResourcePackage hashes, authorization-domain filtering by Discovery nodes, and pre-call verification. It does not use a single platform label as a substitute for every trust judgment.

The problems can be mapped to OAN's handling as follows: dispersed resource endpoints first require common resource identities and service descriptions; difficult origin verification requires control proofs, registration VCs, and Root-verified ResourcePackages; opaque version status requires binding versions, hashes, and lifecycle states to resource records; and difficult cross-organization discovery requires Discovery nodes, authorization domains, capability tags, and semantic queries to work together. OAN therefore does not merely create another resource list. It organizes resource registration, origin verification, network propagation, and pre-call judgment into a traceable process.

```
{
  "problem": "The same task requires finding reusable capabilities across multiple teams and
    runtime environments",
  "resource": {
    "type": "skill",
    "did": "did:oan:example",
```

```

    "version": "1.0.0",
    "capabilities": ["repository-search", "project-structure-summary"]
  },
  "required_evidence": ["DID document", "control proof", "publication status", "endpoint policy"]
}

```

The example only represents a minimal shape for problem analysis and resource description. It is not a fixed production schema or a registration request that can be submitted directly. Actual fields, proofs, and validation rules are defined by the resource model, the DID method, the registration protocol, and the data schemas in the appendices.

2.3 Resource-First Interconnection Model

“Resource first” means that the primary objects of OAN registration, publication, distribution, and discovery are resource versions with a resource type, resource DID, description, capability tags, use cases, endpoints, versions, and states, rather than agent instances that must run on a unified platform. An Agent Service may provide a session- or task-oriented service; a Skill may serve as a reusable capability unit; an MCP Server may expose a collection of tools; and a Tool API may provide a specific interface. They differ in business form, but all enter the network through unified resource identity and metadata.

The unified model standardizes common properties without erasing type differences. The resource type determines required fields, endpoint shape, protocol bindings, invocation constraints, and version handling. The resource DID identifies a specific controllable object; the ResourcePackage binds the representation and proofs for a version; and capability tags and use cases support structured and semantic discovery. Resources may express dependencies and composition through DIDs, versions, or package hashes. However, “discoverable” only means that a resource satisfies directory and trust conditions; it does not automatically mean that every business invocation in a composition relationship is authorized.

The resource-first model can be understood through an abstract object: identity fields answer “which resource is this,” description fields answer “what can it do,” version and hash fields answer “which version of the material is being viewed,” endpoint and protocol fields answer “how can it be integrated,” and proof and status fields answer “why may it be treated as a candidate resource.” Different resource types may share these common fields, but this does not turn a Skill into an Agent Service, an MCP Server into a Tool API, or a discovery result into business invocation authorization.

```

{
  "resourceType": "skill",
  "resourceDid": "did:oa:example",
  "version": "1.0.0",
  "description": "Search a code repository and summarize the project structure",
  "capabilities": ["code-search", "structure-summary"],
  "protocol": "skill",
  "endpoint": "https://example.invalid/skill",
  "status": "published"
}

```

This is an example object for explaining the resource-first model. The formal fields, endpoint format, hashes, and signature materials for each resource type are defined in the corresponding chapters.

2.4 Trusted Registration, Publication, Discovery, and Invocation

The OAN trust path is not a single action completed by one request. It is a continuous process performed jointly by the resource controller, Registrar node, Root platform, content distribution platform, Discovery node, and caller. The resource controller first generates or imports identity materials locally and prepares the resource DID Document, resource metadata, capability tags, endpoint, and control proof. The Registrar node performs admission checks and submits upstream. The Root platform verifies authorization, signatures, hashes, versions, and governance conditions. Downstream nodes only propagate and index trusted facts that have already been formed. The caller independently verifies the relevant evidence before establishing a business connection.

For a resource version, readers can determine “how far the process has progressed” in this order:

1. Whether the local materials have been generated and the controller can use the corresponding private key to create the proof;
2. Whether the Registrar node has accepted the request and returned the registration result;
3. Whether the Root platform has formed the acceptance and trusted-publication facts for the version;
4. Whether the content distribution platform and Discovery node have completed validation, synchronization, and authorization-domain filtering;
5. Whether the caller has completed checks of the DID, VC, version, status, endpoint, and business policy.

Any failed step should retain a locatable status or error so that readers can determine whether the cause is a material problem, a node-authorization problem, propagation delay, an indexing problem, or a caller-side policy rejection.

2.4.1 Resource Registration and Control Proofs

Resource registration begins when the resource provider generates a resource version locally. The client should collect the name, description, capability tags, use cases, protocol bindings, endpoint, version, and dependencies according to the resource type, and establish a verifiable relationship among the resource DID, control key, public key, and DID Document. The material submitted to a Registrar node should prove that the requester controls the resource DID. The Registrar node may validate formats, fields, tags, endpoints, and policies, but its acceptance does not mean that the Root platform has accepted the resource.

A control proof answers “who can represent this resource in a registration request.” It cannot replace judgments about resource quality, endpoint availability, or invocation permissions. On

failure, the service should return locatable field or proof errors while keeping the local draft and key materials outside the service's control. A second submission for the same DID should distinguish a duplicate request for the same version from an authorized update to a new version, and dispatch them respectively to idempotency handling and version-update rules.

2.4.2 Root Verification and Trusted Publication

After receiving a signed upstream request from a Registrar node, the Root platform should first verify request freshness, request-body hash, nonce, and the Registrar node's valid authorization. It then verifies the resource DID Document, resource control proof, resource type, metadata hash, ResourcePackage, and version-transition conditions. Only when these facts are consistent and satisfy governance and publication policies does the Root platform form an acceptance record and Root-verified ResourcePackage for the resource version. The Root acceptance record should bind the DID, version, representation hash, authorization domain, status, time, or sequence information so that downstream nodes do not have to determine the origin from readable text alone.

After successful Root verification, publication normally proceeds asynchronously through persistent tasks, messages, or synchronization cursors. The resource provider may receive a registration result from the Registrar node, but the resource should not be described as discoverable across the network until content distribution, Root publication, and Discovery-node indexing are complete. Rejection, isolation, retry, and republishing should have observable states; one successful HTTP response must not be used to represent completion of the entire propagation path.

2.4.3 Content Distribution and Discovery-Node Indexing

The content distribution platform carries ResourcePackages, DID Documents, metadata, proofs, and materials required for indexing after the Root platform has verified them. A Discovery node obtains candidate versions from the distribution layer or Root notifications. The Discovery node must recalculate hashes, verify the Root proof and governance status, and filter by its own authorization domain before writing qualified resources into structured and semantic indexes. The distribution layer improves accessibility and synchronization efficiency, but it does not acquire the Root platform's trust authority.

Distribution and indexing are asynchronous. It is therefore normal for the Root platform to have published a resource while a Discovery node has not yet made it visible, for structured indexing to be complete while semantic indexing is retrying, or for a node to provide only an older snapshot temporarily. Implementations should retain synchronization cursors, version freshness, rejection reasons, and pending tasks so that operators can distinguish not published, not synchronized, validation failure, authorization-domain mismatch, and search-index failure.

2.4.4 Pre-Call Verification and Integration

After obtaining candidate resources from a Discovery node, the caller must not treat search results directly as invocation permission. At a minimum, it should verify the response origin and freshness, the resource DID Document, the resource control relationship, the registration VC or equivalent registration evidence, the Root-verified ResourcePackage, the ResourcePackage hash,

the current version, lifecycle status, the Discovery node's authorization domain, and the target endpoint's protocol and authentication declarations. For high-risk operations, the actual request should also include a timestamp, nonce, target DID, request-body hash, and signature to prevent replay or replacement of the result.

Successful verification only means that the candidate satisfies the prerequisites of the OAN trust path and the caller's local policy. Whether the call can ultimately execute still depends on authentication, authorization, business state, and security policy at the resource endpoint. A client should distinguish DID or signature errors, credential-status errors, Root-proof mismatches, expired versions, invalid governance states, authorization-domain mismatches, and endpoint invocation failures rather than reducing every problem to “resource not found.”

2.5 Separation of Resource Control, Node Authorization, and Governance

Resource control is expressed through the control key and control proof associated with the resource DID, answering “who may submit or update the document on behalf of this resource.” Node authorization is expressed through Root authorization materials and their governance status, answering “which Registrar node, Discovery node, or other infrastructure node is qualified to perform a particular network action.” On-chain governance records express governance facts such as changes to node authorization, answering “when an authorization becomes effective, is suspended, restored, or revoked.” A Root proof answers “whether a resource version has passed Root verification and entered the publication path”; it is not a credential of resource ownership.

These three categories of fact should be checked separately. Possession of a resource private key does not give an unauthorized node registration privileges. Possession of an old node authorization VC does not bypass the current governance status. Root publication of a resource does not replace the caller's judgment about endpoint and business permissions. This separation allows OAN to manage infrastructure authorization without receiving resource-control private keys, while allowing governance status, resource identity, and publication status to be updated independently.

Judgment question	Main evidence	Judgment it cannot replace
Who may submit or update a resource	Resource DID, control key, and control proof	Whether the node has infrastructure authorization
Which node may perform a network action	Node authorization materials and governance status	Whether that node controls a particular resource
Which version has entered the trusted publication path	Root proof, version, and hash	Whether the resource endpoint meets the caller's business requirements
Whether an invocation may actually be made	The caller's combined verification and local business policy	It cannot be decided only by a search result or page status

2.6 Federation, Interoperability, and Incremental Adoption

OAN is federated because multiple Registrar nodes and Discovery nodes can operate independently under the same Root trust and publication rules. Registrar nodes can provide resource admission for different organizations, teams, or integration scenarios. Discovery nodes can provide indexes based on authorization domain, network location, or service capability. The Root platform unifies verification and publication facts; it does not require every resource to be stored in one business platform. A third-party node may enter the corresponding operating state only after completing identity, authorization, interface, and conformance checks.

Incremental adoption can begin with one organization or an experimental network: first adopt `did:oan` resource identity and the registration-discovery flow, then connect existing Agents, MCP Servers, or Tool APIs, and later add multiple nodes, authorization domains, semantic tags, and governance read models. Interoperability means preserving OAN's boundaries for identity, proofs, versions, and status while connecting existing business systems through protocol adapters. It does not require a business system to become a unified runtime immediately.

An organization can proceed in the order “make visible first, make trusted next, federate later.” In the first phase, it organizes existing resources and establishes resource identities and descriptions. In the second, it integrates a Registrar node and forms control proofs, registration VCs, and Root publication evidence. In the third, it deploys or integrates Discovery nodes and enables structured and semantic queries. In the fourth, it adds nodes from multiple organizations, authorization domains, governance synchronization, and protocol mappings. Existing business endpoints can remain in place at every phase. OAN mainly provides resource identity, registration, publication, discovery, and pre-call judgment; it does not require an immediate replacement of existing business systems.

Resource organization and identity establishment

- > Registration and trusted publication
- > Discovery-node indexing
- > Multi-node integration and authorization-domain governance
- > Mapping to existing Agent, MCP, and Tool/API protocols

2.7 Control Plane, Data Plane, and Semantic Plane

The control plane maintains facts that determine whether a resource can enter the network, propagate, or be treated as valid, including identity, authorization, version, lifecycle, and publication status. The data plane carries DID Documents, ResourcePackages, metadata, proofs, and distribution content for clients. The semantic plane maintains capability trees, tag hierarchies, aliases, descriptions, and query explanations for registration recommendations and semantic matching during discovery. The Root platform performs three bounded hub roles at the same time, but facts maintained by the three planes cannot substitute for one another.

The three planes have a clear dependency order. Semantic matching can help a Discovery node find relevant resources, and data distribution can allow a node to obtain a ResourcePackage. However, a candidate has trusted-discoverable status only after the control plane confirms that the resource version has been accepted and published and is within the Discovery node's authorization

domain. Semantic relevance, endpoint accessibility, or interface presentation cannot grant DID control, node authorization, or governance qualification in reverse.

Plane	Main facts maintained	Typical participating components	Constraint imposed on other planes
Control plane	Identity, authorization, version, lifecycle, and publication status	Root platform, governance components, Registrar nodes, off-chain trust indexer	Determines which materials may enter propagation and trusted discovery
Data plane	DID Documents, ResourcePackages, metadata, proofs, and distribution content	Content distribution platform, Registrar nodes, Discovery nodes	Must preserve hashes, versions, and origins, and must not rewrite control-plane facts
Semantic plane	Capability tags, hierarchies, aliases, descriptions, and query explanations	Root semantic governance, Discovery nodes	Helps understanding and ranking, but cannot replace identity and authorization judgments

2.8 Public Visibility, Discoverability, and Invocation Authorization

Registration completed means that a Registrar node has accepted or processed the request. Associated evidence normally includes the registration response, registration VC, or registration record. Root publication completed means that the Root platform has accepted a resource version and formed the corresponding publication proof. Discovery-node indexing completed means that an authorized Discovery node has validated the ResourcePackage and written it into its local index. Visibility within an authorization domain means that the node allows the resource to be returned to a particular query caller according to its authorization scope. Invocation permission is the caller's local decision after verifying OAN evidence together with endpoint authentication and its own business policy.

These states may advance in sequence, or may temporarily diverge because of asynchronous tasks or faults. An older snapshot may continue to provide a limited set of verified historical results when refresh fails, but it must carry freshness or status information. Resource suspension, revocation, version replacement, and node-authorization changes should affect discovery and pre-call verification after the corresponding state has propagated. Any public portal, statistics page, or cache is only an observation interface; it cannot replace the Root acceptance record, governance status, or signature verification as a trust source.

The following diagram compresses the chapter's goals into a checkable resource-version flow. Each arrow represents the transfer of a fact or material to the next stage. It does not mean that the previous stage automatically grants authority for the next stage. Asynchronous distribution and indexing may create time differences, and the caller must still verify the result before the final connection.



Stage	Main responsible party	Fact produced or confirmed	Basis available to the next stage
Preparation	Resource controller/provider	Resource DID, DID Document, metadata, and control proof	Registration materials that can be submitted
Registration	Registrar node	Request acceptance, field validation, and registration result	Registration credential or upstream submission record
Publication	Root platform	Node authorization, resource version, and Root proof	ResourcePackage that can be distributed
Indexing	Discovery node	Local receipt, validation, authorization-domain filtering, and indexing status	Discovery result with origin status
Pre-call judgment	Caller/verification component	Identity, signature, version, governance, and endpoint checks	Local decision on whether to establish a business connection

Discoverable does not mean callable. A Discovery node returns candidate resources, but the caller must still make the final decision based on its permissions, endpoint authentication, resource status, and business risk.

The design principles in this chapter can be reviewed through three acceptance questions: Does the resource have a verifiable identity? Does publication have traceable evidence? Can the discovery result carry enough origin and status information to support pre-call judgment? If any answer depends only on page text or name matching, the trusted interconnection process is incomplete.

2.9 References

Source	Type	Link
OAN White Paper	arXiv White Paper: project vision, ecosystem value, and rationale	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv Yellow Paper: technical principles, trust boundaries, and system mechanisms	https://arxiv.org/abs/2606.03163
GRAIL Semantic Discovery Paper	arXiv paper: semantic discovery and resource matching	https://arxiv.org/abs/2605.02489
Agentic Overlay Network Architecture	IETF Internet-Draft: agentic network architecture	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

Chapter 3 Terms, Roles, and Trust Boundaries

This chapter defines the subjects, resources, credentials, nodes, and trust boundaries involved in resource interconnection within OpenAget (OAN). The role definitions support the protocols that follow; they are not abstract descriptions of organizational structures or deployment processes. The subject controls identities and resources, the Registrar node accepts and submits registrations, the Root platform authorizes, verifies, publishes, and governs semantics, the content distribution platform provides already-verified materials, the Discovery node synchronizes, filters, indexes, and returns results, and the caller verifies candidate resources and decides whether to connect. Each role's permissions should match the facts it can produce or verify.

Three relationships require particular distinction in OAN: whether the resource controller is authorized to represent the resource DID; whether an infrastructure node is recognized by the Root platform and current governance status; and whether a resource version has passed Root publication and been indexed by a Discovery node. DIDs, signatures, VCs, Root proofs, and on-chain governance events carry different facts. No role can replace a verification performed by another role merely because it stores a JSON document, receives a successful HTTP response, or has a search ranking.

When reading this chapter, place each term into four questions: who or what does it describe, what does it prove, who creates or stores it, and can it replace another piece of evidence? For example, a resource DID describes the controlled resource object, a control proof expresses the resource controller's operational authorization, node authorization expresses an infrastructure node's qualification to operate, and a Root proof expresses that a particular resource version has passed Root verification. All four may appear in one registration or discovery flow, but they must not be collapsed into a generic “trusted” field.

Judgment dimension	Typical question	Main evidence
Object identity	Which subject or resource does the current material describe?	Subject DID, resource DID, DID Document
Control relationship	Who can represent this object in a submission or update?	Control public key, control proof, signature
Infrastructure qualification	Which node is qualified to perform a network action?	Node authorization, governance status, authorization domain
Publication and visibility	Which version has been accepted, distributed, and indexed?	Root proof, synchronization record, discovery response

Judgment dimension	Typical question	Main evidence
Business invocation	Should the caller connect and invoke the resource?	Local verification result, endpoint authentication, business policy

3.1 Core Terms and Abbreviations

3.1.1 Unified Terminology Table

Full English term	Abbreviation
OpenAgenet	OAN
Internet of Agents	IoA
Decentralized Identifier	DID
DID Document	
<code>did: oan</code> Method	
Verifiable Credential	VC
Verifiable Presentation	VP
ResourcePackage	
Resource Metadata	
<code>oanMetadata</code>	
Resource Controller	
Resource Provider	
Agent Service	
Skill	
MCP Server	
Tool/API	
Registrar Service Node	
Discovery Service Node	
Root Platform	
Content Delivery Network	CDN
Trust Indexer	
User Agent	
Model Context Protocol	MCP
Agent2Agent Protocol	A2A
A2A Agent Card	
Software Development Kit	SDK
Application Programming Interface	API
Hypertext Transfer Protocol	HTTP
JavaScript Object Notation	JSON
Request for Comments	RFC
Root-verified ResourcePackage	
Root Proof	
Registration Verifiable Credential	
Resource DID	
Local Identity	
Node Authorization	
Governance Status	
Resource Version	

Full English term	Abbreviation
Service Endpoint	
Capability Tag	
Capability Tag Tree	
Semantic Discovery	
Structured Discovery	
Authorization Domain	
Control Plane	
Data Plane	
Semantic Plane	
Governance Authority	
Verifiable Control Proof	
Public HTTP API	
Message Authentication Code	
Resource Type	
Subject Type	
implementationLinks	
Protocol Binding	
Publication Cursor	
Publication Sequence	
Governance Event Sequence	
Synchronization Cursor	
Freshness	
Governance Event	
Publication Fact	
Discovery Index	
Data Integrity Proof	
Public Key Multibase	
Crypto Suite	
Nonce	
Hash Algorithm	
Package Claims	
Protocol Version	
Version Constraint	
Previous Version	
Challenge	
Proof	
Holder	
Controller	
Credential Subject	
Request ID	
Replay Protection	
Idempotency	
Rate Limiting	
Signature	
Discovery Query	
Query Projection	
Exact Lookup	
Lexical Search	

Full English term	Abbreviation
Hybrid Search	
Search Score	
Query Explanation	
Pagination Cursor	
Publication Manifest	
Notification Batch	
Publication Job	
Watermark	
Backoff	
Signed Request Envelope	
DID Control Challenge	
Subject Control Proof Bundle	
Registration Submission	
Discovery Response	
Index Snapshot	
Query Fallback	
Protocol Envelope	
Capability Vocabulary	
Governance Projection	
Sync Batch	
Correlation ID	
Checkpoint	
Event Cursor	
Capability Set	
Content Digest	
Audience	
Cross-Origin Resource Sharing	CORS
Entity Tag	ETag
Server-Side Request Forgery	SSRF
Web Cryptography API	
OpenAPI Specification	
Node Package Manager	npm
Base64url	
Base58	
JSON Web Key	JWK
Multibase	
Ed25519	
SM2	
SHA-256	
SM3	

DIDs in this manual are decentralized identifiers, and `did:oon` is the method identifier used by OAN. A DID Document is a resolvable document corresponding to a DID. It contains verification methods, service endpoints, and resource-related metadata. The DID control relationship is demonstrated by verifiable keys and signatures. A VC is a verifiable credential used to express registration, node authorization, or other claims for which the issuer assumes responsibility. A VP is a credential presentation organized for verification. A ResourcePackage is a distributable object

binding a resource version, metadata, hashes, and Root publication evidence.

The Registrar node is responsible for resource admission, request validation, control-proof verification, and submission to the Root platform. The Discovery node is responsible for synchronizing Root-published content, building indexes, and returning signed discovery responses. The Root platform performs three hub roles: trust authorization, data distribution, and semantic governance. The content distribution platform carries verified materials and is not a trust authority. MCP Servers, A2A, and Tool/API refer respectively to resource types or services and interfaces in external protocol contexts; similar terms do not imply the same lifecycle or permissions.

Names in the unified terminology table should be used directly in the body, interface descriptions, and examples. For example, registration materials may contain a resource type, resource DID, control proof, and service endpoint at the same time. “Resource type” is not “resource DID,” and a “service endpoint” is not a resource control identity. MCP is the name of an external protocol or a protocol identifier in a resource-type context and must not be generalized as a synonym for every tool resource. If an external protocol uses its own object name, explain the correspondence at the first mapping and use the names in Section 2.1.1 thereafter.

3.2 Resource Controller and Resource Provider

The resource controller holds the control key corresponding to the resource DID and is responsible for proving the authority to create, register, or update the DID Document and resource version. The resource provider supplies the actual capability, resource content, service endpoint, protocol binding, and usage instructions, and assumes business and operational responsibility for those claims. In an individual or small deployment, the two are usually the same subject. In an organizational setting, the controller may be an organization or platform, while the provider may be a team or system operating the service under delegation.

The Registrar node and Root platform verify the controller's control evidence and the consistency of the submitted materials. They must not infer DID control merely because the provider can access an endpoint, nor infer that a business endpoint is genuinely available merely because the controller holds a private key. Resource metadata should make the relationship between controller and provider explicit. When that relationship changes, it should be expressed through an authorization update, ownership transfer, or new resource-version rule rather than by changing only a display name.

```
{
  "resourceDid": "did: oan: example-resource",
  "controller": "did: oan: example-controller",
  "provider": {
    "name": "Example Team",
    "endpoint": "https://example.invalid/resource"
  },
  "controlProof": {
    "createdBy": "controller",
    "verification": "signature over resourceDid and version"
  }
}
```


This example shows that the controller and provider can be separate: the controller creates the signature proof, while the provider is responsible for the endpoint and capability declarations. It does not mean that the provider automatically obtains resource control, nor that a control proof guarantees the business quality of the endpoint.

3.3 Subject Identity and Local Identity

Subject identity is the identity context on which a resource control relationship depends. Local Identity is the client-side collection of subject configuration, control keys, resource keys, and necessary credentials. When a Local Identity is created, keys should be generated or imported in a client environment controlled by the user. A registration request sends only the DID Document, public keys, signatures, and public credentials required for protocol verification; it does not send resource-control private keys or subject private keys. Multiple resources may be controlled by the same Local Identity, but each resource still has its own DID, version, and control-proof boundary.

Subject identity is the identity context on which a resource control relationship depends. Local Identity is the client-side collection of subject configuration, control keys, resource keys, and necessary credentials. When a Local Identity is created, keys should be generated or imported in a client environment controlled by the user. A registration request sends only the DID Document, public keys, signatures, and public credentials required for protocol verification; it does not send resource-control private keys or subject private keys. Multiple resource DIDs may be controlled by the same Local Identity, but each resource still has its own independent control boundary under the protocol. Local Identity is better understood as a client-side container for keys and configuration rather than an account on a Registrar node. It may associate a subject DID, multiple resource DIDs, imported node configuration, and necessary public credentials, but it cannot change the independent control boundary of each DID.

3.3.1 Subject Identity and Resource Control Identity

A subject DID expresses the subject of a control relationship, while a resource DID expresses a resource object that can be registered, published, discovered, and updated independently. Subject identity and resource identity may be associated through the same key system, but they must not be conflated in the protocol. A subject private key proves that the subject can manage related resources; a resource-control signature proves that a particular operation on the resource DID is authorized. A runtime account of the provider, a web-login account, or an account for a Registrar node cannot automatically replace either type of DID control evidence.

3.3.2 Local Key Generation, Import, and Backup

The Local Identity flow should support creating a subject identity in the client, importing an existing identity backup, storing resource-control materials, and restoring them when needed. A backup file may contain the subject DID, public key, control key, resource configuration, and necessary credentials, but its sensitivity and custody responsibility must be explicit. During import, the structure and correspondence among keys must be checked; a filename or display text must not be treated as proof of identity authenticity. Downloads, imports, and restoration in a browser-based

registration flow are client-side operations. The server receives only the public materials required for protocol verification.

```
{
  "subjectDid": "did:oan:example-subject",
  "resources": [
    {
      "resourceDid": "did:oan:example-resource",
      "controlKey": "stored locally"
    }
  ],
  "publicMaterial": ["subject public key", "resource public key", "credential references"],
  "privateMaterial": "client-side only"
}
```

This is a conceptual example of a Local Identity backup. The actual backup format, encryption method, and fields depend on the client implementation. The private material in the example must not be uploaded to a Registrar node, the Root platform, or a Discovery node.

3.3.3 Private-Key Custody and Node Visibility

Resource-control private keys, subject private keys, and node-runtime private keys should be held by their respective controllers. A Registrar node, Root platform, Discovery node, content distribution platform, or off-chain trust indexer should not obtain these private keys from ordinary registration or discovery requests. Nodes may see public keys, signatures, and public credentials in DID Documents, and may retain submission records, Root proofs, and index data, but public evidence cannot be used to derive private keys. Server privileges, log privileges, and backup policies held by node operators remain part of the deployment security boundary and must be managed separately.

3.4 Root Platform

The Root platform is OAN's network-level coordination and verification center, but it is not the execution center for all business capabilities. It receives submissions from authorized infrastructure nodes, verifies identities, control proofs, resource types, versions, hashes, and publication conditions in registration requests, maintains trusted publication facts that nodes can synchronize, and coordinates notifications to the content distribution platform and Discovery nodes. The Root platform stores and issues proofs and statuses required for network verification; it should not hold the control keys of resource-provider business endpoints.

3.4.1 Trust-Authorization Hub

As the trust-authorization hub, the Root platform determines, according to governance status, which Registrar nodes, Discovery nodes, and other infrastructure participants are qualified to perform corresponding network actions, and provides eligible nodes with verifiable authorization materials. The subject of this judgment is node qualification, role, and authorization scope, not

the business quality of a resource. Nodes must also check whether authorization remains valid at runtime; they must not continue restricted operations solely because an old VC exists or a previous request succeeded.

The question answered by this hub is “can this node perform a particular network action,” not “is every item of business content returned by the node trustworthy.” Node authorization materials normally need to be associated with node identity, role, authorization domain, validity period, and governance status. When processing a restricted request, the executing node should check both the authorization material and current status rather than checking only whether the VC signature is valid.

3.4.2 Data-Distribution Hub

As the data-distribution hub, the Root platform organizes verified resource versions, DID Documents, ResourcePackages, hashes, Root proofs, and synchronization cursors, and makes verifiable materials available to authorized Discovery nodes through the content distribution platform. The content distribution platform may cache and transmit these materials in batches, but it must not modify facts accepted by the Root platform. A Discovery node must still recheck hashes, proofs, versions, and authorization domains after receipt.

```
{
  "publication": {
    "resourceDid": "did:oon:example-resource",
    "version": "1.0.0",
    "packageDigest": "sha256:example-digest",
    "rootProof": "available for verification",
    "publicationSequence": 42
  },
  "distribution": {
    "consumer": "authorized discovery service node",
    "verification": ["packageDigest", "rootProof", "governanceStatus"]
  }
}
```

This example distinguishes the publication fact formed by the Root platform from the transmission task performed by the content distribution platform. Successful distribution only means that the materials arrived; it does not mean that the receiver has completed verification, and it does not grant a Discovery node visibility beyond its authorization domain.

3.4.3 Semantic-Governance Hub

As the semantic-governance hub, the Root platform maintains capability tags and their hierarchy, identifiers, aliases, and versions for tag selection and recommendation during registration, and for semantic indexes and query explanations at Discovery nodes. Semantic governance determines how tags are organized and evolved, but it does not replace resource control proofs or node authorization. A Discovery node may use local indexing and ranking methods on the basis of Root vocabulary, but it must not extend its authorization domain through local tags.

The capability tag tree answers “how can capabilities be described and located with shared vocabulary.” It does not answer “who controls the resource” or “which node can publish the resource.” A Discovery node may map a natural-language task to one or more capability tags and return tag matches and relevance explanations, but the final result must still be associated with the resource DID, version, origin, governance status, and Root proof.

3.4.4 Root Platform Responsibility Boundary

The Root platform is responsible for network-level verification, authorization, publication, and coordination. It does not execute specific business invocations for resource providers, guarantee the business quality of resource endpoints, or automatically grant a caller business permission because a resource has been published. Public websites, metrics, maps, and governance-event displays are observation entry points, not replacements for Root facts. When the Root platform is unavailable, nodes may provide limited existing data under explicit freshness and fallback rules, but they must not fabricate new acceptance, authorization, or publication facts.

3.5 Registrar Service Node

The Registrar node is the admission and submission boundary through which resource providers enter OAN. It may provide forms, SDKs, or Skills, check DID syntax, resource types, required fields, capability tags, endpoints, hashes, and control proofs, and package qualified requests into signed submissions to the Root platform. A registration VC or registration result issued by the Registrar node proves only that it processed the relevant registration fact. It cannot replace a Root proof or allow the Registrar node to expand the resource's network visibility by itself.

The Registrar node must perform restricted actions using its own valid node authorization and permitted authorization domain, and must retain enough request IDs, verification results, error reasons, and submission status for auditing. It may reject malformed requests or requests with unclear control. When the Root platform rejects a request or asynchronous publication fails, the Registrar node must return the actual status and must not display local admission as network-wide publication completion.

From the evidence-chain perspective, the Registrar node produces at least three distinguishable results: whether the request arrived and passed local validation, whether it was submitted to the Root platform, and whether the Root platform accepted and published it. When troubleshooting registration, a client or operator should query these levels using the request ID, resource DID, version, and submission status rather than relying only on a page saying “submission successful.”

3.6 Discovery Service Node

The Discovery node is the authorized read side of Root-published content. It obtains candidate ResourcePackages from the content distribution platform or Root notifications, verifies Root proofs, hashes, versions, and governance status, filters resources according to its authorization domain, and builds structured and semantic indexes. Its responses to users, applications, and agents are locally processed discovery responses. It must therefore retain resource origin, version, freshness, and

verification status, and must assume responsibility for response content when signed responses are supported.

A Discovery node may optimize query performance, ranking, and semantic matching, and may retain an older snapshot for limited fallback display. It must not modify Root publication facts, add unauthorized resources to a trusted index, or use local ranking to override revocation, suspension, or version status. Search results are inputs to the caller's subsequent verification, not automatic invocation permission.

```
{
  "resourceDid": "did:oon:example-resource",
  "version": "1.0.0",
  "match": {
    "type": "semantic",
    "capabilities": ["code-search", "structure-summary"],
    "score": 0.91
  },
  "source": {
    "discoveryNode": "authorized-node",
    "publicationSequence": 42,
    "freshness": "known"
  },
  "verificationRequired": true
}
```

The score, tag match, and explanation in a discovery response belong to the index or query layer. `resourceDid`, `version`, `source`, and `freshness` support subsequent verification. Even a high relevance score cannot bypass DID Document, Root proof, governance-status, or endpoint checks.

3.7 Content Distribution Platform

The content distribution platform carries `ResourcePackages`, DID Documents, metadata, proofs, hashes, and synchronization materials that have been verified by the Root platform and permitted for propagation. Its goal is to improve the reachability, batch efficiency, and caching capability of node synchronization. It is a data-plane component, not an authority for governance or identity trust. Recipients must reverify distributed content using the Root proof, hashes, and status. The distribution layer may be operated by the same organization as the Root platform or by independent infrastructure, but that deployment relationship must not change the protocol responsibility boundaries.

3.8 Off-Chain Trust Indexer

The off-chain trust indexer is the read-side projection component for on-chain governance facts. It reads governance events such as node authorization, authorization-domain changes, suspension, restoration, and revocation, and writes authorization status into a queryable model for the Root platform, Registrar nodes, and Discovery nodes according to sequences or cursors. Projection results may be delayed or affected by faults. They must retain the last processed position, event source,

projection time, and error status. Runtime code must not equate “seen by the indexer” with the final on-chain fact.

When the Root platform or another executing node uses indexer status, it must define expiration, staleness, and fallback behavior. For actions that must prevent an unauthorized node from continuing, the node should fail closed when status cannot be confirmed or exceeds the allowed freshness. Public statistics or historical displays may show delay and old data, but must not change the actual authorization decision.

```
{
  "governanceProjection": {
    "nodeId": "did: oan: example-node",
    "role": "discovery",
    "authorizationDomain": "example-domain",
    "status": "active",
    "eventSequence": 128,
    "source": "governance event stream",
    "projectedAt": "2026-09-09T00:00:00Z"
  },
  "decision": "allow indexed discovery within the authorization domain"
}
```

This projection is an off-chain read model of on-chain governance events and must retain the event sequence and projection time. If the sequence is interrupted, the status is stale, or the source cannot be verified, the executing node should follow the failure policy for sensitive actions rather than treating the last successful projection as permanent authorization.

3.9 Caller, User, Application, and Agent

The user is the person expressing an intent to register, discover, or use a business capability. The browser or another client is the tool that carries the Local Identity and requests. The application is software that calls OAN interfaces and executes local policies. The user agent is an agent that can query and initiate calls on behalf of the user. These roles may appear together in one flow, but their sources of authority differ: the user controls local keys, the client protects private materials, the application verifies network evidence, and the user agent selects resources based on the verification result.

During discovery, the caller is a relying party. During an actual business invocation, it must also handle request signing, credential presentation, replay protection, and local authorization policies. It must not trust only search ranking, a website display name, or a resource description forwarded by a third party. Before connecting, it should verify the DID, VC, Root proof, governance status, version, and endpoint declaration, and decide whether to continue according to the concrete business risk.

3.10 Governance Authority and Authorization Domain

The governance authority decides whether an infrastructure node enters, remains within, or exits OAN's trusted operating scope. An authorization domain is the capability or resource scope

that a node is permitted to process. It may constrain which resources a Registrar node accepts, which resources a Discovery node synchronizes and returns, and which network actions a node may perform. An authorization domain is not a resource control relationship or a capability tag: tags describe and match capabilities, while authorization domains limit permissions and visibility.

Governance status may be formed jointly from on-chain events, Root runtime status, and node authorization materials, but an executing node must state the source and priority of each fact. A historical authorization VC, cached status, or manually maintained display must not override a current revocation or suspension. When authorization-domain changes propagate with delay, the node should follow the prescribed freshness and fail-closed rules for sensitive actions.

An authorization domain can be understood as “resource scope + action scope + observation scope.” Resource scope limits which resources or resource types a node can process. Action scope limits whether the node can register, synchronize, index, or perform management actions. Observation scope limits which callers or network areas may receive results. These three scopes jointly determine effective node permissions; a broad `active` status cannot replace them.

3.11 Trust Boundaries and Responsibility Matrix

The OAN responsibility matrix should be established along four dimensions: producing facts, verifying facts, storing status, and making external statements. The resource controller produces resource identity and control proof. The Registrar node verifies admission materials and submits registration. The Root platform verifies node authorization and resource versions and forms publication facts. The content distribution platform transfers verified materials. The Discovery node verifies and indexes them within its authorization domain. The off-chain trust indexer projects governance status. The caller verifies candidate resources and executes local invocation policy. Every action should point to a corresponding signature, credential, hash, governance event, interface response, or persistent record.

3.11.1 Resource Controller and Resource Provider

The resource controller is responsible for the resource DID, control key, and update authorization. The resource provider is responsible for capability descriptions, resource content, endpoints, and business availability. They may be the same subject or may be separated through an explicit delegation or operating relationship. The Registrar node and Root platform should determine resource-operation permission only through a verifiable control proof. They must not infer that the provider controls the resource DID merely because it submitted a service address or successfully logged in.

3.11.2 Registrar Node, Root Platform, and Discovery Node

The HTTP interfaces of these three roles correspond to different trust boundaries. The resource publisher submits a `ResourceRegistrationSubmission` to the Registrar node. The Registrar node submits a `ResourceVerifyAndPublishRequest` containing `registrarDid` and `upstreamAuth` to the Root platform. The Discovery node returns a `ResourceDiscoveryResponse` to the caller. The first interface receives resource-control materials, the second proves that an authorized node is representing the

network, and the third provides local-index candidates. Interface reachability, request parseability, and candidate visibility are different facts and cannot replace one another across roles.

The Registrar node accepts, validates, and submits registrations. The Root platform verifies node authorization and resources, performs trusted publication, and coordinates distribution. The Discovery node synchronizes, verifies, filters by authorization domain, and serves queries over published materials. A Registrar result cannot replace a Root publication fact, and a Discovery result cannot rewrite Root status. Interfaces, proofs, and states among the three must remain traceable.

3.11.3 Caller and Verifier

The caller may be a user, application, or user agent. It submits queries, receives candidate resources, and decides whether to establish a business connection. The verifier may be the caller itself, an SDK, or a verification component trusted by the caller. It checks DIDs, signatures, VCs, Root proofs, governance status, versions, endpoints, and invocation policies. A verification component may provide a verification result, but it must not directly execute a high-risk business action without the caller's policy authorization.

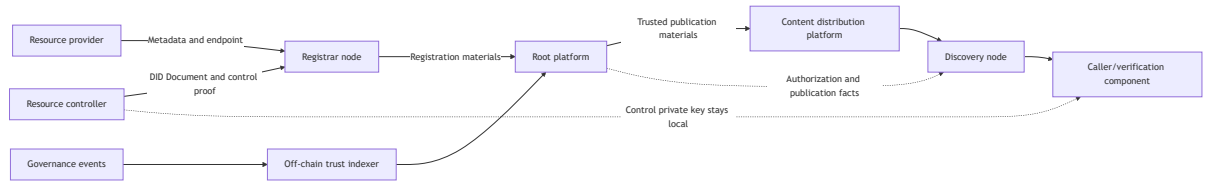
3.11.4 Responsibility for Key Actions

Resource creation and updates belong to the resource controller. Registration acceptance and registration credentials belong to the Registrar node. Root acceptance and trusted publication belong to the Root platform. Material transfer belongs to the content distribution platform. Authorization-domain indexing and discovery responses belong to the Discovery node. Governance-event projection belongs to the off-chain trust indexer. Final business invocation authorization belongs to the resource endpoint and the caller's local policy. A public website, statistics interface, or manually maintained event table is an observation and presentation layer and cannot serve as protocol authority for these key actions.

The boundaries among roles can be checked by asking who produces facts, who verifies facts, who stores status, and who makes external statements. “Responsible” in the table below means protocol responsibility; it does not mean responsibility for all business content or runtime environments.

Role	Main facts produced	Main facts verified	External result provided	Responsibility not replaced
Resource controller	DID Document, control proof, resource update	Own keys and materials to be published	Controlled resource materials	Cannot declare Root publication by itself
Resource provider	Capability description, endpoint, version, usage conditions	Resource business content	Resource description available for registration	Does not necessarily control the DID
Registrar node	Registration acceptance, field validation, registration credential	Control proof, fields, admission qualification	Registration materials submitted to Root	Does not replace Root publication

Role	Main facts produced	Main facts verified	External result provided	Responsibility not replaced
Root platform	Node authorization, publication fact, Root proof	Node authorization, resource version, hash, control proof	Trusted publication materials available for distribution	Does not replace business-endpoint authorization
Content distribution platform	Transfer, cache, and distribution records	Publication-material integrity	Publication artifacts available for retrieval	Does not grant resource or node permissions
Discovery node	Local synchronization, verification, and index records	Publication proof, authorization domain, lifecycle	Discovery results within authorization scope	Does not change Root facts
Off-chain trust indexer	Governance-status read model	On-chain governance events and ordering	Node-authorization status query	Does not create on-chain governance facts
Caller/verification component	Local verification and invocation decision	DID, VC, VP, endpoint, and business policy	Decision on whether to establish a connection	Does not replace business authorization at the resource service



When a key action occurs, the corresponding signature, credential, hash, governance event, synchronization cursor, or interface response should be locatable. If a result has only “a node returned success” as evidence and its source and verification scope cannot be explained, it can be treated only as an observation of runtime status, not as a cross-role trust proof.

When troubleshooting a resource or node, establish the evidence chain in this order: identify the object and version, confirm the control proof or node authorization, check the publication fact and governance status, and finally inspect the discovery index, endpoint, and caller policy. This order applies to registration failures, stale discovery results, and propagation after node-authorization revocation. If any step lacks a source or cannot be verified, limit the conclusion to the observable scope.

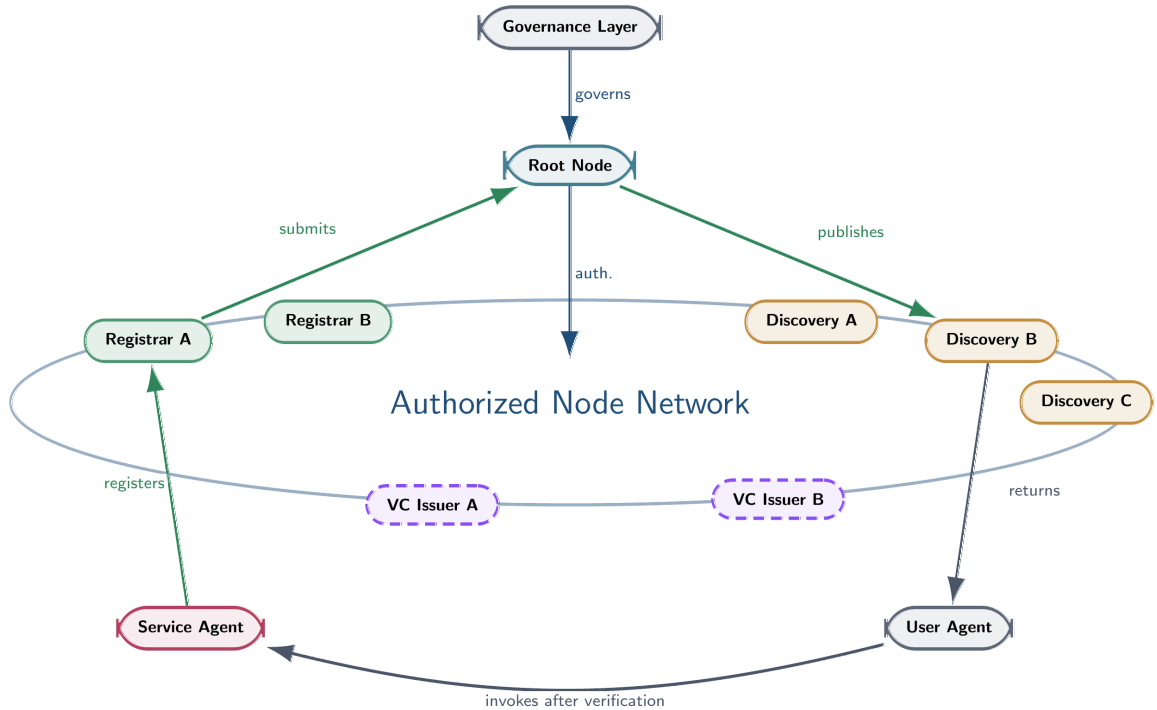
3.12 References

Source	Type	Link
oan-protocol-common	Code repository: shared protocol types, identity objects, and trust materials	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv Yellow Paper: roles, trust boundaries, and verification mechanisms	https://arxiv.org/abs/2606.03163

Source	Type	Link
did:oan DID Method Specification	Standard/method specification: DID method and identity objects	https://github.com/OpenAgenet/ oan-public-docs/blob/main/did- oan-specs/doc/OAN%20DID% 20Method%20Specification.md

Chapter 4 Overall Architecture and Network Topology

This chapter describes the federated architecture adopted by OpenAgetnet (OAN): coordination by a root platform, access through multiple registrar service nodes, propagation through a content distribution platform, indexing by multiple discovery service nodes, and a governance read model provided by an off-chain trust indexer. The OAN network can simultaneously accept registrar service nodes and discovery service nodes operated by the official organization and third parties. Each node participates in the network with an independent DID, role, authorization domain, and runtime status. A resource controller can submit a resource to an appropriate registrar service node, and a caller can query different discovery service nodes. The root platform uniformly verifies node eligibility, resource versions, and publication facts. The content distribution platform propagates verified materials to authorized discovery service nodes, and each discovery service node uses those materials to maintain its own structured and semantic indexes.



Three types of flows are connected within the same resource lifecycle, but they do not replace one another. The data flow carries DID Documents, resource packages, metadata, digests, and proofs; the control flow carries node authorization, request authentication, publication acceptance,

lifecycle, and governance status; and the semantic flow carries the capability tag tree, tag aliases, capability descriptions, use cases, and query explanations. An independent deployment may adjust its storage, messaging, and network implementation, but it must not allow the content distribution platform to acquire governance authority, allow a discovery service node to rewrite root-platform publication facts, or allow semantic relevance to replace identity and authorization verification.

4.1 Logical Architecture

The client is the primary operational boundary for resource controllers and callers. It stores local identities and private keys, generates or imports resource-control materials, and sends requests to any registrar service node or discovery service node that meets its authorization-domain and service requirements. Each registrar service node independently stores its local registration records, registration credentials, and submission status, and packages validated resource registration requests into requests submitted to the root platform. The root platform stores network-wide authorization status, acceptance records, resource versions, publication proofs, and pending tasks. The content distribution platform provides multiple authorized discovery service nodes with root-platform-verified resource packages, DID Documents, and metadata. Each discovery service node separately stores its synchronization cursor, verified resource packages, rejection records, structured index, and semantic index. The off-chain trust indexer stores a projection of on-chain governance events into node authorization status, providing a unified read model for third-party node onboarding, suspension, and authorization-domain changes.

The logical connections can be summarized as follows: the client-to-registrar-service-node connection is the resource access flow; the registrar-service-node-to-root-platform connection is a protected upstream submission flow; the root-platform-to-content-distribution-platform connection is a publication data flow; the content-distribution-platform-to-discovery-service-node connection is a synchronization flow; the discovery-service-node-to-client connection is a query response flow; and the on-chain governance system-to-off-chain-trust-indexer-to-executing-node connection is an authorization-state flow. The root platform also provides discovery service nodes with publication notifications or summaries, allowing them to obtain the current resource package by cursor instead of relying on registrar service nodes to write directly to discovery indexes.

The following logical configuration illustrates component boundaries in one deployment instance. Field names are provided only to explain role relationships. Domains, ports, databases, and messaging components in an actual environment are deployment configuration, not fixed protocol values.

```
network:
  root_platform:
    role: root
    publishes: [node-authorization, resource-publication, capability-vocabulary]
  registrar_nodes:
    - id: registrar-a
      role: registrar
      submits_to: root_platform
      operator: official
    - id: registrar-b
```

```

    role: registrar
    submits_to: root_platform
    operator: third-party
  discovery_nodes:
    - id: discovery-a
      role: discovery
      syncs_from: root_platform
      operator: official
      authorization_domain: domain-a
    - id: discovery-b
      role: discovery
      syncs_from: root_platform
      operator: third-party
      authorization_domain: domain-b
  trust_indexer:
    role: governance-read-model
    projects: governance-events

```

This topology organizes the network by node roles and authorization relationships rather than limiting its scale to a fixed set of service instances. After completing node identity, authorization-domain, interface, security, and conformance checks, a third-party operator can run a new registrar service node or discovery service node. The new node cooperates with existing nodes through common publication facts and governance status recognized by the root platform, while retaining independent deployment, storage, indexing, and service policies.

4.2 Root Platform as the Trust and Authorization Hub

The root platform's trust and authorization hub responsibilities apply to infrastructure participants, not to the business quality of resources controlled by resource controllers. The root platform should maintain verifiable node identities, node roles, authorization domains, and governance status. When processing registration submissions, publication tasks, content distribution, and discovery notifications, it should check whether the relevant nodes remain effectively authorized. The off-chain trust indexer provides the root platform with a read-side projection of on-chain events, but the projection may have synchronization latency. For high-risk writes, the root platform should decide whether to accept, reject, or suspend processing according to explicit freshness, missing-data, and invalidation rules.

Node credentials prove that “the node has a certain type of network qualification”; resource-control proofs prove that “the requester can represent a resource DID”; and root-platform publication proofs prove that “a particular resource version has passed root-platform verification.” These three types of proof cover different facts, and none can replace another. At a minimum, an authorization check should bind the node DID, role, authorization domain, status, validity period or governance sequence, and record the verification result as an auditable accepted, rejected, or pending status.

The minimum input for a root-platform authorization decision can be abstracted as follows:

```
{
```

```

"nodeDid": "did:oon:example-registrar",
"role": "registrar",
"authorizationDomain": "example-domain",
"governanceStatus": "active",
"requestedAction": "submit-resource-registration",
"observedAt": "2026-09-09T00:00:00Z"
}

```

The output should at least distinguish `allow`, `deny`, and `indeterminate`. `indeterminate` means that the authorization status, governance projection, or a critical proof cannot currently be confirmed. It must not be silently converted into an `allow` decision. The specific status codes and error structures are described in the registration, governance, and API chapters.

4.3 Root Platform as the Data Distribution Hub

After a resource version passes verification, the root platform organizes the DID Document, resource metadata, resource package, DID Document digest, metadata digest, resource package digest, resource type, version, lifecycle status, authorization domain, and root-platform trusted publication proof into a publication artifact that can be carried by the content distribution platform. A publication artifact must have explicit version and digest boundaries. After obtaining the materials, a discovery service node must be able to independently recompute digests, verify the proof statement, and determine the current version without relying on an informal statement or interface status from the content distribution platform.

Consistency among different discovery service nodes does not require them to have identical local databases. It requires them to validate the same root-platform publication fact using the same resource version, digests, proof, and governance status. The root platform establishes a traceable order through the publication cursor, resource DID, and version. The content distribution platform may use caching, batching, or retries to improve transmission efficiency, but it must not present an old version as the current version or modify resource-package content without a new publication fact.

```

{
  "publicationArtifact": {
    "resourceDid": "did:oon:example-resource",
    "version": "1.2.0",
    "didDocumentDigest": "sha256:did-document",
    "metadataDigest": "sha256:metadata",
    "packageDigest": "sha256:package",
    "publicationSequence": 108,
    "status": "published"
  },
  "consumerCheck": ["digest", "version", "rootProof", "governanceStatus"]
}

```

This object represents the logical composition of a distributable publication artifact; it does not mean that every deployment must use exactly the same field names. The essential requirement

is that the receiver can bind the resource DID, version, each type of digest, and the root-platform trusted publication proof, and then write the material into its read model only after local verification.

4.4 Root Platform as the Semantic Governance Hub

The root platform maintains the capability tag tree and its versions, hierarchical levels, tag identifiers, Chinese or English labels, aliases, and parent-child relationships, and provides this governed vocabulary to both registrar service nodes and discovery service nodes. During registration, a client can select tags for a resource based on its description or receive tag recommendations. During discovery, a discovery service node can convert the user's natural-language task, tag conditions, and structured filters into local query conditions. The tag tree is used to unify semantic boundaries and support authorization-domain decisions; it does not mean that the root platform evaluates the business quality of a resource on behalf of its provider.

Semantic governance and semantic retrieval have clearly separated responsibilities: the root platform maintains shared vocabulary and governance facts, while discovery service nodes are responsible for local indexing, tokenization, vectors, or other ranking implementations. A discovery service node may retain local semantic aliases and ranking signals, but its returned results should still retain the resource source, version, authorization domain, and verification status. A result with high relevance but without a valid control proof, root-platform publication proof, or governance status must not enter the trusted discovery results.

Tag governance can be expressed as a versioned vocabulary entry:

```
{
  "vocabularyVersion": "2026-09",
  "tag": {
    "id": "code-repository-analysis",
    "preferredLabel": "Code Repository Analysis",
    "aliases": ["Code Search", "Project Structure Summary"],
    "parent": "software-development",
    "status": "active"
  }
}
```

A discovery node can generate candidate conditions based on aliases and natural-language queries, but returned results must still refer to the resource DID, publication version, and verification status. Changes to the vocabulary version affect query interpretation and ranking; they must not silently change the identity or digest of an already signed resource package.

4.5 Publication Path from a Registrar Node to the Root Platform

A normal publication consists of local preparation, access validation by a registrar service node, a signed submission from the registrar service node, root-platform authentication and verification, resource-version acceptance, content distribution, and discovery notification. A registrar service node may return a registration result or registration credential, but a resource is eligible to enter

the propagation path only after the root platform has completed resource-version verification and generated a publication fact. Publication tasks are usually asynchronous. The registration response, root-platform acceptance, completion of content distribution, and completion of discovery indexing should be recorded and displayed separately.

Failure handling must preserve the stage at which the failure occurred. Client field errors, DID Document structure errors, and control-proof errors are access or verification failures. An unauthorized registrar service node, an expired upstream signature, or a request-body digest mismatch is a node-authentication failure. A resource type, version, digest, or governance condition that does not satisfy root-platform requirements is a publication rejection. Temporary unavailability of content distribution or discovery indexing is a propagation-stage failure. Only temporary failures that can be safely retried should be retried automatically, and retry requests must still satisfy freshness and idempotency constraints.

```
Client -> Registrar service node: Resource materials and control proof
Registrar service node -> Root platform: Node-signed upstream submission
Root platform -> Registrar service node: accepted / rejected / pending
Root platform -> Content distribution platform: Verified publication artifact
Content distribution platform -> Discovery service node: Publication materials and cursor
```

Each response in this path describes only the result of the current stage. For example, `accepted` may mean that the root platform accepted the resource version, but it does not mean that a discovery service node has completed indexing. `pending` means that the task is still progressing asynchronously. The caller and operator should continue to determine subsequent results from status interfaces or synchronization records.

4.5.1 Access Validation by a Registrar Node

A registrar service node first checks whether the request conforms to the resource registration model, including the resource DID, DID Document, resource type, subject type, resource name and description, capability tags, use cases, protocol bindings, service endpoints, authorization domain, version, and digest consistency. It should also verify the relationship between the resource DID and the DID Document subject, control keys, and control proof, and confirm that the submitted resource type is consistent with the semantic encoding of the DID. Recommended tags can assist with completion, but cannot replace required fields or proof of control.

4.5.2 Upstream Submission and Identity Authentication

The submission sent by a registrar service node to the root platform is not an unprotected form. It is a protected request containing the resource registration content, request identifier, protocol version, target path, timestamp, nonce, request-body digest, sender DID, and signature. The root platform uses this information to verify the registrar service node's identity, authorization domain, and request freshness, and then verifies the resource registration credential, resource control proof, and resource-package statement contained in the request. The submitter's private key is used only within the registrar service node's own node environment; the resource controller continues to retain the resource-control private key.

4.5.3 Root-Platform Verification and Publication Confirmation

During verification, the root platform simultaneously checks node authorization, the resource DID Document, resource control proof, registration credential, resource-type binding, metadata digest, resource-package digest, version relationship, authorization domain, and lifecycle conditions. After successful verification, the root platform records the resource-version acceptance fact, generates a root-platform trusted publication proof containing the resource DID, version, digests, status, authorization domain, and relevant governance or publication references, and marks subsequent content-distribution and discovery-notification tasks as executable. This confirmation does not mean that a discovery service node has completed indexing.

4.5.4 Failure, Rejection, and Retry

The root platform should distinguish an idempotent retry of the same resource DID, version, and digest from an update request in which content or version changes under the same DID. The former can return the existing acceptance result or publication status. The latter must re-verify control, version relationship, and publication conditions. An invalid signature, replayed nonce, invalid node authorization, digest mismatch, or inconsistent resource type must not be bypassed through simple retries. Temporary errors such as network timeouts or temporary downstream unavailability can enter a backoff-based task retry, while retaining the association with the original request.

4.6 Distribution Path from the Root Platform to Discovery Nodes

The root platform provides publication content and synchronization positions to discovery service nodes through the content distribution platform or protected synchronization interfaces. A discovery service node requests incremental materials according to its locally stored cursor and can perform controlled backfill when the cursor is missing or recovery is required. For every candidate resource, the discovery service node must obtain the current resource package, verify the DID Document, metadata, digests, root-platform trusted publication proof, lifecycle status, and authorization domain, and then write the verified version into its local index. Materials that fail verification should enter a traceable rejection record rather than being silently discarded.

The ultimate goal of the synchronization path is for each discovery service node to form a read model that is “locally queryable, source-traceable, and status-verifiable.” A discovery index may use relational tables, a full-text index, a semantic index, or a cache, but each index record should retain the resource DID, version, package digest, publication time or cursor, source, and verification status. Ranking and summaries in a query response are derived information and cannot override root-platform publication facts.

4.6.1 Distribution Content and Publication Checkpoints

Distributable content includes at least the resource DID, resource type, resource version, DID Document, resource metadata, resource-package digest, metadata digest, DID Document digest, authorization domain, lifecycle status, root-platform trusted publication proof, and the necessary

publication cursor or source reference. A publication checkpoint should answer whether the resource has been accepted by the root platform, whether the content has been written to the distribution layer, whether a discovery service node has obtained and verified it, and at which stage a failure remains.

```
{
  "resourceDid": "did:oa:example-resource",
  "version": "1.2.0",
  "checkpoints": {
    "rootAccepted": true,
    "publishedToDistribution": true,
    "discoveryFetched": false,
    "discoveryVerified": false,
    "discoveryIndexed": false
  },
  "lastError": null
}
```

This checkpoint object is used to explain propagation progress; it does not compress multiple states into a single `available` field. When a discovery service node has not yet obtained or verified the material, the resource may already have completed root-platform publication, but it should not be described as discoverable on that node.

4.6.2 Discovery-Node Synchronization and Indexing

After obtaining materials, a discovery service node first completes source, integrity, and governance verification, then writes the verified resource package to a table or equivalent storage, and subsequently updates the structured and semantic indexes. The structured index can support queries by DID, resource type, version, capability tags, and authorization domain. The semantic index can use capability descriptions, tag aliases, use cases, and protocol bindings for matching. Every index write should be associated with the verification result; the node must not display a result first and perform trust verification later.

4.6.3 Sequence Numbers, Freshness, and Convergence

A publication cursor expresses the propagation order of root-platform publication materials. A discovery service node's synchronization cursor expresses the position that the node has processed. The two are not equivalent. A discovery service node should record the last successfully applied cursor, the time at which the material was received, the resource version, and the verification result, and should determine whether an old snapshot can still be used for limited display according to the permitted freshness. Advancement of the cursor means that the synchronization position has advanced; it does not mean that every resource is ready to be invoked.

```
{
  "nodeId": "did:oa:example-discovery",
  "rootPublicationCursor": 108,
  "localAppliedCursor": 105,
  "lastSuccessfulSyncAt": "2026-09-09T00:00:00Z",
}
```

```

    "freshness": "lagging",
    "pendingRanges": [[106, 108]]
  }

```

When the local cursor falls behind, the node can report lag and continue processing verified old snapshots, but it must not disguise `localAppliedCursor` as the latest root-platform publication sequence. After a gap is recovered, the node should re-verify every publication material in the missing range before advancing the local cursor.

4.6.4 Distribution Failure and Recovery

Content-distribution failure, network interruption, temporary unavailability of a resource package, and discovery-service-node database failure should all support recovery. A recoverable task should retain the resource DID, target version, source cursor, and retry information. When a digest mismatch, proof mismatch, or invalid governance status is encountered, the task should enter a rejected or isolated state and wait for new publication materials or a governance change rather than retrying indefinitely. After recovery, processing should continue from a persisted cursor or controlled backfill point to avoid skipping publication records.

4.7 Multiple Registrar Nodes and Discovery Nodes

Multiple registrar service nodes can provide access for different organizations, resource types, or authorization domains, while multiple discovery service nodes can provide indexing for different network regions, user groups, or query policies. They share the root platform's basis for judging identity, authorization, resource versions, and publication proofs, but do not need to share the same database or local ranking. When a resource is submitted repeatedly through multiple registrar service nodes, the root platform should identify an idempotent request or an update request based on the resource DID, version, digest, and control proof. When a discovery service node receives the same publication material repeatedly, its indexing should remain idempotent.

Federation does not mean that arbitrary nodes write to one another. A registrar service node may accept and submit only within its authorization domain. A discovery service node may synchronize and return only the published resources allowed within its authorization domain. Propagation between nodes should primarily pass through root-platform publication facts and the content distribution path. A failure of one discovery service node does not make root-platform facts disappear; another discovery service node that has completed synchronization can continue providing results, while the failed node's local visibility and freshness should be reported accurately.

4.7.1 Access by Multiple Registrar Nodes

The root platform maintains an independent node identity, role, and authorization domain for each registrar service node. Different registrar service nodes can submit the same resource, but changing the registrar service node cannot grant new resource control; an update to the same resource still requires a valid control proof from the resource controller and must satisfy the version-update rules. A registrar service node's local registration record is evidence on the intake side; the root platform's acceptance record is the basis for determining network publication.

4.7.2 Synchronization by Multiple Discovery Nodes

Each discovery service node has an independent synchronization cursor and local index state, and can obtain the same root-platform publication facts from the content distribution platform according to its authorization domain. Nodes should not treat local search results as authoritative data for another node. When consistency needs to be compared, the comparison should use the resource DID, version, package digest, root-platform proof, and governance status. Different result counts or rankings over a short period may result from synchronization delay or local policy, but not from arbitrary rewriting of root-platform facts.

4.7.3 Data Consistency Between Nodes

Consistency between nodes should distinguish fact consistency from read-model consistency. Fact consistency requires the DID Document, metadata, resource-package digest, and root-platform proof of the same publication version to be consistent. Read-model consistency permits differences in indexing time, ranking, caching, and semantic interpretation, but each difference must be explainable by the cursor, freshness, or authorization domain. When a discovery service node detects a version conflict, it should prioritize the root-platform publication sequence and proof, rather than letting local arrival time override the trusted source.

When comparing two discovery service nodes, the following stable fields should be compared before result counts or rankings:

(resourceDid, version, packageDigest, rootProof, governanceStatus)

When these fields are consistent, the two nodes generally have the same core understanding of the publication fact. Differences in indexing time, semantic score, pagination position, and cache time are read-model differences. If the stable fields differ, the root-platform publication records and governance-event sequence should be consulted to locate the cause.

4.7.4 Node Failure and Isolation

When a node exhibits signature anomalies, expired authorization, persistent synchronization failure, publication-material tampering, or non-conforming interface behavior, it should be isolatable so that restricted writes are no longer accepted from it or new authorization scope is no longer provided to it. Isolation does not mean deleting historical facts. The root platform and other nodes should retain auditable records and decide whether to re-enable the node based on governance status and recovery checks. A failure of an ordinary query node can be handled by another node, but this must not expand the other node's authorization domain.

4.8 Third-Party Node Onboarding Model

Third-party node onboarding starts with node identity materials and an operating role, rather than simply opening an HTTP port. The applicant should submit the node DID Document, public verification methods, service endpoint, proposed node role, authorization domain, and operating contact or operational materials. The governance authority or root platform then performs qualification review and authorization. After obtaining test status, the node must pass tests for identity,

signatures, interfaces, resource-package verification, synchronization, error handling, and status updates. A production node then provides services within its formal authorization domain.

Test nodes and production nodes should be isolated in authorization status, visibility, data retention, and network impact. A test node can use test resources and test governance status to validate an implementation, but must not mix test publication facts into a production discovery index. A production node's authorization revocation, suspension, or scope change must be recognizable at runtime and affect restricted operations.

4.8.1 Pre-Onboarding Authorization

Pre-onboarding authorization should at least confirm that the node DID can be resolved, verification methods are available, the node role is explicit, endpoint ownership is clear, request signatures can be verified, the authorization domain can be expressed, and the operator can explain key custody, data storage, and failure handling. Authorization materials should bind the node identity and role; a domain name, IP address, or human-readable name alone must not serve as proof of node eligibility.

4.8.2 Node Capabilities and Authorization Scope

The node capability description should distinguish between “interfaces the node can provide” and “actions the node is authorized to perform.” A discovery service node may support structured and semantic queries but be authorized to index only certain resource domains. A registrar service node may support access for all four resource types but be authorized to accept only some of them. The authorization scope should map to the node's actual filtering, submission, and response logic. Requests outside the scope must be rejected or omitted from responses.

4.8.3 Protocol Conformance Checks

Conformance checks should cover health checks, DID Document parsing, signature and digest verification, node authorization status, resource registration, root-platform publication, content distribution, discovery synchronization, query responses, and error contracts. Test results should record the input, node version, configuration, expected result, actual result, and evidence location. Passing an interface-connectivity test does not mean that the node has passed trust-boundary tests.

The minimum onboarding record for a third-party node can be organized as follows; actual fields are determined by the onboarding process:

```
node:
  did: did:oon:example-third-party-node
  role: discovery
  endpoint: https://example.invalid/discovery
  authorizationDomain: example-domain
  testProfile: discovery-node-v1
  checks:
    - did-resolution
    - signature-verification
    - publication-sync
    - query-response
```

`status: test-passed`

This record describes only the scope and result of the onboarding assessment; passing the test must not be directly interpreted as a guarantee of long-term operation. Authorization status, dependent services, and continuous synchronization status must still be checked during production operation.

4.8.4 Runtime Status Management

During production operation, a node should continuously check its own authorization, governance status, dependent endpoints, synchronization cursor, task queue, and database status. After authorization is suspended or revoked, the node should stop the corresponding restricted actions. When the content distribution platform or off-chain trust indexer is temporarily unavailable, the node may use existing state according to freshness rules, but must not fabricate new authorization, acceptance, or publication facts. Recovery should include an observable resynchronization and re-verification process.

4.9 Public, Private, and Consortium Network Deployments

A public network allows Internet users to access public registration, discovery, or resolution entry points, but public accessibility does not mean that every resource or every node status is visible to every user. A private network can restrict registrar service nodes, discovery service nodes, the content distribution platform, and resource endpoints to an organizational network or identity domain, while using the same resource identity, control proof, and publication-verification boundaries. A consortium network can be operated by multiple organizations that run nodes and exchange resource publication facts under an agreed root platform, governance authority, and authorization domain.

The three deployment forms can share protocol objects, but they should make the network scope, domains and endpoints, certificates, authorization materials, data retention, and cross-domain propagation conditions explicit. Deployment domains, node counts, and database choices are runtime configuration and should not be written into a resource DID or root-platform proof as immutable protocol conditions. What must be interoperable is identity, signatures, resource packages, digests, status, and node authorization.

Deployment form	Primary reachability	Suitable resource propagation boundary	Common facts that must remain consistent
Public network	Internet users or public clients	Resources and discovery results within public authorization domains	DID, control proof, root-platform publication proof, and governance status
Private network	Internal users and systems of an organization	Internal authorization or business domains	Resource version, digests, node role, and verification order

Deployment form	Primary reachability	Suitable resource propagation boundary	Common facts that must remain consistent
Consortium network	Authorized members of multiple organizations	Resources and semantic vocabulary propagated under consortium rules	Shared trust root, node authorization, and cross-domain verification boundaries

The deployment form changes reachability and the scope of data propagation. It does not change how resource control is proved, nor does it turn the network entry point itself into a trust authority.

4.10 Availability, Isolation, and Failure Boundaries

OAN's degradation strategy must distinguish read-side display from trust-sensitive actions. When the distribution service is temporarily unavailable, a discovery service node can display an old snapshot with a freshness marker, and an operations page can display delayed statistics or historical events. However, resource registration, root-platform publication, node-authorization changes, revocation-state application, and verification before high-risk invocation should be rejected, suspended, or placed in a pending state when critical evidence cannot be confirmed. No degradation may generate a new signed fact or change the resource lifecycle state.

Network interruption, database failure, and index lag should be recorded separately. During recovery, components should continue from persisted tasks, cursors, or event positions and re-verify cross-node materials, avoiding the direct treatment of an old cached version as the latest version. Isolation boundaries include node identity, authorization domain, resource version, and data-storage boundaries. A failure or incorrect data from one node must not automatically spread as a trust fact to other nodes.

Failure symptom	Permitted read-side behavior	Trust-sensitive action	Basis for recovery
Temporary content-distribution unavailability	Display an old snapshot with a freshness marker	Suspend new trusted publication or unverified propagation	Distribution recovery, digest verification, and publication proof
Governance projection delay	Display delayed status or historical data	Apply conservative handling to authorization changes	Governance-event sequence and projection cursor
Discovery-index failure	Provide results through another node or display unavailability	Do not generate a new trusted indexing conclusion	Database recovery and re-indexing
Node identity/signature anomaly	Retain audit records	Reject restricted writes and isolate the node	Authorization recovery and re-onboarding tests

4.11 Interaction of the Control, Data, and Semantic Planes

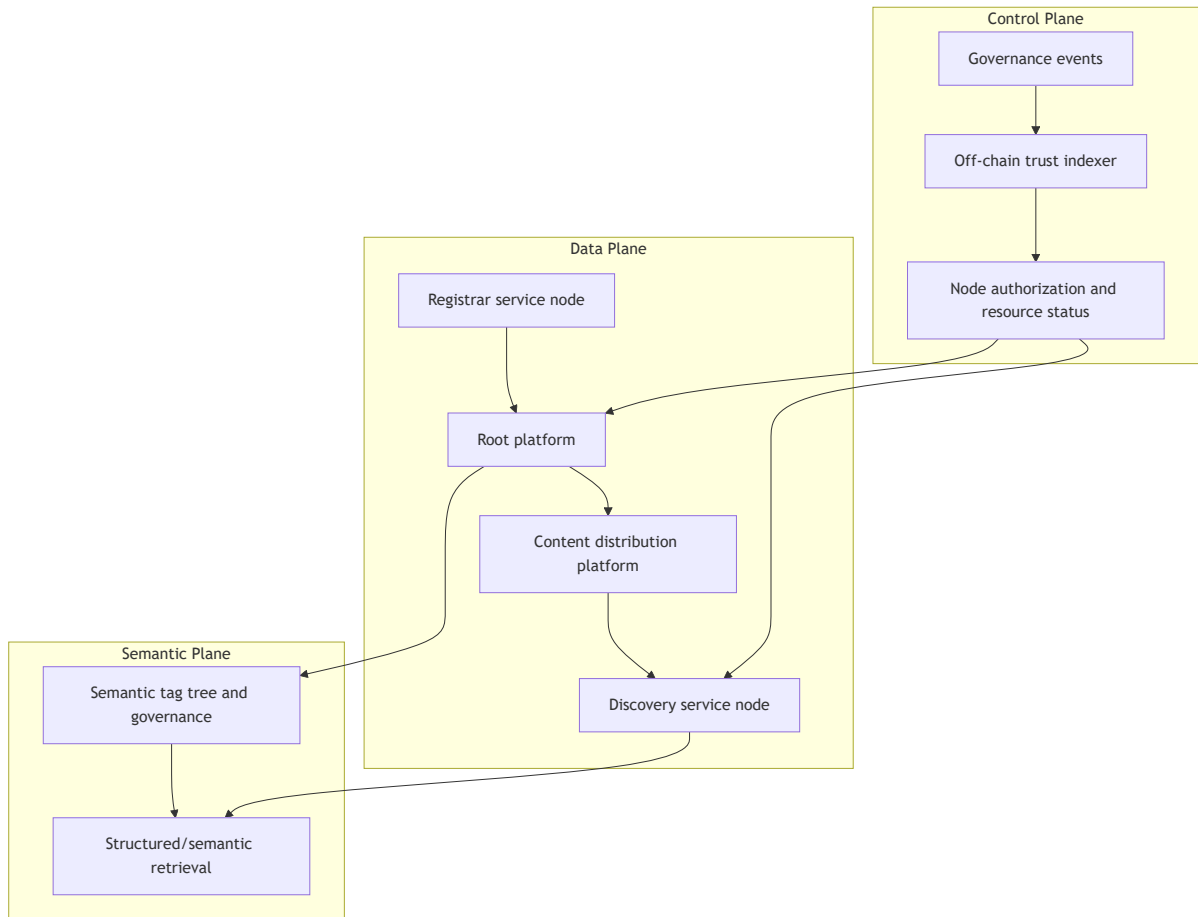
The control plane first decides whether a node and a resource version meet the conditions for entering the network, propagation, and validity. The data plane then transfers the DID Document,

resource package, metadata, digests, and proofs permitted for publication by the control plane. The semantic plane builds capability-tag, description, and use-case indexes for resources that have already passed identity and publication checks. A query can use structured and semantic conditions at the same time, but the result must contain or be traceable to the resource DID, version, source, root-platform proof, and governance status.

Governance-status changes affect continued propagation through the data and semantic planes. After a node is revoked or a resource version becomes invalid, new synchronization should stop or filter the corresponding materials, and existing indexes should be updated after the status has propagated. A change to the tag tree can affect registration recommendations and query interpretation, but cannot retroactively change the identity or digest of a signed resource package. The dependency order among the three planes is that control facts constrain data propagation, data materials support semantic indexing, and semantic results cannot grant control authority in return.

The boundaries among the three planes also determine the order of failure handling. When control-plane status is unknown, the data plane can retain verified materials for limited reads, but must not expand propagation. When data-plane materials are missing or their digests are inconsistent, the semantic plane must not establish or update a trusted index. When the semantic index is unavailable, the system can still provide exact DID queries or other explicitly supported structured queries. After recovery, it should first restore control status and data integrity, and then rebuild or refresh the semantic index.

The three planes and their dependencies are shown below. In the implementation, the data exchanged between the planes is not undefined free-form JSON. It is a JSON/HTTP payload serialized from shared Rust types: resource registration uses `ResourceRegistrationSubmission`; forwarding from a registrar service node to the root platform wraps the request in `ResourceVerifyAndPublishRequest`; content distribution uses `ResourceCdnPublishRequest` or `ResourceCdnBatchPublishRequest`; and discovery queries use `ResourceDiscoveryQuery` and return `ResourceDiscoveryResponse`. These types define field names and nesting relationships, Axum routes define methods and paths, and signature envelopes and resource control proofs define different trust boundaries. Together, they form the node-cooperation contract.



A cross-plane query response should explain both “why it was found” and “why it can be used.” For example, a discovery service node can first use capability descriptions, capability tags, and structured resource types to filter resources, and then associate each result with the resource DID, version, resource-package digest, root-platform trusted publication proof, and current governance status:

```
{
  "resourceDid": "did: oan: SKDM: example-resource",
  "match": {
    "mode": "semantic_and_structured",
    "query": "Search a code repository and summarize the project structure",
    "matchedTags": ["code-repository", "project-structure-summary"]
  },
  "trust": {
    "governanceStatus": "active",
    "rootProof": "present",
    "packageDigest": "sha256:..."
  },
  "freshness": {
    "indexedAt": "2026-09-09T08:30:00Z",
    "sourceCursor": "publish-1842"
  }
}
```

The semantic match in this example can explain the relevance between the resource and the

query. `governanceStatus`, `rootProof`, and `packageDigest` support status judgment, trusted-publication judgment, and content-integrity judgment respectively. If any trust field is missing, the discovery service node may return the resource, but it must lower the result status or require the caller to continue verification. It must not interpret semantic relevance directly as authorization or a security guarantee.

4.12 Sources of Facts and Derived Read Models

The sources of fact for the resource control relationship are the resource DID Document, control keys, and valid control proof. The source of fact for registration acceptance is the registrar service node's registration record or registration credential. The sources of fact for root-platform acceptance and publication are the root platform's persistent records and root-platform trusted publication proof. The source of fact for distribution is the publication artifact and its digests stored by the content distribution platform. The source of fact for discovery visibility is the synchronization, verification, and indexing record of the specific discovery service node. The source of fact for node authorization status is the governance events and their projection in the off-chain trust indexer. Website pages, cached interfaces, and statistics are observation layers and are not the sole authority for any of the facts above.

A derived read model must retain source references, a version or cursor, verification time, freshness, authorization domain, and error status. A read model may combine fields or build a full-text or semantic index for query efficiency, but it must not discard the DID, version, digest, and proof references needed to re-verify the resource. When the read model differs from the authoritative source, it should show a synchronization or verification anomaly rather than allowing the existence of a local database record to override the authoritative status.

Network fact	Authoritative source	References a derived node must retain	Handling when a gap occurs
Resource control relationship	DID Document and control proof	DID, verification method, document digest	Reject or await verification
Registration acceptance	Registrar service node record/registration credential	Node identity, resource DID, time, and status	Must not be written as published
Root-platform publication	Root-platform record and root-platform trusted publication proof	Resource version, three types of digest, proof	Must not enter trusted distribution
Node authorization	Governance events and off-chain read model	Event identifier, sequence, and current status	Fail closed for sensitive actions
Discovery index	Discovery service node's local record	Synchronization cursor, verification time, and freshness	Retain the old snapshot and mark it as lagging

A resource index in a discovery service node can use the following derived structure. It is not a new authoritative record, but a read model that organizes multiple sources for efficient querying. `source` is used to trace back to facts, and `sync` is used to determine whether local data has caught up with the publication position:

```

{
  "resourceDid": "did:oon:SKDM:example-resource",
  "version": "2026-09-09T08:20:00Z",
  "metadata": {
    "resourceType": "skill",
    "capabilityTags": ["code-repository", "summary"]
  },
  "source": {
    "registrarRecord": "registrar-record-901",
    "rootPublication": "publish-1842",
    "rootProof": "root-proof-1842",
    "packageDigest": "sha256:..."
  },
  "sync": {
    "localCursor": "publish-1842",
    "checkedAt": "2026-09-09T08:31:00Z",
    "freshness": "current",
    "error": null
  }
}

```

When `localCursor` is behind, source verification fails, or the governance projection has not yet been updated, the read model may still be retained for limited display, but `freshness` should be changed to `stale` or `verification_required` and the reason for the error should be retained. A row in an index database can only show that the local node once received the material; by itself, it cannot prove that the resource remains valid, propagatable, or invocable.

4.13 Network Topology and Sequence Consistency

The root-platform publication sequence is produced by the root platform and identifies the position of resource publication materials in the data-distribution path. The on-chain governance-event sequence is produced by the on-chain governance system and expresses the order of node-authorization and status changes. The discovery service node's synchronization cursor is maintained by the discovery service node and expresses the distribution position that the node has processed. The three values may look numerically similar, but they are not interchangeable. The implementation must store and check them separately, avoiding the use of one global number to infer resource publication, governance finality, or local indexing status.

When a discovery service node encounters a cursor gap, disorder, or rollback, it should suspend application of the affected scope, retrieve missing materials from the root platform or content distribution platform, and continue only after re-verification. Duplicate materials can be processed idempotently by resource DID, version, and package digest. Duplicate governance events should be deduplicated by event identifier or sequence. A rollback in the publication sequence must not overwrite a newer version that has already been confirmed. During cross-node delay, an old snapshot may be retained for limited read-side display, but freshness must be marked, and sensitive actions must stop when revocation, suspension, or authorization status cannot be confirmed.

The three sequences should be stored separately in implementation and operations records. The configuration-style example below expresses their sources and processing boundaries only; it does not mean that they share one numbering format:

```
resource_publication:
  producer: root-platform
  latest: publish-1842
  applies_to: did-document-and-resource-package
  gap_action: pause-affected-publication

governance_events:
  producer: governance-ledger
  latest: event-771
  applies_to: node-authorization-and-resource-status
  gap_action: fail-closed-for-sensitive-actions

discovery_sync:
  producer: discovery-service-node
  local_cursor: publish-1839
  applies_to: local-validation-and-indexing
  gap_action: fetch-missing-publications-and-revalidate
```

For example, a discovery service node's synchronization cursor falling behind the root-platform publication sequence only means that the local index may be incomplete. It does not mean that governance events are behind by the same amount. If a gap appears in governance events, an application that depends on governance status should be suspended even when resource publication materials are complete. A node may mark a resource as a current discoverable result only after the corresponding sequences are continuous, the materials pass verification, and the status projection is complete.

4.14 References

Source	Type	Link
oan-root-services	Code repository: root platform, publication coordination, and node directory	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	Code repository: registrar service node and access path	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	Code repository: discovery service node, synchronization, and indexing	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	Code repository: governance-event synchronization and off-chain state projection	https://github.com/OpenAgenet/oan-trust-indexer
OAN Yellow Paper	arXiv yellow paper: network architecture and trust path	https://arxiv.org/abs/2606.03163

Source	Type	Link
Agentic Overlay Network Architecture	IETF draft: agentic overlay network architecture	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

Chapter 5 OAN Resource Model

This chapter explains how OpenAget (OAN) uniformly models capability units that can be registered, published, distributed, and discovered as resources. The current core resource types include agent services, skill packages, MCP services, and tool APIs; infrastructure nodes, organizations, and developers may also appear as identity or subject types in associated relationships, but this chapter focuses on the four discoverable resource types. Each resource has, at minimum, an independent resource DID, resource type, description, capability tags, use cases, protocol bindings, service endpoints, version, and lifecycle state, and may use a resource package to carry a verifiable representation of a particular version.

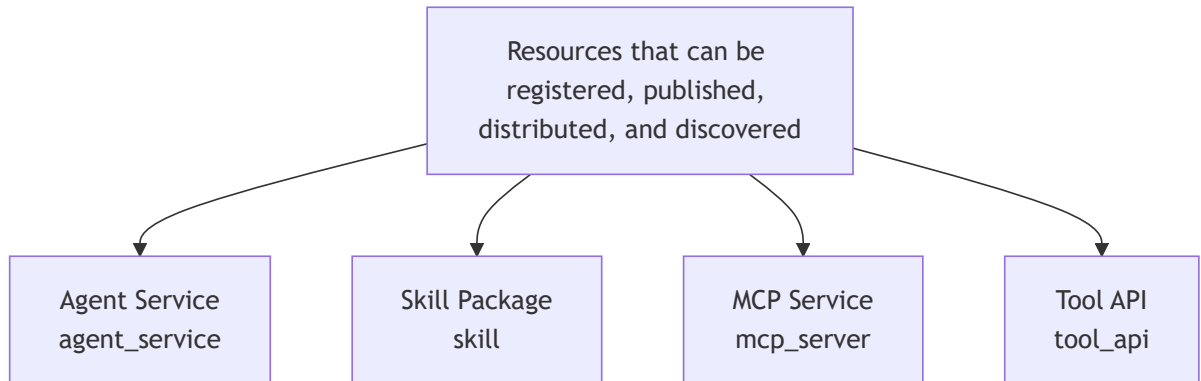
The resource model unifies only the common elements required for cross-type interoperability; it does not force the four resource types into the same business object. The resource type determines field interpretation, endpoint shape, protocol binding, and invocation method; the resource DID establishes the control and registration boundary; resource metadata is used for registration and discovery; and the resource package binds the DID document, metadata, hashes, and the root platform's trusted publication proof into a distributable object. Subsequent chapters specify identity, registration, discovery, publication, and invocation separately, avoiding a duplicate definition of all protocol behavior in this chapter.

5.1 Resource Model Overview

The common identity fields of the resource model include the resource DID, resource type, and subject type, as well as optional controller DID, publisher DID, and issuer DID; common descriptive fields include the name, description, capability description, capability tags, and use cases; common connectivity fields include protocol bindings, service endpoints, and authentication or credential requirements; and common state fields include the version, lifecycle state, authorization domain, network scope, and update time. `oanMetadata` is the primary metadata object in a DID document for carrying OAN resource semantics and lifecycle information, while the resource metadata in a resource package provides a versioned representation for publication and discovery.

The resource type enumeration includes at least `agent_service`, `skill`, `mcp_server`, and `tool_api`. The method-specific encoding of a resource DID should remain consistent with the resource type, and the resource type, subject type, `oanMetadata` in the DID document, and resource package metadata must be mutually verifiable. When a field is absent, empty, or present only in the web presentation layer, it must not be used to infer that the resource has the corresponding capability; actual invocation permission is not automatically granted by an endpoint or description field in the resource model.

The four resource types share the common models of resource DID, resource metadata, version, and lifecycle, while using different protocol bindings, endpoints, and package contents according to their capability forms:



The following example shows only how common fields are composed. A specific resource must still add protocol, endpoint, package, and credential information according to its type. `resourceType`, `subjectType`, and `resourceDid` are the fields used by machines to determine the resource boundary, while the name, description, and capability tags serve human understanding and discovery queries:

```

{
  "resourceDid": "did:oan:SKLG:example-skill",
  "resourceType": "skill",
  "subjectType": "skill",
  "publisherDid": "did:oan:ORLG:example-publisher",
  "name": "Repository Structure Summary",
  "description": "Summarizes the structure of a code repository.",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "authorizedDomains": ["public"],
  "lifecycleState": "active",
  "packageVersion": "1.0.0",
  "hashAlgorithm": "sha256",
  "updatedAt": "2026-09-09T08:30:00Z"
}

```

This object can serve as a minimal reading example of resource metadata, but it cannot independently replace the DID document, control proof, resource package hash, or the root platform's trusted publication proof. A discovery service node can use its structured fields to build an index, while the caller must still obtain the corresponding evidence and perform subsequent verification according to the resource type.

5.2 Agent Services

An agent service is a resource type that provides callable agent capabilities, with the resource type value `agent_service`. It expresses an independent control and publication boundary through its resource DID, describes capabilities, capability tags, use cases, inputs and outputs, and applicable scenarios through its resource description, and expresses how to establish a connection through service endpoints and protocol bindings. An agent service may declare A2A, MCP, or other business

protocols, but declaring a protocol only identifies the resource's interface context; it does not mean that the caller has already obtained business authorization.

The relationship with an A2A Agent Card is an interoperability mapping relationship. OAN capability descriptions, capability tags, use cases, service endpoints, protocol bindings, version, and authentication declarations can provide trusted identity and publication provenance for corresponding A2A Agent Card information; A2A Agent Card fields can also be mapped to OAN resource metadata through an adapter. OAN additionally provides the resource DID, control proof, registration credential, root platform publication proof, governance status, and authorization domain, allowing a capability description to be associated with provenance, version, and network governance facts without changing the interaction semantics of A2A itself.

Interoperability information	Where it is carried in the OAN resource model	Boundary to preserve during mapping
Service name and description	<code>resourceDescription.name</code> , <code>resourceDescription.description</code>	Explains what the resource is; it is not equivalent to invocation authorization
Capabilities and applicable tasks	<code>capabilityDescription</code> , <code>capabilityTags</code> , <code>useCaseExamples</code>	Describes discoverable capabilities; it does not replace runtime capability checks
Service entry point	<code>service</code> in the DID document and <code>serviceRef</code> in the protocol binding	The entry-point address may be public, but the endpoint still requires independent authentication
Protocol and version	<code>protocolBindings.protocol</code> , <code>version</code> , <code>transport</code>	OAN records the access context without changing A2A interaction semantics
Identity and publication evidence	Resource DID, control proof, registration credential, and the root platform's trusted publication proof	The content of an A2A Agent Card alone cannot be treated as an OAN publication fact

5.3 Skill Packages

A skill package is a capability unit that can be published, installed, loaded, or reused independently, with the resource type value `skill`. It should describe its capabilities, inputs and outputs, use cases, dependencies, and usage conditions through its resource description, and explain its acquisition and usage entry points through service endpoints, package information, or implementation links. A skill package may be composed by an agent service or another orchestration system, but its discoverability, version, and provenance should still be based on its own resource DID, resource package, and root platform publication facts.

The “reusable” property of a skill package means that a caller can use it repeatedly after satisfying its dependency, version, and authorization conditions; it does not mean that the caller can bypass resource control, installation security, or business permissions. Installation commands, download addresses, scripts, or dependencies in the skill package description are public declarations only. The caller must still verify the resource package hash, publication proof, version state, and local execution policy.

For example, public metadata for a skill package can place its capability, applicable tasks, dependencies, and version within the same resource boundary:

```
{
  "resourceDid": "did: oan: SKLG: example-skill",
  "resourceType": "skill",
  "subjectType": "skill",
  "name": "Repository Structure Summary",
  "description": "Summarizes a code repository and its project structure.",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "implementationLinks": [
    {
      "relation": "depends_on",
      "targetDid": "did: oan: TLFi: example-tool-api",
      "targetType": "tool_api",
      "versionConstraint": ">=1.0.0"
    }
  ],
  "lifecycleState": "active",
  "packageVersion": "1.0.0",
  "hashAlgorithm": "sha256",
  "updatedAt": "2026-09-09T08:30:00Z"
}
```

Here, `implementationLinks` expresses a verifiable resource relationship, and the version constraint expresses a composition condition; it does not mean that the target tool API has been installed or that the caller has obtained access permission. When the actual resource package is published, this metadata must also be bound to the DID document hash, metadata hash, and resource package hash.

5.4 MCP Services

An MCP service is a service resource that provides tool or resource access capabilities through the Model Context Protocol, with the resource type value `mcp_server`. Its resource DID identifies the service's publication and control boundary; its resource metadata describes capability tags, use cases, protocol version, transport method, service endpoint, and authentication requirements; and its MCP capability description can be expressed through protocol bindings or a dedicated capability object. A tool list or resource list is a capability declaration and cannot independently replace endpoint authentication, invocation authorization, or runtime status checks.

The mapping between OAN and MCP metadata should preserve two boundaries: MCP is responsible for tool, resource, and interaction semantics at the protocol layer, while OAN provides the MCP service with resource identity, control proof, trusted publication, governance status, and a discovery entry point. An MCP service endpoint being reachable does not mean that the endpoint has been published by the root platform or is open to every caller; discovery results should not interpret an MCP capability declaration directly as invocation permission.

The resource description of an MCP service can provide protocol bindings and service endpoints together, for example:

```

{
  "resourceType": "mcp_server",
  "subjectType": "mcp_server",
  "protocolBindings": [
    {
      "id": "mcp-http",
      "protocol": "MCP",
      "version": "2025-06-18",
      "transport": "streamable-http",
      "serviceRef": "#mcp-service"
    }
  ],
  "services": [
    {
      "id": "#mcp-service",
      "type": "MCPServer",
      "serviceEndpoint": "https://example.invalid/mcp",
      "protocol": "MCP"
    }
  ]
}

```

The address, version, and identifiers in this example are illustrative values. `protocolBindings` specifies the protocol through which the resource is accessed, while `services` specifies the network entry point; both still need to be associated with the resource DID, publication version, hashes, and authentication requirements. A discovery service node should not mark a resource as directly callable merely because an MCP tool list exists.

5.5 Tool APIs

A tool API is a resource that provides a specific operation capability through a network interface, with the resource type value `tool_api`. It should describe the interface address, operation or capability name, input parameters, output results, protocol, version, authentication method, and invocation constraints; complex parameters may be expressed through input and output schemas, protocol schema references, or external documentation. A tool API can be referenced by an agent service, skill package, or MCP service, but a reference relationship does not mean that the caller has passed authentication and authorization at the target endpoint.

The resource description of a tool API and the actual service implementation should maintain a verifiable association. The resource controller is responsible for declaring the interface and version; the publication path is responsible for binding the DID document, metadata, hashes, and root platform proof; and the caller is responsible for checking the endpoint, version, evidence, and its own policy before connecting. The existence of an interface, a normal HTTP response, or the presence of a discovery index entry cannot independently prove that the tool's business result is correct or that invocation is permitted.

The capability description of a tool API can separate “what it can do” from “how to invoke it”

by using input and output schemas and authentication requirements:

```
{
  "resourceType": "tool_api",
  "resourceDescription": {
    "name": "Repository Structure Summary API",
    "capabilityDescription": "Accepts a repository reference and returns a project structure summary.",
    "inputSchema": {
      "type": "object",
      "required": ["repository"],
      "properties": {
        "repository": { "type": "string" }
      }
    },
    "outputSchema": {
      "type": "object",
      "required": ["summary"],
      "properties": {
        "summary": { "type": "string" }
      }
    }
  },
  "credentialRequirements": [
    {
      "credentialType": "Bearer",
      "presentationMode": "header",
      "required": true
    }
  ]
}
```

The schema constrains the shape of inputs and outputs, while the credential requirements describe the authentication material that must be prepared before invocation. Actual permission decisions, rate limiting, business validation, and result quality remain the responsibility of the target API and the caller's policy. The resource model provides the identity boundary within which these declarations can be registered, published, and discovered.

5.6 Resource Relationships and Composition

Resource relationships express verifiable dependencies, provision, invocation, containment, or composition between resources. The current implementation uses structured links such as `implementationLinks` to carry relationships. A link should include at least the relationship type and target resource DID, and may include the target resource type, target service, and version constraint. The credibility of a relationship comes from the identities, publication evidence, and current states of both resources, not from an uncheckable natural-language statement.

Before runtime, a composition system should verify the DID, resource type, version or package hash, root platform's trusted publication proof, lifecycle state, and authorization domain of every

referenced resource. If a referenced resource is undiscoverable, does not satisfy the version constraint, lacks proof, has been revoked, or has been suspended, the caller or orchestration system should choose according to risk among rejecting the composition, waiting for synchronization, or displaying it only without invocation eligibility. It must not treat the existence of an index summary as proof of composition integrity.

A resource relationship can be represented in the data as an explicit link object rather than embedding a dependency in free text:

```
{
  "relation": "provides",
  "targetDid": "did:oan:TLFI:example-tool-api",
  "targetType": "tool_api",
  "targetService": "#repository-summary",
  "versionConstraint": "^1.0.0"
}
```

Relationship verification should check at least the target DID, relationship type, target type, version constraint, and current publication state of the target resource. `targetService` is only a reference for locating the target service; it cannot replace endpoint reachability and authentication. When the target resource's version or package hash changes, the original relationship must be matched and verified again.

5.6.1 Dependencies Between Resources

A dependency relationship means that normal use of one resource requires another resource or external capability. The dependency declaration should point to the target resource DID and, where necessary, provide the target type, service reference, and version constraint; the business meaning, installation method, and runtime authorization of the dependency can be supplemented through the resource description or protocol binding, but the identity and version boundaries must be expressed structurally. When the dependency target is updated, the caller should recheck the version constraint and publication state.

5.6.2 Composition of Agent Services and Skill Packages

An agent service may depend on or compose multiple skill packages, using them as reusable capabilities for task execution, information processing, or tool orchestration. The composition record should distinguish the agent service DID, skill package DID, skill package version or package hash, and relationship type; publication of the agent service does not automatically extend the control and publication facts of the skill package. The caller must verify the evidence and compatibility conditions of both the agent service and the skill package.

5.6.3 Invocation Relationships Between MCP Services and Tool APIs

An MCP service may provide a collection of tools or reference one or more tool APIs through protocol bindings; a tool API may also be called directly by an agent service or skill package. The relationship declaration should specify the target DID, target service or operation, protocol, and

version, avoiding a statement that only says “supports a particular tool.” Tools discoverable at the MCP protocol layer still require OAN resource identity, publication state, endpoint authentication, and the caller's authorization decision.

5.6.4 Composition Integrity and Version Constraints

Composition integrity requires at least that the DIDs, resource types, versions or package hashes, relationship declarations, root platform trusted publication proofs, and current lifecycle states of both sides of a relationship are consistent, and that the target resource is within the authorization domain visible to the caller. When a version constraint is not satisfied, the package hash changes, the proof expires, or the resource is suspended, the old composition declaration must not be used as an unconditional basis for invocation; where necessary, the caller should rediscover, reverify, or fall back to a clearly compatible version.

5.7 Resources as Independently Identifiable Objects

Each of the four resource types can have an independent resource DID, using that DID as the identity boundary for creation, registration, publication, discovery, and update. A subject DID can express the control relationship of a resource, and a publisher DID can express the registration or operational provenance, but neither can replace the resource's own DID. The method-specific encoding of the resource DID should be consistent with the resource type, and the public keys, service endpoints, and `oanMetadata` in the DID document should jointly support verification of the resource identity and capability declaration.

An independent DID allows a skill package, MCP service, or tool API to be discovered and reused independently of a particular agent service, and also allows it to be updated, suspended, revoked, and versioned independently. When resources have composition relationships, each resource must still be verified separately through its own DID and publication evidence; the trusted state of one resource does not automatically transfer to its dependencies.

There is a code-level validation relationship between resource types and the method-specific encoding of `did:oan`. The current common library uses `AG`, `SK`, `MC`, and `TL` for agent services, skill packages, MCP services, and tool APIs, respectively. The following DIDs are provided only to illustrate the form:

Resource type	Machine value	Example method-specific encoding	Independent identity boundary
Agent service	<code>agent_service</code>	<code>did:oan:AG...:<method-specific-id></code>	The service's own control, publication, and version
Skill package	<code>skill</code>	<code>did:oan:SK...:<method-specific-id></code>	Skill package content, dependencies, and version
MCP service	<code>mcp_server</code>	<code>did:oan:MC...:<method-specific-id></code>	MCP service entry point and tool collection
Tool API	<code>tool_api</code>	<code>did:oan:TL...:<method-specific-id></code>	API operations, schemas, and endpoints

The encoding prefix is an identifier currently used by the implementation to check consistency

with the resource type. It should not be understood as an independent DID method outside the rules of the `did: oan` method. When the resource type is inconsistent with `oanMetadata.resourceType` in the DID document, a node should reject further processing of the object as that resource type.

5.8 Resource Metadata and Capability Descriptions

Resource metadata is a structured description used for registration, publication, and discovery in addition to resource identity. Common fields include the resource DID, resource type, subject type, controller or publisher DID, name, description, capability description, capability tags, authorization domain, protocol bindings, service endpoints, credential requirements, package information, network scope, lifecycle state, version, and update time. `oanMetadata` in the DID document and the resource metadata in the resource package should remain consistent in key fields such as resource DID, resource type, version, hash, authorization domain, and state.

The capability description answers what the resource can do; capability tags answer which governed capability categories the resource belongs to; and use cases answer in which tasks those capabilities are used. Names and free text help people and semantic engines understand a resource, but cannot replace structured tags, protocol bindings, endpoints, or control proofs. Metadata is a declaration made by the resource provider. The root platform and discovery service nodes can verify its structure, hash, and publication provenance, but complete fields do not replace an independent assessment of business quality.

Capability fields in `oanMetadata.resourceDescription` can be organized into a minimal object for retrieval:

```
{
  "name": "Repository Structure Summary",
  "description": "A reusable capability for examining code repositories.",
  "capabilityDescription": "Finds repository files and summarizes the project structure.",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "useCaseExamples": [
    "I need a tool that can search code repositories and summarize the project structure."
  ],
  "version": "1.0.0"
}
```

Natural-language fields are suitable for semantic discovery and user understanding, tags are suitable for structured filtering, and the version associates the description with the resource package and publication evidence. Together they describe the resource's capabilities, but do not mean that a node has verified runtime behavior, nor do they replace identity, publication, and authorization checks before invocation.

5.8.1 Basic Identity and Description Fields

The basic fields should be sufficient to identify the resource DID, resource type, subject type, name, description, and controller or publisher relationship, and should be consistent with `id`, `oanMetadata.resourceType`, `oanMetadata.subjectType`, and related DID fields in the DID document.

Resource type values should use the protocol-defined enumeration; a Chinese display name or alias in the presentation layer must not be the sole basis for machine judgment.

5.8.2 Capability Tags, Capability Descriptions, and Use Cases

Capability tags should preferably use canonical identifiers from the capability tag tree maintained by the root platform. Extension tags may also be retained when their source is clearly identified. Capability descriptions and use cases should explain executable capabilities, inputs and outputs, and applicable tasks. A registration service node may recommend tags based on a description, and a discovery service node may combine tags, aliases, descriptions, and use cases for structured or semantic search, but relevance results still require identity, publication, and authorization-domain verification.

5.8.3 Protocol, Endpoint, and Authentication Information

Protocol bindings should specify the protocol name, version, transport method, service reference, or schema reference, and associate these with one or more service endpoints. A service endpoint should specify its address, type, version, protocol, and any necessary port or server information. Authentication information should express the public authentication method, credential requirements, or prerequisites for invocation. Endpoint and authentication declarations are part of the resource capability description; they do not mean that the caller has already obtained access permission.

5.8.4 State, Version, and Governance Information

Resource metadata should distinguish the resource lifecycle state, resource package version, update time, expiration time, previous version or version constraint, authorization domain, and network scope. The lifecycle state expresses facts such as whether the resource is active, suspended, revoked, or replaced; propagation progress is expressed by root platform publication, content distribution, and discovery synchronization records. These concepts must not be conflated in a single `available` field.

5.9 Use Cases, Protocol Bindings, Endpoints, and Authentication Declarations

Use cases, protocol bindings, service endpoints, and authentication declarations together form the public explanation of “how to understand and use this resource.” A resource may have multiple use cases, protocol bindings, and endpoints. Each binding should, as far as possible, specify the applicable capability, protocol version, transport method, service reference, and schema; authentication declarations should, as far as possible, be associated with a target endpoint, operation, or version. If an endpoint is unavailable, the protocol version is incompatible, or authentication information is incomplete, a discovery service node may return a result with the corresponding state, but should not describe it as directly callable.

5.9.1 Use Case Descriptions

A use case description should state the task objective, input conditions, expected output, and applicable resource capabilities, and should preferably correspond to the capability description and capability tags. A natural-language use case can support semantic retrieval and user understanding, but cannot independently prove that the resource has the capability; precise constraints should be combined with input and output schemas, protocol bindings, and version information.

5.9.2 Protocol Bindings

Protocol bindings should record the protocol name, protocol version, transport or interaction method, and references to endpoints or schemas. A2A and MCP may enter the resource description as external protocol contexts. OAN binding fields only express how the resource connects to these protocols; they should not turn OAN resource registration state into a mandatory clause of an external protocol.

5.9.3 Service Endpoints

A service endpoint is the network entry point through which a resource provides capabilities or obtains materials. It may contain an endpoint identifier, endpoint type, address, version, protocol, server type, and port. When a resource has multiple endpoints, the protocol or version differences between them should be stated. When an endpoint is unreachable, its certificate is invalid, or the service is inactive, the discovery result should preserve that fact or freshness information rather than considering only the static address.

5.9.4 Authentication and Invocation Requirements

Authentication and invocation requirements explain what identity, credential, signature, permission, or protocol context a caller must provide. A resource may publicly declare the credential type, issuer, scope, and presentation method, but final business authorization is jointly determined by the resource endpoint and the caller's policy. OAN VCs, the root platform's trusted publication proof, and node governance status are materials for verification before invocation; they are not automatic authorization for every business operation.

Authentication declarations in the resource model primarily answer “what must the caller prepare,” such as credential type, issuer, scope, and presentation method. The endpoint must still check credential validity, request subject, business permissions, and runtime policy after receiving a request. The public declaration of the same resource can be understood in two layers:

Layer	What the resource model can express	Conclusion that cannot be derived from this field alone
Public declaration	Credential type, issuer, scope, presentation method, and applicable endpoint	That the caller definitely possesses the credential
Invocation decision	Endpoint authentication result, authorization policy, protocol context, and business conditions	That the resource is definitely available or that the business result is definitely correct

5.10 Resource Packages and Publication Artifacts

A resource package is a distributable object bound to a particular resource version. It usually contains the resource DID, resource type, resource version, DID document, DID document hash, metadata hash, resource package hash, hash algorithm, resource metadata, the root platform's trusted publication proof, and creation time. The DID document expresses identity, public keys, service endpoints, and OAN metadata; resource metadata expresses capabilities, use cases, protocols, endpoints, authorization domain, and lifecycle; the control proof expresses the resource controller's authorization for an operation; the registration VC expresses the registration fact processed by the registration service node; and the root platform's trusted publication proof expresses that the root platform has verified and accepted that resource version.

At node interfaces, the resource model corresponds to different payload types. The client submits `ResourceRegistrationSubmission`, whose `metadata` remains dynamic JSON because the registration service node must validate its structure according to the current resource metadata rules. After root platform verification succeeds, these fields are organized into a `ResourcePackage`, which is then carried and distributed by `ResourceCdnPublishRequest` or a batch request. Therefore, a “resource object,” a “registration request,” and a “distributable resource package” are not the same JSON object; a client must not treat a registration request as a resource package that already has a root platform proof.

These objects form a layered verification path rather than parallel attachments. A caller or discovery service node should first resolve the DID document and verify the control relationship; then check consistency of the resource type, DID, authorization domain, and version between the metadata and DID document; recompute the DID document hash, metadata hash, and resource package hash; and verify the registration credential, the root platform's trusted publication proof, and the current governance state. Registration acceptance, root platform publication, and discovery indexing are produced by different components, and success at any one stage cannot replace the others.

A resource package can be understood through the following structure. The hashes, DIDs, times, and proof contents in this example are placeholders used to show the reference relationships between objects and do not represent official fixed values:

```
{
  "packageVersion": "1.0.0",
  "resourceDid": "did:oan:SKLG:example-skill",
  "resourceType": "skill",
  "didDocument": {
    "@context": ["https://www.w3.org/ns/did/v1"],
    "id": "did:oan:SKLG:example-skill",
    "verificationMethod": [],
    "authentication": [],
    "assertionMethod": [],
    "service": [],
    "oanMetadata": {
      "subjectType": "skill",
      "resourceType": "skill",
      "capabilityTags": ["code-repository", "project-structure-summary"],
```

```

    "authorizedDomains": ["public"],
    "lifecycleState": "active"
  }
},
"didDocumentHash": "sha256:example-did-document",
"metadataHash": "sha256:example-metadata",
"packageHash": "sha256:example-package",
"hashAlgorithm": "sha256",
"metadata": {
  "resourceDid": "did:oan:SKLG:example-skill",
  "resourceType": "skill",
  "subjectType": "skill",
  "name": "Repository Structure Summary",
  "description": "Summarizes a code repository.",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "authorizedDomains": ["public"],
  "services": [],
  "lifecycleState": "active",
  "packageVersion": "1.0.0",
  "packageHash": "sha256:example-package",
  "metadataHash": "sha256:example-metadata",
  "hashAlgorithm": "sha256",
  "updatedAt": "2026-09-09T08:30:00Z"
},
"rootProof": {
  "rootDid": "did:oan:INLG:example-root",
  "packageClaims": {
    "resourceDid": "did:oan:SKLG:example-skill",
    "resourceType": "skill",
    "version": "1.0.0",
    "didDocumentHash": "sha256:example-did-document",
    "metadataHash": "sha256:example-metadata",
    "packageHash": "sha256:example-package",
    "hashAlgorithm": "sha256",
    "lifecycleState": "active",
    "authorizedDomains": ["public"]
  }
},
"createdAt": "2026-09-09T08:30:00Z"
}

```

The verifier can first use `resourceDid` and `resourceType` to determine the object's ownership, then recompute the three types of hashes, check consistency of key fields between `metadata` and the DID document, and finally verify that `rootProof.packageClaims` is consistent with the current package declaration. This order helps distinguish “content is complete,” “resource is controllable,” “registration was accepted,” and “root platform publication was completed,” rather than interpreting a downloaded file as the complete network fact.

5.10.1 ResourcePackage Contents

The resource package content should be organized around one resource version and bind at least `resourceDid`, `resourceType`, `packageVersion`, the DID document, DID document hash, metadata hash, resource package hash, hash algorithm, resource metadata, and the root platform's trusted publication proof. The `rootProof` in the resource package should associate the resource DID, type, version, hashes, lifecycle state, authorization domain, and necessary governance or publication references through its claims.

5.10.2 Metadata and Resource Content

Metadata is a structured description of the resource content, capabilities, and runtime entry points. The resource content may be skill package files, a service description, a tool interface, or an MCP capability declaration. Package information, download addresses, manifest addresses, implementation links, and version fields in the metadata should point to or describe the current resource version; historical metadata must not override the identity and hash of the current resource package.

5.10.3 Hashes and Integrity Proofs

The resource package should express the DID document hash, metadata hash, and resource package hash separately, and declare the hash algorithm used. The verifier should recompute the corresponding hashes according to the canonical representation, and check whether the internal fields of the resource package are consistent with the declarations in the root platform proof. A matching hash proves content integrity only; it does not independently prove control, node authorization, or trustworthiness of the business endpoint.

5.10.4 Publication, Download, and Verification Artifacts

A registration service node may return a registration result, registration credential, or downloadable material; the root platform produces an acceptance record and the root platform's trusted publication proof; the content distribution platform provides downloadable resource packages and related materials; and a discovery service node stores a verified index object. Downloading a resource package does not mean that invocation authorization has been completed. The caller should reverify it according to provenance, version, freshness, governance state, and local policy.

5.11 Resource Lifecycle and Visibility State

The resource lifecycle describes the state of the resource version itself, while network propagation progress describes whether that version has passed registration acceptance, root platform acceptance, content distribution, and discovery indexing. A resource may be registered but not yet published by the root platform, published but not yet synchronized by a particular discovery service node, or present in a discovery index but no longer visible to a particular caller because of a change in authorization domain or governance state. State judgments should cite both the component that produced the state and the corresponding evidence.

Resource updates should distinguish an idempotent retry of the same version from publication of a new version. A repeated submission with the same resource DID, version, and hashes may reuse the existing result; content changes, version changes, control-key changes, or lifecycle changes must enter a new verification and publication decision. An old snapshot may be used for limited display, but must not override current revocation, suspension, replacement, or authorization-domain restrictions.

Resource version and propagation state can be viewed along two dimensions:

Resource version/lifecycle	Registration service node	Root platform and content distribution platform	Discovery service node	Meaning for the caller
Draft	Not accepted	No publication fact	Should not enter a trusted index	May only be edited or previewed locally
Registered	Registration record or registration credential exists	May not yet be published	May be invisible	Does not establish network-wide publication
Published	Registration evidence is retained	Root platform trusted publication proof and distribution materials exist	May still be synchronizing	The caller can wait for discovery or directly verify the materials within its authorization domain
Indexed	Registration evidence is retained	Publication materials are verifiable	Local validation and indexing are complete	The caller must still check current governance state and invocation authorization
Suspended/revoked	The state record must be retained	Further propagation or use is restricted	Query results should be updated or marked stale	An old snapshot must not continue to be treated as current and usable

5.11.1 Registration State

A local draft means that the resource materials have not been accepted by a registration service node; submitted means that the registration service node has received and processed the request; and registered means that the registration service node has formed a registration record or registration credential. These states do not directly indicate that the root platform has published the resource or that a discovery service node has indexed it. The resource controller should preserve the local DID document, keys, and submission materials so that they can be reverified after a failure or during an update.

5.11.2 Publication and Indexing State

Root-platform published means that the root platform has accepted a particular resource version and formed a publication proof; content distributed means that the resource package has entered the serving scope of the content distribution platform; and indexed by a discovery service node means that a particular discovery service node has verified the resource package and written

it to its local read model. These three states progress asynchronously and may be temporarily inconsistent, so they should be expressed through cursors, state, and freshness.

5.11.3 Suspension, Revocation, and Recovery State

Suspension means that a resource or its version may temporarily no longer be propagated or invoked as `active` within a specified scope; revocation means that the corresponding resource, credential, publication fact, or node authorization is no longer valid; and recovery means that a new governance or publication decision has again permitted the corresponding action. A state change should be associated with the version, resource DID, governance fact, or publication record, rather than merely changing web display text. A discovery service node should update its index and query results after the state has propagated.

5.11.4 Publicly Visible and Trustfully Discoverable State

Publicly visible means that a portal or node can return resource information to a particular query party; trustfully discoverable means that the discovery service node has verified the resource identity, publication proof, hashes, lifecycle, and authorization domain and permits the resource to be returned within its authorized scope. A resource may be displayed publicly without being trustfully discoverable, or may have been published by the root platform but be invisible to a particular authorization domain. The caller must still perform its own verification before connecting.

5.12 Resource Ownership, Control, and Delegation Relationships

Resource ownership is a rights relationship at the business or organizational layer; DID control is a cryptographic relationship in which keys and signatures prove the ability to represent a resource DID and perform prescribed operations; operational management is responsibility for an endpoint, content, or runtime environment; publication delegation is the qualification for a registration service node or another authorized participant to submit or process materials on behalf of someone else; and invocation authorization is a business permission jointly determined by the resource endpoint and the caller's policy. These roles may be held by the same subject or by different subjects.

Resource metadata may express some relationships through `controllerDid`, `publisherDid`, `subjectDid`, `implementationLinks`, or credential requirements, but the fields themselves are not sufficient proof that the relationships are genuine. The verifier should separately check the resource DID control proof, credentials of the publisher or registration service node, the root platform publication proof, governance state, and business endpoint authentication. It must not infer other permissions merely because a party can upload a file, provide an address, or issue a registration result.

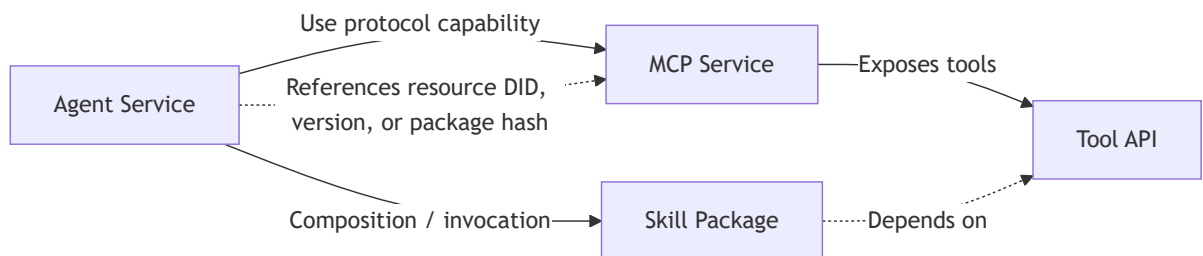
5.13 Resource Dependencies and Composition Integrity

Dependency integrity requires every resource in a composition to be independently identifiable, locatable, verifiable, and usable according to its version. The minimum check set includes the target resource DID, resource type, relationship type, target version or package hash, consistency between the DID document and metadata, the root platform's trusted publication proof, lifecycle state, authorization domain, and necessary protocol compatibility conditions. When a discovery service node returns only an index summary, the caller should obtain the complete resource package or verifiable proof before establishing a high-risk composition.

When a discovery node is behind, the target resource is temporarily unavailable, or proof is temporarily missing, the composition may be marked as pending synchronization or unverifiable. When the target resource has version drift, is suspended, revoked, outside the authorization domain, or has an inconsistent hash, the composition should not continue to be used unless a clearly constraint-satisfying alternative version exists. Composition integrity is a pre-invocation verification condition, not an incidental result of registration or search ranking.

The four resource types share common information such as resource DID, resource type, capability description, version, endpoint, protocol binding, and lifecycle, but differ in content form and invocation method. The goal of the resource model is to provide a common registration, publication, and discovery framework, not to force the four resource types into the same runtime object.

Resource type	Main content carried	Common discovery basis	Key checks before invocation or use
Agent service	Agent capabilities for tasks or sessions	Capability tags, use cases, interaction protocols, and service endpoints	Service identity, protocol, authentication, and business permissions
Skill package	Reusable instructions, workflows, or capability components	Capability description, applicable scenarios, dependencies, and version	Package hash, provenance, compatibility, and execution boundary
MCP service	MCP-oriented tool collection or service entry point	MCP protocol binding, tool capabilities, and endpoint	MCP version, server identity, authentication, and tool declaration
Tool API	Directly callable interface capability	API capability, input/output schema, and endpoint	API version, schema, authentication, and invocation authorization



Resource relationships must explicitly state the target DID, relationship type, and version or package hash. A name, search relevance score, or reachable endpoint alone cannot prove dependency integrity. When a referenced resource is suspended, revoked, has version drift, or falls outside the

authorization domain, the composition party should refuse further use or move it to a pending-verification state.

Boundary of the resource model: OAN is responsible for the identity, description, publication, discovery, and verification associations of resources. Model inference, business workflows, tool execution, and final business authorization inside a resource remain the responsibility of the corresponding resource and the caller's policy.

5.14 References

Source	Type	Link
oan-protocol-common	Code repository: resource types, core domain objects, and resource packages	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	Code repository: resource DIDs, resource packages, and client-side material construction	https://github.com/OpenAgenet/oan-sdk-ts
OAN White Paper	arXiv white paper: resource ecosystem and interoperability positioning	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv yellow paper: resource model and trusted publication mechanism	https://arxiv.org/abs/2606.03163
did: oan DID Method Specification	Standard/method specification: resource identity expression	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md

Chapter 6 `did: oan` Method and Resource Identity

This chapter defines the stable rules for `did: oan` as the resource identity method of OpenAgent (OAN). The method layer answers “whether an identifier is valid, what object it identifies, how to obtain the corresponding DID Document, and who can prove control.” It does not answer whether a resource has been registered, whether the Root platform has published it, or whether a Discovery Service Node has indexed it. Registration, publication, distribution, and discovery states in the resource network should refer to the objects and evidence defined in other chapters of this manual.

An implementation of the `did: oan` method must simultaneously satisfy four conditions: syntactic parsability, explainable semantic encoding, verifiable DID Document structure, and verifiable control signatures. Its DID Document also uses `oanMetadata` to place resource identity and capability semantics in one parsable object: capability tags provide governed classification keys, capability descriptions explain the tasks that the resource can perform, use cases describe typical usage contexts, and protocol bindings and service references explain how to connect to the resource. This design allows the core descriptions required for semantic discovery to be obtained immediately after resolving the DID. It also binds semantic declarations to resource identity and version through the document hash, control proof, and `ResourcePackage`, making this an important innovation of `did: oan` for intelligent-agent resource discovery.

6.1 Method Purpose and Identifier Scope

`did: oan` can identify Agent Services, Skills, MCP Servers, and Tool APIs. It can also identify subjects, organizations, and infrastructure nodes when these objects are permitted by the method definition. The first two positions of the method-specific semantic code distinguish the object category, while the latter two positions represent the application or semantic domain. Resource DIDs, subject DIDs, Registrar Service Node DIDs, Discovery Service Node DIDs, and Root platform DIDs serve different purposes and must not be conflated merely because they use the same DID method.

Uniqueness first means string uniqueness within the method namespace. Random generation depends on cryptographically secure random numbers, while deterministic derivation is jointly determined by semantic encoding, control material, and a nonce. When a resource is propagated through different Registrar Service Nodes or Discovery Service Nodes, its resource DID does not change because the entry point changes. Historical versions of the same DID should be associated through versions, hashes, publication sequences, and historical DID Documents rather than by

arbitrarily generating a new DID for every update.

In the OAN network topology, the entry node and the identity object are two different relationships. A client may submit a resource through one Registrar Service Node and query the same resource from multiple Discovery Service Nodes, but the DIDs of those service nodes identify the nodes themselves. The resource DID is still generated or held by the resource controller. A resolver should obtain the corresponding DID Document from the resource DID instead of embedding the submission entry point into the DID.

Object	DID subject	Primary purpose	Object it must not replace
Subject DID	Organization, developer, or controlling subject	Express the control relationship and signing context	Specific resource DID
Resource DID	Agent Service, Skill, MCP Server, or Tool API	Express the registration, version, and control boundary of a resource	Subject DID or node DID
Registrar Service Node DID	Registration infrastructure node	Express the identity of a registration entry point and node	Resource control
Discovery Service Node DID	Discovery infrastructure node	Express the identity of an indexing and query entry point	Resource publication fact
Root platform DID	Root platform infrastructure identity	Express the source of Root-related proofs and publication	Resource ownership

6.2 DID Syntax and Method-Specific Identifier

The method-specific identifier uses the form `did: oan: <semantic-code>: <suffix>`. `semantic-code` must contain 4 ASCII uppercase letters or digits. Its first two positions are the subject code, and its last two positions are the application-domain code. `suffix` must contain 32 Base58 characters and must use the Base58 character set defined by OAN. Easily confused characters such as 0, o, I, and 1 are not permitted. A method implementation validates the complete DID, the number of segments, the semantic code, and the suffix separately.

For example, `did: oan: SKFI: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu` is a syntactically parsable Skill DID. A string with incorrect case, a suffix whose length is not 32, an illegal Base58 character, or a missing segment must be rejected. SDK normalization only converts the semantic code to uppercase according to the method rules and leaves the suffix unchanged. Normalization cannot repair invalid characters or change the object category of a DID.

The method library separates the parsing result into the complete value, semantic code, subject code, application-domain code, and suffix, so that subsequent resource-type checks do not require callers to guess meanings from string positions:

```
{
  "input": "did: oan: skfi: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "normalized": "did: oan: SKFI: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "semanticCode": "SKFI",
```

```
"subjectCode": "SK",  
"appDomainCode": "FI",  
"suffixLength": 32,  
"parseResult": "valid"  
}
```

This normalization applies only when the case of the semantic code can be converted according to the implementation rules. If the number of segments is wrong, the suffix length is invalid, or an illegal character is present, parsing must fail directly. The implementation must not generate another DID through “best-effort repair.”

6.3 Relationship Between Resource DID and Subject/Controller

The method-specific code can indicate only the object category to which a DID belongs. It cannot independently prove the legal or business relationship among the subject, controller, provider, or operator. The control relationship of a resource should be expressed in the DID Document's verification methods, `oanMetadata.controllerDid`, `publisherDid`, `issuerDid`, and the resource control proof. A verifier must check the correspondence among these fields and the signature.

A subject may control multiple resource DIDs, and the resource provider may differ from the controller. The DID of a Registrar Service Node represents infrastructure qualification and cannot replace the resource DID. When an update uses the same resource DID, it must include an update proof that can be verified against the current control relationship. The DID string itself does not carry ownership, and public resolution of a DID does not transfer control.

6.3.1 Subject DID

A subject DID identifies the subject that controls resources or represents an organization or developer. It establishes the context of the control relationship and signing subject. It is not the DID of a specific resource and is not equivalent to a user login account. The DID Document of a subject DID should expose the public keys and service information required for verification. The private key is kept by the subject in the Local Identity.

6.3.2 Resource DID

A resource DID establishes an identity boundary for a resource that can be registered, published, discovered, updated, and revoked independently. Each of the four product resource types can have an independent resource DID. The resource type must be consistent with the DID semantic code and with `oanMetadata.resourceType` and `subjectType` in the DID Document. Otherwise, the Registrar Service Node and Root platform should reject the material.

6.3.3 Control and Signature Relationships

The control relationship is jointly supported by the verification methods in the DID Document and verifiable control proofs for specific operations. A verifier should check the signer, verification

method, target DID, purpose, challenge, time, and DID Document hash in the proof. It must not ignore purpose, target, or freshness checks merely because the proof's public key is identical to a public key in the document.

6.3.4 Separation of Subject, Resource, and Provider

The subject controls the identity and resource operations, the resource provider is responsible for capabilities, content, and service endpoints, and the operator is responsible for the runtime environment and availability. These roles may belong to one subject or may be separate. `publisherDid` or `subjectDid` is only a relationship declaration; the truth of the relationship still requires corresponding credentials, delegation materials, and control proofs.

6.4 DID Document Structure

An OAN DID Document is based on the DID Core data model and must at least contain `@context`, `id`, `verificationMethod`, `authentication`, `assertionMethod`, `service`, and the OAN-resource-oriented `oanMetadata`. `id` must be the normalized `did:oan`. The controller of each verification method, service-endpoint references, and resource identity in the metadata must be mutually verifiable. For Agent Services, Skills, MCP Servers, and Tool APIs, `oanMetadata` is the structured carrier for the resource type, capability semantics, protocol relationships, and publication references.

`oanMetadata.resourceDescription` and `oanMetadata.agentDescription` can express capability descriptions, capability tags, and use cases. A resource description can also carry input and output schemas, examples, audiences, domains, languages, and versions. Top-level `authorizedDomains`, `protocolBindings`, `implementationLinks`, `credentialRequirements`, and `packageInfo` supplement visibility scope, interaction protocols, composition relationships, invocation credential requirements, and publication materials respectively. A Discovery Service Node can use these fields to build tag, lexical, and vector indexes that map natural-language tasks to candidate resources. After obtaining a candidate, a caller can continue checking the DID control relationship, resource version, and publication proof.

The following is a minimal DID Document that illustrates the relationships among fields. The public key, service address, and resource DID in the example are placeholder values. An actual document must use a real public key and proof materials matching the controller:

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/oan/v1"
  ],
  "id": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "verificationMethod": [
    {
      "id": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
      "type": "Ed25519VerificationKey2020",
      "controller": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
      "cryptoSuite": "Ed25519Sha256",
      "publicKeyMultibase": "zExamplePublicKey"
    }
  ]
}
```

```

    }
  ],
  "authentication": [
    "did: oan: SKFI: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"
  ],
  "assertionMethod": [
    "did: oan: SKFI: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"
  ],
  "service": [],
  "oanMetadata": {
    "subjectType": "skill",
    "resourceType": "skill",
    "controllerDid": "did: oan: DVFI: 7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
    "capabilityTags": ["code-repository", "project-structure-summary"],
    "resourceDescription": {
      "name": "Repository structure analyzer",
      "description": "Searches code repositories and summarizes project structure.",
      "capabilityDescription": "Locate repository content, identify modules and summarize their relationships.",
      "capabilityTags": ["code-repository", "project-structure-summary"],
      "useCaseExamples": [
        "Inspect an unfamiliar repository before development",
        "Summarize modules, entry points and dependencies"
      ]
    }
  },
  "lifecycleState": "active"
}
}

```

A verifier should at least check that `id`, the `controller` of verification methods, and the authentication and assertion references belong to the same document. It should also check that the resource type is consistent with the `SK` subject code. Capability tags, capability descriptions, and use cases should describe the same resource capability. A Discovery Service Node can use them respectively for category filtering, semantic retrieval, and result explanation. Because these semantic properties are located in the DID Document, the resource controller can generate a control proof over a document containing the semantic declarations. The `ResourcePackage` can also lock this version through `didDocumentHash`, preventing the discovery index from combining the identity with an external description of unknown origin.

6.4.1 DID Identifier and Method Metadata

`id` is the subject identifier of the DID Document and must exactly match the resource DID in the request or `ResourcePackage`. `@context` should include the DID Core context and the OAN method context. A parser may interpret the method-specific semantic code to determine the resource category, but it must not use that code alone to replace consistency checks for `oanMetadata` and the `ResourcePackage`.

6.4.2 verificationMethod

`verificationMethod` describes public-key methods that can be used to verify signatures. Each item must contain at least a method ID, type, controller, and one public-key representation. OAN core types support representations such as `publicKeyMultibase` and `publicKeyJwk`, and can use `cryptoSuite` and `publicKeyFormat` to specify the cryptographic suite and encoding. A private key must never appear in a DID Document.

6.4.3 authentication and assertionMethod

`authentication` identifies verification methods that can be used for identity authentication or request proofs. `assertionMethod` identifies verification methods that can be used to issue or verify assertions. Each reference must resolve to a method ID in the same DID Document. A key used only for assertions must not automatically be treated as an authentication key, and a public key present in `verificationMethod` must not automatically be considered valid for every purpose.

6.4.4 service Endpoint

Each service item in `service` should contain a unique service ID, service type, and `serviceEndpoint`. It may also specify a version, protocol, server type, and port. An endpoint may point to a resource business service, resolution service, `ResourcePackage`, or another protocol description. The fact that an endpoint can be resolved proves only that an address exists; it does not prove that the business is available or that the caller is authorized.

6.4.5 Resource Metadata and Extension Fields

`oanMetadata` carries OAN resource type, subject type, controller, publisher, capabilities, authorization domains, protocol bindings, implementation links, credential requirements, package information, lifecycle, and other semantics. Extension fields must not override the identity and verification meanings of DID Core. If the DID, type, version, authorization domain, or hash conflicts among resource metadata, the `ResourcePackage`, and the registration submission, the material should be rejected or isolated according to the conflict-handling rules.

The relationship between `oanMetadata` and DID Core fields can be understood as follows:

Field area	Main question answered	Verification focus
<code>id</code> , <code>verificationMethod</code>	Which DID is this, and which public key verifies it?	DID method syntax, method ID, controller, and public-key format
<code>authentication</code> , <code>assertionMethod</code>	Which methods can be used for authentication or assertion?	Whether references exist and purposes match
<code>service</code>	Where can the service or protocol entry point be obtained?	Service ID, protocol, version, and endpoint format
<code>oanMetadata</code>	What is the resource, and what capabilities and status does it have?	Resource type, subject type, authorization domain, version, and package binding

Therefore, a parser may preserve unknown OAN extension fields, but it must not allow extension fields to change the meaning of `id`, verification methods, or purpose references. After successful

method resolution, a resource-network verifier should continue checking consistency between meta-data and the ResourcePackage.

6.5 Verification Methods and Public Keys

The `id` of a verification method should belong to the corresponding DID, and its `controller` should match the control relationship. The public-key representation and cryptographic suite must be parsable by the verifier. A control signature should specify `proofPurpose`, `verificationMethod`, the target object, and the signature coverage. The verifier first resolves the public key from the DID Document, and then checks the signature, purpose, hash, and time conditions.

Multiple verification methods may separately serve authentication, assertion, update, or recovery purposes. Key rotation must be authorized by the currently valid control relationship and must preserve the state required for historical verification or old-key invalidation. An old key must not continue issuing valid new updates. Merely writing an attacker's public key into a DID Document that has not been accepted by the current control relationship does not grant control of the DID.

The acceptability of a verification method is not the single condition that “a public key exists.” It is the joint result of multiple fields:

```
{
  "verificationMethod": "did: oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
  "expectedController": "did: oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "proofPurpose": "authentication",
  "targetDid": "did: oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "documentHash": "sha256:example-did-document",
  "signatureVerification": "required"
}
```

The verifier should first confirm that `verificationMethod` can be resolved from the target DID Document, and then confirm that its controller, purpose, target DID, and signed document hash match. If the public key belongs to another DID, the method is not referenced for the applicable purpose, or the document has been replaced, verification fails. Comparing only whether public-key bytes are identical is insufficient.

6.6 Service Endpoints and Discovery References

Service endpoints in a DID Document provide location information for resources or protocols. Typical uses include business services, Registrar Services, Discovery Services, DID resolution, and ResourcePackage retrieval. The `id` of a service item should be associable with the resource DID, and `serviceEndpoint` should conform to the declared address or object format. A discovery response may reference an endpoint, but the caller must still verify the Root publication fact, endpoint freshness, and business authentication requirements.

The interfaces of Registrar Service Nodes, Discovery Service Nodes, and resolvers describe network-component behavior. They do not change their authorization boundaries merely because an endpoint is written into a DID Document. If an endpoint is missing, its protocol is incompatible, its address is unreachable, or the referenced object is incompatible, resolution may still successfully

return the DID Document. Before discovery or invocation, however, the verifier must clearly mark the corresponding problem.

Resolution results should distinguish “the document was retrieved successfully” from “the endpoint is available.” A client-oriented result summary may contain:

```
{
  "did": "did: oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "resolution": {
    "status": "resolved",
    "documentRetrieved": true,
    "methodSyntaxValid": true,
    "documentIdMatchesRequest": true
  },
  "services": [
    {
      "id": "#resource",
      "type": "OANResource",
      "serviceEndpoint": "https://example.invalid/resource",
      "availability": "not_checked"
    }
  ],
  "nextVerification": ["control-proof", "publication-proof", "endpoint-authentication"]
}
```

`availability: not_checked` means that the resolution phase did not assess business-endpoint availability. A resolver must not rewrite the existence of an HTTP address or successful DID Document retrieval into endpoint health, published-resource status, or invocation authorization.

6.7 Resource Metadata Binding

The resource DID Document is associated with resource metadata and the `ResourcePackage` through `oanMetadata` and package information. For the same resource version, the resource DID, resource type, capability tags, capability description, use cases, authorization domains, protocol bindings, version, and hashes should remain verifiably consistent across the DID Document, registration submission, and `ResourcePackage`. The `ResourcePackage`'s `didDocumentHash` binds the exact document content; binding only the DID string is insufficient. Semantic discovery can therefore refer simultaneously to a stable DID, governed semantic declarations, and a published version instead of building an index only over free text separated from identity.

After a resolver returns the DID Document and `oanMetadata`, a Discovery Service Node can extract capability tags, capability descriptions, use cases, input and output schemas, and protocol bindings. It can combine the shared capability vocabulary with a local semantic search engine to build an index. During a query, a natural-language task is used for semantic retrieval, capability tags and resource types for structured filtering, and use cases and capability descriptions for explaining the match. Authorization-domain, lifecycle, and publication-status checks are then applied. This chain connects the semantic extension of the DID method with OAN semantic governance and the discovery protocol.

The binding relationship for one resource version can be checked through the following summary. The three hashes serve different purposes: a resource DID only locates an object and cannot replace integrity checks on the document and content:

```
{
  "resourceDid": "did: oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "packageVersion": "1.0.0",
  "didDocumentHash": "sha256:example-did-document",
  "metadataHash": "sha256:example-metadata",
  "packageHash": "sha256:example-package",
  "checks": {
    "didDocumentIdMatchesResourceDid": true,
    "resourceTypeMatchesSemanticCode": true,
    "metadataHashRecomputed": true,
    "packageHashRecomputed": true
  }
}
```

If the DID Document changes, `didDocumentHash` should change even when the resource DID remains the same, and a new control proof and resource-network verification should be triggered. If the caller queries only the DID string and does not compare the document hash, it cannot determine which document version it has received.

6.8 Control Proofs and DID Document Proofs

A resource control proof is generated by the controller using the private key corresponding to a verification method in the DID Document. It proves control over a particular DID, document hash, operation purpose, and challenge. The Registrar Service Node verifies this proof, and the Root platform verifies it again before accepting a resource version. A registration credential proves the fact that registration was processed; a Root proof proves the fact of network publication. These three kinds of evidence cannot replace one another.

A DID Document proof should cover the normalized document content or a prescribed document hash, and should include a locatable verification method, creation time, proof purpose, and signature value. Verification must fail if the proof is absent or expired, the target DID does not match, the document hash does not match, the purpose does not match, or the signature cannot be verified. No published state may be produced from such a failure.

The verification input for an OAN control challenge should bind the request context to the signature instead of verifying only a context-free signature. The key checks of a security library can be abstracted as follows:

```
{
  "challenge": {
    "challengeId": "challenge-example",
    "draftId": "draft-example",
    "subjectDid": "did: oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "registrarDid": "did: oan:INRG:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
    "purpose": "register_resource",
  }
}
```



```

    "verificationMethod": "did:oa:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
    "didDocumentHash": "sha256:example-did-document",
    "expiresAt": "2026-09-09T09:00:00Z"
  },
  "verification": {
    "signatureValid": true,
    "challengeMatchesRequest": true,
    "notExpired": true,
    "proofPurposeMatches": true
  }
}

```

The Registrar Service Node should compare the subject DID, target Registrar Service Node, purpose, document hash, and validity period in the challenge with the current request item by item. Passing the challenge proves only that the controller authorized this particular operation with its associated context. It does not mean that the Registrar Service Node has formed a registration record or that the Root platform has published the resource.

6.9 DID Generation and Local Key Handling

The client is the primary generation and custody boundary for DIDs and control keys. A random DID is generated from cryptographically secure random numbers, while a deterministic DID is derived from semantic encoding, control material, and a nonce. In either case, the result must pass method parsing and resource-type consistency checks. A Registrar Service Node should receive only the public DID Document, signatures, and proofs required for verification. It should not obtain subject or resource-control private keys.

A Local Identity backup is sensitive material for restoring a control relationship, not a public credential. During import, the DID, keys, public keys, control relationships, and file structure must be checked for consistency. A newly generated control proof after restoration must be rebound to the current operation and document hash; a historical signature cannot be treated as unconditional authorization for every future version.

When handling Local Identity materials, the client can reduce key-leakage and binding errors through the following sequence:

1. Generate or import subject and resource-control keys locally, and check that public and private keys match.
2. Generate or resolve `did:oa` according to the resource type, and check that the semantic code matches `resourceType`.
3. Construct the DID Document and confirm that verification methods, purpose references, and `oaMetadata` point to the same resource DID.
4. Generate a new challenge control proof for the current document and operation instead of reusing a signature bound to an old document.
5. Submit only the public DID Document, proof, and necessary resource metadata to the Registrar Service Node; never upload private keys.

A Local Identity backup downloaded by a browser is user-side recovery material and cannot be replaced by storing keys on the server. A page indicating “created” or “download complete” only means that the client completed the file operation. The user must still verify that the file is complete and keep it according to local security requirements.

6.9.1 DID Generation

The client can call a method library's random-generation or deterministic-derivation interface to create a DID. The semantic code in the generation parameters must be valid. Deterministic derivation produces the same result only when the control material and nonce are identical and the implementation uses the same method version. The derivation algorithm must not be treated as a mechanism for recovering a private key from a DID.

6.9.2 Key Generation and Storage

Control keys should be generated or imported in a client environment controlled by the user, with private keys stored through local secure storage or encrypted backup. DID Documents, registration requests, and discovery responses expose only public keys and materials required for verification. Server logs, request bodies, and error responses must not contain private keys or plaintext materials from which private keys can be recovered.

6.9.3 Local Identity Backup

A Local Identity backup may contain a subject DID, resource DIDs, key materials, resource configuration, and necessary credentials. It should clearly state its sensitivity level, creation time, and recovery purpose. The backup filename, download location, and page text do not provide identity authentication. The user must independently protect the backup and prevent unauthorized copying.

6.9.4 Import, Recovery, and Key-Use Boundaries

After import or recovery, the client should confirm that public and private keys correspond, that the DID Document subject is consistent, and that the resource type is consistent. It should regenerate the current challenge and hash before signing. A Registrar Service Node, Root platform, or Discovery Service Node may verify public proofs, but should not obtain private keys from the import process. Node-runtime private keys must also be separated from user resource-control private keys.

6.10 DID Resolution Requirements

A resolution request first validates the DID method and syntax, and then uses a resolution service to obtain the currently available DID Document or an explicitly requested historical version. A successful response should contain the document and its verifiable metadata. “Not found,” unsupported method, invalid format, inactive resource, and unavailable historical version should use

distinguishable errors or statuses instead of returning an empty document and making the caller guess the cause.

Successful resolution means only that the resolver can return a DID Document. It does not mean that the resource is registered, the Root platform has published it, the credential remains valid, or the business endpoint is available. The return type of the current resource-detail or DID Document HTTP interface is determined by the specific node. The content distribution platform's `GET /cdn/documents/{did}` directly returns `DidDocument`, while some directory interfaces place the DID in a `did` field and the document in a `document` field. Clients must parse according to the actual interface contract and must not assume that every DID read interface uses the same wrapper structure. The caller should continue judging based on resource-network evidence, governance status, version, and local policy.

Resolution errors should allow the caller to distinguish input errors, method capability, and network status. The following result categories can be used; the specific HTTP mapping is determined by the resolution-service interface:

Result category	Method-layer judgment	Caller handling
<code>invalid_did</code>	DID prefix, segments, semantic code, or suffix is invalid	Correct the input and do not start a business call
<code>method_not_supported</code>	The current resolver does not support <code>did:oan</code>	Switch to a resolver that supports the method
<code>not_found</code>	The method is valid but no DID Document can be returned	Distinguish not registered from temporarily unreachable
<code>resolved</code>	The document has been retrieved and passed basic structural checks	Continue with control, publication, and endpoint verification
<code>status_restricted</code>	The document or resource status restricts current use	Retain historical evidence and stop sensitive actions

A resolver must not conceal `not_found` or `method_not_supported` by returning an empty object. It must not treat an old snapshot from a Discovery Service Node as the current result of DID method resolution.

Interoperability with external protocols such as A2A and MCP is a matter of consuming and adapting resolution results. It does not change the identifier, DID Document, or control-relationship rules of `did:oan`. A resolver may preserve `oanMetadata`, service endpoints, and protocol bindings for an adapter to map them into an external protocol description. The caller must still independently verify the DID, `ResourcePackage`, control proof, and Root publication evidence.

6.11 DID Update, Version Management, Suspension, Recovery, and Deactivation

An update to the same resource normally keeps the resource DID unchanged and expresses the change through a new DID Document version, `ResourcePackage` version, hash, and control proof. The updater must sign with the currently valid control relationship. The Root platform and relevant nodes should check version order, reference to the previous version, document hash, and resource

lifecycle state. The method library itself provides DID resolution and type validation only; it does not replace the network update protocol.

The minimum decision for an update to the same DID can be summarized as follows:

```
current DID + current control proof + new document hash
-> verify current control and version relation
-> accept a new resource version or reject the update
```

If the DID in an update request differs from the existing record, it should be treated as a new resource or an invalid replacement. The object to be overwritten must not be determined by receipt time. If the DID is the same but the control proof is invalid, the old document must not be overwritten merely because the resource name, endpoint, or submitter is the same.

Suspension, recovery, and deactivation are resource-network state actions. They must be bound to explicit control proofs, governance facts, or authorization conditions. Historical documents may be retained for auditing and historical verification, but current-state resolution must not treat an old document or old publication proof as the latest state.

6.11.1 Update with the Same DID

An update using the same DID requires the new material's id to equal the existing resource DID and requires the current control key to sign the new document or prescribed document hash. If the control relationship changes, the key-rotation or recovery rules must be satisfied first. Submitting the same DID with only a new public key cannot overwrite the existing document.

6.11.2 Version Selection

The current version should be determined jointly by the resource version, document hash, package hash, and publication sequence. A resolver or Discovery Service Node should select the latest version according to trusted publication facts and lifecycle status, not solely according to receipt time or an auto-incrementing local database number. Historical-version queries require an explicit version identifier or publication reference.

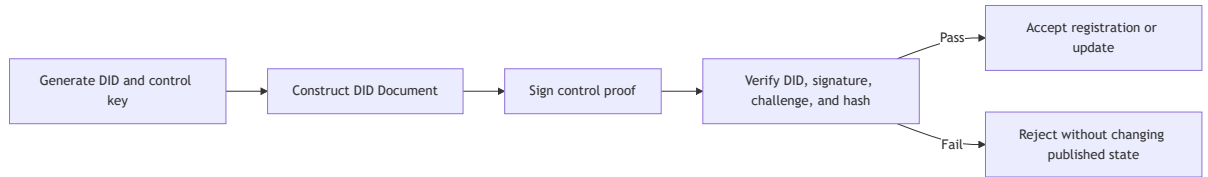
6.11.3 Suspension and Recovery

Suspension should prevent discovery, publication, or pre-call verification within the specified scope from treating the resource as active. Recovery requires new valid status evidence and a renewed check of the version and publication evidence. A cached old document may be used for historical display, but it must not bypass the current suspension state.

6.11.4 Deactivation and Historical State

Deactivation means that the DID or resource no longer accepts new valid use within the specified scope. Historical documents and proofs may still be retained for auditing. Deactivation, resource revocation, registration-credential revocation, and endpoint shutdown should be recorded separately. One page-status field cannot replace all of these facts.

The basic control path for same-DID updates, suspension, recovery, and deactivation is shown below. The diagram expresses the verification order between the method layer and the resource-network layer; it does not mean that the method library completes registration or publication by itself:



6.12 DID Method Security and Privacy Considerations

Security comes from private-key confidentiality, signature coverage, DID Document public-key binding, challenge, and freshness checks, not from a DID string being hidden or complicated. An attacker can copy a public DID Document and public key, but without a valid control private key, current challenge, and corresponding authorization, the attacker cannot produce an acceptable update proof. A random nonce, request time, document hash, and purpose field should participate in replay protection.

A method implementation should avoid unconditionally exposing subject relationships, resource content, and endpoint privacy. A DID Document should contain only the information required for public verification and discovery. Sensitive credentials, private keys, and internal addresses should be handled through controlled endpoints or external authorization mechanisms. Failed verification must not write a published state, and error logs should retain sufficient audit information without revealing private materials.

6.13 DID Method Conformance Requirements and Test Vectors

Conformance tests should at least cover valid DID resolution; rejection of incorrect prefixes, segment counts, semantic codes, suffix lengths, and illegal characters; mapping of the four resource codes; random-generation results that can be parsed again; deterministic derivation that is stable for identical inputs and changes when control material or nonce changes; rejection of mismatches between DID and resource type; and semantic preservation before and after normalization.

Signature tests should also cover a verification-method purpose mismatch, target-DID mismatch, document-hash mismatch, expired challenge, reused nonce, wrong public key, and update with an old key. Test evidence should record fixed inputs, method version, expected errors, and actual outputs. Passing method-layer tests does not mean that the registration, publication, or discovery chain has passed.

The method implementation can use the following minimum test vectors as examples of input classification:

Input	Expected result	Basis for rejection or acceptance
did:oan:SKFI:5 HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu	Pass	Four-segment structure, four-character semantic code, and 32-character Base58 suffix are valid
did:oan:skfi:5 HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu	Normalize before judgment	Only the semantic-code case may be normalized according to SDK rules
did:oan:SKFI:0 HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu	Reject	The suffix contains the forbidden Base58 character 0
did:oan:SK:5 HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu	Reject	The semantic code does not contain four characters
Valid DID + resourceType: tool_api	Reject	SK does not match the resource type

The DIDs and suffixes in the test vectors are used only to verify the method implementation and must not be used directly as production resource identities. Tests should also record the resolver version, input before and after normalization, error type, and whether any persistent side effect was produced. Returning an error string alone is insufficient to prove that publication state was not modified.

6.14 DID Method Interoperability and Resolver Behavior

did:oan should follow the basic data model of DID Core for DIDs, DID Documents, verification methods, and services, while the method implementation defines its method-specific identifier and resolution behavior. An external resolver should at least recognize the method name and reject identifiers that do not conform to the method syntax. A resolver supporting the OAN method should also preserve oanMetadata and its extension fields; it must not lose resource type, capabilities, or publication references during conversion. When consuming public information, an external system must still verify the control relationship, ResourcePackage, and Root publication evidence according to OAN rules.

A did:oan identifier consists of the method name and a method-specific identifier. The following example illustrates the structure and field relationships only. The example values do not represent real network objects and must not be treated as control materials that can be published directly.

```
did:oan:<subject-type-or-resource-type-code>:<method-specific-identifier>
```

```
{
  "id": "did:oan:SKDM:example-resource-id",
  "verificationMethod": [
    {
      "id": "did:oan:SKDM:example-resource-id#key-1",
      "type": "JsonWebKey2020",
      "controller": "did:oan:SKDM:example-resource-id",
      "publicKeyJwk": { "kty": "OKP", "crv": "Ed25519", "x": "..." }
    }
  ]
}
```

```

    }
  ],
  "authentication": ["did:oa:SKDM:example-resource-id#key-1"],
  "service": [
    {
      "id": "#resource",
      "type": "OANResource",
      "serviceEndpoint": "https://example.invalid/resource"
    }
  ]
}

```

Check object	Question that must be answered	Consequence of failure
DID string	Do the method name, code, characters, and type conform to the rules?	Reject resolution or enter an invalid state
DID Document id	Does it exactly match the resource DID in the request?	Reject the binding
Public key and control relationship	Is the verification method declared by the DID controller and capable of verifying the signature?	Do not accept the control proof or update
Signature coverage	Does the signature cover the target DID, document/package hash, challenge, and purpose?	Reject the signature fact
Service and metadata	Are the endpoint, protocol, and resource description consistent?	Do not enter trusted discovery or invocation

Method-layer conformance testing should separately record valid resolution, invalid-input rejection, DID-type mapping, document binding, signature purpose, challenge freshness, and update control. Passing method tests proves only that DID-method behavior matches the test vectors. It does not replace business-chain verification by the Registrar Service Node, Root platform, or Discovery Service Node. The specific generation, signing, verification, and rejection paths are described in Sections 5.9, 5.10, 5.11, and 5.13; this section only specifies that cross-resolver consumption must not discard method semantics.

6.15 References

Source	Type	Link
oan-protocol-common	Code repository: core did:oa types and method implementation	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	Code repository: TypeScript DID resolution, generation, and verification	https://github.com/OpenAget/oan-sdk-ts

Source	Type	Link
OAN Yellow Paper	arXiv Yellow Paper: identity, control, and resource-publication mechanisms	https://arxiv.org/abs/2606.03163
did:oan DID Method Specification	Standard/method specification: did:oan identifier format and resolution rules	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Resource Identity and Discovery	IETF Internet-Draft: resource identity and discovery	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

Chapter 7 Resource Metadata and Resource Package Types

This chapter defines the data boundaries, field relationships, deterministic hashing, and recipient-side verification methods for resource metadata and `ResourcePackage`. Resource metadata describes what a resource is, what it can do, and how it can be connected. `ResourcePackage` organizes the DID document, metadata, hashes, and root platform trusted publication proof for a specific resource version into a distributable object. A registration submission is the input to the verification process, while a root-platform-verified resource package is the publication product after network-level verification. These three objects are not interchangeable.

This chapter primarily relies on the resource types, serialization implementation, and resource package validation code in `oan-protocol-common`. The current registration payload schema only provides object-level entry constraints; the semantics of individual fields are jointly determined by the protocol types and validation logic. Where the schema, implementation, and deployment configuration differ, the applicable version and the difference should be stated. A single example must not be treated as mandatory behavior for every implementation.

7.1 Resource Metadata Schema

In the Rust implementation, resource metadata is represented by `ResourceMetadata`. Its core fields are as follows:

Field	Type	Semantics and validation focus
<code>resourceDid</code>	string	The resource DID; it must correspond to the <code>id</code> of the <code>ResourcePackage</code> and DID document
<code>resourceType</code> , <code>subjectType</code>	enum	The resource and subject types; current resource package validation requires the two values to match
<code>publisherDid</code> , <code>subjectDid</code>	optional string	Declarations of the publisher-subject relationship; they do not independently prove control
<code>name</code> , <code>description</code>	string	The resource name and summary description
<code>capabilityTags</code>	string array	Capability tags for structured and semantic discovery

Field	Type	Semantics and validation focus
<code>authorizedDomains</code>	string array	Authorized domains in which the resource may be propagated or discovered
<code>protocolBindings</code>	object array	Protocol, version, transport method, and service or schema references
<code>services</code>	service endpoint array	The resource's business entry points or protocol entry points
<code>lifecycleState</code>	string	The lifecycle state of the current resource version
<code>packageVersion</code>	string	The current resource package version
<code>packageHash, metadataHash</code>	string	Integrity references for the resource package and metadata
<code>hashAlgorithm</code>	string	Hash-suite identifier, which must be consistent with the hash references
<code>updatedAt</code>	UTC time	The update time of the current metadata version

These fields may be exposed for registration, distribution, and discovery, but control private keys, internal access credentials, and operational details that should not be public must not be placed in the metadata. Unknown extension fields may be retained only when they do not change the meaning of core fields, the hash input, or the verification result. A recipient should reject core-field type errors, a missing resource DID, or unsupported enum values.

The following fragment shows the field relationships in a `ResourceMetadata` instance. The hash values are examples and must not be used directly in production materials. During metadata hash computation, `packageHash` and `metadataHash` must first be cleared and then populated with the computed results.

```
{
  "resourceDid": "did:oon:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "resourceType": "skill",
  "subjectType": "skill",
  "publisherDid": "did:oon:ORLG:8LcR3Vn5YpQw2Tx7Mb9Zd4Fa6GhKsEuJ",
  "subjectDid": "did:oon:DVLG:7YpQm9Kx2VnRb6TsWfHa4Cd5Ej8LgNz",
  "name": "Repository Structure Summary",
  "description": "Summarizes the structure of a code repository.",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "authorizedDomains": ["public"],
  "protocolBindings": [],
  "services": [],
  "lifecycleState": "active",
  "packageVersion": "1.0.0",
  "packageHash": "sha256:example-package",
  "metadataHash": "sha256:example-metadata",
  "hashAlgorithm": "sha256",
  "updatedAt": "2026-09-09T08:30:00Z"
}
```

```
}

```

7.2 Required and Optional Fields

The structurally required fields of `ResourcePackage` include `packageVersion`, `resourceDid`, `resourceType`, `didDocument`, `didDocumentHash`, `metadataHash`, `packageHash`, `hashAlgorithm`, `metadata`, `rootProof`, and `createdAt`. In the metadata, the resource DID, resource type, subject type, package version, three key hashes, and hash algorithm are the core fields for forming a verifiable publication object. Capability tags, authorized domains, protocol bindings, and service endpoints may be empty arrays or may be undeclared for a given resource type, but their omission must not be presented as evidence that the resource has the corresponding capability.

The four resource types share common identity, description, version, integrity, and governance fields. An agent service generally needs capability descriptions, use cases, interaction protocols, and service endpoints. A skill package needs package entry points, installation or loading information, and dependencies. An MCP server needs transport methods, tool or resource inventories, and authentication requirements. A tool/API needs operations, parameters, outputs, and invocation constraints. Exact requiredness is determined by the protocol model for the applicable version; web form inputs cannot directly replace the underlying schema.

When a field required by the protocol is empty, the recipient must reject the object. A missing optional field is handled as “undeclared”. Unknown extension fields may be retained in compatibility mode, but they must not be used for permission decisions that the recipient cannot interpret. The registration service node should perform these checks before registration, and the recipient of a `ResourcePackage` should perform them again.

The four resource types share the resource DID, resource type, subject type, description, capability tags, version, authorized domains, and integrity fields, but the resource type determines which capability descriptions and endpoint information have practical meaning. When fields are missing, the recipient should distinguish “undeclared” from “structurally incomplete”. For example, no optional service endpoint may mean that no invocation entry point has yet been declared, while a missing `resourceDid`, resource type, DID document, or resource package hash prevents the formation of a verifiable resource package.

Field scope	Common requirement for all four resource types	Type-specific supplement
Identity and type	<code>resourceDid</code> , <code>resourceType</code> , <code>subjectType</code>	The three values must be consistent with the semantic encoding of <code>did: oan</code>
Capability description	Name, description, tags, and use cases may be used for discovery	Agent services emphasize tasks and interaction; skill packages emphasize reuse and dependencies
Connectivity	Protocol bindings and service endpoints are declared as needed	MCP servers and tool APIs generally need more explicit protocol or interface descriptions

Field scope	Common requirement for all four resource types	Type-specific supplement
Integrity and version	Package version, three classes of hashes, and hash algorithm	A content change must result in a new verification and publication decision

7.3 Resource Type and Subject Type Constraints

The resource type must remain consistent with the semantic encoding of `did:oon: agent_service` corresponds to AG, `skill` to SK, `mcp_server` to MC, and `tool_api` to TL. In current resource package validation, `resourceType` and `subjectType` in the resource metadata must be identical, and both should correspond to the subject code in the resource DID. Infrastructure nodes, organizations, and developers may be identified as method-level identity objects, but a node DID must not be directly used as a product resource DID.

A subject may control multiple resource DIDs, and one resource provider may operate multiple resources. The identity boundaries between the subject DID, resource DID, and node DID are not merged by relationship fields. A verifier must reject a resource package when its resource type, subject type, DID document id, or package metadata is inconsistent.

7.4 Name, Description, Tags, and Use Cases

The name and description are public claims by the resource provider about the resource's capabilities. Capability tags and use cases are used for registration recommendations, semantic governance, and discovery queries. Tags should preferably reference identifiers in the root platform capability tag tree; an extended tag should state its source or namespace. Natural language may use Chinese, English, or another declared language, but field types and array structures must remain stable. HTML, scripts, or unparseable objects must not be mixed into these fields. A detailed description does not mean that the capability has been verified, and relevance ranking is not proof of control or business quality.

The name and description are public claims by the resource provider about the resource's capabilities. Capability tags and use cases are used for registration recommendations, semantic governance, and discovery queries. Tags should preferably reference identifiers in the root platform capability tag tree; an extended tag should state its source or namespace. Natural language may use Chinese, English, or another declared language, but field types and array structures must remain stable. HTML, scripts, or unparseable objects must not be placed in text fields.

A more detailed description or use case does not mean that the capability has been verified. Relevance ranking by a discovery service node is also not proof of control, endpoint availability, or business quality. Length, quantity, and content filtering should be specified by deployment or conformity configuration and enforced before registration and indexing.

When capability tags and use cases enter the discovery process, they may serve both as structured conditions and as semantic retrieval text, while the original values and their sources should be retained in the result. For example, a query such as “I need a tool that can inspect a code

repository and summarize its project structure” may match the relevant capability description and use case. Whether the resource can be invoked still requires checking the resource DID, publication proof, endpoint, and credential requirements. A tag match is evidence of relevance, not proof of resource control or business effectiveness.

7.5 Semantic Tags and Protocol Bindings

`capabilityTags` expresses the resource capability classification, while `authorizedDomains` expresses the scope of propagation and discovery permissions. They cannot substitute for one another. A binding object in `protocolBindings` must at least state the protocol name and may include a version, transport method, `serviceRef`, and `schemaRef`. A protocol binding describes how to connect; it does not automatically grant invocation permission. A registration service node may use these fields to recommend tags, and a discovery service node may use them for structured or semantic retrieval, but returned results must still retain their source and verification status.

`capabilityTags` expresses the resource capability classification, while `authorizedDomains` expresses the scope of propagation and discovery permissions. They cannot substitute for one another. A binding object in `protocolBindings` must at least state the protocol name and may include a version, transport method, `serviceRef`, and `schemaRef`. A protocol binding describes how to connect; it does not automatically grant invocation permission. The tag-tree version, tag identifier, and aliases should retain their source so that different nodes can interpret the same tag.

A registration service node may provide recommendations based on capability descriptions and the tag tree. A discovery service node may use them for structured or semantic retrieval, but externally returned results must still retain the original tags, protocol bindings, source, and verification status. A tag match without a root platform publication proof must not enter the trusted discovery result set.

The minimum expression of a protocol binding may be written as the following object. `serviceRef` and `schemaRef` are internal resource references or external schema references; they must not be interpreted as new identities:

```
{
  "id": "mcp-http",
  "protocol": "MCP",
  "version": "2025-06-18",
  "transport": "streamable-http",
  "serviceRef": "#mcp-service",
  "schemaRef": "https://example.invalid/schema/mcp.json"
}
```

7.6 Service Endpoints and Invocation Requirements

Resource endpoints are expressed as a service endpoint array. The core structure includes an endpoint ID, service type, and `serviceEndpoint`, and may include a version, protocol, server type, and port. An endpoint may be an agent interaction entry point, an MCP service entry point, a tool API address, a resource package address, or a protocol description address. The endpoint type

and protocol should explain its purpose, and version or authentication differences between multiple endpoints should be explicit. An authentication declaration may express the credential type, issuer, scope, presentation method, or invocation prerequisite, but it is only a public statement of invocation requirements. The existence of an endpoint or HTTP reachability does not mean that the caller has been granted access. Sensitive tokens, private keys, and internal network credentials must not be published as public metadata.

7.7 Resource Versions and Compatibility Fields

`packageVersion` identifies the resource package version. A `version` in the resource description or a version in a protocol binding may express a resource capability version or protocol version; they must not be substituted for one another without explanation. `previousVersion`, version constraints, or release notes may establish an update chain and compatibility relationship, but a version string alone does not prove the source of the content. Every version must be bound to the resource DID, DID document hash, metadata hash, resource package hash, and hash algorithm. A new version under the same resource DID should enter the verification process through a new resource package and control proof.

Version comparison should at least keep the resource package version, protocol version, and resource lifecycle separate:

Information	Example	Object being judged
Resource package version	1.2.0	Whether the current resource materials and their hashes are a newer version
Protocol version	MCP 2025-06-18	Whether the caller and endpoint can interact under the same protocol
Version constraint	>=1.0.0	Which target versions are permitted by the composition relationship
Lifecycle state	active	Whether the current version remains available for continued propagation

The same version string does not mean that the content is identical. A recipient should still check the resource DID, three classes of hashes, root platform proof, and governance state. If content changes while an old hash is retained, the recipient should treat it as an integrity failure.

7.8 Package Hash and Metadata Hash

The current implementation separately computes the DID document hash, resource metadata hash, and resource package hash. A hash value is expressed with an algorithm prefix, such as `sha256:<digest>`. `hashAlgorithm` must be consistent with the algorithm prefix in the hash references. Before computing the metadata hash, the metadata's own `metadataHash` and `packageHash` are cleared

to avoid circular dependencies. The resource package hash is computed only from fixed declarations such as the package version, resource DID, resource type, DID document hash, metadata hash, and hash algorithm. A verifier should recompute all three hashes using the same cryptographic suite and serialization method. A matching hash alone does not prove control, node authorization, or endpoint availability.

The input to the resource package hash is not a recursive serialization of the entire resource package. It is the set of package declaration fields explicitly selected by the current implementation:

```
{
  "packageVersion": "1.0.0",
  "resourceDid": "did:oon:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "resourceType": "skill",
  "didDocumentHash": "sha256:example-did-document",
  "metadataHash": "sha256:example-metadata",
  "hashAlgorithm": "sha256"
}
```

This input illustrates the field boundary of `verify_package_hash`. The actual digest must be computed by the implementation using the unified cryptographic suite; the example digest must not be written into a production package. The metadata hash is computed over `ResourceMetadata` after its two own hash fields have been cleared. The two input sets must not be mixed.

7.9 Canonical Serialization and Deterministic Hashing

Deterministic hashing requires implementations to use the same field names, enum serialization, array order, time format, null handling, and character encoding. The current Rust implementation serializes the object with `serde` and then passes it to a unified hash function. Cross-language implementations must not change field names, add arbitrary default values, or sort arrays before hashing unless the specification version explicitly requires it. The metadata hash and package hash must not include their own output fields in the input again. If canonicalization fails, an object cannot be serialized, or the algorithm is unsupported, construction should stop and return an error.

When reproducing the process across languages, the object before hashing should be recorded so that “different fields” and “different serialization” can be distinguished:

```
ResourceMetadata for hashing
-> clear metadataHash and packageHash
-> serialize with protocol field names and stable enum values
-> hash with the selected crypto suite
-> write metadataHash back to the package

Package hash input
-> packageVersion, resourceDid, resourceType
-> didDocumentHash, metadataHash, hashAlgorithm
-> serialize and hash
```

If only the description, tag order, authorized domains, or time field in the metadata changes, the canonicalized metadata input changes and the recipient should obtain a different metadata hash.

If only the computed result is written back without changing any other input, the existing hash must not change.

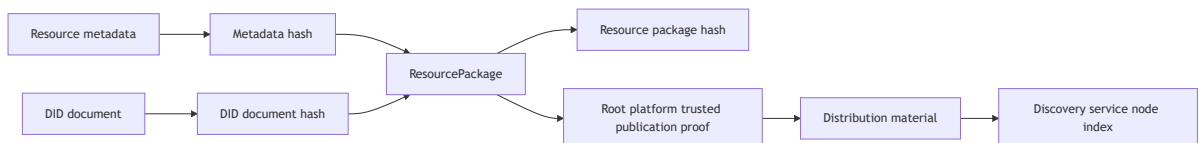
7.10 ResourcePackage Construction Rules

The construction order for `ResourcePackage` is: determine the resource DID, resource type, DID document, and metadata; compute the DID document hash and metadata hash; compute the resource package hash; organize the version, hashes, lifecycle state, and authorized-domain declarations; have the resource controller submit the control proof; and let the registration service node and root platform perform their respective registration and publication duties. A root platform trusted publication proof can be produced only after the root platform has completed verification. A client must not construct one in advance as if publication had already occurred.

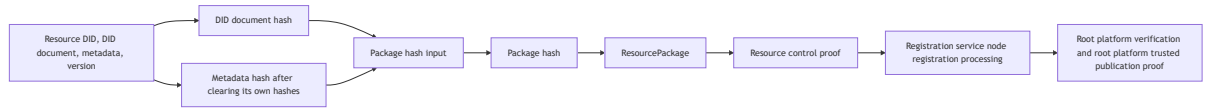
The responsibility boundaries among resource metadata, the DID document, the resource package, the root platform trusted publication proof, and the discovery index record are as follows:

Object	Main content	Producer	Core checks by the recipient
Resource metadata	Type, description, tags, use cases, endpoints, version, and authorized domains	Resource provider/controller	Fields, types, semantics, and metadata hash
DID document	DID, verification methods, public keys, and services	Resource controller	DID resolution, control relationship, and document hash
Resource package	DID document, metadata, version, hashes, and state	Constructor	Structure, three classes of hashes, and field consistency
Root platform trusted publication proof	The root platform's verification and publication statement for a specific version	Root platform	Issuer, scope, signature, and state
Discovery index record	Local verification, authorized domain, freshness, and synchronization position	Discovery service node	Source reference, governance state, and index freshness
Content distribution material	Verified resource package and its propagation references	Content distribution platform	Source, hash, version, and publication authorization

The relationship between construction and publication can be represented as follows:



The construction process can be represented as a reviewable dependency chain:



The registration processing and root platform verification shown in the diagrams belong to the resource network stage and must not be written into an ordinary resource package constructed by the client in advance. Each step should retain the input version and output hash. If any input changes, all subsequent dependent steps must be performed again.

7.10.1 Input Materials

Input materials include the resource DID, resource type, DID document, resource metadata, resource package version, hash algorithm, resource control proof, and any necessary registration credentials or source references. The DID document `id`, metadata `resourceDid`, package `resourceDid`, and the DID targeted by the control challenge must be consistent. The resource type and subject type must be consistent with the semantic encoding of the DID.

7.10.2 Field Canonicalization

The constructor should use protocol field names and deterministic serialization rules, represent time as a parseable UTC value, and keep array order and enum values stable. Before computing the metadata hash, clear `metadataHash` and `packageHash` in the metadata. When computing the package hash, include only the specified package declaration fields.

These fields are mapped to HTTP JSON through `serde`; a separate interface schema is not automatically generated at compile time. `ResourceRegistrationSubmission.didDocument` uses the DID document type, while `metadata` uses dynamic JSON. `ResourcePackage.metadata` uses the resource metadata type, and `rootProof` uses the root platform proof type. The recipient therefore needs both Rust-type-based deserialization and business validation of resource types, hashes, and proofs. Successful deserialization does not mean that the object can be published.

7.10.3 Hash Computation

Compute the DID document hash, metadata hash, and resource package hash in sequence, and write the algorithm prefix into the corresponding fields. After hashes are populated, none of the other fields participating in the hash may be changed. A change in any recomputed result indicates that the content of the current version has changed.

7.10.4 Signature and Proof Generation

The resource controller signs the control challenge and the prescribed resource-control materials. A registration service node may issue a registration credential for the registration fact. After checking node authorization, the control proof, hashes, and publication conditions, the root platform generates a root platform trusted publication proof. The issuer, verification method, target DID, purpose, and covered hashes of each proof must be verifiable.

7.10.5 Post-construction Validation

After construction, check resource type consistency, metadata consistency, the DID document hash, metadata hash, resource package hash, and root platform statement binding in sequence. A failure in any check must prevent the object from being marked distributable or trusted-published. A failed object may be retained as local diagnostic material, but it should not enter a public index.

The post-construction check result can be recorded as a machine-readable diagnostic summary:

```
{
  "resourceDid": "did: oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "packageVersion": "1.0.0",
  "checks": [
    { "name": "resource_type_consistency", "status": "passed" },
    { "name": "metadata_consistency", "status": "passed" },
    { "name": "did_document_hash", "status": "passed" },
    { "name": "metadata_hash", "status": "passed" },
    { "name": "package_hash", "status": "passed" },
    { "name": "root_claim_binding", "status": "passed" }
  ],
  "publishable": true
}
```

`publishable` means only that the local object passed the current package checks. It does not replace acceptance by a registration service node or publication by the root platform. If any check fails, the diagnostic result should include the corresponding error category, and the package must not enter trusted distribution or a discovery index.

7.11 Structure of a Root-Platform-Verified ResourcePackage

The resource package after root platform verification is still based on the DID document, resource metadata, and hashes provided by the resource controller, but it adds or binds the root platform's verification result for that specific resource version. The root platform DID, `packageClaims`, signature or data-integrity proof in `rootProof`, together with any necessary announcement-event reference, expresses which facts the root platform has checked and accepted. The publication sequence, publication time, and synchronization cursor belong to the publication or propagation layer and must not be written back as original claims by the resource provider.

7.11.1 Resource Identity Section

The resource identity section includes `resourceDid`, `resourceType`, the DID document in the package, and `didDocumentHash`. The recipient should check that the package fields are consistent with the DID document `id`, resolve `did: oan`, and verify the resource-type mapping. A valid identity section only means that the package is bound to a resolvable resource DID; it does not mean that the resource has been published by the root platform.

7.11.2 Resource Metadata Section

The resource metadata section includes the name, description, capability tags, use cases, authorized domains, protocol bindings, service endpoints, lifecycle, and version information. The recipient should check that the metadata hash, resource type, resource DID, package version, authorized domains, and lifecycle are consistent with the declarations outside the package, and should retain the metadata source and update time.

7.11.3 Root Platform Verification Proof

The root platform verification proof must at least associate the root platform DID, resource DID, resource type, package version, DID document hash, metadata hash, package hash, hash algorithm, lifecycle state, and authorized domains. If `packageClaims` is inconsistent with the package contents, the proof signature cannot be verified, or the proof source is outside the trusted scope, the package must not be treated as a root-platform-verified resource package.

7.11.4 Fields Required for Distribution and Indexing

Distribution and indexing require at least the resource DID, resource type, package version, three classes of hashes, root platform proof, lifecycle state, authorized domains, source reference, and publication or synchronization cursor. A discovery service node should also record the local receipt time, verification time, current synchronization position, and rejection reason, so that an old snapshot, indexing lag, or cross-node inconsistency can be explained.

7.12 ResourcePackage Integrity and Provenance Verification

The recipient should first verify the structure and resource identity, then verify the DID document hash, metadata hash, resource package hash, field consistency, and root platform proof in that order. Finally, it should combine lifecycle, authorized domain, version, and governance state to decide whether the object may be written to a distribution cache or discovery index. A hash or proof failure must prevent the object from entering the trusted read model. Temporary network unavailability may place it in a pending-retry state, but an object with incomplete verification must not be marked as published.

The recipient may classify verification results into three categories: “passed”, “rejected”, and “pending retry”:

Result	Typical reason	Enters the trusted read model?	Follow-up action
Passed	Structure, hashes, source, and state are all verifiable	Yes	Write to distribution or index according to version and authorized domain
Rejected	Hash mismatch, invalid proof, type conflict, or untrusted source	No	Save the error and evidence; request correction or resubmission

Result	Typical reason	Enters the trusted read model?	Follow-up action
Pending retry	An upstream service is temporarily unreachable or the governance projection has not synchronized	Must not be marked as published	Retain a pending task and retry after recovery

“Pending retry” is not a verified state. Even when the JSON structure of a package is correct, if the root platform proof or a necessary source cannot be confirmed, the recipient may retain only an incomplete state.

7.13 ResourcePackage Provenance Chain and Attribution

The resource provenance chain contains at least five classes of facts: the resource controller produces the DID document, metadata, and control proof; the registration service node produces the registration acceptance and registration credential; the root platform produces the resource-version acceptance, root platform trusted publication proof, and publication reference; the content distribution platform forwards or caches materials whose propagation is permitted by the root platform; and the discovery service node produces the local verification and indexing record. Each fact should identify the source subject, target resource DID, version, time or cursor, and preserve its signature or hash association.

The provenance chain can be read in five stages: “claim, registration, publication, transport, and indexing”:

```
Resource controller
  -> DID document, resource metadata, control proof
Registration service node
  -> registration record, registration credential
Root platform
  -> acceptance record, root platform trusted publication proof, publication reference
Content distribution platform
  -> transport or cache record for verified materials
Discovery service node
  -> local validation, synchronization position, and discovery index record
```

Each later stage may add facts about its own processing, but it cannot issue a proof that belongs to an earlier stage. For example, an index record from a discovery service node may show that the node verified and stored materials for a version; it cannot replace the resource controller's control proof or the root platform's publication proof.

7.13.1 Resource Controller Provenance

Resource controller provenance is determined by the DID document's control relationship, the resource control proof, and its signature verification method. `publisherDid` or `subjectDid` may

supplement a business relationship, but cannot replace the control proof. The recipient should check the resource party's claim against the current DID document and proof together.

7.13.2 Registration Service Node Provenance

Registration service node provenance is expressed by the receipt record for the registration request, the node DID, node authorization materials, the registration credential, and the processing state. The scope of a registration credential is the registration fact; it does not automatically prove that the root platform has accepted the resource or that a discovery service node has indexed it.

7.13.3 Root Platform Publication Provenance

Root platform publication provenance is expressed by the root platform DID, root platform trusted publication proof, resource package declarations, and publication or announcement references. The root platform proof must be bound to a specific resource version and its hashes. A proof for another version under the same resource DID cannot be reused directly.

7.13.4 Provenance Chain Verification

Provenance-chain verification requires checking the control proof, registration credential, root platform trusted publication proof, and distribution record layer by layer. A content distribution platform proves only transport or caching behavior, while a discovery service node proves only local retrieval, validation, and indexing behavior. A missing layer must be reflected in the response; the existence of a later layer must not conceal the failure of an earlier one.

7.14 Package Size, Content Limits, and Validation Strategy

Size, length, nesting depth, number of endpoints, number of tags, and number of protocol bindings should be specified by deployment or conformity configuration and enforced by the registration service node and `ResourcePackage` recipient using the same limits or limits no lower than the network requirements. When the current schema does not define a unified numerical business limit, this chapter must not invent a fixed number. The implementation must return an identifiable error when a limit is exceeded, preventing oversized text, deeply nested structures, or large numbers of endpoints from exhausting parsing, indexing, and hashing resources. Limit checks must occur before hash computation, so that truncation, omission, or automatic compression cannot produce a different hash.

Limit checks should be applied to the original input, and the failure reason should be associated with a field path rather than silently truncating the content:

```
{
  "validation": "rejected",
  "errors": [
    {
      "path": "metadata.capabilityTags",
      "code": "limit_exceeded",
      "message": "capability tag count exceeds the active profile"
    }
  ]
}
```

```

    }
  ],
  "hashingStarted": false
}

```

The limit values in the example are determined by the actual deployment configuration. Once input has been truncated, a field has been discarded, or an array has been automatically reordered, the object being hashed has changed and the original signature or hash cannot continue to be used.

7.15 Schema Evolution and Backward Compatibility

Schema evolution should distinguish the addition of an optional field, a change to existing semantics, field removal, and a change to the hash input or signature coverage. If a new field does not affect parsing by older implementations or existing hash rules, it may be introduced through a versioned extension. If it changes resource identity, resource type, version, hash input, root platform statement, or the verification process, the schema or protocol version must be advanced and the behavior of older nodes must be stated. An older node may ignore an unknown optional field, but it must not silently discard a field covered by a signed object when recomputing it. An unknown required field, an unrecognized hash algorithm, or an unverifiable proof should cause rejection or entry into a compatibility-pending state.

This section specifies only the effects of schema changes on parsing and integrity validation. The responsibility boundaries of resource metadata, the DID document, the resource package, the root platform trusted publication proof, and the discovery index record are described in Sections 6.1, 6.4, 6.10 through 6.13, and Chapters 9 and 10 respectively. A schema-version change does not automatically change resource control, publication state, or discovery visibility. The recipient must still recompute the relevant hashes according to the established validation order and retain the reason for any failure.

7.16 References

Source	Type	Link
oan-protocol-common	Code repository: resource metadata, resource package, and schema	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	Code repository: resource package construction, hashing, and shape validation	https://github.com/OpenAgenet/oan-sdk-ts
OAN Yellow Paper	arXiv yellow paper: resource packages, hashing, and trusted publication	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF draft: resource identity and discovery materials	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

Chapter 8 Identity, Keys, Signatures, VCs, and VPs

This chapter provides a unified description of the semantic boundaries of OpenAget (OAN) identity materials and cryptographic proofs. Identity materials answer who an object is and who can represent the object in making a statement; a signature answers whether specified content was produced by the corresponding private key; a verifiable credential (VC) answers which facts an issuer has made a verifiable statement about; and a verifiable presentation (VP) answers which credentials a holder presents for a specific verification purpose. These four concepts are related, but they cannot substitute for one another.

Resource registration commonly involves a subject DID, a resource DID, a resource control key, a Registrar node key, a node authorization VC, a resource control proof, a registration VC, and a Root proof. A client can generate and hold private keys, a node can verify public keys and proofs, and the Root platform can verify node authorization and publication conditions; these verification capabilities do not mean that a node or the Root platform obtains the resource control private key, nor do they mean that the resource has obtained permission for business invocation.

8.1 Subject Identity and Resource Identity

A subject is an identity object capable of managing one or more resource control relationships. A resource identity is an object boundary that can be registered, published, discovered, updated, and revoked independently. The subject DID, resource DID, Registrar node DID, Discovery node DID, and Root platform DID may all use the `did: oan` method, but their object categories, key purposes, and trust sources differ. They must not be mixed merely because they use the same method.

Object	Primary question proved	Typical materials	Facts not directly proved
Subject	Who manages the resource control relationship	Subject DID Document, subject control key	That the resource endpoint is necessarily available
Resource	Which resource and version have been registered	Resource DID Document, ResourcePackage, version, and hash	That the caller has been permitted to invoke it
Registrar node	Whether the node is qualified to accept registration	Node DID Document, node authorization VC	Resource business quality

Object	Primary question proved	Typical materials	Facts not directly proved
Discovery node	Whether the resource has been synchronized and indexed, and whether it is returned according to the current node state	Node DID Document, index record, discovery response, and freshness information	Resource control, Root platform publication, or business quality
Root platform	Whether a network-level publication fact has been formed	Root proof, governance state	The legal relationship of the resource provider

At registration time, the minimum public materials required for verification should be disclosed, such as the resource DID Document, public key, resource metadata, hash, and control proof. The subject's name, contact information, local identity backup, and private key are not public fields required for resource discovery. Registration submission must not by default place them in node logs, indexes, or public responses.

Identity materials in the same registration request can be divided into three layers according to their purposes:

Layer	Typical materials	Description
Identity layer	Subject DID, resource DID, DID Document, and public key	Identifies the object and indicates where a verifier can obtain the public key
Control layer	Control challenge, control proof, and document hash	Shows whether the current controller authorized this specific operation
Network-fact layer	Registration VC, Root proof, and governance state	Shows which node processed or published which fact

A verifier should start from the identity layer and check the control layer and network-fact layer in sequence. Taking the resource DID text, a registration VC, or a signature from a particular node alone cannot reconstruct the complete control and publication relationship.

8.2 Key Generation, Custody, Backup, and Recovery

Keys should be generated, imported, and used in a client or node secure execution environment managed by the controller. The current `oan-crypto` provides Ed25519 and SM2/SM3-related key processing and supports multiple public-key encodings and JWK public-key representations. The cryptographic suite, public-key representation, and proof type must be identifiable by the recipient. When materials are unknown, truncated, or incompatible, the operation should fail explicitly rather than silently switching to another algorithm suite.

Key material	Holder	Typical purpose	External materials
Subject control private key	Subject or authorized controller	Managing the subject and related resources	Subject DID, public key, and control proof

Key material	Holder	Typical purpose	External materials
Resource control private key	Resource controller	Creating or updating the resource DID Document and versions	Resource DID, public key, and operation proof
Registrar node private key	Registrar node	Node authentication, signing upstream requests, and issuing registration VCs	Node DID, public key, and authorization VC
Root platform key	Controlled Root platform execution boundary	Node authorization, Root proof, and service signatures	Root platform public key and verifiable proofs

A local identity backup may contain subject configurations, resource configurations, public keys, private keys, and credentials at the same time, and is therefore sensitive material. During import or recovery, the client should check the file structure, the correspondence between DIDs, public keys, and private keys, the resource type, and the control relationship. Successful recovery only means that the client has regained local control materials; it does not mean that the resource has been registered, published, or re-indexed.

The boundary between a local identity backup and a public DID Document can be represented by the following minimal structure:

```
{
  "publicIdentity": {
    "did": "did:oa:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
    "verificationMethod": "did:oa:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz#key-1",
    "publicKey": "public-key-material"
  },
  "localSecret": {
    "privateKey": "private-key-material",
    "storage": "browser-local-backup"
  },
  "controlledResources": ["did:oa:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu"]
}
```

This structure is for illustrating the boundary only; the actual backup fields are determined by the client implementation. Public identity materials can be sent to a verifier, while `localSecret` must remain within the local secure boundary managed by the controller. Obtaining a public key from a server does not allow a private key to be derived.

8.3 Resource Control Signatures

A resource control signature must be bound to a specific operation rather than being an isolated signature over arbitrary text. `DidControlChallenge` combines a challenge identifier, draft identifier, target resource DID, DID Document hash, Registrar node DID, operation purpose, verification method, nonce, and validity period. `SubjectControlProofBundle` then associates the control proof, verification time, actual verification method, and proof hash. Both the Registrar node and the Root platform should check these bindings.

At minimum, the following conditions should be checked:

1. `subjectDid` is consistent with the submitted resource DID.
2. `didDocumentHash` is consistent with the submitted DID Document.
3. `verificationMethod` belongs to the target DID Document and its purpose permits the current operation.
4. `purpose` is consistent with the current registration, update, or publication operation.
5. `nonce`, issuance time, and expiration time satisfy freshness and replay-prevention requirements.

Successful proof verification only shows that the corresponding private key signed the specified payload. Whether the resource provider is authorized to represent an organization, whether the endpoint is secure, and whether the caller has obtained business authorization must still be determined separately through delegation materials, governance state, and business policies.

The core of a control proof is not the signature value itself, but the context covered by the signature. A verifier can check a registration operation according to the following relationship:

```
{
  "operation": "register_resource",
  "subjectDid": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "registrarDid": "did:oan:INRG:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "didDocumentHash": "sha256:example-did-document",
  "challengeId": "challenge-example",
  "nonce": "nonce-example",
  "proofPurpose": "assertionMethod",
  "signature": "signature-example"
}
```

The DID, target Registrar node, document hash, purpose, and nonce are all binding relationships to be checked. The example values do not represent a real request. If any binding field differs from the current request, registration or update must not be accepted merely because the signature algorithm verification succeeds.

8.4 Infrastructure Node Identity Keys and Node Signing Keys

The node DID Document expresses the node identity. A communication signing key proves that a request comes from the corresponding node execution boundary. A node authorization VC proves the node's role and authorization domain. Runtime credentials support service startup, key loading, or access to controlled infrastructure. A deployment may host multiple categories of materials within the same security boundary, but protocol semantics should still separate them by purpose.

When sending a resource verification request to the Root platform, `SignedRequestEnvelope` binds `requestId`, protocol version, purpose, HTTP method, path, audience, request time, request nonce, request-body hash, and proof. The recipient should check the audience, path, time window, nonce

replay, node DID Document, public key, and node authorization before processing the business content.

The verification sequence for `SignedRequestEnvelope` can be summarized as follows:

1. Read the node DID, audience, HTTP method, path, and request-body hash from the request.
2. Parse the node DID Document and locate the verification method and public key used by the request.
3. Verify the purpose, creation time, request ID, nonce, and signature payload of the proof.
4. Check the role, authorization domain, and current status of the node authorization VC.
5. Process resource verification, registration, or publication business only after the checks pass.

This separates the question of whether “the request comes from a particular node” from the question of whether “the node is authorized to perform the current action.”

8.5 Root Platform Authorization VC for Infrastructure Nodes

The core relationships in a node authorization VC are as follows: `issuer` is the Root platform or a governance-approved issuer, `subject` is the authorized node DID, `role` indicates the node role, `claims` carries the authorization domain and capability scope, and `status`, `issuedAt`, and `expiresAt` describe status and the validity window. The proof of `NodeAuthorizationCredential` must be issued by the issuer over the credential content without a proof. The verifier should obtain the public key from the issuer DID Document and verify the proof.

```
Node DID Document -> obtain node verification public key
    -> verify authorization VC proof
    -> check issuer, subject, role, status, issuedAt, expiresAt
    -> check authorization domain and capability scope
    -> decide whether the current node action is authorized
```

When the proof is absent, the issuer cannot be resolved, the subject does not match, the role is not permitted, the authorization has expired, the status is not active, or the authorization domain does not cover the requested scope, the restricted action must be rejected. A node authorization VC proves infrastructure eligibility; it does not prove resource control.

The relationships among node authorization VC fields can be checked using the following example:

```
{
  "type": "NodeAuthorizationCredential",
  "issuer": "did:oa:INRT:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "subject": "did:oa:INRG:8LcR3Vn5YpQw2Tx7Mb9Zd4Fa6GhKsEuJ",
  "role": "registrar",
  "status": "active",
  "issuedAt": "2026-09-09T08:30:00Z",
  "expiresAt": null,
```

```

"claims": {
  "authorizedDomains": ["public"],
  "capabilities": ["resource_registration"]
},
"proof": {
  "type": "DataIntegrityProof",
  "proofPurpose": "assertionMethod",
  "verificationMethod": "did: oan: INRT: 7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz#key-1",
  "proofValue": "signature-example"
}
}

```

The role, authorization domain, and capability values in this example are used to illustrate the validation relationships. The verifier must confirm that the issuer, subject, proof, and governance state match one another, and then compare the authorization scope with the current request. The validity of a node authorization VC does not change the controller of a resource DID.

8.6 Registration VC Issued to the Resource Controller

In the current Registrar node flow, the Registrar node first calls the Root platform resource verification and publication interface. After the upstream operation succeeds, the Registrar node issues a registration VC and writes the registration record to storage. The credential contains the OAN credential context, credential type, issuer, resource credential subject, resource DID, resource type, DID Document hash, metadata hash, resource package hash, ResourcePackage version, hash algorithm, authorization domain, lifecycle state, and credential status.

The following is an illustrative structure; field order has no additional semantic meaning:

```

{
  "type": ["VerifiableCredential", "OANResourceRegistrationCredential"],
  "issuer": "did: oan: ...",
  "issuanceDate": "2026-09-06T00:00:00Z",
  "credentialSubject": {
    "id": "did: oan: SKFI: ...",
    "resourceDid": "did: oan: SKFI: ...",
    "resourceType": "skill",
    "didDocumentHash": "sha256: ...",
    "metadataHash": "sha256: ...",
    "packageHash": "sha256: ...",
    "packageVersion": "1.0.0",
    "hashAlgorithm": "sha256",
    "lifecycleState": "active"
  },
  "credentialStatus": {"status": "active"},
  "proof": {"type": "DataIntegrityProof", "proofValue": "..."}
}

```

A registration VC proves that the Registrar node processed the specified registration materials and the registration facts explicitly listed in the credential. It does not independently prove that

the Root platform has completed network-wide publication, that a Discovery node has completed indexing, or that the caller has obtained permission for business invocation.

The most important binding relationships in a registration VC are **issuer**, **subject**, the resource version, and the three classes of hashes. A verifier should compare the credential claims again with the received DID Document, resource metadata, and ResourcePackage:

```
{
  "type": "AgentRegistrationCredential",
  "issuer": "did:oon:INRG:8LcR3Vn5YpQw2Tx7Mb9Zd4Fa6GhKsEuJ",
  "subject": "did:oon:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "status": "active",
  "issuedAt": "2026-09-09T08:31:00Z",
  "claims": {
    "resourceDid": "did:oon:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "resourceType": "skill",
    "packageVersion": "1.0.0",
    "didDocumentHash": "sha256:example-did-document",
    "metadataHash": "sha256:example-metadata",
    "packageHash": "sha256:example-package",
    "hashAlgorithm": "sha256",
    "lifecycleState": "active"
  },
  "proof": {
    "type": "DataIntegrityProof",
    "proofPurpose": "assertionMethod",
    "proofValue": "signature-example"
  }
}
```

The example values do not represent a real credential. If **subject**, **resourceDid**, the version, or any hash differs from the current materials, the registration VC cannot be used to prove that the current materials have been registered. Even when the credential signature itself is valid, it only shows that the issuer signed another set of claims.

8.7 Governance and Authorization Evidence

Evidence	Producer	Verifier	Fact proved
Resource control signature	Resource controller	Registrar node, Root platform	The controller authorized a specific DID, Document, version, and operation
Node authorization VC	Root platform or governance issuer	Node, Root platform, and trusted components	Node role, authorization domain, and valid status
Registration VC	Registrar node	Controller, auditor, and client	Registration processing and the material relationships listed in the credential

Evidence	Producer	Verifier	Fact proved
Root proof	Root platform	Content distribution platform, Discovery node, and caller	The specified version passed Root platform verification and entered the publication flow
Governance event	Governance system or on-chain record	Off-chain trust indexer, node, and auditor	A time-sequenced fact about authorization or governance state

A verifier must combine evidence according to the type of fact being established. A valid registration VC only means that registration processing was completed. Without a Root proof, the registration result must not be described as globally discoverable. When the resource control proof does not match, a valid node authorization VC is not sufficient to accept a resource update.

8.8 VC and VP Structural Requirements

A VC is a verifiable expression of facts asserted by an issuer. A VP is a presentation organized by a holder for a verification purpose. The current core code directly models node authorization VCs and resource registration VCs. When a VP is used, it should add a holder, presentation purpose, challenge, validity period, and presentation proof. An arbitrary collection of VC JSON objects must not be called a VP directly.

8.8.1 Basic VC Structure

A VC should be able to identify its context, type, issuer, credentialSubject, issuance time, optional expiration time, status, and proof at minimum. OAN's Rust credential structures use `type`, `issuer`, `subject`, `status`, `issuedAt`, `expiresAt`, `claims`, and `proof`. An HTTP JSON structure may use interoperability names such as `credentialSubject` and `issuanceDate`, but the serialization mapping must be stable and must not change the meaning of the claims.

8.8.2 Basic VP Structure

A VP should contain a presentation type, holder, one or more `verifiableCredential` objects, a presentation purpose, and a proof for the current verification. When replay prevention is required, it should also bind a challenge or nonce provided by the verifier. A collection of credentials without holder proof, purpose binding, or the current verification context should not be accepted as a fresh VP.

The minimum VP example can be represented as follows:

```
{
  "type": ["VerifiablePresentation"],
  "holder": "did: oan: DVFI: 7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "verifiableCredential": [
    {
      "type": ["VerifiableCredential", "AgentRegistrationCredential"],
      "issuer": "did: oan: INRG: 8LcR3Vn5YpQw2Tx7Mb9Zd4Fa6GhKsEuJ",

```

```

    "subject": "did:oon:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "status": "active",
    "claims": { "packageVersion": "1.0.0" },
    "proof": { "proofValue": "credential-signature-example" }
  }
],
"purpose": "resource_verification",
"challenge": "verifier-challenge-example",
"proof": {
  "proofPurpose": "authentication",
  "proofValue": "presentation-signature-example"
}
}

```

This example distinguishes the issuer proof of a VC from the holder presentation proof of a VP. The verifier must still validate `issuer`, `subject`, the VC proof, `holder`, the VP proof, `challenge`, and status according to the actual credential structure. The credential object in the example must not be used directly as an available credential.

8.8.3 Issuer, Holder, and Subject

The issuer is responsible for the facts asserted by the VC, `credentialSubject` is the object described by the assertion, the holder is the party currently controlling the presentation materials, and the controller is the party controlling verification methods in a DID Document. These four roles may be the same or different. A Registrar node may be the issuer of a registration VC, the resource DID may be its `credentialSubject`, and the resource controller may be the holder; the node operator therefore does not automatically become the resource controller.

8.8.4 Credential Status and Verification

Verification must not stop at proof verification. It should also check the credential type, issuer and DID Document, public key, subject, purpose, validity period, status, and required claims. When the status source is unavailable, the verifier should apply an explicit failure or degradation policy according to the credential purpose. “Temporarily unable to query” must not be interpreted as active.

For different materials, the verification result should preserve two levels: “signature valid” and “business fact valid”.

Check result	Fact that can be confirmed	Content that still requires checking
Proof verification succeeds	The specified key signed the specified payload	Issuer identity, purpose, validity period, and status
VC status is active	The credential is not currently marked invalid by that status source	Whether the claims match the current resource materials
Node authorization is valid	The node has infrastructure eligibility within the declared scope	The role, authorization domain, and resource control proof for the current request

Check result	Fact that can be confirmed	Content that still requires checking
Resource control proof is valid	The controller authorized a specific resource and operation	Registration acceptance, Root platform publication, and business invocation permission

8.9 Relationship among Issuer, Holder, Controller, and Credential Subject

These fields express different relationships. The complete ownership and authorization chain cannot be inferred merely from the presence of a field. Verification should check the field relationships, DID Document, public key, proof purpose, governance policy, and business policy together.

8.9.1 Issuer

The issuer signs the credential content without a proof using the issuance key and bears traceable responsibility for the claims. The verifier should obtain the verification method from the issuer DID Document and check the consistency of the proof creator, verification method, and issuer.

8.9.2 Holder

The holder is the party currently controlling the credential presentation materials. It may be a resource controller, client, or delegated user assistant agent. Holding a public VC does not mean that the holder has obtained control of the credentialSubject; the relationship of representation still requires control proof, delegation, or business authorization.

8.9.3 Resource Controller

The resource controller controls registration, update, or other protected operations for the resource identified by the resource DID. The controller may differ from the resource provider, registration VC holder, or endpoint operator. Its permission decision depends on the target DID, verification method, operation-bound proof, and current version rules.

8.9.4 Credential Subject

`credentialSubject` is the object described by the VC. The subject of a node authorization VC is normally an infrastructure node DID, while the subject of a registration VC is normally a resource DID. If the subject is replaced, the DID Document does not match, or the claims conflict with the subject type, the credential must not be used for the corresponding decision.

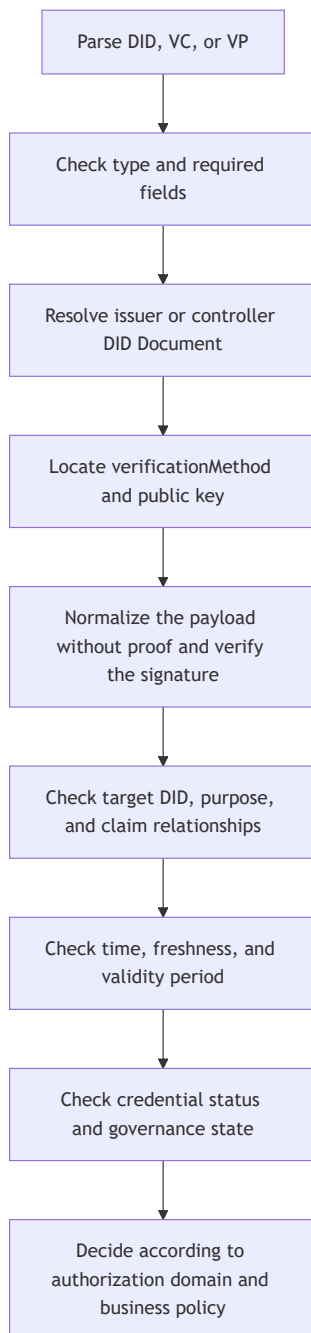
8.10 Signature Verification and Status Verification

After a verification service receives JSON, it should first parse it according to the type or field structure declared by the interface, and then separately verify the signature coverage and the status source. The `upstreamAuth` of an inter-node request proves that the request comes from a node

with the corresponding authorization. `subjectControlProof` in resource registration proves that the resource controller authorized the current registration or update. A registration VC describes the registration fact produced by the Registrar node. A Root proof describes that a specific resource version passed Root platform verification. These materials may appear consecutively in the same HTTP business flow, but they must not be conflated into one credential object.

Request can be parsed

- > request signature and audience are valid
- > resource control proof is valid
- > registration VC issuer and subject match
- > Root publication proof matches the version and hashes
- > governance state and freshness are acceptable



Any failure must terminate the flow before a side effect such as publication, authorization, or

invocation occurs, and should return a locatable failure reason. Verification records may retain the verification object, proof type, verification method, verification time, status source, and error code, but must not retain private keys or materials from which private keys can be recovered.

8.11 Credential Privacy and Disclosure Boundaries

The resource DID, public key, capability tags, resource description, endpoint, version, hash, and Root publication evidence required for resource discovery may enter a public ResourcePackage or discovery response. Subject personal information, private keys, local identity backups, undisclosed delegation materials, and unnecessary internal configuration should remain within a controlled boundary.

Material	Default scope	Processing requirement
Resource DID Document	Publicly resolvable	Must not contain private keys; verify core and extension fields
Registration VC	As required for registration and audit	Check issuer, subject, status, and hash relationships
Node authorization VC	For nodes and verification components	Expose publicly verifiable parts and protect runtime credentials
Root proof	For distribution and discovery verification	Bind the resource version and ResourcePackage hash
VP	Only to the designated verifier	Bind holder, purpose, challenge, and validity period
Local identity backup	Retained only by the controller	Highly sensitive material; prohibit entry into logs and public indexes

Minimal disclosure must not remove binding fields required for verification. If hiding a field prevents the verifier from confirming the issuer, subject, version, hash, or proof, controlled presentation or a verifiable digest should be used instead of unverifiable display text.

8.12 Key Rotation and Compromise Response

Key rotation should first establish a verifiable control relationship between the new public key and the existing DID, and only then make the old public key unavailable for new operations. A resource DID update should describe the new verification method, its purpose, and its effective version in the new DID Document. Node key rotation must also synchronize the node authorization VC, node configuration, and upstream trust materials. Root platform key rotation must take downstream caches, historical proofs, and verification windows into account.

When a key is compromised, the affected key should be suspended, the old public key and status required to verify historical materials should be retained, and the revocation, replacement, or recovery fact should be published through a new control relationship. If the control private key is completely lost and no recovery key or transfer mechanism exists, the system cannot restore control merely from the DID string; it must enter a governance intervention or re-registration process.

8.13 Registration VC and Resource Publication Evidence

Two types of proof form a continuous but independent fact layer. After the Registrar node receives a successful response from the Root platform interface, it generates a registration VC. The Root platform produces a Root proof after verifying the resource control proof, metadata, ResourcePackage, version, hashes, and publication conditions. An HTTP success code or `rootResponse` is only a transport result and cannot replace verification of proof content and status.

```
Resource control proof -> Registrar node accepts the registration
    -> Root platform verifies resource, version, hashes, and publication conditions
    -> Root proof
    -> Content distribution platform distributes the material
    -> Discovery node verifies and indexes it
    -> Caller verifies it again and invokes it according to policy
```

Registration acceptance, Root platform acceptance, material distribution, Discovery node indexing, and caller authorization must be expressed separately. No single VC or publication proof may compress these states into “the resource is already trusted and available.”

8.14 Node Authorization VC and Infrastructure Permissions

Node actions are jointly constrained by role, authorization domain, capability scope, and governance state. A Registrar node may process only resources covered by its authorization domain, and a Discovery node may only synchronize and return resources within its permitted scope. A node's ability to verify a material does not mean that the node is authorized to perform every network action. The authorization domain should be checked for format, ordering, duplicates, hierarchical coverage, and consistency with the node VC.

```
allow(action) = valid(node authorization VC)
    && active(governance status)
    && role permits(action)
    && granted domains cover(requested domains)
    && freshness is acceptable
```

This expression represents a decision relationship rather than fixed implementation code. Errors should distinguish node authorization failure, resource control failure, resource content failure, and upstream service failure so that auditing, client handling, and operations diagnosis remain possible.

8.15 Credential Revocation, Suspension, and Status Propagation

A credential with a valid signature may still be unusable for a new authorization decision because of a governance event, a status record, or `expiresAt`. Status propagation may pass through on-chain governance events, Root platform status, the off-chain trust indexer, and node caches. Therefore, the verification result should carry the status source, observation time, and freshness information.

8.15.1 Credential Status Changes

Every status change should be associated with the credential identifier, subject, old status, new status, reason, event sequence, and time, and should be produced by an authorized party. Restoring a suspended credential and issuing a new credential are different operations. Whether a new credential may be issued after revocation should be determined by governance and issuance policies.

8.15.2 Node Authorization VC Status

After a node authorization VC is suspended or revoked, a Registrar node or Discovery node must not continue to perform new restricted actions. Whether existing materials may continue to be provided can only be determined according to cache freshness and risk policy. A historical VC must not be used to infer that the node still has permission to submit or index new content.

8.15.3 Resource Registration VC Status

A change in registration VC status affects the current validity of the registration proof, but does not necessarily mean that the resource DID must be deleted. Historical Documents and audit materials may be retained while the current version is marked suspended, revoked, or undiscoverable. The verifier should check the VC status, resource lifecycle, and Root publication status separately.

8.15.4 Status Propagation and Cache Updates

When a node receives a newer governance event or status cursor, it should verify sequence continuity, event completeness, and proof before updating its local cache. When the cursor is behind, events are missing, the status source is unavailable, or the cache exceeds the allowed freshness, sensitive actions should be suspended or return an indeterminate status rather than continuing to use the old active judgment.

8.16 Signature Formats, Normalization, and Verification Test Vectors

Cross-language implementations must unify the signed source text, field names, field ordering, null handling, time format, hash algorithm, public-key encoding, and proof fields. `signature_input`, `build_data_integrity_proof`, and `verify_payload_with_proof` in the reference implementation show the boundaries for payload construction, proof generation, and verification. Other language implementations should produce an equivalent byte sequence rather than compare only in-memory objects.

Vector category	Normal example	Failure example	Acceptance criterion
Public-key parsing	Multiple encodings and JWK restore the same public key	Invalid prefix, length, or coordinate	Explicit public-key encoding error

Vector category	Normal example	Failure example	Acceptance criterion
Payload signature	Verification succeeds over the same normalized source	Any field or <code>proofValue</code> is modified	Verification fails
Control proof	DID, hash, purpose, and challenge match	Any binding field differs	Reject before publication
VC status	Active and not expired	Revoked, suspended, or expired	Authorization decision fails
Node request	Audience, path, nonce, and time are valid	Replayed nonce or wrong audience	Request rejected with no publication side effect

Test vectors should be versioned and should cover the Rust reference implementation, TypeScript SDK, and node HTTP layer. Test private keys must use dedicated materials; production private keys must not enter test files or logs. Logs may retain non-sensitive evidence such as public keys, hashes, request IDs, and error codes.

8.17 Reference Sources

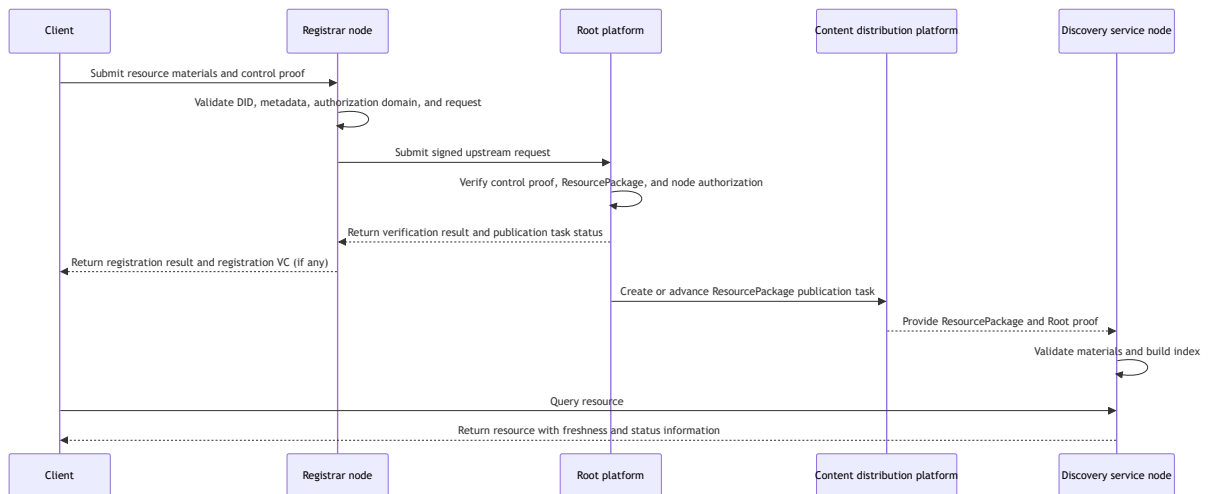
Source	Type	Link
<code>oan-protocol-common</code>	Code repository: cryptography, credentials, signatures, and identity structures	https://github.com/wolfbrother/oan-protocol-common
<code>oan-sdk-ts</code>	Code repository: identity bundles, key, and credential client processing	https://github.com/OpenAgenet/oan-sdk-ts
OAN Yellow Paper	arXiv yellow paper: identity, VCs, proofs, and verification boundaries	https://arxiv.org/abs/2606.03163
<code>did:oan</code> DID Method Specification	Standard/method specification: DID Documents and verification methods	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md

Chapter 9 Registration Protocol

This chapter specifies the process by which resources enter OpenAget (OAN) from a client. After the client creates the resource DID, DID Document, resource metadata, ResourcePackage, and resource control proof, it submits them to a Registrar node. The Registrar node performs entry validation and authorization-domain validation, and then calls the Root platform's resource verification and publication interface through a signed upstream request. The current reference implementation uses `/resources/register` and the compatible `/resources/submit` entry points. The successful path includes a successful Root platform request, registration VC issuance, registration-record persistence, and response delivery.

When the Registrar node returns `status: submitted`, this only means that the registration process has obtained successful results from that node and its upstream call. It does not mean that the resource has completed network-wide publication, content distribution, or discovery indexing. The client should save and display the registration result, Root proof, and the indexing status of Discovery service nodes separately.

The registration path can be summarized as “the client prepares the materials, the Registrar node accepts and validates them, the Root platform verifies and publishes them, and the Registrar node returns the registration result.” A client must not infer discovery visibility solely from a success message on a page. It should determine the three stages separately from the resource registration record, the content returned by the Root platform, and the query result from a Discovery service node. The public interface of a Registrar node is the resource submission entry point; upstream calls between nodes belong to a protected service-to-service protocol. Their request formats and signing subjects must not be mixed.



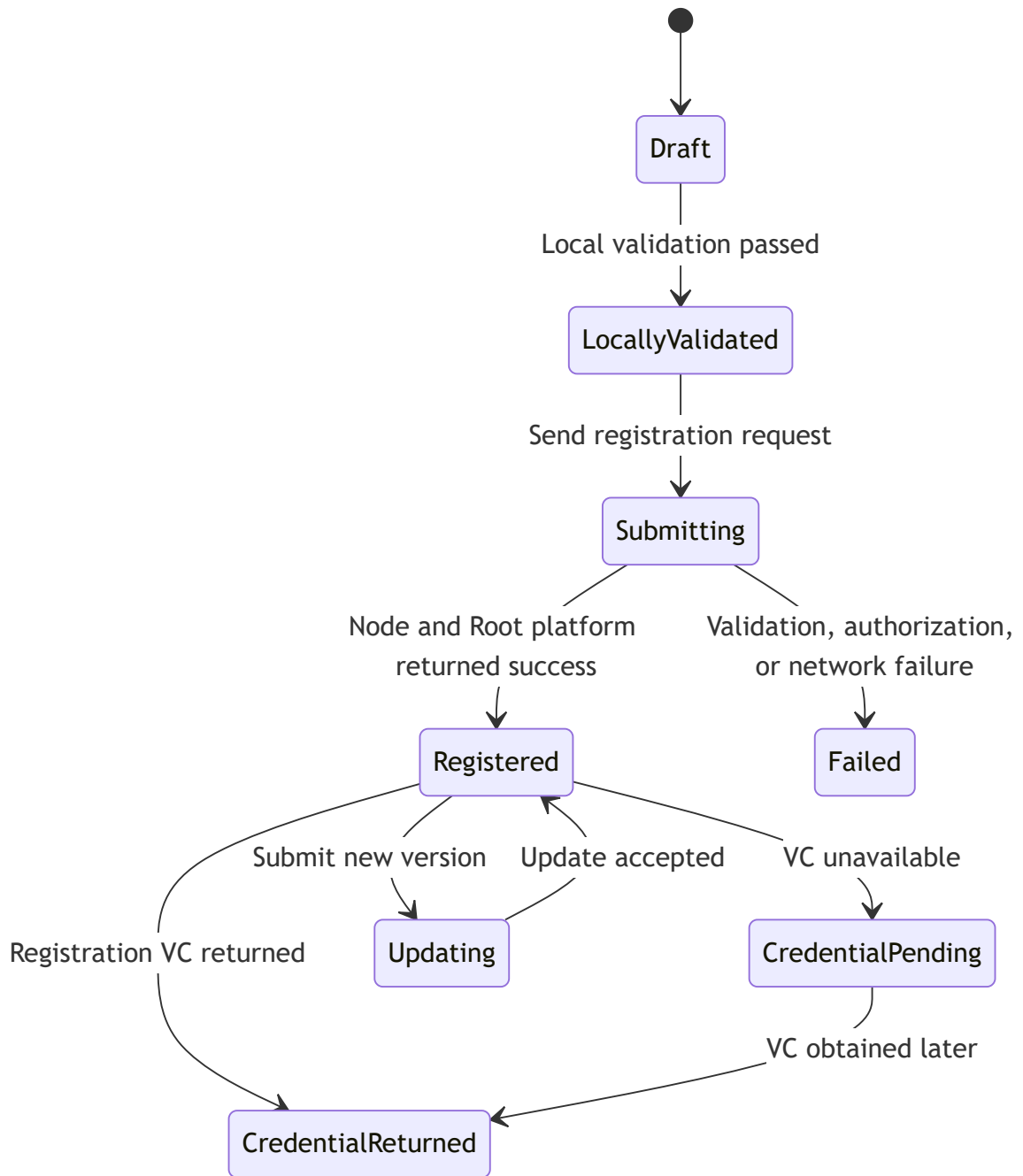
The Root platform verification, content distribution, and discovery indexing shown above are

consecutive stages that may complete asynchronously. `resource-verified-and-queued` or `status: submitted` returned by a Registrar node only represents the corresponding registration or queueing fact. It cannot be used to infer that the content distribution platform has completed hosting, or that a Discovery service node has built a queryable index.

9.1 Registration Targets and States

Registration state should distinguish the client's local state, the Registrar node's processing state, and the Root platform's publication state. The following logical states are recommended:

State	Entry condition	Permitted operations	Termination or transition condition
Draft	Form or SDK resource materials have not been submitted	Edit, preview, regenerate	Materials pass local validation or are modified
Locally validated	Local Identity, fields, hashes, and control proof pass validation	Submit or continue editing	Returns to Draft when any bound field changes
Submitting	The client has sent a registration request	Wait for response or query status	A success, failure, or timeout result is received
Registered	The Registrar node has obtained the required upstream success result	Save VC and query publication status	Subsequent Root publication and discovery proceed independently
Credential pending	A registration result exists but the VC is missing or empty	Query or retry according to the interface rules	VC is obtained and verified, or exception handling begins
Duplicate	DID, version, and hashes are identical to existing materials	Return the existing result	No new business version should be created
Conflict or failed	Validation, control, authorization, version, or network conditions are not satisfied	Correct materials or handle according to retryability	Must not be displayed as network-wide publication success



Passing browser preview or local validation does not create a registration record; a network timeout must not directly generate a new DID or new version. A retry with the same business materials should reuse the original resource DID, version, and hashes, and the server or query interface should be used to confirm the actual result.

9.2 Registration Request Structure

The core payload of the registration entry point is `ResourceRegistrationSubmission`. Its main fields and purposes are as follows:

Field	Source	Validation and purpose
resourceDid	Resource identity generated by the client	Serves as the resource identity and primary key of the registration record
resourceType	Selected by the client	Must be consistent with the DID semantic encoding and DID Document metadata
didDocument	Client draft	Provides public keys, verification methods, service endpoints, and OAN metadata
didDocumentHash	Computed by the client according to the protocol	Binds the control challenge to the publication materials
metadata	Client form or resource description	Validates the resource declaration and authorization domain
packageVersion	Client resource version	Distinguishes different versions under the same DID
packageHash, metadataHash	Computed by the client	Binds the integrity of the ResourcePackage and metadata
hashAlgorithm	Protocol and client configuration	Explains the algorithm prefix used by hash fields
registrationCredential	Protocol compatibility field	Cleared by the Registrar node when submitting to the Root platform
subjectControlProof	Signature from the resource controller	Proves that the current registration or update has control authorization

These fields can be divided into four groups according to validation order:

Field group	Fields	Main determination
Identity and type	resourceDid, resourceType, didDocument	Whether the resource identity, resource type, and DID Document are mutually consistent
Integrity binding	didDocumentHash, metadataHash, packageHash, packageVersion, hashAlgorithm	Whether the submitted materials correspond to the same version and the same content
Business declarations	metadata	Whether capability, endpoint, authorization-domain, and lifecycle descriptions are complete
Authorization proof	subjectControlProof, registrationCredential	Whether the resource controller authorized the operation; the Registrar node does not forward an existing registration VC to the Root platform

An illustrative request fragment is:

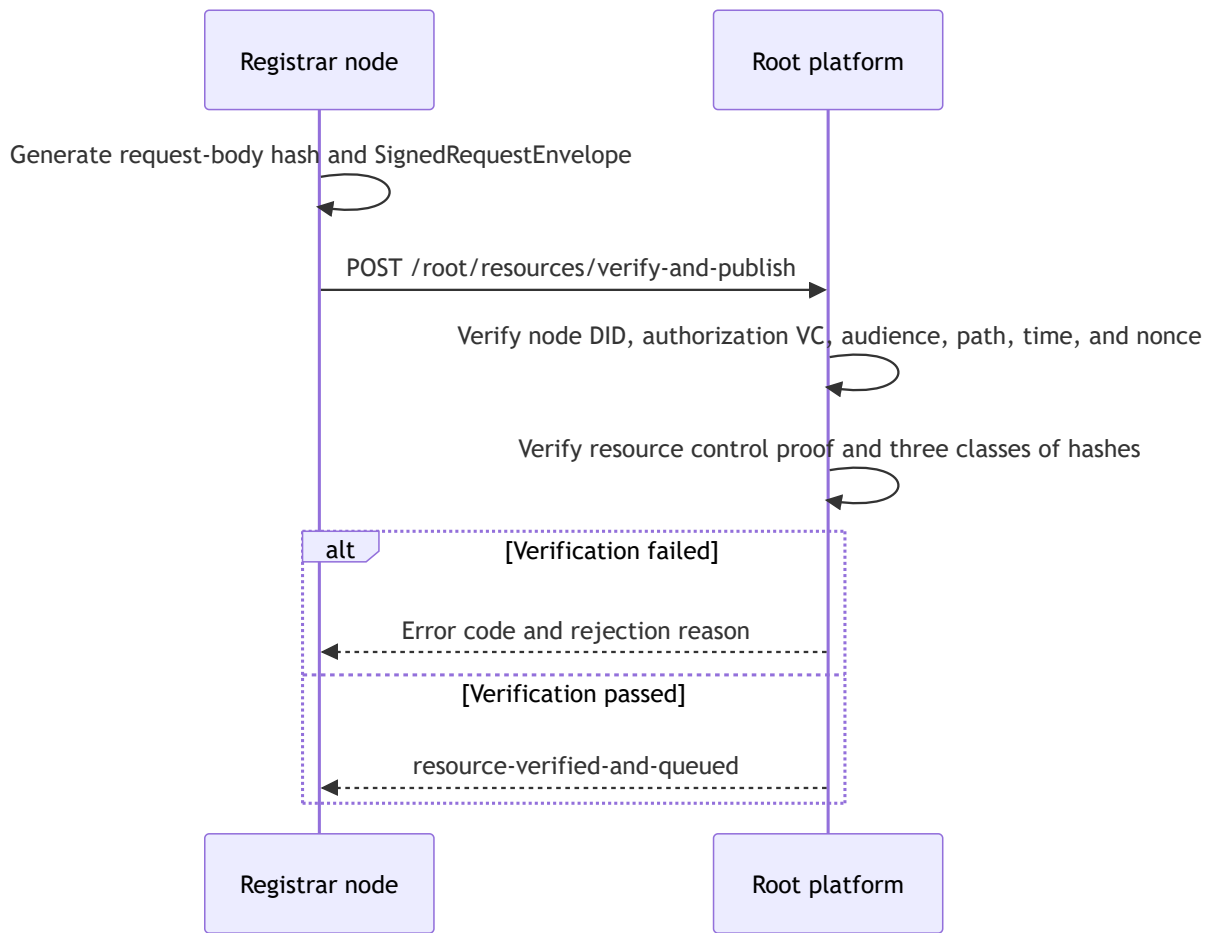
```
{
  "resourceDid": "did: oan:SKFI:...",
  "resourceType": "skill",
  "didDocument": {"id": "did: oan:SKFI:...", "verificationMethod": []},
```

```
"didDocumentHash": "sha256:...",
"metadata": {"resourceType": "skill", "authorizedDomains": ["software"]},
"packageVersion": "1.0.0",
"packageHash": "sha256:...",
"metadataHash": "sha256:...",
"hashAlgorithm": "sha256",
"registrationCredential": null,
"subjectControlProof": {"challenge": {}, "proof": {}}
}
```

Before calling the Root platform, the Registrar node should reject requests whose resource DID, DID Document identifier, challenge target, or hashes are inconsistent. Whether unknown extension fields are retained is determined by the protocol version. Unknown core fields, invalid types, or unparseable structures must not be treated as valid requests.

A valid request must at least satisfy all of the following relationships: `didDocument.id == resourceDid`; the resource type declared by the DID Document must be consistent with `resourceType`; the three hash fields must be interpretable according to `hashAlgorithm`; and the subject and document hash in the control challenge must correspond to the resource DID and `didDocumentHash`, respectively. For example, a request that changes `resourceType` from `skill` to `agent_service` without simultaneously updating the DID Document and metadata should be rejected during local validation by the Registrar node. `registrationCredential` remains in the shared request model for compatibility and record-keeping, but the Registrar node clears it when constructing the upstream request to the Root platform, preventing a credential previously issued by the Registrar node from being used as the Root platform's verification basis.

The key interaction for forwarding a request from the Registrar node to the Root platform can be summarized as follows:



This interaction describes only the protected upstream call between the Registrar node and the Root platform. The client submission entry point, registration VC issuance, and subsequent content distribution are handled separately in the other sections of this chapter. A successful Root platform response does not change the fact that the Discovery service node may not yet have completed indexing.

9.3 Local Identity Initialization and Import

The Local Identity is a collection of materials in which the client stores subject configuration, resource control keys, and necessary credentials. It is not a user account on a Registrar node. It is used to create or restore control relationships and to issue control proofs for specific registration operations. A Registrar node should receive only the DID Document, public keys, proofs, and registration materials required for verification. It must not receive subject or resource control private keys.

The resource control relationship can be represented as follows: “the Local Identity holds the control key, the resource DID Document publishes the verification method, and the control proof binds one specific operation to that resource and document version.” Therefore, a Local Identity and a resource are not the same object. One subject configuration may control multiple resources, but each registration request must identify one `resourceDid` and use a proof matching that resource's DID Document. The Registrar node verifies public materials and signature results, rather than the client's assertion that “this is my resource.”

9.3.1 Create a New Local Identity

The client generates a subject control key and subject DID, and saves the public verification material and local configuration. It can then create an independent resource DID and resource control material for each resource. Creating a Local Identity does not itself create a Registrar node record and does not mean that a resource has obtained a network publication state.

9.3.2 Import a Local Identity

The client parses a Local Identity backup and validates the correspondence among the subject DID, public key, private key, resource configuration, and credentials. After a successful import, the client can continue controlling existing resources, but a new registration, update, or retry must still create a new proof for the target DID, version, hash, and operation purpose.

9.3.3 Private Keys and Subject Configuration

Private keys are used only on the client to issue control proofs or other protected signatures. Subject configuration describes the relationship between the subject DID and resources. Server requests and logs may contain public DIDs, public keys, signatures, hashes, and credentials, but must not contain private keys or plaintext materials from which private keys can be recovered.

9.3.4 Identity Backup and Recovery

An identity backup should contain the structured information required to restore control relationships, and should indicate its sensitivity and version. After recovery, the client should check the key correspondence and DID Document consistency again. The existence of a backup file does not mean that the resource has been registered, and a recovery failure must not overwrite an already valid identity on the client.

9.4 DID Document Draft Generation

The client generates a DID Document from the resource type, name and description, capability tags, authorization domains, endpoints, version, Local Identity, and control key. `id` must equal `resourceDid`; `oanMetadata.resourceType` and `subjectType` must match the request; verification methods must provide resolvable public keys; and service endpoints and `ResourcePackage` information must be bound to the resource metadata.

The preview button may generate or reuse a local DID Document and display validation results, but expanding the preview must not issue a registration network request. If the form, Local Identity, advanced access settings, or `ResourcePackage` information has not changed, the client may reuse the generated document. When an input affecting the DID, document hash, version, or control proof changes, it must regenerate the document and sign it again.

To prevent the client from repeatedly producing different resource identities when the user has not changed the materials, the client should form an internal snapshot from the form values, the Local Identity subject DID, resource control configuration, endpoints, authorization domains, version, and `ResourcePackage` digest used to generate the DID Document. If the snapshot is identical

and a preview document already exists, clicking the button only toggles the expanded state of the preview area. If the snapshot changes, the client regenerates the document and related hashes. This cache is only a client-experience optimization and cannot replace the Registrar node's revalidation of the request materials.

9.5 Pre-submission Local Validation

Before submission, the client should at least check the following conditions: a Local Identity has been created or imported; a resource type has been selected; required fields such as name, description, endpoint, authorization domain, capability tags, and version are complete; the DID syntax matches the resource type; the DID Document, metadata, ResourcePackage, and hashes are consistent; the target, purpose, document hash, and verification method of the control challenge are correct; and the control proof can be verified with a public key in the DID Document. Error messages should identify the missing condition, such as “Local Identity is required,” “Resource endpoint is required,” “No authorization domain selected,” or “DID Document hash is inconsistent.”

The preview button performs only local generation and validation. Before sending a request, the submit button reuses the validation result. Neither button should submit automatically when the page is opened. A failed validation must not write a registration record to the Registrar node, a publication record to the Root platform, or a successful registration VC.

Local validation results should preferably be returned by field so that the UI and SDK can take deterministic actions:

```
{
  "valid": false,
  "errors": [
    {"field": "localIdentity", "code": "required", "message": "Local Identity is required"},
    {"field": "serviceEndpoint", "code": "required", "message": "Resource endpoint is required"}
  ]
}
```

When `valid: false`, the client must not enter the submission request. Network unavailability is a submission-stage error and must not be disguised as a local-field error. A successful preview proves only that the current browser can generate and validate the draft; it does not prove that the Registrar node or Root platform has accepted the materials.

9.6 Resource Control Proof Verification

The Registrar node should verify the proof payload, signature, and business context together. The target resource DID, DID Document hash, verification method, operation purpose, challenge Registrar node, time window, and nonce must match the current request; the public key and signature algorithm must be parseable. A missing, expired, wrong-purpose, wrong-target, hash-inconsistent, or reused proof should be rejected before the Root platform is called.

A failed control proof must not produce a successful registration state or registration VC. The node may save the challenge ID, resource DID, document hash, verification method, verification

time, and error code for auditing, but must not save the signing private key.

The validation order should first check structure and context, then verify the signature, and only then begin the publication call:

```
Request is parseable
-> challenge.subjectDid == resourceDid
-> challenge.didDocumentHash == request.didDocumentHash
-> challenge.registrarDid == current Registrar node DID
-> purpose, issuedAt, expiresAt, and nonce are valid
-> verificationMethod is in the DID Document's assertionMethod
-> Resolve public key and verify signature
-> Continue with the registration flow
```

Any failure must stop the subsequent Root platform call. A successful signature check proves only that the signer possesses the corresponding private key and that the signed content has not been modified. Node authorization scope, resource type, version, and Root platform publication conditions must still be verified separately.

9.7 Registrar Node Authorization and Scope Validation

The current Registrar node reads the authorization domains in its own DID Document, checks consistency between the authorization domains in the resource DID Document and those in the request metadata, and then determines whether the node's authorization domains cover the resource domains. Authorization domains must be non-empty, correctly formatted, deterministically ordered, and free of privilege escalation. When they are inconsistent or exceed the node's scope, the node should return an error before calling the Root platform.

The node signing key proves only that the request came from the node's operating boundary. The node authorization VC proves that the node has the corresponding role and scope. The resource control proof is a separate condition.

The request sent by a Registrar node to the Root platform is bound by `SignedRequestEnvelope` to a request ID, protocol version, purpose, HTTP method, path, audience, timestamp, nonce, request-body hash, and node proof. The Root platform can use these values to check which Registrar node submitted which materials, for what purpose, and at what time. The node-to-node signature does not replace the resource control proof, and does not automatically make the Registrar node the resource controller.

9.8 Resource Type, Metadata, Endpoint, and ResourcePackage Validation

The resource type, DID semantic encoding, DID Document metadata, resource metadata, and ResourcePackage should form one consistency chain. The resource DID, resource type, version, DID Document hash, metadata hash, and ResourcePackage hash must reference one another. Endpoints should conform to the declared protocol and format. Capability tags should be interpretable through the capability tag tree or permitted extension rules. Authorization domains must not be bypassed through free-form text.

The current `validate_shape()` covers DID syntax, resource-type relationships, DID Document identifiers, resource structure, ResourcePackage version, hash algorithm, three classes of hash references, challenge target DID, document hash, and operation purpose. Whether an endpoint is reachable, whether the business is safe, and whether the resource meets the caller's quality requirements are judgments for later publication or pre-invocation stages and cannot be replaced by shape validation.

9.9 Duplicate Registration and Idempotency

Business idempotency should be determined by the resource DID, resource type, version, DID Document hash, metadata hash, and ResourcePackage hash, rather than by submission time or VC issuance time. The request ID and nonce provide request correlation, freshness, and replay protection; they do not alone determine whether two requests represent the same resource version.

9.9.1 Repeated Registration of the Same DID

When the same DID is submitted again, the server should verify control first, and then compare the version and hashes. Identical materials should return the existing registration result or an explicit idempotent result. The same DID with a different version or any different hash should enter the same-DID update decision. Invalid control proof must not overwrite the existing record.

9.9.2 Basis for Idempotency Decisions

An idempotency decision depends at least on the resource DID, resource type, version, DID Document hash, metadata hash, and ResourcePackage hash. The system should also save the request ID, processing time, and result to distinguish a business retry from a new version. The current Registrar node writes records using an upsert by resource DID to SQLite, PostgreSQL, or file storage. This can preserve the current record, but it is not by itself evidence of a complete version-level idempotency history.

9.9.3 VC Reissuance

A retry for the same business version may return the saved registration VC, or may issue a content-equivalent VC with a different issuance time according to an explicit policy. The client should identify the credential by the resource DID, version, hashes, and state, and must not treat a new issuance time as a resource version update.

9.9.4 Duplicate Publication Control at the Root Platform

The Root platform should make repeated publication of the same DID, version, and hashes produce one business result. A different hash or version should enter an update, conflict, or rejection path. A successful upstream HTTP response received by a Registrar node is not sufficient to prove that the Root platform completed duplicate-publication deduplication; the publication proof and sequence record are the evidence for that determination.

9.10 Resource Updates and Same-DID Version Updates

An update under the same DID preserves the resource identity, but must produce a new version, document hash, or ResourcePackage hash and express the predecessor relationship through fields such as `previousVersion`. The new version must use the current control relationship to generate a new control proof. An old registration VC or old ResourcePackage cannot automatically authorize a future update. Version increments, concurrent conflicts, history retention, and rollback are jointly enforced by the Registrar node and Root platform. The specific publication replacement rules are described in Chapter 10.

9.10.1 Update Authorization Verification

The verifier must confirm that the submitter is still authorized by the current DID control relationship, and must check that the control proof is bound to the new document hash, version, purpose, and challenge. An update supported only by an old credential without a new control proof for the new version must be rejected.

9.10.2 DID Document Updates

An updated DID Document continues to use the original resource DID, and its document hash must be included in the registration submission and ResourcePackage. Adding, replacing, or deactivating verification methods requires a verifiable control chain. If the update cannot be proven to have been authorized by the current controller, the current document must not be replaced.

9.10.3 Resource Metadata and Endpoint Updates

Changes to the resource description, capability tags, authorization domains, protocol bindings, or endpoints should produce a new metadata hash and ResourcePackage hash. Expanding the authorization domain cannot be achieved by changing text fields alone. Endpoint reachability also cannot replace business security and invocation-authorization checks.

9.10.4 New Version Publication and Old-Version Handling

After a new version is accepted, the Root platform should clearly identify the current version, historical versions, and old-version states. Discovery service nodes synchronize according to the publication sequence. An old version may be retained for auditing or rollback, but must not be confused with the latest version in query results.

9.11 Registration VC Issuance and Retrieval

In the current reference implementation, the registration credential generation logic is called only after the Root platform's resource verification and publication interface returns success, and the registration VC is written to the registration record and successful response. The issuer of the registration VC is the Registrar node, and its credential subject is the resource DID. Its claims include registration facts such as resource type, version, hashes, authorization domains, and lifecycle.

9.11.1 Conditions for VC Issuance

Before issuance, entry-shape validation, resource authorization-domain validation, control-proof processing, and the Root platform request must be completed. Passing local preview, clicking Submit, or the Registrar node receiving a request is not by itself a condition for issuance.

9.11.2 VC Delivery

The current successful response contains `status`, `resourceDid`, `resourceType`, `registrationCredential`, and `rootResponse`. The client should save the original VC and associate it with the DID Document, version, and hashes. `rootResponse` must still be parsed according to the Root proof and status rules.

9.11.3 Handling a Missing or Empty VC

If the registration result is successful but the VC is missing or empty, the client should record the registration state and credential state separately, and clearly indicate that the credential has not yet been obtained. It must not fabricate an empty VC or treat an empty value as a valid credential. Queries or retries should retain the original DID, version, and hashes.

9.11.4 Local VC Storage

The registration VC should be associated with the resource DID Document and resource version in local files. When the client packages and downloads the VC, DID Document, and Local Identity backup, the private key remains sensitive material that may be kept only by the controller. It must not be sent to a node or public storage.

9.12 Registration Responses and Error Handling

The current Registrar node handler receives `Json<ResourceRegistrationSubmission>` and returns dynamic JSON on success. The stable core fields of a successful response are `status`, `resourceDid`, `resourceType`, `registrationCredential`, and `rootResponse`. `rootResponse` preserves dynamic content returned by the Root platform and must not be assumed by the client to be a fixed VC or an indication that network-wide publication is complete.

Stage	Typical result	Client interpretation
Local validation by Registrar node	<code>4xx</code> or structured error	The request did not enter the Root publication flow; correct fields, hashes, or control proof first
Root verification and queueing	<code>status: resource-verified-and-queued</code>	The Root platform has verified the request and created a subsequent task; content distribution and discovery indexing are not necessarily complete
Successful registration by Registrar node	<code>status: submitted</code>	The Registrar node has recorded the registration result; the response may contain a registration VC and Root response

Stage	Typical result	Client interpretation
Temporary network or upstream failure	5xx, connection error, or timeout	Query the status using the resource DID, version, hashes, and request identifier before deciding whether to retry

The following handler relationship describes the actual input and output types. It is a concise representation of the interface contract, not a complete implementation that can be substituted directly:

```
async fn register_resource(
  State(state): State<AppState>,
  Json(submission): Json<ResourceRegistrationSubmission>,
) -> ApiResult<serde_json::Value>;
```

The `ResourceVerifyAndPublishRequest` exchanged between the Registrar node and Root platform also contains `registrarDid`, the original registration submission, and `upstreamAuth`. A client must not construct this internal forwarding object itself, and must not treat the Registrar node's `registrationCredential` as the Root platform's Root proof. The `error`, status code, and request-correlation identifier in an error response should be used together to decide whether to correct, query, retry, or stop.

Result	Meaning	Client handling
Success with VC	The Registrar node completed the current flow and returned the registration credential	Save the VC and separately confirm Root publication and discovery status
Success but VC unavailable	The registration result exists but the credential is incomplete	Preserve the DID, version, and hashes; query or retry
Validation failure	The request structure, type, hash, or proof is invalid	Correct the specific field; do not retry blindly
Authorization failure	The node scope or resource control authority is insufficient	Check the authorization VC, authorization domains, and control relationship
Upstream failure	The Root platform rejected the request or is unavailable	Handle according to retryability; do not declare publication success
Network timeout	The client did not receive a definitive response	Preserve the original materials, query, or retry with the same business materials
Duplicate or conflict	The same version already exists or different content is present	Return the existing result, or enter update and conflict handling

In the current implementation, errors are returned mainly through the HTTP status and `error` text. A stable future protocol should add a request ID, error code, registration state, upstream state, and retryable flag so that clients do not have to infer network state from free text.

Response handling is recommended to follow these deterministic rules:

```
{
  "status": "submitted",
```

```

"resourceDid": "did:oon:SKFI:...",
"resourceType": "skill",
"registrationCredential": null,
"rootResponse": {"status": "published"}
}

```

When an HTTP request times out without a response, the client can record only “result unknown,” retain the original DID, version, and hashes, and query or retry with the same materials. It must not generate a new DID on that basis. When a non-success HTTP status is received, the result must not be saved as a successful registration. When a success status is received but the VC is empty, the registration state and credential state should be displayed separately; only a credential that was actually obtained and verified may be downloaded as a VC file.

9.13 Registration Audit Records

A registration record should at least associate the resource DID, resource type, version, DID Document hash, metadata hash, ResourcePackage hash, authorization domains, Registrar node, registration VC, Root response summary, submission time, and processing result. The current `write_resource_record()` writes by resource DID to SQLite, PostgreSQL, or file storage. A complete audit implementation should therefore also retain the operation type, request ID, previous and new versions, and failure reason, so that an upsert does not erase the historical context.

Audit records must not store resource control private keys, Local Identity backups, or recoverable secrets. DID, public key, hash, challenge ID, request ID, state, and error code may be used as trace evidence. Audit-read permissions must be separated from the public discovery interface.

9.14 Resource Update Authorization and Same-DID Replacement

After receiving new materials for the same DID, the server should confirm the control proof, version relationship, and hash relationship in sequence. The same version with the same hashes is an idempotent retry; the same version with different hashes is a conflict; a new version with a valid predecessor relationship may enter the update flow; version rollback, a missing predecessor, an unapproved authorization change, or an invalid control proof should be rejected or sent for manual handling. The Registrar node may save the acceptance record, but the Root platform determines the network publication fact.

The following table can be used to check replacement decisions:

DID	Version	Content hash	Control proof	Handling
Same	Same	Same	Valid	Handle idempotently as the same business version
Same	Same	Different	Valid	Reject as a conflict; do not overwrite the current version

DID	Version	Content hash	Control proof	Handling
Same	Updated	Different	Valid and bound to new materials	Process through the update flow
Same	Updated or same	Any	Invalid	Reject without replacement side effects
Different	Any	Any	Valid	Process as a new resource registration

9.15 Submission Retries, Idempotency Keys, and Duplicate Handling

The upstream request from a node to the Root platform uses `SignedRequestEnvelope` to bind the request ID, protocol version, purpose, method, path, audience, request time, request nonce, request-body hash, and proof. The security layer uses nonce storage and a time window for replay protection. Business idempotency depends on the resource DID, version, and content hashes. After a client timeout, it should preserve the original materials, first query the existing result or retry with the same business materials, and must not immediately generate a new DID to conceal an unknown result.

9.16 Registration Rate Limiting and Abuse Control

The public registration entry point should limit the request rate per time period, request-body and `ResourcePackage` size, failure count per source, same-DID conflict count, and upstream-forwarding concurrency. Control-proof, node-authorization, and authorization-domain checks should be performed early to reduce the storage, network, and computation pressure caused by forged materials. Rate limiting must not misclassify a legitimate same-DID idempotent query as a new version, and throughput must not be gained by weakening control-proof checks.

The current reference implementation explicitly includes shape validation, authorization-domain coverage checks, upstream request signing, and nonce replay protection. If a reverse proxy, WAF, or service-level rate limiter is enabled in deployment, the actual policy, response codes, and recovery method should be documented in the deployment documentation.

Registration Flow Acceptance Checklist

- ☐ For an empty form, missing Local Identity, missing endpoint, or missing authorization domain, the client identifies the specific condition.
- ☐ Before calling the Root platform, the Registrar node completes request-shape, resource-authorization-domain, and control-proof validation.
- ☐ When the Root platform fails or the network times out, the client does not display network-wide publication success and does not lose the original DID and version materials.
- ☐ The registration VC in a successful response can be associated with the resource DID, version, and three classes of hashes.

- Retries with the same DID, version, and hashes receive different handling from updates with a different hash or version.
- Registration, Root publication, content distribution, and discovery indexing states can be observed and verified separately.
- Audit records can trace requests without containing private keys, Local Identity backups, or other sensitive secrets.

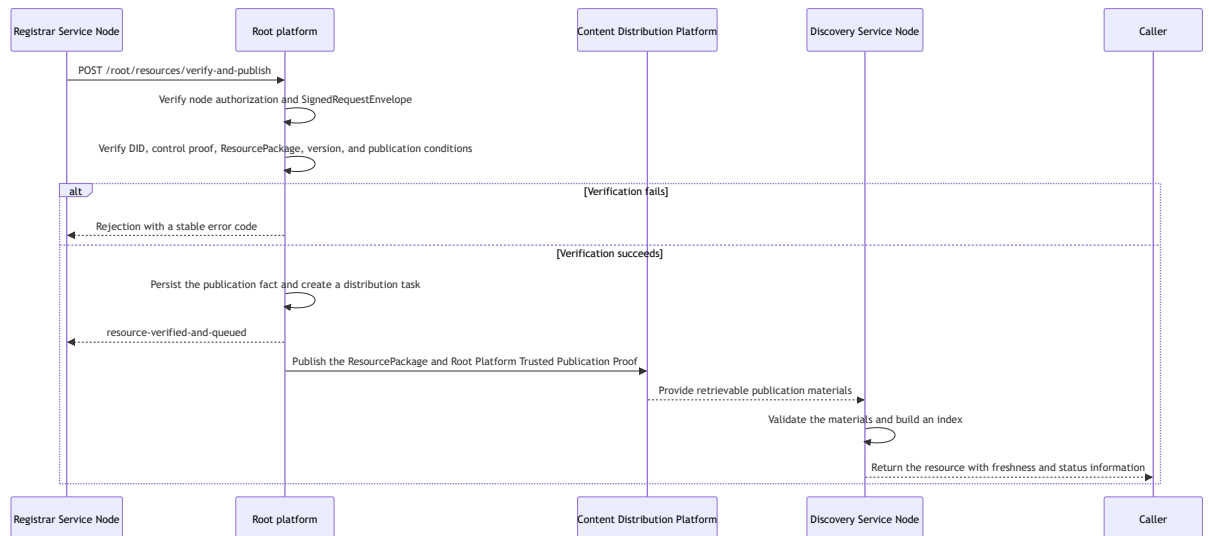
9.17 References

Source	Type	Link
oan-registrar-node	Code repository: registration entry point, validation, registration records, and registration VC	https://github.com/OpenAgenet/oan-registrar-node
oan-root-services	Code repository: Root platform resource verification and publication interfaces	https://github.com/OpenAgenet/oan-root-services
oan-protocol-common	Code repository: registration payloads, error codes, and cross-node protocol types	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	Code repository: client registration materials and registration calls	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	Code repository: registration-flow scripts and developer invocation examples	https://github.com/OpenAgenet/oan-community-skill
OAN Yellow Paper	arXiv yellow paper: registration, publication, and verification flow	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF draft: relationship between resource identity, registration, and discovery	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

Chapter 10 Root Platform Verification and Trusted Publication

This chapter explains how the Root platform converts resource materials submitted by a Registrar Service Node into verifiable network-level publication facts. It covers upstream request authentication, resource DID and control-proof verification, node authorization, ResourcePackage integrity, publication policy, issuance of the Root Platform Trusted Publication Proof, version selection, distribution tasks, and failure recovery. It continuously distinguishes among “verified and queued,” “publication fact formed,” “content distributed,” and “indexed by a Discovery Service Node.”

10.1 Root Platform Publication Responsibilities



“Verified and queued,” content distribution, and discovery indexing in this flow correspond to different completion conditions. A successful response from the Root platform means only that the resource has passed Root-platform processing and entered the distribution task chain. The Content Distribution Platform and Discovery Service Node must still complete their own validation, publication, or indexing, and the caller should independently verify the DID, registration VC, Root Platform Trusted Publication Proof, and status information.

The Root platform is an independent verification boundary before a resource enters the network-level publication path. It receives registration materials submitted by an authorized Registrar Service Node, re-verifies the resource DID, DID Document, resource control proof, ResourcePackage, node authorization, version, and publication conditions, and writes the verified resource version to

the publication record. Pre-validation by a Registrar Service Node is an access-layer check and cannot replace the Root platform’s independent confirmation of the publication fact.

In operational terms, Root-platform “acceptance” contains two consecutive but distinguishable facts: first, the resource materials pass Root-platform verification and are persisted; second, a publication task for the Content Distribution Platform is created. The current successful response is `resource-verified-and-queued`. Therefore, a Registrar Service Node or client should interpret it as “verified and queued,” not directly display it as “network-wide publication completed.”

10.2 Upstream Submission by a Registrar Service Node

A Registrar Service Node submits a resource registration request with upstream authentication, rather than uploading an unproven `ResourcePackage`. The request should be associated with the Registrar Service Node DID, request identifier, protocol version, target path, time, nonce, request-body hash, and signature, and should carry the resource registration submission and its control proof. The Root platform should first verify the submission source and request integrity, and only then process the resource content. A request that is duplicated, expired, or addressed to the wrong audience must not enter the publication state.

The current entry point is `POST /root/resources/verify-and-publish`. After receiving the request, the Root platform loads authorization materials according to the Registrar Service Node DID, verifies the `SignedRequestEnvelope` signature, method, path, audience, time, nonce, and request-body hash, and only then performs structural and content validation of the resource submission. The upstream signature proves “which Registrar Service Node submitted these materials.” It is neither a resource control proof nor a Root Platform Trusted Publication Proof.

The request type for this entry point is `ResourceVerifyAndPublishRequest`. Its minimum relationship can be represented as follows:

```
{
  "registrarDid": "did:oan:...",
  "submission": {
    "resourceDid": "did:oan:SKDM:...",
    "resourceType": "skill",
    "didDocument": {},
    "metadata": {},
    "packageVersion": "1.0.0",
    "didDocumentHash": "sha256:...",
    "metadataHash": "sha256:...",
    "packageHash": "sha256:...",
    "hashAlgorithm": "sha256",
    "subjectControlProof": {}
  },
  "upstreamAuth": {}
}
```

The current successful response is dynamic JSON. Its core result is `status: resource-verified-and-queued`, together with the resource DID, resource type, package version, three classes of hashes, lifecycle state, and `cdnDispatchStatus: queued`. This means that the Root platform has completed

the current verification and created a Content Distribution Platform task. Receipt by the Content Distribution Platform and indexing by a Discovery Service Node must be confirmed through their respective status or synchronization records.

10.3 Resource DID and Control-Proof Verification

The Root platform should first resolve the resource DID and confirm that its resource type matches the DID’s semantic encoding. It should then confirm that the DID Document `id`, resource type, verification methods, and service declarations satisfy the method rules, and finally use the public key declared in the document to verify the control proof. The control proof must bind the current challenge, target resource DID, DID Document hash, purpose, and validity period. A verification failure must prevent the version from entering the publication record and must not overwrite an existing valid version.

A reproducible negative test is to prepare a submission that passes validation, record the Root platform’s current resource version and publication-task count, and then modify only one of `resourceDid`, `didDocument.id`, `didDocumentHash`, or the control challenge’s `subjectDid`. The Root platform should return the corresponding error, and the failed request should not add or overwrite an acceptance record or Content Distribution Platform task. This verifies that the Root platform does not mistake “the request was reachable” for “resource control was proved.”

10.4 Registrar Service Node Authorization Verification

The object of Registrar Service Node authorization is infrastructure-node qualification, not resource control. The Root platform should determine whether a node may submit the applicable resource by using the node DID, target role, authorization VC, authorization domain, stable DID Document hash, issuer, and current governance state. If authorization is expired or revoked, the role does not match, or the authorization domain does not cover the target resource, the submission should be rejected or isolated. The check must not be relaxed merely because the request came from a known address.

The Root platform’s decision contains at least two steps: whether the node remains in an available governance state, and whether the node’s authorization scope covers the resource’s authorization domain. The first handles node revocation, suspension, or authorization expiry. The second handles cases in which a Registrar Service Node may serve only part of the resource authorization domain. Even if a Registrar Service Node has successfully issued a registration VC, the Root platform must still re-evaluate the current authorization state before accepting publication. A registration VC cannot replace the Root platform’s node-authorization check.

10.5 ResourcePackage and Hash Verification

The Root platform should recompute the DID Document hash, metadata hash, and ResourcePackage hash according to the unified canonicalization rules, and check consistency among the hash-algorithm prefix, resource DID, resource type, package version, and nested fields. Any hash mismatch, inconsistent treatment of circular fields, or inconsistency between package content and

its declarations is an integrity failure. Failed materials may be retained as diagnostic records, but must not enter the distributable-resource set.

After successful recomputation, the Root platform organizes the resource DID, resource type, version, three hashes, lifecycle, and authorization domain into `packageClaims`, and writes the Root proof into the `ResourcePackage`'s `rootProof`. A verifier should use consistency between these claims and the actual `ResourcePackage` fields as evidence. Merely seeing a client-submitted `packageHash`, or checking only its string format, is insufficient to prove `ResourcePackage` integrity.

10.6 Metadata and Endpoint Policy Checks

Policy checks determine whether materials that have passed identity and integrity checks are suitable for the network publication path. The Root platform should check consistency among resource type, capability tags, authorization domain, protocol bindings, service endpoints, lifecycle, and version declarations, and prevent private keys, access tokens, and internal credentials that should not be public from entering public resource materials. Passing a policy check does not guarantee endpoint quality; endpoint invocation is still authorized jointly by the caller and the resource service.

Policy-check results should be recorded separately from subsequent propagation stages. Materials that pass the checks may enter the Root platform acceptance record and Content Distribution Platform task. Whether an endpoint is online, whether the Content Distribution Platform has completed hosting, and whether a Discovery Service Node has completed indexing must be confirmed by the respective later services. If a resource contains a private key, access token, or an internal address that should not be public, it must be rejected or isolated before public distribution materials are formed; the system must not rely on a Discovery Service Node to remedy this during indexing.

10.7 Issuance of the Root Platform Trusted Publication Proof

10.7.1 Issuance Preconditions

The Root Platform Trusted Publication Proof may be issued only after the submission source, Registrar Service Node authorization, resource control proof, DID Document, three classes of hashes, resource version, and publication policy have all passed verification. Before issuing the proof, the Root platform should also confirm that the current governance state does not prevent the node or resource version from entering the publication path. Pending verification, pending review, or distribution failure must not be converted directly into published.

10.7.2 Proof Contents

The proof should bind at least the Root platform DID, resource DID, resource type, `ResourcePackage` version, DID Document hash, metadata hash, `ResourcePackage` hash, hash algorithm, lifecycle state, and issuance time. If the proof contains an authorization domain, publication cursor, or announcement reference, it must clearly state whether those fields cover a publication fact or a

propagation fact. They must not include business conclusions that were not declared by the resource controller.

10.7.3 Proof Signature

The Root platform should sign the prescribed proof contents with its protected signing key and specify the cryptographic suite, signature coverage, and verification-method reference. A verifier should first confirm that the issuer belongs to the trusted Root platform, and then check that the signature input exactly matches the current ResourcePackage. Being able to parse JSON or see a Root platform DID is insufficient to prove that the signature is valid.

10.7.4 Proof Retrieval and Verification

The resource provider, Content Distribution Platform, and Discovery Service Node may obtain the proof through the Root platform publication interface, publication materials, or an associated address. A verifier should check the proof status, target resource DID, version, hashes, issuance time, and Root-platform authorization state, and retain the verification result. Successfully obtaining the proof does not mean that a Discovery Service Node has completed indexing.

The Root Platform Trusted Publication Proof should be used with one determined resource version. A verifier should at least associate the Root platform DID, resource DID, package version, DID Document hash, metadata hash, ResourcePackage hash, proof, and verification time. The verifier must recheck the hashes and proof status when retrieving the same resource again. A registration VC issued by a Registrar Service Node, the Root Platform Trusted Publication Proof, and a Discovery Service Node index record prove different stages and cannot replace one another.

10.8 Publication Sequence and Version Selection

10.8.1 Publication Sequence

The publication sequence identifies the order of resource publication facts in the distribution path. It should advance monotonically and be bound to the resource DID, resource version, and package hash. Re-reading or retrying the same publication task must not create a new logical publication fact, and a Discovery Service Node synchronization cursor must not be treated as the Root platform publication sequence.

10.8.2 Version Selection for the Same DID

Version selection for the same resource DID should consider the ResourcePackage version, predecessor-version relationship, package hash, control proof, and Root platform publication sequence. Identical materials can be confirmed idempotently. Materials with different content but no valid control proof or version relationship should be rejected or isolated, rather than allowing the arrival time to overwrite an existing published version.

10.8.3 Duplicate Submission Handling

When the Root platform receives a submission whose resource DID, package version, and package hash are exactly the same as an existing resource, it may return the existing publication result or a duplicate-submission status while keeping the original publication cursor and proof unchanged. The duplicate-submission record should still be associated with the new request identifier so that network retries can be distinguished from actual version updates during audit.

10.8.4 Conflicting-Version Handling

If the same DID has the same version with a different hash, the same publication sequence with different content, an inconsistent predecessor version, or a proof with inconsistent coverage, the submission should enter conflict or isolation handling. The system must not automatically choose the first-arriving version. The Root platform should retain a conflict summary and rejection reason. Only new materials from the controller that satisfy the version and control conditions may be verified again.

When investigating a version problem, inspect the resource DID, package version, three hashes, predecessor relationship, and publication cursor together. Do not select a version only by request arrival time. The publication cursor is ordering evidence in the Root platform publication record and distribution path; it is neither resource control nor a Discovery Service Node synchronization cursor. Even after a Discovery Service Node sees the cursor advance, it must still validate the specific ResourcePackage and Root proof.

10.9 Publication Status and Rejection Reasons

The current successful response from the Root platform is `resource-verified-and-queued`. It means that the resource has passed Root-platform processing and entered the Content Distribution Platform task chain; it does not mean that the Content Distribution Platform has completed publication. External processing should distinguish at least the following states:

Status	Meaning	Allowed into distribution	Fail
Pending verification	Received, but necessary checks are not complete		No Que
Verified and queued	Resource verification is complete and a publication task has been created		Yes Wa
Published	The publication fact is persisted and the proof is verifiable		Yes Cor
Duplicate submission	The same DID, version, and hash arrived again	Reuse the existing fact	Ret
Rejected	Control, authorization, hash, or policy checks failed		No Cor
Pending retry	The external publication path temporarily failed	Use the existing task	Ret

Rejection reasons should preferentially use stable codes such as `registrar_not_authorized`, `did_document_hash_mismatch`, `metadata_hash_mismatch`, and `package_hash_mismatch`. An invalid control proof, unauthorized node, and inconsistent hash are verification failures that block publication. Event-delivery failure or temporary Content Distribution Platform unavailability is a recoverable subsequent failure. These two categories must not both be displayed only as “publication failed.”

10.10 Content Distribution Preparation

After accepting a resource, the Root platform persists the ResourcePackage, publication task, and event-publication intent, and hands the task to the Content Distribution Platform through a content-distribution event. The event may carry only replayable references such as the resource DID, package version, publication cursor, Root platform DID, and hashes; the Content Distribution Platform then retrieves the ResourcePackage from the Root platform. The current design uses a transactional outbox to keep the resource acceptance record, publication task, and event intent consistent. If event sending fails, the outbox relay retries it.

```
{
  "jobKey": "did: oan:SKDM:example-resource:1.0.0",
  "resourceDid": "did: oan:SKDM:example-resource",
  "packageVersion": "1.0.0",
  "publicationCursor": 123,
  "rootDid": "did: oan:ROOT:example-root",
  "packageHash": "sha256:<example-package-hash>",
  "didDocumentHash": "sha256:<example-did-document-hash>",
  "metadataHash": "sha256:<example-metadata-hash>"
}
```

Acceptance by the message system does not mean that content publication is complete. The Content Distribution Platform should confirm the task after retrieving the ResourcePackage, verifying the Root proof and hashes, and completing its own publication. A Discovery Service Node should also independently validate the ResourcePackage and proof after receiving propagation materials.

10.11 Root Platform Publication Audit Trail

The audit chain should trace a complete publication decision from a request identifier or task key: what the Registrar Service Node submitted, which fields the Root platform verified, what the ResourcePackage and hashes were, which publication cursor was generated, whether a Content Distribution Platform task was created, whether the event was delivered, and how the task was ultimately processed. At least the following field groups are recommended:

Field group	Typical fields	Traceability purpose
Request association	<code>requestId</code> , Registrar Service Node DID, receipt time	Locate one upstream submission
Resource version	Resource DID, <code>packageVersion</code> , three hashes	Fix the materials being processed
Verification result	Control proof, node authorization, policy-check result	Explain acceptance or rejection
Publication evidence	Root platform DID, proof, <code>publicationCursor</code>	Prove the Root platform publication fact
Distribution status	<code>jobKey</code> , event status, retry count, last error	Track subsequent propagation

Audit records may retain summaries and references. They must not contain subject private

keys, resource-control private keys, identity backups, or access tokens. Public resource fields and internal diagnostic fields should have separate read permissions so that an audit interface does not become a side channel for sensitive materials.

10.12 Republish, Rollback, and Recovery

When distribution fails, first retry the publication task with the same resource DID, package version, hashes, and task key. Do not generate a new resource identity merely to resend a transmission, and do not interpret a transport retry as a new version. After data recovery, restore the Root platform acceptance record, publication task, and outbox state first, and then rebuild incomplete event delivery. When rolling back to a historical version, the historical version's DID, version, hashes, and Root Platform Trusted Publication Proof still correspond to the original fact. If the resource content changes, a new control proof must be generated, hashes recomputed, and Root-platform verification performed again.

Distribution failure

- > Retain the original jobKey, resource DID, version, and hashes
- > Check the task lease, retry count, and last error
- > Retrieve the same ResourcePackage and verify the Root proof
- > Confirm the task after the Content Distribution Platform succeeds
- > Let Discovery Service Nodes continue synchronization by publication cursor

10.13 Same-DID Update Authorization and Version Acceptance

An update under the same DID must not be decided by comparing version strings alone. The Root platform should confirm that the new materials remain authorized by the current control relationship, that the DID Document, resource metadata, and ResourcePackage hashes correspond to the new materials, and that the new version can establish a predecessor relationship with the existing version. The same version with the same hash is handled as an idempotent retry. The same version with a different hash is a conflict. A new version whose control proof remains bound to the old DID Document hash must be rejected before the acceptance record is written.

A reproducible acceptance test can fix an accepted `v1` and submit, separately, an identical `v1`, a `v1` with a different hash, a `v2` with a valid control proof, and a `v2` without a predecessor relationship. The acceptance criteria are: the first case produces no second logical publication fact; the second does not overwrite `v1`; the third enters new-version processing; the fourth is rejected or isolated; and every result can be explained from the request, version, and hash fields.

10.14 Publication Rejection, Isolation, and Reprocessing

A clear verification failure should be rejected directly with its reason. Isolation is appropriate only when the materials cannot yet be determined, depend on external governance state, or require review. An isolation record should retain the original submission summary, resource DID, version,

hashes, Registrar Service Node, failure stage, and reprocessing conditions, but must not appear in a Content Distribution Platform manifest or discovery index. Reprocessing must check current authorization, control-proof validity, hashes, and governance state again; it must not merely change the old status to “published.”

10.15 Root Platform Publication State Machine

10.15.1 Pending Verification

Pending verification means that the Root platform has received a submission or publication task but has not completed all necessary checks. This state may allow processing-status and diagnostic queries, but must not enter the trusted distribution manifest or be presented externally as registration success, Root-platform publication, or discovery visibility.

10.15.2 Verification Passed

Verification passed means that the resource materials, control proof, node authorization, and publication conditions have passed Root-platform checks. If the Root proof has not been issued or the publication record has not been persisted, however, this state is still not equivalent to published. The transition should have a clear submission identifier, verification time, and evidence reference.

10.15.3 Published

Published means that the Root platform has formed a publication fact and Root Platform Trusted Publication Proof bound to a specific resource version, and permits the related materials to enter the Content Distribution Platform path. It does not mean that the Content Distribution Platform has completed hosting, that a Discovery Service Node has completed indexing, or that the caller has obtained permission to invoke the business service.

10.15.4 Rejected, Isolated, and Recovered

Rejected means that the current submission does not satisfy a necessary condition. Isolated means that the materials require review, completion, or a change in external state. Recovery must recheck current authorization, signatures, hashes, and governance state. A distribution failure may trigger pending retry; it must not alter the existing resource version or forge a new Root Platform Trusted Publication Proof.

During recovery, use the original submission’s resource DID, version, and material summary as the association basis, and run the current authorization, control-proof, hash, and governance-state checks again. Only after passing verification may isolated materials return to the publication flow. Network retries must not change the resource version or generate a new Root Platform Trusted Publication Proof. The Root platform’s “published” state still means only that a publication fact has been formed; it does not mean that every Discovery Service Node has indexed the resource.

10.16 References

Source	Type	Link
<code>oan-root-services</code>	Code repository: Root-platform resource verification, publication proofs, and distribution tasks	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: Registrar Service Node upstream submission and registration result	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-protocol-common</code>	Code repository: shared ResourcePackage, proof, and publication-event types	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv Yellow Paper: Root platform, trusted publication, and security boundaries	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF draft: resource identity, publication, and discovery relationships	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

Chapter 11 Content Distribution and Network Propagation

This chapter explains how, after the Root platform has formed publication facts, resource packages, DID documents, metadata, hashes, credentials, and publication proofs are propagated through the content distribution platform to authorized Discovery nodes. The chapter distinguishes the states of publication tasks, propagation materials, synchronization cursors, receipt validation, and local indexing, and explains how batch transfer, retries, partial failures, idempotency, and recovery keep propagation traceable. The fact that content can be retrieved must not be mistaken for the fact that the resource has been applied by every Discovery node.

11.1 Distribution Model and Content Categories

Content distribution follows the model of “publication facts are formed first, propagation materials are transferred afterward, and Discovery nodes validate again.” The propagation set for a resource version usually includes the resource package (**ResourcePackage**), DID document, resource metadata, hashes, registration credential, Root-verified ResourcePackage, lifecycle status, authorization domain, and necessary summaries or index hints. A summary may improve synchronization efficiency, but it cannot replace validation of the complete resource package and proofs.

There are three types of objects that are easily confused in the propagation path: the Root platform creates publication tasks and publication cursors, the content distribution platform stores retrievable resource materials, and Discovery nodes convert validated materials into local indexes. They respectively represent “what the Root platform decides to propagate,” “where the content can be retrieved,” and “what a node has already applied.” The same resource DID may run through all three, but the three states must not be merged into one network-wide state.

11.2 Content Distribution Service Responsibilities

The content distribution platform is responsible for hosting, retrieving, batch-transferring, caching, and reporting the transfer status of publication materials. A content distribution publisher can obtain publication tasks from the Root platform, organize resource packages according to the publication cursor, write them to the content distribution platform, and report successful or failed results back to the Root platform. This service does not create resource control rights, node authorization, or Root-verified ResourcePackage; Discovery nodes must still decide whether to apply materials based on their own validation results.

The current reference implementation's batch publication response contains published items, failed items, and the corresponding publication cursor. A publisher confirms the result of a batch only when the response is parseable and the number of items is consistent with the request. If a request succeeds but the response lacks a cursor or contains incomplete item statuses, it must be handled as an exception rather than being treated as confirmation that the entire batch has been published.

11.3 Distribution Manifest and Publication Checkpoints

The publication manifest is the entry point through which a Discovery node determines the synchronization scope and validation objects. Publication checkpoints divide propagation state into sent, received, validated, and applied. The resource DID, package version, package hash, and publication cursor in the manifest associate a resource version; the target nodes and sequence range in a notification batch associate one propagation operation. A Discovery node may report that an item has been applied only after it has completed validation and written the item to its index.

11.4 Distribution of Resource Packages, Metadata, Proofs, and Index Data

11.4.1 Resource Package Distribution

The resource package is the primary complete payload for Discovery nodes. It must at least be associated with the publication manifest through the resource DID, resource package version, and package hash. The content distribution platform may provide a complete package or incremental materials according to the manifest, but the receiver must recompute the hash and validate the Root-verified ResourcePackage before writing the material to a trusted discovery index.

The current content distribution platform provides `POST /cdn/resources` as the single-item publication endpoint and `POST /cdn/resources/batch` as the batch publication endpoint. A single-item request is carried by `ResourceCdnPublishRequest` and contains `ResourcePackage` and `upstreamAuth`; a batch request is carried by `ResourceCdnBatchPublishRequest` and contains entries with `publicationCursor` and shared upstream authentication. A successful single-item response contains `status: published`, the resource DID, resource type, package version, and `publicationCursor`. A batch response additionally distinguishes `acceptedCount`, `failedCount`, `successful items`, and `failed items`. Batch HTTP success cannot be used as a substitute for every item having been applied.

Discovery nodes can use `POST /cdn/resources/batch-get` to read resource packages in batches by resource DID, or use the resource index endpoint to read incrementally by `afterCursor`. A batch read returns the requested count, hit count, and resource package items; an incremental index response returns `items`, `count`, `afterCursor`, `nextCursor`, and `hasMore`. These cursors serve batch reads, list pagination, and propagation synchronization respectively and cannot replace one another.

11.4.2 DID Document and Metadata Distribution

DID documents and resource metadata may be transferred together with the resource package, or retrieved separately by resource DID, version, and hash. When they are sent separately, a Discovery node must not apply one part while losing the association between the parts. If the DID document, metadata, and package-external summary are inconsistent, the node should reject the combination and record the missing or conflicting fields.

11.4.3 VC and Root Proof Distribution

The registration credential describes the registration fact established by the registration service node, while the Root-verified ResourcePackage describes the Root platform's acceptance of a specific resource version. They should be transferred with associations to the resource DID, version, and hash. The receiver should separately validate the issuer, signature, coverage, and current status of each, and must not skip Root proof checks merely because a registration credential is present.

11.4.4 Index Data Distribution

Index data is a derived read model formed by a Discovery node from published materials. It may contain resource summaries, capability tags, endpoint references, lifecycle status, and freshness information. Index data must not be returned as a new Root platform fact. A Discovery node must be able to identify the publication cursor, validation time, and authorization domain corresponding to an index record.

11.5 Distribution-Boundary Integrity Validation

After receiving materials, a Discovery node should first validate the notification target, manifest scope, and payload structure, and then validate the resource DID, resource type, version, the three categories of hashes, Root-verified ResourcePackage, lifecycle, and authorization domain. Objects that fail validation must not be written to the trusted discovery index. A rejection record may be written for diagnosis and retry. Successful transfer with corrupted content is an integrity failure; missing material is a recoverable retrieval failure; a revoked governance status is a state failure that blocks application.

Negative acceptance tests at the receiver can independently change only the package content, the resource DID bound to the proof, or the authorization domain of the current Discovery node. In the first two cases, the receiver should reject the material respectively at the hash or proof-binding stage; in the third case, it should block index application. A failure may leave a record containing the resource DID, cursor, failure stage, and error code, but the affected synchronization cursor must not advance.

11.6 Version Freshness and Synchronization Cursors

The publication cursor describes the position of Root platform publication facts in the propagation path; the synchronization cursor describes the position already processed by a Discovery

node; the resource version describes the version of the resource content itself; freshness describes the time relationship between publication or receipt and the current observation moment. They serve ordering, recovery, version selection, and latency explanation respectively, and one field cannot replace the others.

11.6.1 Synchronization Cursor

A synchronization cursor is maintained by a Discovery node to represent the content distribution position that the node has processed. Advancing the cursor must be conditional on the corresponding materials completing structural, hash, proof, and status validation. Advancing the cursor to a position does not mean that all resources after that position have been applied successfully.

11.6.2 Version Freshness

Version freshness should consider the Root platform publication cursor, resource package version, publication or update time, local receipt time, and index application time together. Time differences are used to explain propagation latency and must not alone determine version priority. Content selection for the same DID must still be based on a verifiable version, hash, and Root platform publication fact.

11.6.3 Expired Data Determination

A Discovery node should determine whether data is expired according to the permitted delay, governance status, and resource lifecycle. Ordinary information display may retain an old snapshot marked with freshness information. Before processing revocation, suspension, node authorization changes, or high-risk calls, if the latest status cannot be confirmed, the node should pause the sensitive action instead of continuing to use an unconfirmed old status.

11.6.4 Gap Compensation

When a cursor gap, discontinuous batch sequence, or missing resource material is detected, the node should pause cursor advancement for the affected scope, retrieve the missing content according to the manifest or update endpoint, and continue after revalidation. Before the gap is filled, the node may report synchronization lag but must not mark subsequent items as applied without evidence.

For example, if a node has applied cursor 100 and receives cursor 102 while 101 is missing, it may save the material for 102 as pending and report the gap, but its local “applied position” should remain 100. After retrieving and validating 101, it should process 102 in order. If 101 is explicitly rejected, it should retain a traceable rejection record and determine whether the cursor may cross that item according to the implementation. Receiving a larger cursor value alone is not sufficient to advance the cursor.

11.7 Initial Synchronization and Incremental Synchronization

11.7.1 Initial Full Synchronization

Initial synchronization should first obtain a verifiable publication manifest, starting cursor, and target authorization domain, and then retrieve resource packages and their proofs according to the manifest. The node should validate, persist, and apply each item to the index. After the entire scope is completed or every rejection has been explicitly recorded, it may report the synchronization result for that scope.

11.7.2 Incremental Synchronization

Incremental synchronization continues from the persisted synchronization cursor or notification-batch sequence, retrieves publication materials after that position, and processes them in publication order. Duplicate batches and duplicate resource packages should be processed idempotently. A new cursor may advance only after the corresponding items reach an applied or explicitly rejected state.

11.7.3 Synchronization Confirmation

Synchronization confirmation should distinguish at least received, validated, indexed, and externally queryable. Confirmation responses, logs, and database records should associate the target Discovery node, sequence range, successful or rejected items, current cursor, and last error. An HTTP request succeeding cannot substitute for confirmation that content has converged.

11.7.4 Recovery After Interruption

After a node restarts or the network is interrupted, it should recover from the last persisted cursor or incomplete batch, replay necessary materials, and process them according to idempotency rules. If the cursor is invalid, the manifest has changed, or governance status cannot be confirmed, the node should retrieve the manifest again or enter a review-pending state instead of skipping the missing interval.

11.8 Retry, Backoff, and Interruption Recovery

Retries apply only to recoverable transfer, timeout, service-unavailable, or temporary-lock failures. The publisher and Discovery node should retain the task or batch identifier, target cursor, processed items, retry count, and last error, and use backoff to avoid continuously impacting the upstream service. Invalid signatures, hash mismatches, resource-type errors, and authorization-domain mismatches are material or permission failures; they should be recorded and repeated infinite retries of the unchanged payload should be stopped.

A retry record should at least answer three questions: which resource version was retried, whether the failure occurred during retrieval, validation, writing, or result reporting, and what condition the next retry depends on. For batch processing, results should be updated separately for successful and failed items. A successful overall batch response must not cause already successful

items to be applied again, and one failed item must not erase the processing evidence for other items. Repeated delivery of the same task should use the resource DID, version, hash, and publication cursor for idempotency checks.

11.9 Cache Behavior and Expired-Data Handling

11.9.1 Browser or Client Cache

A browser or client cache stores public materials, network snapshots, or resource summaries already retrieved by the page or client. It is not the authoritative publication record of the Root platform. Cache keys should include the interface scope, resource DID, or page-data version and other necessary context. Clearing browser data may remove the cache, but it does not affect the server-side publication fact. When reading cached data, the client should include the retrieval time, source, and freshness marker so that a local hit is not displayed as if it had just been confirmed by the network.

11.9.2 Node-Side Cache

The content distribution platform and Discovery nodes may cache resource packages or indexed read models that have passed the corresponding validation. The content distribution platform caches materials available for propagation, whereas a Discovery node caches query data after local application. Node-side caches should retain the resource version, package hash, publication cursor, and validation time, and replace entries in a verifiable order when updating. A cache must not bypass Root-verified ResourcePackage, resource-package hash, or current governance-status checks.

11.9.3 Preferential Display of Old Snapshots

When new page or query data has not yet been retrieved, the client may first display the most recently saved old snapshot whose source can be identified, while marking the snapshot time, data source, and possible synchronization delay. An old snapshot can improve the waiting experience for ordinary reads, but it is not suitable as the real-time basis for revocation, suspension, authorization changes, or high-risk calls. Snapshot display should be separated from background refresh status; the display time of old data must not be updated to the current confirmation time.

11.9.4 Background Refresh and Failure Degradation

After a background refresh succeeds, the client or node should complete structural, hash, proof, and status checks on the new materials and then atomically replace the old snapshot. When refresh fails, it should retain the old snapshot and record the failure reason, last successful time, and freshness. If the failure concerns revocation or the authorization status cannot be confirmed, the node should reduce the availability state or pause sensitive actions rather than unconditionally continuing to use the old snapshot. Refresh logic must not block basic page rendering or clear valid existing data on failure and thereby create unnecessary blank space.

11.10 Propagation Observability and Audit Records

Propagation audits should associate the resource DID, package version, package hash, publication task, publication cursor, notification batch, target Discovery node, send/receive/validation/application times, retry count, and error reason. The Root platform and publisher should be able to state which items completed content distribution; a Discovery node should be able to state which items were rejected or remain pending synchronization. These records trace the propagation process and do not change the original proof of the resource.

For batch propagation, both the overall batch result and each resource-item result should be recorded. An overall successful response does not mean that every item has been applied, and partial success must not erase the evidence of completed items. During operational troubleshooting, `jobKey`, request ID, and publication cursor can be used to associate Root platform tasks, content distribution platform history, and Discovery node synchronization records. These fields trace propagation and are not equivalent to a resource controller signature or governance authorization.

For batch propagation, both the overall batch result and each resource-item result should be recorded. An overall successful response does not mean that every item has been applied, and partial success must not erase the evidence of completed items. During operational troubleshooting, `jobKey`, request ID, and publication cursor can be used to associate Root platform tasks, content distribution platform history, and Discovery node synchronization records. These fields trace propagation and are not equivalent to a resource controller signature or governance authorization.

11.11 Publication Ordering and Conflict Resolution

Propagation order is based on the Root platform publication cursor and notification sequence. Resource-version selection is based on the resource package version, predecessor-version relationship, package hash, and Root-verified `ResourcePackage`. Duplicate batches containing the same materials should be processed idempotently. If different hashes appear at the same position, an old version attempts to overwrite a new version, or predecessor materials are missing, application of the affected scope should be paused and the conflict recorded. Materials must not be overwritten arbitrarily according to arrival time.

11.12 Distribution Consistency and Freshness Guarantees

Consistency metrics should separately state the scope that the Root platform has published, the scope available from the content distribution platform, and the scope received, validated, and applied by Discovery nodes. Freshness metrics should state the publication or receipt time from which they are calculated and the observation or application time to which they are calculated. Metrics must include the sample scope, failed items, and authorization-domain definition; success by one node or a cache hit cannot represent network-wide convergence.

11.13 Partial Failure and Recovery Semantics

Partial failures are isolated by resource item and target node. Items that have completed validation and application may advance their own state. Failed items should retain independent error, retry, and missing-material information, and must not be skipped because the overall batch response succeeded. During recovery, the node should first fill missing materials, then complete item-level validation and persistence, and finally advance the corresponding cursor or record an explicit rejection. Applied items are not rolled back merely because another item in the same batch failed, and failed items must not be hidden because an old snapshot remains available.

Final convergence requires every item in the target sequence range to reach “applied” or have a clear and traceable rejection record. If hashes, proofs, lifecycle, or authorization domains change during recovery, the corresponding validation process must be entered again. An old snapshot may be used only for limited read-side results marked with freshness information; it must not conceal revocation, suspension, or authorization changes.

11.14 References

Source	Type	Link
oan-root-services	Code repository: publication tasks, content distribution coordination, and publication status	https://github.com/OpenAgenet/oan-root-services
oan-discovery-node	Code repository: propagation-material receipt, validation, and indexing	https://github.com/OpenAgenet/oan-discovery-node
oan-protocol-common	Code repository: resource-package, publication-event, and hash structures	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv yellow paper: trusted publication, propagation, and discovery path	https://arxiv.org/abs/2606.03163
Agentic Overlay Network Architecture	IETF draft: node coordination and overlay-network propagation	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

Chapter 12 Discovery Protocol and Query Semantics

This chapter explains how a discovery service node processes exact DID queries, structured queries, tag queries, text queries, and semantic queries from a local index that has been verified and filtered by the applicable authorization scope. It covers query input, version selection, provenance and freshness, result deduplication, pagination, explanations, errors, and degradation boundaries. A discovery response is explicitly a candidate result produced from a node's index; the caller must still perform pre-invocation verification based on the DID, credentials, root platform trusted publication proof, governance status, and endpoint information.

12.1 Discovery Objectives and Query Types

A discovery service node provides five query paths for resources that have entered its local index: exact lookup by a complete resource DID; structured queries by resource type, version, protocol, and capability tags; tag queries by capability tags; text or natural-language queries based on resource names, descriptions, tags, use cases, and protocol materials; and combined queries that join natural-language and structured conditions. Query results are derived responses from the discovery service node rather than the root platform's publication records. The response should retain the resource DID, resource type, version, lifecycle state, provenance, and necessary verification references.

The current reference implementation accepts a unified query object through `POST /discovery/resources/query`. When the query text is a complete, whitespace-free `did:oa:` identifier, it enters the exact DID branch; other text and explicit fields enter structured, lexical, or semantic paths. The query node constructs candidates only from locally indexed material. It must not directly forward the query to a registration service node or the root platform to manufacture a temporary result.

Before retrieval begins, query input should pass boundary validation and then be converted into an internal query object. `resourceType`, `capabilityTags`, `protocol`, and `version` form explicit conditions; `versionMode` specifies the version selection method; `limit` restricts the response size; and `query` carries a DID or a text/natural-language query. When text and structured fields are supplied in the same request, text performs recall and ranking while structured fields narrow the candidate set. The two kinds of conditions must not override each other.

```
{
  "query": "I need a tool that can search code repositories and summarize the project structure
  .",
  "resourceType": "tool_api",
```



```
"capabilityTags": ["code-repository"],
"protocol": "MCP",
"versionMode": "latest",
"limit": 10
}
```

The query text, tags, protocol, and limit in the example illustrate a combination of fields. The actual resource types and tags must follow the protocol values accepted by the target discovery service node. The `score` in a response candidate is a query-relevance result, not a resource trust level.

12.1.1 Exact DID Discovery

A complete resource DID must be interpreted as an exact lookup condition. It must not degrade into a name, prefix, or semantic-similarity search. The node should read the local index object corresponding to the DID and continue checking explicit conditions, lifecycle state, and the node's authorization scope.

12.1.2 Structured-Condition Discovery

Fields such as resource type, version, protocol, and capability tags should be converted into reviewable filter conditions. Multiple explicit conditions use intersection semantics by default. An invalid enumeration, an unresolvable version mode, or an unsupported protocol value should produce a clear request error.

12.1.3 Capability-Tag Discovery

Capability-tag queries use `capabilityTags` in the resource metadata. Parent-child relationships, aliases, and extension tags in the tag tree may participate in the query projection, but the original tags and their sources must be retained. A tag match does not represent control or invocation authorization.

12.1.4 Text and Semantic Discovery

Text and natural-language queries may use lexical retrieval, vector retrieval, or a hybrid of both. When the semantic index is unavailable, the node may fall back to lexical and structured paths, but the query explanation or diagnostic information should state which backend was actually used.

12.1.5 Combined-Condition Discovery

A combined query should first apply authorization-scope, lifecycle, and explicit structured-condition filters, and then rank the remaining candidates lexically or semantically. Relevance must not remove hard filters or allow unpublished or unauthorized resources to enter a trusted result.

12.2 Exact DID Queries

An exact query first validates the DID method syntax and resource-type semantics, and then reads the resource package from the local index. A hit must use the DID document, metadata, endpoint, hash, and root platform proof reference for the same resource version. A miss must not be interpreted simply as proof that the resource does not exist on the entire network, because it may not yet have propagated to this node or may have been filtered by the authorization scope. The node should distinguish the following results:

Result	Explanation	Caller handling
Candidate returned	This node has a visible indexed object that satisfies the conditions	Continue verifying the DID, proof, and endpoint
Empty candidates	The resource does not exist here, has not been indexed, or is outside this node's authorization scope	Do not infer that it does not exist network-wide
Suspended/revoked	The indexed object exists, but its current state restricts use	Do not invoke it as a currently valid resource
Service error	The database, index, or governance read model is temporarily unavailable	Retry or degrade according to temporary-failure handling

The current exact-query implementation identifies a DID through the `query` field and passes the remaining explicit conditions to resource-type, version, capability-tag, and protocol matching. Therefore, a query containing a DID may still return empty candidates when its version or authorization-scope conditions are not satisfied. An exact hit only means that the discovery service node has a visible index record. The caller must still obtain the complete DID document and proof materials for pre-invocation verification.

The result boundary of an exact DID query can be checked using the following table. Here, “not found” means the result under the current index and authorization scope of this node; it is not a statement about the state of the Internet of Agents.

Query stage	Object to check	Meaning of the result
Input identification	Whether query is a complete did: oan: string	Determines whether the exact path or the ordinary retrieval path is used
Index read	Whether this node has an indexed resource-package record	Reflects only whether this node has stored the record
Condition matching	Explicit conditions such as version, resource type, protocol, and capability tags	Any unsatisfied hard condition can produce empty candidates
Visibility check	This node's authorization scope and the resource lifecycle	An invisible, suspended, or revoked resource must not be returned as an ordinary candidate
Pre-invocation decision	DID document, registration credential, root platform trusted publication proof, and endpoint	Performed by the caller; not replaced by score or an index hit

12.3 Structured Resource Queries

A structured query maps fields in the request to database or in-memory filter conditions. The current reference implementation filters by resource lifecycle, resource type, version, capability tags, and protocol. For an ordinary query without an explicit resource type, it uses a default scope oriented toward resources intended for user consumption. Explicit capability tags require the resource to contain all requested tags. A protocol condition requires the resource service endpoint or protocol binding to declare or explain that protocol. An empty query with structured conditions may be executed, while a query with no conditions at all should be constrained by result count and resource scope limits.

After filtering, results still pass the node's authorization-scope check and latest-valid-version selection. Display names, descriptions, and retrieval scores are index fields; they are not evidence of identity, publication, or governance.

Explicit filters should be applied before candidates enter relevance ranking. The current implementation performs resource-level matching for `resourceType`, `version`, `capabilityTags`, and `protocol`. Capability tags use intersection matching against the requested set; a protocol may match through a protocol binding or a service endpoint declaration. A missing field is not automatically a wildcard, and an unresolvable request value must not be silently converted into another condition.

Structured filtering must be reviewable: request fields, resource fields, and the decision should correspond item by item. For example, if a request specifies both `resourceType = "mcp_server"` and `capabilityTags = ["file.search", "desktop.automation"]`, a candidate must first be of the requested resource type and contain both tags before it participates in text-relevance ranking. A candidate that is semantically close to “file search” but has the wrong resource type must not enter the result through ranking. If a field is absent from the resource package, that condition should be treated as unsatisfied rather than as an arbitrary match.

For unknown enumerations, blank fields, duplicate tags, and conflicting version conditions, the node should define deterministic handling during request parsing. Duplicate tags may first be normalized into one condition; an unknown resource type or an unresolvable version mode should be a request error; and a conflict between version and lifecycle should narrow the result under version-selection and state rules rather than be relaxed by the retrieval score.

12.4 Capability-Tag Queries

Capability tags are declarations by a resource provider that classify the resource's capabilities. The root platform capability-tag tree provides governance context for tag identifiers, hierarchy, aliases, and versions. A discovery node may project aliases in a query to canonical tags or maintain local retrieval aliases, but the response should retain the resource's original tags and the tags that actually matched. Whether a parent tag covers a child tag and whether an extension tag enters the trusted index must be defined by the tag-tree version and node configuration, not implicitly determined by model relevance.

A tag query should distinguish three kinds of information: the resource's original `capabilityTags`; the retrieval terms produced by expanding aliases in query text or explicit parameters; and the tags ultimately used to explain the match. Alias expansion can improve recall, but it must not modify

the resource's original declaration. If an extension tag is not supported by the current node's tag tree or configuration, it should be treated as an ordinary miss rather than inferred to be a parent tag or synonym.

12.5 Text and Semantic Queries

Semantic queries may retrieve public information such as resource names, descriptions, capability tags, use cases, protocol bindings, and resource types. The reference implementation may organize resource material into tag, context, and intent views, and combine vector, lexical, and structured scores to rank candidates. `score` expresses relevance to the current query and index snapshot; it does not express a trust level, service quality, or security level.

When the semantic index, embedding service, or pgvector is unavailable, the node may fall back to lexical and structured queries. Degradation must not relax authorization-scope, lifecycle, or explicit conditions. Empty text should not force a semantic backend call. Input that is too long, cannot be encoded, or uses an unsupported language should produce a clear error or an explainable degraded result.

The current semantic implementation projects resource type, protocol, capability tags, and query text into retrieval terms, and constructs search documents from public material such as resource names, descriptions, tags, context, examples, and use cases. When the semantic backend is available, results may combine BM25-like lexical scores, tag scores, context scores, intent scores, and structured scores. These internal scores are used for ranking and explanation and must not be treated by external callers as a governed quality assessment.

Text used to build a semantic index must come from resource description material that has already been made public. Index construction should retain the association among the resource DID, resource-package version, and semantic source content so that the corresponding index can be rebuilt after a resource update and old descriptions do not continue to produce highly relevant results. Multilingual queries may improve recall through normalization, aliases, or embedding models, but a model's inferred capability must not be written back into resource metadata, and model similarity must not replace resource control, a registration credential, or the root platform trusted publication proof.

A discovery node should be able to distinguish the semantic backend, lexical path, and structured path through runtime logs or a diagnostic interface. For the same hard conditions, a semantic-service failure may change candidate recall and ranking quality only. If the index database, authorization-scope read model, or resource-package read fails, the failure must be handled as unavailable data or a service error rather than returning an apparently normal but incomplete empty result.

12.6 Combined Semantic and Structured-Condition Queries

A combined query can be abstracted into the following processing order:

```
visible = authorizedDomain AND active AND explicitFilters
ranked  = rank(queryText, visible)
result  = stableSort(ranked)[0:limit]
```

Here, `explicitFilters` includes resource type, version, protocol, and explicit capability tags; `rank` may use lexical, vector, or hybrid retrieval; and `stableSort` must retain a stable secondary key. If no resource satisfies the hard conditions, the node should return empty candidates rather than a “similar” resource that does not satisfy them.

An important boundary of combined queries is that semantic results must still pass resource-package validity, lifecycle, and authorization-scope checks at the node. A record in the semantic index does not mean that it is visible at the time of the query. For implicit natural-language requirements such as “latest”, “trusted”, or “callable”, the node may convert the portion that maps to protocol fields into structured conditions. A portion that cannot be mapped to fields may be used only as ranking or explanation information; it must not automatically produce an authorization conclusion.

A minimal reproducible example of combined-query processing is:

```
{
  "query": "I need a tool that can search code repositories and summarize the project structure",
  "resourceType": "tool_api",
  "protocol": "MCP",
  "capabilityTags": ["code-repository"],
  "versionMode": "latest",
  "limit": 10
}
```

A qualifying result for this request must satisfy the resource type, protocol, and capability-tag conditions simultaneously. Natural language only supplements relevance judgments over names, descriptions, use cases, and related fields. If the query text expresses “trusted” or “callable”, the node may make the corresponding decision only when the relevant structured fields and verifiable facts exist. Otherwise, it should preserve the phrase as retrieval intent and leave the subsequent decision to the caller's verification stage.

12.7 Authorization-Scope Filtering

A discovery node obtains its own authorization scope from its DID document or governance read model and compares it with `authorizedDomains` in resource metadata. The resource domain must be covered by a domain for which the node is authorized. A domain mismatch must keep the resource out of the query result. This filter determines discovery visibility provided by this node to the caller; it does not change the resource DID control relationship or mean that the resource is invisible at other discovery nodes. When governance status or node authorization cannot be confirmed, ordinary display should mark the result as not fresh, while sensitive queries and pre-invocation verification should apply restrictive handling.

12.8 Latest Valid Version Selection

By default, a query selects, for each resource DID, the latest version that satisfies the node's authorization scope, has an active lifecycle state, has completed the necessary validation, and has

a confirmable synchronization position. Version selection for the same DID should consider the resource-package version, root-platform publication cursor, package hash, predecessor relationship, and governance state. It must not rely only on update time or query arrival order. If a discovery node is behind, it may return a verified older version for limited display, but it must include freshness and synchronization position and must not claim that version to be the network latest.

Version selection must verify at least three conditions: the candidate belongs to the same resource DID; its version satisfies the requested `versionMode` and `version` conditions; and its lifecycle and governance state still allow discovery. For a `latest` query, the selected item is the latest currently verifiable valid version, not the record most recently written to the database. For a specified-version query, a missing version record, inconsistent hash, or unconfirmable state should produce an empty result or a clear data-unavailable error; it must not silently substitute another version.

A reproducible check can use two records with the same DID: record A has an earlier publication sequence but an active state, while record B has a later update time but is suspended. A conforming implementation should select record A or return a restricted result; it must not treat B as the latest valid version merely because its timestamp is newer. The check should also verify that repeated queries over the same index snapshot return the same resource DID, version, and state, while retaining the publication sequence, package hash, and state source as decision evidence.

12.9 Search Ranking and Deterministic Result Ordering

12.9.1 Matching Conditions

Matching conditions are the first basis for candidate ranking. Explicit conditions such as resource type, version, protocol, and capability tags are hard filters. Text matches in names, descriptions, use cases, and protocol materials are relevance signals. A candidate must satisfy explicit conditions before it participates in relevance ranking. A resource that is textually similar but has the wrong resource type or version must not enter the result.

12.9.2 Trust and Governance State

Only candidates with completed root-platform trusted publication, valid governance state, and confirmable node authorization may enter a trusted discoverable result. Trust and governance state are not ordinary features that merely increase a retrieval score; they determine whether the candidate is eligible to be returned. A candidate with unknown state, missing proof, or suspended or revoked status should be filtered out or returned with an explicit restricted state. High relevance must not conceal these conditions.

12.9.3 Version and Freshness

By default, only the current valid version of a resource DID is retained. When several candidates have similar relevance, the node should prefer the newer valid version whose publication sequence and governance state can both be confirmed. Freshness describes how far the index is from publication and governance facts; it must not be equated with retrieval relevance. An old snapshot may be used for limited display, but it must carry `stale` or an equivalent state.

12.9.4 Ranking Stability

Under the same index snapshot, authorization scope, and query conditions, ranking should produce repeatable results. Ranking first uses the actually enabled relevance combination and then uses a deterministic resource DID, version, publication sequence, or other stable key as a secondary key. The node should not use database natural order, concurrent completion order, or random perturbation as an implicit ranking rule; otherwise pagination and client-side deduplication cannot work reliably.

12.9.5 Duplicate-Result Handling

Resource-identity deduplication must occur either before or after ranking; duplicate synchronization records for the same DID must not occupy multiple result positions. Deduplication retains the current valid version, the newer publication sequence, and the record with more complete verifiable evidence, while retaining the source node and duplicate diagnostics. If duplicate records have conflicting hashes or governance states, they must enter exception handling rather than being arbitrarily selected as a trusted result.

12.10 Search Explanations and Evidence Display

12.10.1 Match Reasons

Match reasons should point, as far as possible, to observable fields such as capability-tag intersection, resource-type match, protocol match, text-field match, or natural-language intent match. An explanation may be a summary and need not expose the internal model or index implementation, but it must not invent a capability that the resource did not declare. When a query uses a lexical, vector, or hybrid path, the response should identify the actual path or at least state the determinacy boundary of the explanation.

12.10.2 Resource Provenance

The result should identify the discovery service node, index update time, resource source node or source reference, and available publication sequence and resource-package hash. Provenance fields trace where a fact was synchronized from; they do not mean that the source node guarantees the resource's business quality. When the resource is obtained through a content distribution platform, the response should retain the reference relationship to the root platform publication material rather than recording only a cache address.

12.10.3 DID, VC, and Root Platform Proof

The resource DID identifies the resource; the DID document provides public keys, service endpoints, and resource metadata; the registration credential expresses the registration service node's processing of a registration fact; and the root platform trusted publication proof expresses the root platform's verifiable validation and publication fact for a particular resource version and its materials. A discovery response may return summaries or references to these pieces of evidence, but

before connecting, the caller must obtain and verify the complete materials. No reference field alone constitutes completed verification.

Response fields should be understood in two layers: “index display” and “verifiable evidence”. `resourceDid`, `resourceType`, `score`, `version`, `tags`, and `endpoints` are candidate-display information. The DID document, registration credential, root platform trusted publication proof, and their signature materials are the basis for the caller's verification of identity, control, and publication facts. A discovery node returning a candidate does not mean that it has performed endpoint connectivity testing, business compatibility assessment, or authorization on behalf of the caller.

12.10.4 Governance State and Version Information

The explanation area should display the resource lifecycle, resource version, publication sequence, governance state, and index freshness together, allowing the caller to distinguish “highly relevant” from “currently verifiable”. Governance state comes from a governance-event projection or another explicit fact source, while version information comes from the resource publication chain. When they conflict, the response should mark the conflict and apply restrictive handling; a newer display time must not override a revocation or suspension fact.

Search explanations should keep verifiable facts and retrieval inferences separate. `matchReason`, tag intersection, resource-type match, and protocol match are query explanations. The DID document, public key, registration credential, root platform trusted publication proof, publication sequence, and governance state are verification materials. The former answer “why did this appear in the result?”; the latter answer “which facts can the caller use for the next decision?” They must not be merged into one recommendation score.

12.11 Discovery Response Model

A discovery response should allow the caller to identify at least four things: which discovery service node processed the query, which resource DID corresponds to each candidate, which index state produced the candidate, and why the candidate was returned. A reference implementation response may contain top-level fields such as `discoveryDid`, `candidates`, `createdAt`, and `proof`. A candidate should include the resource DID, resource type, retrieval score, version, lifecycle state, capability tags, authorization domains, service endpoints, protocol bindings, resource-package information, and root platform proof reference. A deployment may add fields, but display fields must not be mistaken for proof fields.

```
{
  "discoveryDid": "did:oa:...",
  "createdAt": "2026-09-07T10:00:00Z",
  "candidates": [
    {
      "resourceDid": "did:oa:...",
      "resourceType": "skill",
      "score": 0.87,
      "version": "1.0.0",
      "lifecycle": "active",
```



```

    "capabilityTags": ["code-repository", "project-structure"],
    "serviceEndpoints": ["https://example.invalid/endpoint"],
    "protocolBindings": ["MCP"],
    "rootProofRef": "...",
    "freshness": {
      "publicationSequence": 42,
      "indexUpdatedAt": "2026-09-07T09:59:30Z"
    }
  }
]
}

```

`proof` or `rootProofRef` means only that the response or candidate refers to proof material. The caller must still obtain and verify the corresponding content according to Chapters 7 and 9. When a field is missing, the node should explicitly return “unavailable” or omit the field; it must not insert an unverified default value. Unknown extension fields should be safely ignorable by compatible clients, while core identity, state, and provenance fields must not be renamed in a way that changes their meaning.

The minimum response skeleton corresponding to the current protocol structure is `discoveryDid`, `candidates`, `createdAt`, and optional `proof`. In a candidate, `resourceDid`, `resourceType`, `score`, `version`, `lifecycleState`, `capabilityTags`, `authorizedDomains`, `services`, `protocolBindings`, `packageInfo`, and `rootProof` respectively provide display responsibilities for identity, retrieval, state, capability, scope, service, and proof references. The current interface may not return a separate `matchReason` or a complete freshness object. A client must not fill absent fields with facts that have supposedly been verified.

The current discovery-query entry point is `POST /discovery/resources/query`, and its request body corresponds to `ResourceDiscoveryQuery`. This type may contain `query`, `resourceType`, `capabilityTags`, `protocol`, `version`, `versionMode`, and `limit`; `limit` defaults to 10 and `versionMode` defaults to `latest`. The response corresponds to `ResourceDiscoveryResponse`, whose candidates use `ResourceDiscoveryCandidate`. The returned `score` is a relevance score, not an authorization or trust level.

`POST /discovery/query/explain` also accepts `ResourceDiscoveryQuery`, but returns dynamic explanation JSON, including matched resource DIDs, resource types, match flags, scores, tag overlap, resource-type and protocol match status, and semantic-search status. The explanation response is for diagnosing the retrieval process. It is not an ordinary discovery result and cannot replace verification that the resource control relationship or publication proof is valid.

Field names in the current public protocol should be treated as the compatibility baseline for clients. When a resource candidate uses fields such as `lifecycleState`, `services`, `packageInfo`, and `rootProof`, clients should treat optional information according to field presence. Fields such as `lifecycle`, `serviceEndpoints`, `rootProofRef`, or `freshness` in examples are illustrative extensions; they must not be used to claim that all current deployments return those names. New response fields should preserve the ability of older clients to ignore unknown fields and should describe the source and verification method of each fact.

12.12 Discovery-Result Freshness and State

The freshness of a discovery result is described jointly by the index update time, the processed publication cursor, the governance-event sequence, and the local error state, rather than by the HTTP response time alone. A response should provide, where possible, `createdAt`, index update time, publication sequence or synchronization cursor, governance-state read position, and source-node DID. The caller uses these values to determine whether the result is a current snapshot, an acceptable old snapshot, or a partially available result.

State	Meaning	Permitted use
current	Necessary publication and governance-state synchronization is complete, with no known update gap	May proceed to pre-invocation verification
stale	A verifiable local snapshot remains, but publication or governance synchronization is behind the permitted range	Read-only reference with an explicit freshness mark; state must be reconfirmed before invocation
partial	Some resources or index shards are available, and the completeness scope has been reduced	Use only within the scope stated by the response; do not claim a full result set
unknown	Synchronization position, governance state, or proof freshness cannot be confirmed	Must not be used as a trusted basis for sensitive actions

When version state and governance state conflict, restrictive states such as revocation, suspension, or loss of node authorization take precedence over relevance and time ordering. A discovery service node may retain historical snapshots to improve access during network failures, but it must mark the snapshot state explicitly and must not describe a cache hit as the network latest state.

State judgment for a discovery result should be based on local index material and its synchronization records. `createdAt` represents only the time when the response was generated; it is not the resource publication or index-application time. The publication cursor represents propagation position, the index update time represents processing time at this node, and the governance-event sequence represents the position of the governance read model. The caller must compare these sources and positions to determine whether the result may be behind.

An error response should allow the caller to distinguish a request error, a resource miss, a temporarily unavailable index, and a temporary backend failure. An invalid DID format, an unknown resource type, an unresolvable version mode, or an excessive query limit is a request-side error. A legal query with no visible candidate is an empty result. A database, semantic-index, governance-read-model, or upstream-synchronization failure is a service- or data-side error. The error should contain a machine-readable category, a request identifier, and necessary retry guidance, but should not disclose detailed information about restricted resources.

```
{
  "error": {
    "code": "INDEX_STALE",
```

```

    "message": "The discovery index is temporarily behind the publication cursor.",
    "requestId": "...",
    "retryable": true
  },
  "freshness": {
    "status": "stale",
    "lastKnownPublicationSequence": 41
  }
}

```

When the semantic backend is unavailable, the node may fall back to lexical and structured paths. The fallback changes ranking capability only; it must not relax resource type, lifecycle, authorization scope, version, or explicit tag conditions. If the base index itself is unavailable, the node must not use an empty array to pretend that there are “no resources”. It should return an identifiable service error or a partial result with an explicit scope. The client should retry based on `retryable`, backoff guidance, and its own request budget; the node must not amplify requests through automatic retries.

Error handling should preserve the boundary between “invalid request” and “data temporarily unavailable”. A format error, unknown enumeration, or excessive parameter must not cause the same request to be retried. A temporary semantic-backend failure may use lexical degradation under the same hard conditions. If the base index or governance state cannot be read, a partial result must state its scope and state explicitly. A discovery node may record query statistics and backend names for operations, but its public error must not disclose the existence of resources outside the authorization scope.

Error handling for the discovery interface should keep the response structure stable and allow clients to decide whether to correct the request, wait and retry, or stop using the result. The following boundary may be used:

Error category	Typical cause	Client action
Invalid request	DID format, resource type, version mode, or <code>limit</code> is invalid	Correct the request and issue it again; do not repeat the original request
No visible result	The conditions are valid, but this node has no candidate that both satisfies them and is visible	Adjust the query conditions if appropriate, but do not infer that the resource does not exist network-wide
Semantic degradation	The semantic index or embedding service is temporarily unavailable while the base index remains readable	Accept lexical/structured results and lower expectations for relevance
Index unavailable	The resource index, database, or necessary governance read model cannot be read	Handle as a service error, wait for recovery, or switch to a known node
Synchronization lag	The local publication cursor is behind or the snapshot is incomplete	Use only within the scope marked by the response and retry later if necessary

Semantic degradation and base-index failure must be distinguished in implementation and operational observation. The former can usually still return candidates satisfying hard conditions, with reduced relevance-ranking capability. The latter cannot establish that the candidate set is complete or that visibility was correctly determined and must not be disguised as an ordinary empty result. Error logs may record the backend, latency, candidate count, and failure reason, while the public response should contain only the minimum information needed for client handling.

12.13 Discovery Errors and Degraded Responses

The fact that a resource is reachable through an HTTP endpoint means only that the endpoint is reachable for the current request. Entering a discovery index means that a discovery service node has stored a corresponding index record. Having a root platform trusted publication proof means that the root platform has formed a verifiable publication fact for a particular version and its materials. Passing pre-invocation verification additionally requires the caller to recheck the DID control relationship, proof, governance state, endpoint, and its own business policy. These are progressively stronger facts; no level automatically replaces another.

Fact	Primary source	Conclusion that cannot be drawn
Endpoint is reachable	Resource service endpoint	The resource is registered or trustworthy
Entered a discovery index	Discovery service node index	The resource is network-latest or permitted for invocation
Has a root platform trusted publication proof	Root platform publication material	The business endpoint is necessarily available or suitable for a particular scenario
Passed pre-invocation verification	Caller and its verification components	The caller has obtained all business permissions

A public discovery response may display the necessary DID, resource summary, endpoint, and proof reference. For a restricted resource, it should return only the minimum summary within the authorized scope. Before establishing a connection, the caller should obtain or verify the complete DID document, registration credential, root platform trusted publication proof, latest governance state, and endpoint declaration according to the risk of the resource.

Public visibility is jointly determined by the current authorization scope and index state of the discovery service node. A resource absent from public results may not yet be synchronized, may have been filtered, may be suspended, or may be affected by an unavailable index. An empty result alone cannot establish that the resource does not exist. When the caller needs a deterministic conclusion, it should use an exact DID query, read state information, and perform complete pre-invocation verification instead of inferring a hidden inventory from tag enumeration or paginated results.

Visibility involves at least three independent questions: whether the resource has been registered, whether it has propagated to this node, and whether the current caller can see it. A discovery service node can answer only from locally verifiable index material and authorization scope. Therefore, “candidate returned” means that the resource has a discovery record within the visible scope of

the current node, while “not returned” means only that the current query did not produce a visible candidate. For sensitive resources, success, empty results, and permission filtering must not reveal additional information through error codes, field counts, or response latency.

Trusted discoverability also requires the candidate to have a DID document, registration credential, or root platform trusted publication proof reference corresponding to the current version, with no known suspension, revocation, or authorization-loss state. The discovery node provides an index and an evidence entry point. Whether the caller accepts the resource also depends on its own risk policy and its verification of control, proof validity, endpoint security, and business authorization.

12.14 Public Visibility and Trusted Discoverability

12.14.1 DID-Level Deduplication

Within one query response, the same resource DID should normally have only one representative record. The representative should be a current valid version with a newer publication sequence, while retaining a traceable source node, root platform proof reference, and synchronization position. If governance states conflict between records, the response should retain conflict diagnostics and use a restrictive state; arbitrary ordering must not hide the conflict.

12.14.2 Version-Level Deduplication

Different valid versions of the same DID should be returned together only when the query explicitly requests historical versions or specifies a version constraint. A default query returns the current valid version. A historical query should include the version identifier, predecessor version, and publication sequence so that the client cannot mistake a historical record for the current version. The same version identifier with different package hashes is a consistency anomaly; the records should not be merged and should enter diagnostic or quarantine handling.

12.14.3 Duplicate Records from Registration Nodes

Repeated submission of the same resource to one or more registration service nodes must not create multiple resource identities. A discovery node should deduplicate according to the root platform publication fact, resource DID, version, and package hash. A registration service node's local acceptance record may be retained for source auditing, but it must not by itself create a public candidate. Whether a repeated submission causes a new registration credential to be issued is a registration-protocol behavior and does not change discovery-side resource-identity rules.

12.14.4 Handling Identical Content with Different DIDs

Records with the same content, name, or endpoint but different DIDs remain different resource identities and must not be merged solely because their content hashes match. A node may indicate content similarity or a shared package hash in an explanation field, but it must display each control relationship, registration state, version, and root platform proof separately. Whether the same con-

tent may be represented by multiple DIDs is determined by the resource controller and governance rules.

The primary deduplication key is the complete resource DID. Records from multiple registration service nodes, content distribution platforms, or synchronization batches are normally the same resource identity when they point to the same resource DID. If records also contain versions, the resource DID and version identify the version. Package hashes, root-platform publication sequences, and source nodes are used to confirm content consistency and retain evidence; records must not be incorrectly merged merely because they have the same name or similar descriptions.

A deduplication decision should retain the “basis for merging” and the source of each discarded record so that the reason a candidate appears only once can be explained. A default query may use the following priority when selecting a representative record: first exclude records with restricted lifecycle states, mismatched authorization scopes, or unverifiable proofs; then compare the requested version mode and publication sequence; finally use package hash, source node, and deterministic DID order to resolve ties. This priority only constructs the discovery result. It does not change the original acceptance records of a registration service node or delete historical audit material.

Record relationship	Discovery-result handling	Evidence that must be retained
Same DID, same version, same package hash	Merge into one candidate	Source nodes and synchronization batches
Same DID, different versions	Select or return separately according to versionMode	Version, publication sequence, and predecessor relationship
Same DID, same version, different package hashes	Do not automatically merge into a trusted record	Conflicting hashes, sources, and diagnostic state
Different DIDs, identical content	Return as different resources	Each DID, control relationship, and proof

Deduplication must occur before pagination. Otherwise, duplicate records for the same resource consume multiple result positions and cause omissions when the client merges pages. Historical-version queries are an exception: they may return multiple versions of the same DID, but every version must carry an explicit version and state so that the client cannot mistake a historical candidate for the current valid version.

12.15 Result Deduplication and Resource-Identity Matching

A node should limit query-text length, the number of structured conditions, the number of results returned per request, and semantic-retrieval resource consumption. **limit** must be validated against a server-side maximum; a client must not bypass the limit through repeated requests or abnormal parameters. An invalid, expired, or query-mismatched cursor should produce a clear error rather than silently returning the first page, so that the client does not mistake an incomplete pagination result for a complete one.

Within one index snapshot, pagination must use the same filtering, deduplication, and ranking rules. Ranking should include at least the retrieval score and a deterministic resource DID or other stable key. It must not use a random value per request, undefined database natural order, or unstable concurrent completion order. If the snapshot changes during pagination, the response

should communicate this through a snapshot identifier, cursor invalidation, or freshness field so that cross-snapshot duplicates or omissions are not mistaken for a protocol error.

Situation	Recommended handling
Results are fewer than <code>limit</code>	Return all visible candidates that satisfy the conditions and indicate that no more results are available
Results reach the limit	Return a stable pagination cursor and current snapshot information
Query exceeds length or resource budget	Return a non-retryable request error
Cursor cannot be parsed or does not match the query	Return a cursor error and require the client to start the query again
Index updates during pagination	Continue using the original snapshot or explicitly invalidate the cursor

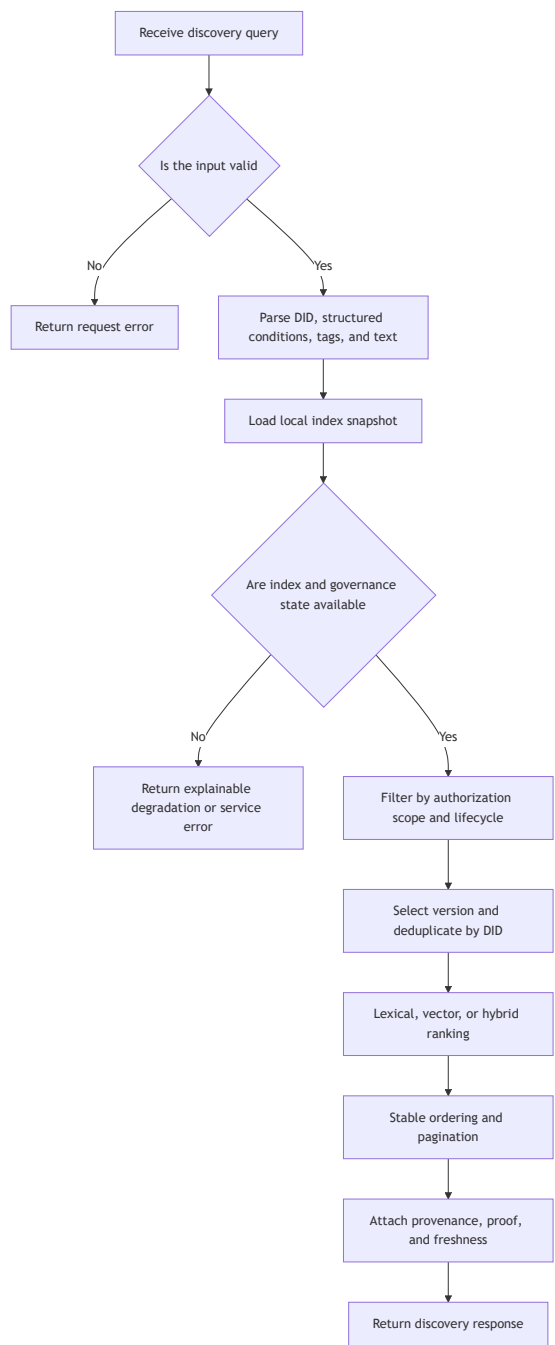
Pagination solves only transport and display scale. It does not change authorization scope or trusted-discoverability decisions. When merging pages, a client should use resource DID and version as deduplication keys and retain the source and snapshot information for each page.

12.16 Query Limits, Pagination, and Result Stability

Authorization-scope filtering must be completed before results are returned, rather than returning candidates first and relying on the client to hide them. For resources outside the requester's or node's authorization scope, the node must not disclose their existence through names, tags, endpoints, proof references, or error differences. A management interface that needs to distinguish insufficient permission from a missing resource may provide more detailed reasons to an authorized administrator. A public query should by default return only the minimum necessary fields of resources that are published, discoverable, and within this node's service domain.

Index material should be classified by field sensitivity. Resource DID, resource type, and a public capability summary may enter public results. Private endpoints, internal tags, sensitive configuration other than the control public key, the submitter's private identity information, and unpublished proof material must not be exposed through semantic indexing. Vector indexes, lexical indexes, and query logs must follow the same authorization boundary; a retrieval backend's need for text does not justify copying content that should not be public.

Query logs should minimize storage of original natural-language input, caller identifiers, and filter conditions, and should use the deployment's privacy policy to define retention, access control, and redaction. A requester must not infer the complete inventory of restricted resources by enumerating DIDs, tag combinations, pagination cursors, or error responses. During propagation of authorization-scope changes, resource revocation, or node suspension, the discovery node should restrict visibility first and retain the state source and processing time for audit.



This flow expresses the order of processing responsibilities: input validation and hard filtering precede relevance ranking; version selection and deduplication precede pagination; and provenance, proof, and freshness are returned with the response. Use of a semantic backend provides ranking capability and must not change the preceding identity, authorization, or lifecycle decisions. Every degradation path must state which validations it completed and which validations it did not complete.

Check item	Conformance criterion
DID query	A complete resource DID uses the exact path and does not degrade into fuzzy search
Structured filtering	Resource type, version, protocol, capability tags, and lifecycle conditions use explicit intersection semantics

Check item	Conformance criterion
Authorization scope	Unauthorized resources do not appear in public results; an unknown scope state is handled restrictively
Relevance explanation	The retrieval score expresses relevance only and is not called a trust level or invocation permission
Version selection	The default returns the current valid version; historical-version queries do not confuse historical state with current state
Deduplication	The same DID does not create duplicate candidates through multi-node synchronization or repeated registration
Response provenance	The result can point to the discovery service node, index state, publication sequence, and proof reference
Freshness	Old snapshots, partial results, and unknown governance state are explicitly marked
Error degradation	Semantic-backend failure does not relax hard conditions, and base-index failure is not disguised as an empty result
Pagination stability	Ordering is stable within one snapshot, and the cursor is bound to the query and snapshot
Privacy boundary	Queries, indexes, and error responses do not disclose sensitive information about resources outside the authorization scope
Pre-invocation verification	The discovery result is only candidate input; the caller re-verifies identity, proof, and business permissions

12.17 Discovery Privacy and Authorization-Scope Visibility

Authorization scope is the boundary of discovery visibility, not a substitute for resource control or invocation permission. A discovery service node should first determine whether a candidate is visible by comparing the node's authorized service domains with `authorizedDomains` in the resource package, and then perform structured and semantic processing only over visible candidates. If the resource and node domains do not intersect, the candidate must be filtered before the public response. A caller must not infer detailed information about a filtered resource from its name, tags, endpoint, result count, or error differences.

Indexing and display should follow minimum disclosure. A public candidate may provide the resource DID, resource type, public capability tags, necessary service endpoints, version, and proof references. Control private keys, the registrant's local identity backup, internal governance materials, unauthorized endpoints, and configuration outside the public resource package must not enter the index or query response. Text, tag aliases, and vector representations used by semantic indexing inherit the authorization boundary of the original fields; entering a retrieval backend must not expand visibility.

While authorization scope, lifecycle, or governance state is synchronizing and the local node

cannot confirm the latest state, public visibility should be restricted. For management and audit purposes, a controlled interface may record the filtering reason, source node, governance-event sequence, and processing time. For ordinary discovery requests, it should return only a uniform result that does not disclose the existence of a restricted resource. Query logs should also be retained according to minimum-necessary principles, with limits on access to and retention of the original natural-language input, caller information, and query conditions.

Information category	May enter a public discovery result	Handling requirement
Resource identity	Resource DID and resource type	Bind to the currently visible version
Capability description	Public capability tags, name, and summary	Must not expand into a capability the resource did not declare
Service information	Service endpoints and protocol bindings within the authorized scope	The endpoint must still be verified before invocation
Trust material	Necessary reference to the root platform trusted publication proof	Do not treat a reference as completed verification
Private material	Private keys, local identity backup, internal configuration, and unpublished proof	Must not enter public indexes, responses, or error details

12.18 References

Source	Type	Link
oan-discovery-node	Code repository: synchronization, indexing, structured queries, and semantic queries	https://github.com/OpenAgenet/oan-discovery-node
oan-protocol-common	Code repository: discovery request, response, and query types	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	Code repository: discovery client and query-result handling	https://github.com/OpenAgenet/oan-sdk-ts
GRAIL Semantic Discovery Paper	arXiv paper: hybrid semantic retrieval and discovery explanations	https://arxiv.org/abs/2605.02489
Efficient Agent Discovery Profile	IETF draft: agent-discovery metadata and query configuration	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Chapter 13 Semantic Governance and Semantic Discovery

This chapter explains how OpenAgenet (OAN) governs capability tags, aliases, authorization domains, and semantic configuration, and how it uses semantic information such as resource names, descriptions, use cases, and protocol bindings for registration recommendations, structured filtering, text retrieval, semantic ranking, and result explanation. It also draws a clear boundary between semantic relevance and identity, control, publication proof, lifecycle, and governance status, ensuring that semantic discovery improves resource matching without changing trusted publication or authorization decisions.

13.1 Semantic Model and Capability Vocabulary

In OpenAgenet (OAN), the semantic model separates what a resource “can do” into several types of information that can be reused by registration and discovery: resource names and descriptions provide natural-language context, capability tags provide relatively stable classification keys, use cases describe applicable tasks, protocol bindings describe interaction methods, and resource types describe object categories. These elements jointly participate in recommendation and retrieval, but they cover different kinds of facts. The capability tag tree answers “which capability category does this term belong to,” the resource package answers “which capabilities does a specific resource declare,” and the DID Document and credential materials answer “who controls the resource, whether it has completed registration, and whether the publication fact can be verified.”

Therefore, semantic matching results should be understood as candidate discovery clues rather than trusted conclusions. A Registrar node may generate tag recommendations based on names, descriptions, use cases, and protocols, and the provider may confirm them before they are written into Resource Metadata. A Discovery node may reuse these fields for filtering, lexical retrieval, or semantic ranking. No matter how high the recommendation score is, it cannot replace control verification, registration credentials, Root proof, lifecycle status, or authorization-domain checks.

13.2 Root Platform-Managed Capability Tag Tree

The capability tag tree is published by the Root platform together with its version and snapshot for use as shared semantic material by registration recommenders and Discovery nodes. The tag tree object in the current public implementation contains `version`, an optional `snapshotHash`, and `tags`; each tag node contains at least a stable `id` and a display `label`, and may also contain `aliases` and

recursive `children`. The following is a minimal example organized according to this data structure; all values are illustrative placeholders:

```
{
  "version": 1,
  "snapshotHash": "sha256:example",
  "tags": [
    {
      "id": "code.repository.search",
      "label": "Code Repository Search",
      "aliases": ["Search Codebases"],
      "children": []
    }
  ]
}
```

The structure of the tag tree organizes semantics and makes queries reproducible; it is not a resource authorization list. After the Root platform adds or adjusts tags, registration recommendations may use the new version, and Discovery nodes may rebuild their indexes accordingly. The original descriptions and original tags submitted by existing resources must not be silently rewritten because the tag tree changed. If a node uses an older snapshot, it should record the version used so that differences in recommendations or discovery results can be explained.

13.2.1 Tag Tree Organization

The capability tag tree uses the organization of “stable identifier + parent-child relationship + display name + vocabulary version.” A tree node expresses a capability category; it does not directly express whether a resource has been registered, published, or trusted. A resource may be associated with multiple leaf nodes or only with a higher-level node, but the Registrar node should store the tags actually selected by the resource rather than only the recommended path.

```
{
  "id": "code.repository.search",
  "parent": "developer.tools",
  "labels": {"zh-CN": "代码仓库检索", "en-US": "Code Repository Search"},
  "vocabularyVersion": "v1"
}
```

13.2.2 Tag Identifiers and Hierarchical Relationships

The tag ID is a cross-node association key. Parent-child relationships organize the retrieval scope, but capability inheritance must not be inferred without vocabulary rules. Query results should retain the matched tag ID, matching path, and vocabulary version so that cross-node results can be reproduced rather than returning only localized names.

Information	Purpose	Can it serve as trusted evidence
Tag ID	Cross-node association and stable queries	No
Parent-child path	Classification and filtering	No
Tag alias	Query expansion	No
Resource control proof and publication proof	Control and publication facts	Yes, subject to independent verification

13.2.3 Tag Recommendation and Discovery Use

During registration, recommendation results enter Resource Metadata only after confirmation by the resource provider. During discovery, the same tag ID, aliases, and hierarchical paths may be used for filtering or ranking. Recommendation hits, tag hits, and trusted-resource verification should be expressed separately in the response.

```
Resource description -> Recommended candidates -> Provider confirmation -> Written to
    ResourcePackage
                                                    -> Discovery index
Query text -> Intent and alias parsing -> Tag conditions -> Candidate ranking
```

13.2.4 Tag Tree Version Management

When a tag tree is published, it should have a comparable version identifier and effective time. Tags from an older version may retain parsing mappings during a migration period, but the original resource declarations must not be silently rewritten. Until index rebuilding is complete, query results should identify the vocabulary version used.

13.3 Tag Hierarchies, Identifiers, Aliases, and Localization

A tag object should at least distinguish its normative identifier, display name, aliases, language, and version:

Field	Example	Processing rule
<code>id</code>	<code>code.repository.search</code>	Stable; does not change with translation
<code>label</code>	Code Repository Search	Used for display
<code>aliases</code>	Search Codebases	Used for query expansion
<code>language</code>	en-US	Identifies the language of the text
<code>vocabularyVersion</code>	v1	Used for reproduction and migration

An unknown tag may be retained as ungoverned provider text, but it must not be presented as an official Root platform tag.

The responsibilities of tag IDs, display names, aliases, and hierarchical paths should remain separate. A query may first use a localized name or alias to find the normative ID and then use

the normative ID for structured filtering; the returned result should retain both the actual resource declaration and the matched normative tag. This supports Chinese, English, and synonym queries without treating a display translation as a new capability declaration.

13.4 Tag Selection and Recommendation During Registration

13.4.1 Resource Provider Selection

The resource provider may accept, cancel, or modify recommended tags. The final submission should retain both the provider's original description and the selected normative tags so that it remains possible to explain “what the resource declared” and “what the system recommended.”

13.4.2 Root Platform Recommendation

The Root platform or its governance materials provide the tag tree, aliases, and version. A recommender generates candidates according to the resource type, name, description, use case, and protocol. The recommender does not issue a resource control proof and does not replace registration validation.

13.4.3 Custom Tag Supplement

Custom tags express business terms that are not yet covered by the official vocabulary. A custom tag must carry a source or namespace and must be stored separately from official tags. It may participate in text retrieval, but it cannot directly change the resource type, authorization domain, or publication status.

13.4.4 Tag Validation and Persistence

Before saving, the system should at least validate the tag ID, vocabulary version, parent-node relationship, language, duplicates, and compatibility with the resource type. The validation result should be associated with the resource DID, resource version, and ResourcePackage digest. A recommendation failure or an expired tag must not cause a silent rewrite.

13.5 Custom Tags and Provider Descriptions

Custom tags are suitable for specific business terms, organization-specific names, or capabilities that have not yet entered the public vocabulary. They should be stored separately from official Root platform tags and should carry provider namespace or source information. Names, descriptions, use cases, and other free text should also retain the original text and language marker. An official tag ID must come from the currently available tag tree; a custom vocabulary must not be promoted to an official tag merely because its format or semantics appear similar.

A Discovery node may use custom tags and provider descriptions for lexical or semantic retrieval, but the result explanation must distinguish among “provider declaration,” “official tag hit,”

and “retrieval-model inference.” Duplicate tags, excessively long text, invalid characters, tags that clearly conflict with the resource type, and keywords intended to manipulate ranking should be rejected, truncated, or degraded to ordinary text according to the node's input validation and indexing strategy. The processing result should retain a traceable reason and must not overwrite the original submission.

13.6 Semantic Governance Change Process

Semantic governance changes should be expressed through new versioned materials rather than by directly rewriting old tags or historical resources. A change record should at least identify the change type, affected tag IDs, old and new versions, alias or hierarchy mappings, effective time, and the effects on registration recommendations and discovery indexes. Additions and renamings can usually be made compatible through new mappings; merges and splits require an explicit explanation of how old tags are queried; deprecated tags should stop being recommended for new resources while the original values and parseable relationships of historical resources are retained.

After a change is published, the registration recommender may load the new vocabulary, and the Discovery node may gradually rebuild related indexes by resource DID and resource version. Until rebuilding is complete, query results should be able to state the vocabulary or index version used. If an old tag cannot be mapped, the node may retain its original text for text retrieval, but must not present it as a definite hit of a new tag. Tag governance changes semantic interpretation and retrieval materials only; they do not change the resource DID, control relationship, registration credential, or Root platform publication fact.

13.7 Semantic Index Construction by Discovery Nodes

13.7.1 Index Inputs

The index inputs of a Discovery node fall into three categories:

1. **Resource facts:** resource DID, version, public DID Document, and Resource Metadata.
2. **Semantic materials:** normative tags, aliases, names, descriptions, use cases, protocols, and languages.
3. **Governance materials:** lifecycle, Root proof reference, authorization domain, tag tree version, and source node.

Private keys, unpublished drafts, and internal configuration must not enter the index.

13.7.2 Tag Index

The tag index should at least support filtering by normative tag ID, hierarchical path, language, and vocabulary version, and should retain the tag source. An index hit only means that the resource declaration or index record matches; the node must still check whether the resource is in a discoverable state.

13.7.3 Text and Semantic Indexes

The text index may cover names, descriptions, and use cases; the semantic index may use a recommender or an optional vector implementation. Both should return the matched fields, index version, and matching basis, and neither may add capabilities that the resource did not declare.

13.7.4 Index Update and Deletion

When the resource version, lifecycle, Root proof, or tag tree version changes, the node should perform an incremental update by resource DID and version. After a revocation, suspension, or deletion event arrives, the index may retain an audit record, but the resource must not continue to be returned as a normal discoverable candidate.

13.8 Matching Tags, Text, Protocols, and Resource Types

The four types of conditions have different roles in a query: resource type and an explicitly specified version are generally used to limit candidate categories; capability tags and protocols may serve as explicit filters; text in names, descriptions, and use cases is mainly used for relevance ranking. Multiple explicit conditions are generally processed as an intersection. A tag query requires a candidate to satisfy the requested tag set, while a protocol may match a protocol binding or service declaration. Semantic similarity may improve ranking, but it must not bring a candidate that fails a hard condition back into the result set.

Candidate eligibility = Visibility and lifecycle + Explicit structured conditions
 Relevance ranking = Tag/protocol hits + Text relevance + Semantic similarity
 Trust decision = DID Document + Credentials/Root proof + Caller policy

The first two lines define the discovery retrieval scope, while the last line is an independent pre-invocation check for identity, publication, and calling policy. When the semantic service is unavailable, a Discovery node may fall back to lexical or structured retrieval, but it must not relax the authorization domain, lifecycle, or explicit conditions.

13.9 Query Explanation and Matching Details

13.9.1 Query Intent Parsing

Query parsing should output auditable intermediate conditions, such as resource type, capability tags, protocol, language, and status filters. An uncertain term may be retained as a text condition and must not be forcibly converted into an official tag.

```
{
  "query": "I need a tool that can search code repositories and summarize project structures",
  "parsed": {
    "resourceType": "tool",
    "capabilities": ["code.repository.search", "code.structure.summarization"]
  }
}
```


13.9.2 Tag and Text Matching

Tag hits, alias hits, original-text hits, and semantic-inference hits should be returned in separate categories. When a hard filter is not satisfied, a relevance score must not cause the candidate to re-enter the result set.

13.9.3 Structured Condition Matching

Resource type, protocol, authorization domain, lifecycle, and publication status are structured conditions. The result for a structured condition should be explicitly expressed as satisfied, unsatisfied, or unknown; unknown must not be treated as satisfied.

13.9.4 Result Explanation

The query explanation interface of the current Discovery node accepts the same `ResourceDiscoveryQuery` as an ordinary discovery query, but returns dynamic JSON rather than `ResourceDiscoveryResponse`. The explanation may contain `items`, `candidateCount`, `matched`, various matching scores, tag overlap, structured-condition hits, and semantic-search status. These fields are observable retrieval results, not new resource proofs. A client should use the explanation to answer “why was this returned,” while relying on the candidate resource's DID Document, `ResourcePackage`, Root proof, and governance status to answer “can this be trusted or invoked.”

An explanation should at least contain the resource DID, matched fields or tags, vocabulary/index version, and the source of trusted status. The distinction between the two types of information can be expressed as follows:

“Relevant” means that the query matches the resource description; “trusted” additionally requires independent verification of the DID, control proof, Root proof, authorization status, and lifecycle.

Query explanations should retain the observable path from natural language to structured conditions. For example, when the user enters “I need a tool that can search code repositories and summarize project structures,” the system may identify “tool” as a candidate resource type, and “search code repositories” and “summarize project structures” as capability or text-retrieval clues. If the current vocabulary has no reliable mapping to an official tag, the system should retain the original sentence and ordinary text hit rather than create a new tag based on model inference. The explanation may display matched tags, fields, aliases, and index version, but should not expose unnecessary internal model details.

13.10 Multilingual Semantic Discovery

13.10.1 Chinese and English Resource Descriptions

Chinese and English descriptions may be stored together, and the original text should carry a language marker. Translated or machine-generated text may serve as a retrieval projection only and must not replace the provider's original description.

13.10.2 Multilingual Tags

Tag names in different languages should map to the same normative tag ID. If no reliable mapping exists, the name should be retained as ordinary text rather than used to create an ungoverned new synonym relationship.

13.10.3 Query Language Identification

Language identification selects only the lexical, alias, and tokenization paths. It must not change resource type, status, or trust-filter conditions. A mixed-language query should retain the parsing basis for each segment.

13.10.4 Cross-Language Matching

Cross-language hits should return the normative tag ID, the language and alias actually matched, and an indication of whether the hit came from tag mapping or text translation. A cross-language hit does not mean that the resource has an additional capability.

13.11 Extensibility for Vector and Hybrid Retrieval

The semantic extension of the current Discovery node may use PostgreSQL/pgvector and an HTTP embedding service to build resource-level semantic indexes; a local semantic recommender provides auxiliary recommendations for capability tags, resource types, and protocols. The choice of backend is a deployment and reference-implementation choice and must not change the processing rules for resource DID, ResourcePackage, authorization domain, lifecycle, or Root proof. A semantic index should also record the model name, model version, vector dimension, index version, and semantic-source hash so that results before and after rebuilding can be explained.

Stage	Information to retain	Boundary on failure
Resource projection	DID, version, public description, tags, use cases, and protocols	Do not write private materials to the semantic index
Embedding generation	Model identifier, model version, dimension, and source-content association	A failure for one resource must not fabricate a vector
Query retrieval	Query text, hard conditions, index version, and backend	Semantic failure may degrade, but must not relax hard conditions
Result explanation	Relevance, matched fields, and evidence references	Relevance is not trust or invocation permission

When the model, dimension, or semantic vocabulary changes incompatibly, the corresponding index should be rebuilt first, and the new results should then be served as a single index version to callers. An old index may be retained during migration, but scores produced by different models must not be compared directly.

13.12 Semantic Governance Compatibility and Migration

During migration, three types of information should be retained together: the original tags and descriptions submitted by the resource provider, the vocabulary version used at that time, and the parseable mapping in the new version. A compatibility mapping may allow old queries to continue finding a resource, but it must not rewrite the original fields in historical ResourcePackages. After a Discovery node completes index migration, it should update the index version. If migration is incomplete or a tag cannot be mapped, a result may continue to serve as limited text-based reference, but its semantic-governance status should be stated.

13.13 Semantic Vocabulary Versioning and Change Impact

Vocabulary versions and semantic index versions should be recorded separately: the former identifies the version of the tag tree and alias materials, while the latter identifies the inputs actually used by a particular Discovery node to construct its retrieval data. The effects of the two types of changes can be understood as follows:

Change type	Registration recommendation	Original value of existing resources	Discovery index
Add a tag or alias	Available for new recommendations	Not affected	Incremental update is possible
Rename or localize	Retain the normative ID	Retain the original text	Update display and retrieval mappings
Merge tags	Establish an old-to-new query mapping	Do not rewrite automatically	Rebuild projections for related resources
Split tags	Require additional information from the provider	Do not infer automatically	Retain and mark the old value when uncertain
Deprecate a tag	Stop new recommendations	Retain a historical parseable value	Degrade or remove from normal recommendations

When rolling back, the system should restore the previous verifiable vocabulary and index versions rather than deleting resource history. Query results and operational diagnostics should at least identify the vocabulary version, index version, and update time used, so that it is possible to explain why the same query produces different rankings on different nodes.

13.14 Semantic Governance Auditability

The audit records for semantic governance need to answer “who changed what in which vocabulary version and at what time, and which recommendations and indexes were affected.” At minimum, the records should associate the operator or governance subject, operation time, change type, old and new values, vocabulary version, reason, impact scope, index rebuilding status, and rollback basis. Audit records explain changes in semantic materials; a tag-change record must not be mistaken for a resource registration credential or resource control proof.

Registration recommendations and discovery queries should also retain necessary references. For example, a recommendation may be associated with an input digest, vocabulary version, and recommendation candidates; a discovery explanation may be associated with matched tag IDs, aliases or text fields, index version, and resource DID. Natural-language originals, caller information, and internal model outputs should follow the deployment operator's access-control and minimum-retention policies. A public interface should expose only the minimum content needed for explanation.

13.15 Mapping Natural-Language Queries to Structured Conditions

Natural-language mapping should distinguish between conditions that can be assigned to structured fields and expressions that can only serve as retrieval clues. For example, “find a tool that can search code repositories and summarize project structures” may produce a candidate resource type, candidate capability tags, and text-retrieval terms. Words such as “latest,” “trusted,” and “invokable” may participate in deterministic filtering only when the protocol and governance materials provide corresponding fields or facts; otherwise, they can serve only as ranking or explanation hints. An uncertain term must retain its original text and must not be forcibly mapped to an official tag or authorization condition.

```
{
  "query": "I need a tool that can search code repositories and summarize project structures",
  "structuredConditions": {
    "resourceType": "tool_api",
    "capabilityTags": ["code.repository.search"]
  },
  "textTerms": ["summarize project structures"],
  "mappingStatus": "suggested"
}
```

The object above is an illustrative intermediate result for explanation and is not the fixed response format of the current discovery interface. A formal request must still use the discovery-query fields specified in Chapter 12. The value `suggested` indicates a semantic suggestion; the caller or resource provider must still confirm the tag, resource type, and protocol conditions. If mapping fails, the vocabulary version is incompatible, or the language does not match, the system may fall back to original-text retrieval or return an explicit result indicating that no usable mapping is available, but it must not conceal the failure as an incorrect formal condition.

13.16 References

Source	Type	Link
oan-discovery-node	Code repository: semantic aliases, indexes, and semantic queries	https://github.com/OpenAgenet/oan-discovery-node

Source	Type	Link
oan-protocol-common	Code repository: capability tags, authorization domains, and semantic support materials	https://github.com/wolfbrother/oan-protocol-common
GRAIL Semantic Discovery Paper	arXiv paper: semantic discovery, hybrid retrieval, and ranking explanations	https://arxiv.org/abs/2605.02489
Efficient Agent Discovery Profile	IETF draft: discovery metadata and query profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Chapter 14 Node Authorization and On-Chain Governance

This chapter explains how OpenAget (OAN) converts on-chain governance facts into authorization states that infrastructure nodes can use. Governance contracts record node authorization, authorization domains, capability scopes, and status changes; the off-chain trust indexer reads and stores these events and produces queryable subject states; the Root platform, Registrar nodes, and Discovery nodes use these states to decide whether to accept requests, propagate resources, or use governance materials produced by a node. On-chain authorization, node authorization credentials, resource registration, and resource publication prove different facts and cannot substitute for one another.

In this chapter, “current state” means the judgment made by a component based on its synchronized read model. The read model may temporarily lag behind on-chain events. Therefore, when handling new authorization, suspension, resumption, revocation, or high-risk actions, the event sequence, synchronization position, state validity period, and evidence source should all be considered.

14.1 Governance Objectives and the On-Chain/Off-Chain Boundary

The on-chain governance layer and the off-chain business layer are connected through “read, verify, project, and execute”; they do not share one state table. On-chain governance contracts produce reviewable raw events, the indexer stores events and subject states, and the Root platform or node services read this projection and make runtime decisions together with the node DID, authorization credentials, and request content. Resource control proofs, resource packages, and publication status belong to the resource business path. Their existence does not allow a service to skip node governance-state checks.

14.1.1 On-Chain Governance Facts

On-chain governance facts include node authorization, suspension, resumption, and revocation, together with their times, sequences, and sources. On-chain facts are the authoritative source of governance state, but an off-chain service may retain an old read model before synchronization is complete. Responses should distinguish the on-chain position, indexing time, and state currently adopted by the service.

In the current governance contract, governance events are expressed by `GovernanceExecutionEvent`. The event contains fields such as the action type, event sequence, target node DID, node type, status, authorization domain, effective time, expiration time, policy hash, metadata hash, and the previous-event and current-event digests. The raw payload, transaction, event sequence, and on-chain position such as the checkpoint should be stored together. In this way, the read side can obtain both a node's current state and the governance execution that produced that state.

The relationship between on-chain events and off-chain services can be summarized as follows: the chain records that “a governance action has occurred, together with its order and parameters”; the indexer constructs a queryable state from these events; and the Root platform and other nodes use that state according to their own policies. The indexer cannot modify on-chain events, reissue governance permissions, or fill in missing events on its own. When the raw event and the read model disagree, the original position and event digest should be the starting point for verification.

Field	Purpose
Node DID	Identifies the governed node
Event type	Indicates authorization, suspension, resumption, or revocation
Authorization domain and capabilities	Limits the operations that may be performed
On-chain position	Supports ordering and verification
Effective time	Distinguishes the on-chain occurrence time from the service observation time

14.1.2 Root Platform Runtime State

Root platform runtime state indicates whether the service has read and applied the current governance result, including whether it accepts node submissions, permits publication, and permits downstream distribution. It is a runtime projection of on-chain governance facts and must not replace those facts.

At runtime, the Root platform must distinguish at least between “the governance state is known” and “this request has been authorized.” The former depends on the state, sequence, and freshness provided by the off-chain trust indexer or another governance read side. The latter also requires matching the requesting node DID, requested action, authorization domain, and capability scope. Even if a node is in `active` status, this only indicates that it may have the corresponding infrastructure responsibility. Resource control, resource content, version, and publication materials still require separate verification.

14.1.3 Registration and Publication Status

Registration acceptance, Root platform publication, content distribution, and Discovery visibility are different states. Valid node authorization is a qualification for executing infrastructure actions; it does not mean that a resource has passed control, content, and version verification.

These states are produced by different components: the Registrar node records the resource registration request and its validation result; the Root platform decides whether to form a Root-verified publication fact; the content distribution platform propagates approved materials; and the Discovery node adds visible resources to its local index. On-chain node authorization only

determines whether an infrastructure node is qualified to perform a certain type of action. It cannot replace the resource provider’s signature, resource package integrity checks, publication process, or Discovery index update.

```
Node authorized -> may submit a request
Request validated -> may enter the publication process
Root platform publication -> may enter the distribution process
Discovery node indexing -> resource may become visible to queries
```

14.1.4 Discovery Node Read Model

The Discovery node uses the off-chain trust indexer or governance synchronization result to filter resources. The read model must record its source and synchronization progress. When a governance event has not yet been synchronized, the node should use a conservative state and must not interpret an old cache directly as a current authorization conclusion.

The Discovery node consumes a governance read model in normal operation, rather than scanning the entire on-chain history for every query. The read model must at least associate the node DID, node type, current status, authorization domain, last governance sequence, event digest, and update time. The source status in a query result should also remain separate from the resource’s own lifecycle and publication proof. If the indexer is behind, a Discovery node may retain a read-only snapshot to improve access continuity, but it must not label that snapshot as the current governance fact.

14.2 Infrastructure Node Authorization Model

Infrastructure nodes use the node DID as the governance subject and distinguish Registrar nodes, Discovery nodes, and VC issuers by node type. The current indexer’s `SubjectState` stores the node type, node DID, status, authorization domain, policy hash, metadata hash, effective time, expiration time, state version, last update time, last governance sequence, and last event digest. A node authorization credential can be used as off-chain evidence, but it and the on-chain state are two evidence chains that must be checked together: the credential states the issuer’s claim about the node’s qualification, while the on-chain read model states the current governance status and authorization scope.

A minimal state judgment can be represented as follows:

```
{
  "subjectType": "discovery",
  "subjectDid": "did:oa:INDS:example",
  "status": "active",
  "authorizedDomains": ["technology.software_engineering"],
  "effectiveFromMs": 0,
  "expiresAtMs": 0,
  "lastSequence": 12
}
```

The DID, sequence, and authorization domain in the example are illustrative values. At run-time, the state can be used as an input for “governance-level authorization” only when the node type

matches, the status is valid, the effective time has been reached, and the expiration time has not been exceeded. The requested action must then still be checked against the authorization domain and capability scope.

14.3 Authorization Domains and Capability Scopes

The authorization domain answers “which business or resource scopes may the node process,” while the capability scope answers “which infrastructure actions may the node perform.” For example, a Discovery node may be authorized to serve a particular capability domain, but this does not mean that it may represent the resource controller to modify a resource DID Document, nor that it can issue a registration credential. Authorization domains should use normalized and comparable values. The current contract also constrains empty domains, quantity, length, canonical order, duplicate entries, and the mixed use of wildcards.

Multiple authorization domains normally represent the union of the node’s service scopes. Whether a specific resource may be processed still requires a combined judgment over node status, the resource authorization domain, and governance state. A change to the authorization domain should produce a new governance event and state version. After the scope is reduced, the node must not continue accepting new requests under the old scope. Historical publication facts that have already been formed should retain their source and time and must not simply be deleted.

14.4 Governance Event Model

The governance contract handles proposal creation, voting, and final execution separately; the indexer focuses on the emitted initialization events and governance execution events. After parsing, the indexer stores both raw event records and structured governance events. Raw records retain the transaction digest, event sequence, checkpoint, package identifier, event type, time, and original JSON. Structured records support queries by node DID, governance sequence, status, and authorization domain. Event names and descriptions shown on a page are projection fields and cannot replace the raw event payload.

The key fields of a governance execution event can be understood as follows:

Field category	Typical fields	Purpose
Identity relationship	<code>subjectDid</code> , <code>subjectType</code> , <code>targetAddress</code>	Associates the governed node with its on-chain target
Action and status	<code>eventType</code> , <code>eventLabel</code> , <code>subjectStatus</code>	Indicates authorization, suspension, resumption, revocation, or domain change
Scope and time	<code>authorizedDomains</code> , <code>effectiveFromMs</code> , <code>expiresAtMs</code>	Constrains the applicable scope and validity window
Ordering evidence	<code>sequence</code> , <code>previousEventDigest</code> , <code>eventDigest</code>	Supports event-chain and state-replay verification
Change basis	<code>proposalId</code> , <code>policyHash</code> , <code>metadataHash</code>	Associates the event with governance proposals and policy materials

Field category	Typical fields	Purpose
On-chain source	Transaction digest, event sequence, checkpoint	Points to the original on-chain record and read position

The official Network page exposes the event list in the current read model through the public endpoint `/api/public/network/governance-events`. At the time of this review, the endpoint's `generatedAt` was `2026-09-12T02:01:05.463553057+00:00`. The following table records the time, node, and page description in the order currently returned by the endpoint. It gives readers a direct view of publicly observed governance actions, but does not replace the raw event payload, transaction, or checkpoint in the governance contract. The list changes as on-chain governance and indexer synchronization change:

Time (UTC)	Node	Event description
2026-08-25 03:11:30	Test Discovery	Updated the authorization domain of the test Discovery node to the full domain.
2026-06-29 02:16:44	Test Discovery	Updated the authorization domain of the test Discovery node to a set of 28 industry/domain categories.
2026-06-29 02:15:47	Registrar 1	Updated the authorization domain of the Registrar node to a set of 28 industry/domain categories.
2026-06-29 01:26:06	Test Discovery	Updated the authorization domain of the test Discovery node to a set of 16 industry/domain categories.
2026-06-29 01:25:10	Registrar 1	Updated the authorization domain of the Registrar node to a set of 16 industry/domain categories.
2026-06-29 01:12:12	Test Discovery	Updated the authorization domain of the test Discovery node to technology.
2026-06-29 01:11:05	Registrar 1	Updated the authorization domain of the Registrar node to technology.
2026-06-28 22:48:58	Official Discovery	Updated the authorization domain of the official Discovery node to the full domain.
2026-06-28 22:47:41	Test Discovery	Updated the authorization domain of the test Discovery node to the full domain.
2026-06-28 22:46:14	Test Registrar	Updated the authorization domain of the test Registrar node to the full domain.

Time (UTC)	Node	Event description
2026-06-28 22:45:59	Official Registrar	Updated the authorization domain of the official Registrar node to the full domain.
2026-06-28 22:45:35	Registrar 1	Updated the authorization domain of the Registrar node to the full domain.
2026-06-04 09:16:16	Official Discovery	Resumed the official Discovery node with active status.
2026-06-04 09:15:47	Official Discovery	Suspended the authorization of the official Discovery node.
2026-06-04 09:15:17	Test Discovery	Resumed the test Discovery node with active status.
2026-06-04 09:14:48	Test Discovery	Suspended the authorization of the test Discovery node.
2026-06-04 09:12:10	Test Registrar	Resumed the test Registrar with active status.
2026-06-04 09:11:41	Test Registrar	Suspended the authorization of the test Registrar.
2026-06-04 09:11:12	Official Registrar	Resumed the official Registrar with active status.
2026-06-04 09:10:42	Official Registrar	Suspended the authorization of the official Registrar.
2026-06-04 09:10:14	Registrar 1	Resumed the Registrar node with active status.
2026-06-04 09:09:46	Registrar 1	Suspended the authorization of the Registrar node.
2026-06-04 07:21:26	Legacy Discovery 2	Revoked the Discovery node authorization.
2026-06-04 07:19:09	Legacy Discovery 1	Revoked the Discovery node authorization.
2026-06-04 07:13:42	Legacy Registrar 3	Revoked the Registrar node authorization.
2026-06-04 07:12:51	Legacy Registrar 2	Revoked the Registrar node authorization.
2026-06-04 07:11:21	Legacy Registrar 1	Revoked the Registrar node authorization.
2026-06-04 07:06:52	Official Discovery	Authorized the official Discovery node with active status; authorization domains: genesis.openagenet.local , openagenet.local .
2026-06-04 07:05:45	Test Discovery	Authorized the test Discovery node with active status; authorization domains: genesis.openagenet.local , openagenet.local .

Time (UTC)	Node	Event description
2026-06-04 07:04:36	Test Registrar	Authorized the test Registrar with active status; no explicit authorization domain was set.
2026-06-04 07:03:36	Official Registrar	Authorized the official Registrar with active status; no explicit authorization domain was set.
2026-06-04 07:02:43	Registrar 1	Authorized the Registrar node with active status; no explicit authorization domain was set.
2026-06-04 06:24:16	Legacy Discovery 2	Authorized the Discovery node with active status; authorization domains: <code>genesis.openagenet.local</code> , <code>openagenet.local</code> .
2026-06-04 06:23:35	Legacy Discovery 1	Authorized the Discovery node with active status; authorization domains: <code>genesis.openagenet.local</code> , <code>openagenet.local</code> .
2026-06-04 06:21:59	Legacy Registrar 3	Authorized the Registrar node with active status; no explicit authorization domain was set.
2026-06-04 06:21:40	Legacy Registrar 2	Authorized the Registrar node with active status; no explicit authorization domain was set.
2026-06-04 06:19:58	Legacy Registrar 1	Authorized the Registrar node with active status; no explicit authorization domain was set.

14.4.1 Node Authorization Events

An authorization event places a node in the candidate set of usable nodes. The recipient must still check finality, authorization expiration, capability scope, and the binding of the node DID. It must not allow a request solely because of the event name.

In the current action encoding, authorization of a Registrar node is 21, authorization of a Discovery node is 31, and authorization of a VC issuer is 41. These codes come from the action mapping in the governance contract and indexer. The actual effect of an authorization event is still determined jointly by status, the time window, the authorization domain, and whether the read model has been synchronized. If the target DID and target address in the authorization event do not match the node DID Document and runtime connection identity, automatic acceptance should stop and the case should enter verification.

14.4.2 Node Suspension and Resumption Events

Suspension events take precedence over ordinary business requests. A resumption event becomes effective only after governance conditions are met and the indexer observes it in the correct

order. The service should retain the raw suspension and resumption events rather than only replacing a current Boolean state.

Suspension and resumption do not delete or overwrite historical events. The indexer should store them as a continuous governance sequence and project the current state from the last valid sequence. After resumption, the system only allows subsequent actions that meet the conditions again. Whether a request rejected during suspension should be resubmitted is decided by the caller based on the request identifier and business result; state recovery must not resubmit it automatically.

14.4.3 Node Revocation Events

A revocation event indicates that a node should no longer perform new operations within its authorization scope. Completed resource publications and historical queries should not be rewritten unconditionally, but downstream services must stop adopting new governance conclusions produced by the revoked node.

Revocation affects the node's subsequent governance qualification; it does not erase all data previously produced by that node from history. After processing a revocation event, downstream services should stop accepting new registrations, publications, distributions, or governance-state updates from that node, and should recheck materials from the node that have not completed verification. Historical resource records independently confirmed by other facts may be retained, but must preserve their source node, event position, and state at that time.

14.4.4 Event Time, Sequence, and Source

Event ordering should prioritize the on-chain checkpoint, transaction sequence, event sequence, or an explicitly defined composite key. The event occurrence time, indexing time, and page display time must be recorded separately.

After paginated reads, the current event source orders events by checkpoint, transaction digest, and event sequence, and saves the read cursor together with the last processed position in the indexer state. `emittedAtMs` represents the on-chain time in the event payload. The indexer write time represents read-side observation and persistence, while the page generation time represents only the display moment. These values cannot replace one another. If an event has no usable time, ordering and diagnosis should rely on the on-chain position and indexing time.

14.5 Node Authorization, Suspension, Resumption, and Revocation

Node status is the current projection of a governance sequence onto a subject state. Authorization, suspension, resumption, and revocation all change subject availability through governance events, but their concrete effect is also constrained by the authorization domain, node capability, effective time, expiration time, event order, and index freshness. Before executing an action, a service node should combine the status judgment with the request source, request type, and resource facts. A successful historical request must not be treated as proof that current authorization remains valid.

14.5.1 Authorization

After authorization, a node obtains an explicit authorization domain and capability scope. When receiving a node request, the Root platform should validate the current status and scope. It must not interpret an authorization event as proof that resource content has already passed verification.

14.5.2 Suspension

While suspended, a service may retain read-only status queries and audit capabilities, but must not continue accepting new submissions or propagation actions that require the node's authorization, unless governance policy explicitly permits them.

14.5.3 Resumption

Resumption should re-confirm the node identity, authorization scope, and governance event order. Resumption does not automatically resubmit requests rejected during suspension. The client must re-confirm them according to the request ID and current status.

14.5.4 Revocation

After revocation, new actions within the authorization scope must stop. The revocation reason, time, operator, and on-chain position should be retained. The Discovery node and distribution service need to refresh their governance read models and converge progressively.

14.5.5 Effects of Status on Registration, Distribution, and Discovery

Node status	Registration submission	Content distribution	Discovery indexing
active	Process according to the authorization domain	Process approved materials	Provide results that satisfy the filtering conditions
suspended	Reject or pause new submissions	Pause related propagation	Do not adopt new governance conclusions from the node
revoked	Reject new submissions	Stop its authorized propagation	Revoke or isolate records sourced from the node
unknown	Conservatively reject high-risk actions	Do not advance an uncertain state	Read-only results must carry an uncertainty marker

State-transition decisions should not rely on a single Boolean field. When projecting state, the indexer must also check whether the event sequence is continuous, whether the previous-event digest matches, whether events are duplicated, and whether effective and expiration times are valid. Runtime authorization checks must compare the current time with `effectiveFromMs` and `expiresAtMs`: an action cannot be allowed before the effective time, and a nonzero expiration time makes the state invalid once reached. When the state is unknown, the sequence has a gap, or the digest chain is inconsistent, high-risk new actions should be blocked rather than guessing the next state.

14.6 Governance Notifications and Authorization State

Governance notifications are suitable for communicating that a state has changed and providing an entry point for verification to service nodes and operations interfaces, but a notification itself is not a new authorization credential. A notification or snapshot should contain at least the node DID, node type, current status, authorization domain, last governance sequence, event digest, on-chain position, and indexer observation time. After receiving a notification, the recipient should read the status interface or original on-chain evidence according to its security requirements and verify it. It must not execute a high-risk action solely from the notification text.

When notification delivery fails, the read model lags, or a state snapshot disagrees with the raw event, a service may temporarily retain the known state for low-risk read-only display. It should stop advancing new high-risk authorization actions and record the synchronization position that needs recovery. After notification delivery resumes, the service should recheck according to the governance sequence rather than processing only the last notification.

14.7 The Off-Chain Trust Indexer as a Governance Read-Side Projection

The off-chain trust indexer is a component for reading and projecting governance information. It reads target governance events from the configured chain endpoint, stores raw events, structured events, and current subject states in local storage, and provides query endpoints for the Root platform and node services over HTTP. Through chain adapters and a unified state projection, the indexer decouples the blockchain implementation from OAN nodes and business logic, allowing the same node-authorization, governance-reading, and state-judgment mechanisms to support consortium chains such as Xinghuo Chain Network and well-known public chains such as Ethereum. The event model, finality, query interface, and deployment adaptation of each chain still require separate implementation and verification; the current reference implementation mainly covers Sui. The value of the indexer is to convert on-chain events into an operational read model. Its boundary is that it does not hold governance keys, submit governance transactions, or create or revoke node authorization based on its own judgment.

14.7.1 Reading On-Chain Events

Event reading should support initial synchronization, cursor-based incremental reading, and resume from a checkpoint. When reading fails, the last confirmed position must be retained. An empty result must not overwrite an existing valid read model.

The indexer's event source supports paginated queries by event type and combines the GraphQL cursor, transaction digest, and event sequence associated with the event type into a resumable cursor. The configured event-type package, accepted emitting packages, and synchronization mode must remain consistent with the actual on-chain upgrade history. Otherwise, the service may appear healthy while being unable to read target events. Initial synchronization establishes the complete history, incremental synchronization continues from the saved cursor, and checkpoint recovery must reread from a safe position and use idempotent writes to eliminate duplicates.

14.7.2 State Projection

State projection updates node status, authorization domains, and capability scopes according to event order and retains the necessary event history. Duplicate events should be handled idempotently, and an old event must not overwrite a confirmed newer state.

The current storage layer uses the governance sequence as the primary key for governance events and uses “node type + node DID” as the primary key for subject state. A repeated event with the same sequence and event digest can be treated as an idempotent replay. If the same sequence arrives with a different digest, the service should report a governance-event conflict and stop silent overwriting. Only an event with a sequence greater than the subject state’s last sequence may update the current projection.

14.7.3 Synchronization Cursor and Lag

The cursor is used for recovery and observation; it is not equivalent to on-chain finality. The interface should distinguish the currently read position, confirmed position, and position adopted by the service, and should make synchronization lag computable.

`/v1/status` may provide the package identifier, governance-object identifier, last processed checkpoint, event cursor, event source, latest governance sequence, latest event digest, sequence-gap flag, digest-gap flag, and last synchronization time. `latestSequence` only indicates the highest governance sequence written by the indexer. It does not by itself prove that the indexer has caught up with the latest on-chain checkpoint. Lag judgment must also consider the latest position returned by the event source and the synchronization result.

14.7.4 Query Interfaces

Query interfaces should return the state source, on-chain position, indexing time, and reason for the state. Callers may use these values for filtering, but high-risk authorization must still be verified against original facts or verifiable proofs.

The public subject query, authorization-state query, and governance-event query provided by the indexer serve different purposes: subject queries are suitable for listing node states, authorization-state queries are suitable for runtime decisions, and event queries are suitable for auditing and fault tracing. Query responses come from the indexer read model. Callers should judge freshness by combining the last governance sequence, event digest, and synchronization status. An `authorized` result must not be interpreted as meaning that the indexer has verified all node keys, endpoints, or resource behavior on behalf of the caller.

The current off-chain trust indexer provides read interfaces including `GET /v1/status`, `GET /v1/summary`, `GET /v1/subjects`, `GET /v1/registrars`, `GET /v1/discoveries`, `GET /v1/vc-issuers`, and `GET /v1/events`. Subject interfaces usually return a `subjects` array, governance-event interfaces return an `events` array, and the status interface returns an index-status structure. These responses use dynamic JSON because index state and on-chain event fields are serialized from the indexer read model. `POST /v1/admin/sync-once` is a synchronization management operation and should not be mixed with public query interfaces.

The indexer’s chain-access implementation uses official Sui gRPC/protobuf types, such as check-

point and service-information requests. These types belong to the external Sui RPC protocol and are not OAN's own node-coordination `.proto`. OAN nodes therefore continue to use JSON/HTTP protocols serialized from shared Rust types as business interfaces, while chain endpoints and protobuf remain at the boundary where governance facts are read.

14.7.5 Synchronization Failure and Recovery

When synchronization fails, the service should record the error, retry count, and last successful cursor. After recovery, it should replay from a safe position and perform consistency checks. A temporary indexer outage must not cause all nodes to be marked as revoked.

Recovery should at least include identifying the reason for the read failure, retaining the last successful cursor, rereading affected pages, checking the event sequence and digest chain, confirming that a subject state has not been overwritten by an old event, and recording the new cursor and processed count after recovery. When the indexer is unavailable, dependents should mark the state as unknown or temporarily unavailable rather than writing `revoked` in bulk. Revocation must come from an actual governance event or a verifiable governance state.

14.8 Enforcing Governance State at Root Platform Runtime

When processing a request, the Root platform should perform the following steps in order:

1. Identify the requesting node DID and verify the communication signature.
2. Query the current governance state and its synchronization freshness.
3. Check whether the requested action belongs to the authorization domain and capability scope.
4. Then verify resource control, version, hash, and content.
5. Write the acceptance, rejection, or delay reason to traceable logs.

Valid authorization is only a prerequisite for entering the resource-verification process; it is not a resource-publication conclusion.

The first step confirms which node sent the communication request. The second confirms that node's state and freshness on the governance read side. The third confirms that the action, node type, authorization domain, and capability scope match. Only the fourth step enters verification of the resource DID Document, control signature, resource package, version, and content. Insufficient evidence at any step cannot be compensated for merely by success in a later step. For example, a valid resource-package signature may show that the resource control proof is potentially valid, but it does not prove that the Registrar node submitting it still has current authorization.

14.9 Governance Filtering on the Discovery Side

Discovery filtering must distinguish at least the source node status, resource lifecycle, Root platform publication status, and index freshness. Semantic relevance or tag matches may participate in ranking only after these hard conditions pass.

The Discovery node may use node authorization status to filter sources, but it should not write that authorization status directly into the resource lifecycle field. When a source node is suspended, revoked, or unknown, newly synchronized materials should be rejected, isolated, or marked as restricted. Existing resource records confirmed by other evidence may retain their historical presence, but they must not continue to be treated as normal new candidates without current governance confirmation. When the authorization domain does not match, the resource is not visible to that Discovery node; this does not mean that the resource DID does not exist across the entire network.

14.10 Governance Event History and Public Observability

Public event queries should use stable pagination and explicit ordering, and should display the on-chain position and indexing time together:

Field	Description
Event type	Authorization, suspension, resumption, or revocation
Node DID	Affected node
Authorization domain/capability	Scope involved in the event
On-chain position	Checkpoint, transaction, or event sequence
<code>observedAt</code>	Indexer observation time
State reason	Explanation for operations and auditing

Public display may use `eventLabel` as a readable action name, but should retain the raw `eventType`, governance sequence, and source position. “Recent” in an event list should be interpreted by governance sequence or on-chain position, not by the response time of the web interface. Pagination boundaries should also be tied to stable ordering so that a client does not display the same event twice or miss an event when new events arrive.

14.11 Chain Endpoints, Packages, Objects, and Deployment Configuration

Indexer configuration must record the network, current operation package, event-type definition package, allowed emitting packages, governance bulletin object, event-source mode, and synchronization parameters together. OAN currently uses Sui Mainnet. The governance bulletin object is `0xb900fb0dc1ebd58fbc4d0c697d644c861578ed948e26638836e338de0d1a3b46`; the initial package `0x1b602d575ddf1ced075f9d2986bfe53cd7d4e59967ec26a0d949009c7b2c9d6c` defines the governance event types; and the current operation package is `0x232f984a25ceb8358290a8bae594078deca2d670a80c8d57d291d64a491d7fd5`. During upgrades, `0x2fd18248c0cd42f371409eb1003176dc0d5744cccfd2e49e5224b4249a5ffb4` also emitted governance events. Therefore, the indexer must accept events produced by all three packages to reconstruct the governance sequence continuously across the upgrade.

```
[chain]
network = "sui-mainnet"
grpc_endpoint = "https://fullnode.mainnet.sui.io"
event_query_endpoint = "https://graphql.mainnet.sui.io/graphql"
```

```

package_id = "0x232f984a25ceb8358290a8bae594078deca2d670a80c8d57d291d64a491d7fd5"
event_type_package_id = "0x1b602d575ddf1ced075f9d2986bfe53cd7d4e59967ec26a0d949009c7b2c9d6c"
emitting_package_ids = [
    "0x1b602d575ddf1ced075f9d2986bfe53cd7d4e59967ec26a0d949009c7b2c9d6c",
    "0x2fd18248c0cd42f371409eb1003176dc0d5744ccccdf2e49e5224b4249a5ffb4",
    "0x232f984a25ceb8358290a8bae594078deca2d670a80c8d57d291d64a491d7fd5",
]
bulletin_object_id = "0xb900fb0dc1ebd58fbc4d0c697d644c861578ed948e26638836e338de0d1a3b46"

[sync]
event_source = "event_query"
poll_enabled = true
sync_on_start = true
poll_interval_ms = 10000
event_page_size = 50
max_pages_per_sync = 2

```

The values above correspond to the current Mainnet event-synchronization configuration. `package_id` points to the package that executes current governance operations. `event_type_package_id` remains directed to the package that originally defined the event structure. `emitting_package_ids` covers the packages that actually emitted governance events during the upgrade history. `bulletin_object_id` points to the governance bulletin object that evolves continuously. These four values have different roles: updating only `package_id` to the latest package while omitting the old event-type package or an intermediate emitting package creates gaps in event queries; configuring only the bulletin object without checking the event source may produce a service that starts normally while its governance sequence has stopped growing.

14.12 Governance Upgrade and Migration Rules

Before an upgrade, the compatibility relationship among the contract package, event structures, indexer, and node services should be declared, followed by migration and replay. Old events must remain parseable or have an explicit conversion rule. If conversion fails, the raw data must be retained and an uncertain state must be prevented from overwriting a valid state.

An upgrade should also distinguish a “package-address change” from an “event-type-definition change.” An upgradeable Sui package’s current operation package may differ from its event-type definition package. Therefore, event-query configuration, checkpoint-audit configuration, and the set of emitting packages accepted by the indexer cannot be inferred from a package name. After migration, a set of known governance events should be used to verify the raw event count, structured event count, latest sequence, digest chain, and subject states, confirming that the upgrade introduced neither duplicate projections nor skipped events.

14.13 Governance Event Ordering and Finality

Event processing can be understood through the following states:

```
observed -> ordered -> confirmed -> projected -> exposed
```

An event that arrives later but has an earlier on-chain position should be inserted into the history, and affected projections should be recalculated. A confirmed higher-version state must not be directly overwritten by an old event.

`observed` means that the event has been read from the event source. `ordered` means that the indexer has established an order by on-chain position or governance sequence. `confirmed` means that the event has passed the implementation-defined integrity and position checks. `projected` means that it has been written to subject state. `exposed` means that the query interface can read the projection. The fact that an event has entered the public read model does not mean that its on-chain occurrence time equals the interface observation time. When a sequence gap or digest gap exists, public state should carry uncertainty so that incomplete history is not treated as a continuous governance chain.

14.14 Consistency Among On-Chain Facts, Off-Chain Publication State, and Read Models

Consistency does not require on-chain events, the indexer, the Root platform, and the Discovery node to update at exactly the same moment. It requires every layer to explain the fact position and delay it is using. On-chain events are the source of governance facts; the indexer provides a read model with cursors and times; the Root platform uses it to make publication decisions; and the Discovery node uses it to filter sources and resource visibility. When any layer lags or conflicts, confirmed facts should be retained, differences and their sources should be marked, and restrictive handling should be applied to high-risk actions.

14.14.1 Final On-Chain Facts

Final on-chain facts are based on a verifiable event position and contract state and should retain at least the object, action, operator, time, and reference to the original event. An indexer interface can provide an access path for reading, but cannot change the original fact.

`SubjectState` in the governance contract stores the node's current status, authorization domain, policy and metadata hashes, effective and expiration times, state version, and update time. `GovernanceExecutionEvent` stores the event context that caused the state change. Together, they can answer “what is the current state” and “which governance execution formed it,” but the result should still be verified against the original on-chain position of the transaction, event, and governance object.

14.14.2 Root Platform Publication State

Root platform publication state represents the result of the off-chain verification and publication process. It may lag behind on-chain authorization, or may not be produced because resource verification failed. Therefore, resource publication cannot be inferred from node authorization.

The Root platform may read node governance state before accepting a submission from a Registrar node, but resource publication must still pass checks for the resource DID Document, control signature, resource package integrity, version relationship, content policy, and publication process. Publication state should retain the resource DID, resource version, publication-proof reference, pro-

cessing time, and source node. It is stored separately from on-chain node authorization state so that downstream propagation eligibility can be reconsidered when the node's governance state changes.

14.14.3 Discovery Node Index State

Discovery index state indicates whether a Discovery node has received and indexed a propagatable resource. It should associate the resource DID, version, publication-proof reference, indexing time, and governance filtering result.

Discovery index state is a local read-model state, not an on-chain resource state. An index record should identify the publication or distribution source, the resource version used, the time written to the local index, the node governance state adopted at that time, and whether the resource was filtered by authorization domain or lifecycle. Semantic-index hits, capability-tag hits, and governance-authorization hits should be explained separately. A retrieval score must not conceal that the source node has been revoked or that its governance state is unknown.

14.14.4 State Inconsistency and Convergence

When states are inconsistent, the system should act conservatively: pause high-risk actions, retain confirmed facts, mark the source of the difference and synchronization lag, and update the external state after the governance read model, Root platform, and Discovery index converge.

Common inconsistencies include an on-chain governance sequence that the indexer has not read, an event read by the indexer that the node service has not refreshed, a Root platform publication that the Discovery node has not indexed, and an old local resource record whose source-node status has changed. Each inconsistency should record the compared sequences, times, sources, and handling action. During recovery, the system should recompute from the fact source rather than overwriting the difference with the web access time or the last successful request.

14.15 Governance Authority Key Management and Emergency Procedures

Governance-key operations must be strictly separated from the local control private keys of resource providers. Emergency procedures should at least include pausing high-risk governance operations, rotating or isolating suspected leaked keys, confirming on-chain events, resynchronizing the indexer, notifying nodes, and verifying state convergence. An operations marker that has not been confirmed through on-chain or authorization procedures must not be used directly as the final governance fact.

Governance keys are used to initiate or approve governance actions. Resource control private keys prove the control relationship for a specific resource DID or resource version. The indexer and ordinary operations processes should not hold governance keys. After emergency handling, the system should separately verify the permissions of the governance account or object, the on-chain event sequence, the indexer state, node runtime decisions, and downstream Discovery results. If any one of these remains unknown or lagging, the external state should retain a restrictive marker; service-process recovery alone is not enough to declare governance recovery complete.

14.16 References

Source	Type	Link
<code>oan-trust-indexer</code>	Code repository: on-chain governance event synchronization and off-chain trust-state projection	https://github.com/OpenAgenet/oan-trust-indexer
<code>oan-root-services</code>	Code repository: node authorization, governance state, and Root platform decisions	https://github.com/OpenAgenet/oan-root-services
<code>oan-protocol-common</code>	Code repository: shared authorization, governance, and state types	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv yellow paper: governance, authorization, and trust boundaries	https://arxiv.org/abs/2606.03163
Agentic Overlay Network Architecture	IETF draft: network-node and governance coordination architecture	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

Chapter 15 Network Node Protocols and Interoperability

This chapter explains how inter-node protocols deliver an interaction to the correct node while preserving verifiable context; it does not extend "request delivered" into "resource published," nor "node authenticated" into "resource control verified."

```
Envelope -> node authentication -> request signing -> business validation -> status handling ->
response recording
```

In the OpenAgenet (OAN) implementation, inter-node operations are usually provided with structured request and response types by a shared protocol crate, while the concrete node service is responsible for HTTP routing, storage, and background tasks. Therefore, when reading a request path, verify all three layers together: the HTTP method and path, the JSON envelope fields, and the node's internal acceptance or publication state. `oan-protocol-common/schemas/agent-contract/openapi.yaml` is currently the contract entry point and a version placeholder, not a complete automatically generated description of every node interface; this file alone cannot establish that an interface is enabled in every deployment.

15.1 Common Protocol Envelope and Request Headers

The common envelope may use the following field semantics, while the business body remains defined by the corresponding interface:

```
{
  "protocolVersion": "oan-resource-2026",
  "requestId": "req-01",
  "purpose": "cdn-publish",
  "method": "POST",
  "path": "/cdn/resources",
  "aud": "did:oa:...",
  "requestTimestamp": "2026-09-07T10:00:00Z",
  "requestNonce": "random-value",
  "bodyHash": "sha256:...",
  "proof": {
    "type": "Ed25519Signature2020",
    "creator": "did:oa:...",
    "created": "2026-09-07T10:00:00Z",
    "proofPurpose": "assertionMethod",
```

```

    "proofValue": "z...",
    "cryptoSuite": "Ed25519Sha256",
    "hashAlgorithm": "SHA-256",
    "verificationMethod": "did:oon:...#key-1"
  }
}

```

The example corresponds to the fields of `SignedRequestEnvelope`, rather than adding another set of `sourceDid`, `targetDid`, `timestamp`, or `sequence` fields. `aud` in the envelope denotes the target audience, and `proof` carries the request signature; business payloads such as a `ResourcePackage`, registration submission, or batch item are carried outside the envelope by the specific request structure. The sender generates `requestId`, `requestTimestamp`, `requestNonce`, and `bodyHash`; the receiver validates the audience, time, nonce, digest, signing subject, and signature coverage. Whether unknown fields are accepted is determined by the protocol version and the deserialization rules of the specific structure. Missing required fields or an inconsistent digest must be rejected.

15.2 Inter-Node Authentication

Check	What it proves	Failure handling
Node DID	Who the communicating party is	Reject authentication
Node authorization VC/governance status	Whether it has the current communication or business qualification	Reject or degrade to read-only
Validity period and status	Whether the credential is still valid	Do not use an expired or revoked credential
Authorization domain and capabilities	Whether the requested action is within scope	Reject an unauthorized action
Resource control proof	Whether the resource is controlled by the submitter	Enter business-validation failure

Authentication decisions should be completed in order: first confirm the connection target and node DID, then resolve the verification method in the node DID Document, next verify the node authorization VC or governance status, and finally check that the requested action falls within the authorization domain. Node authorization and resource control are two different paths: the former indicates that a node may communicate as a particular network role, while the latter is established by the control proof in the resource submission, showing that the submitter can control the resource DID. Even after node authentication succeeds, the receiver must still perform business validation on the `ResourcePackage`, DID Document, and control proof.

15.3 Signing Upstream Requests

The signature must cover at least the canonicalized method, path, target node, time, nonce, request ID, and `bodyHash`. The verifier should resolve the public key according to `proof.verificationMethod` or a compatible verification-method identifier, and confirm that `proof.creator`, the verification-method controller, and the trusted upstream node identity are consistent.

The current shared protocol structure uses `proof`, rather than defining separate `kid` and `signature` fields in `SignedRequestEnvelope`. `proof.verificationMethod` points to a verification method in the DID Document, while `proof.creator` identifies the signature creator; `oan-crypto` canonicalizes the signable value according to the cryptographic suite before signing or verifying. For `Ed25519Sha256` and `Sm2Sm3`, the signature input is the bytes of canonicalized JSON; legacy compatibility suites may first hash the canonicalized content, so the protocol version and cryptographic suite must jointly define the signature input. The HTTP header example applies only when a specific deployment maps envelope fields to request headers; it cannot replace the JSON envelope itself.

Signature verification failure, a mismatched target `aud`, a mismatched `bodyHash`, a verification method that does not belong to the signing subject, or an expired request should all be rejected before entering the resource-write and publication queues, with the `requestId` and an error category useful for operations recorded.

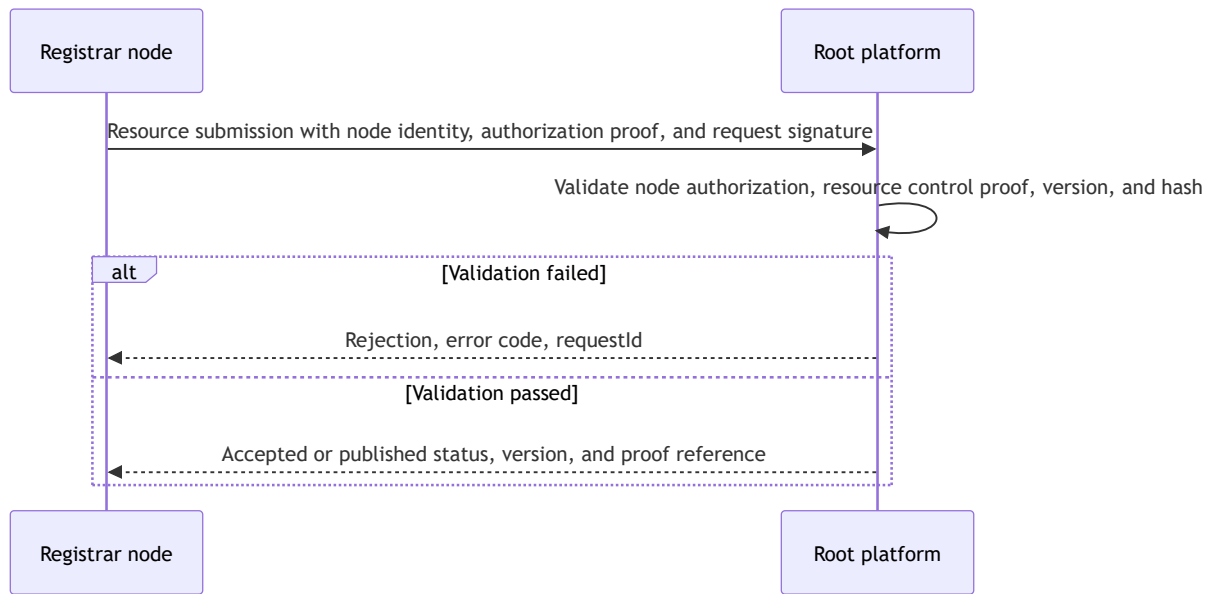
```
X-Request-ID: req-01
X-Node-DID: did:oan:...
X-Timestamp: 2026-09-07T10:00:00Z
X-Nonce: random-value
X-Body-Hash: base64url(...)
X-Signature: jws-or-equivalent
```

15.4 Node Authorization VC Exchange

Node authorization VC exchange should follow the order "carry or reference -> resolve issuer DID -> verify signature -> query status -> check authorization domain and capabilities." Caching may reduce repeated resolution, but it cannot bypass validity periods, revocation status, or governance-event refreshes.

The issuer of a node authorization VC is a trusted institution with governance responsibility or the Root platform. The receiver should verify the issuer DID Document and signature, and check the subject node DID, role, authorization domain, capabilities, validity period, and revocation or suspension status in the VC. The VC may be carried as a complete credential in the request or obtained through a reference agreed by both parties; in the latter case, the referenced content must be traceable and its digest verifiable, and a cache hit must not cause the status check to be skipped. A resource registration VC is the result of resource registration; it cannot in turn prove that a Registrar node is qualified for inter-node communication.

15.5 Registrar node to Root platform Protocol



A Root platform response of "accepted" only means that the request has entered the processing flow; the registration VC, Root proof, content distribution, and Discovery node index status should be expressed separately.

The business payload submitted by a Registrar node to the Root platform is represented by `ResourceVerifyAndPublishRequest`, which includes the Registrar node DID, the resource registration submission, and `upstreamAuth`. The registration submission includes the resource DID, resource type, DID Document, metadata, version, individual digests, registration credential, and subject control proof. The Root platform first verifies the upstream envelope and node qualification, then validates the binding among the resource control proof, DID Document, and digests; only after that does it enter the Root platform's persistence and subsequent publication flow. On timeout or connection interruption, the Registrar node should query the acceptance status using the original `requestId`; it must not determine whether the resource has been published solely from receiving a response again or from the number of local retries.

15.6 Root platform to content distribution platform Protocol

The distribution request should be associated with the resource DID, version, `ResourcePackage` digest, Root proof reference, and distribution sequence. After receiving it, the content distribution platform recomputes the digest, confirms the source and version, and then returns a received, completed, or failed status; successful receipt does not mean that a Discovery node has indexed the resource.

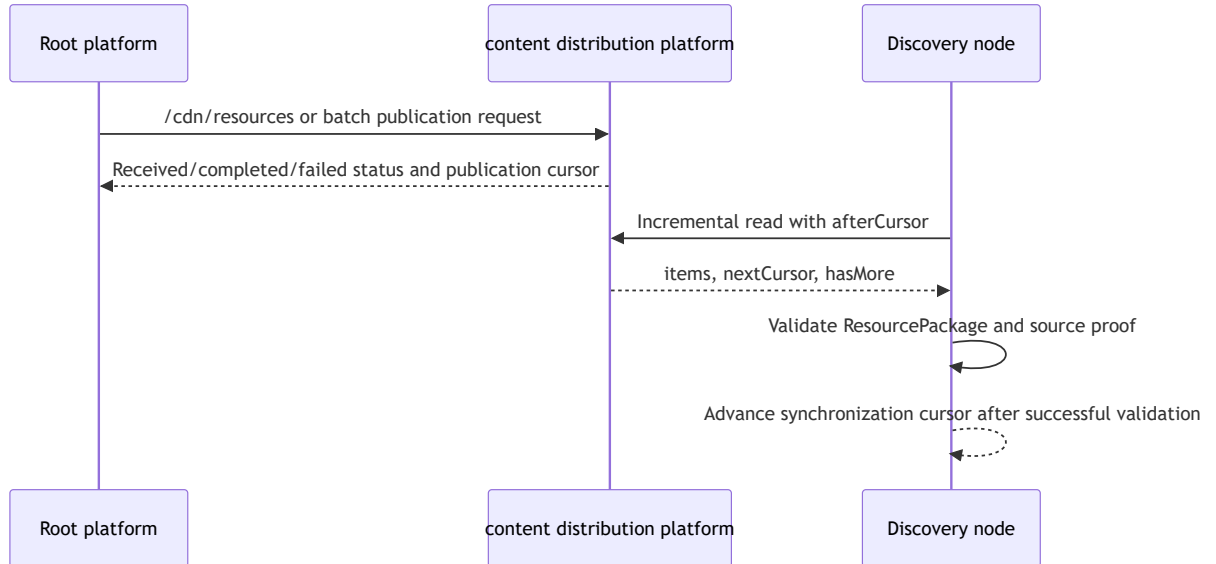
In the shared protocol, a single content distribution request uses `ResourceCdnPublishRequest`, while a batch request uses `ResourceCdnBatchPublishRequest`; batch items are associated with the Root platform's publication order through `publicationCursor`, and the common envelope proves the upstream call context through `upstreamAuth`. The corresponding content distribution path con-

starts in the code are `/cdn/resources` and `/cdn/resources/batch`, but the actual accessible address still depends on the service route prefix and reverse-proxy configuration. After validation, the content distribution platform forms its own readable resource or manifest, and the Root platform's publication-completed status and the Discovery node's indexing-completed status should be recorded separately.

15.7 content distribution platform to Discovery node Protocol

Discovery node synchronization should use a cursor or batch sequence, and should revalidate the digest and Root proof reference for each resource version. Duplicate batches should be processed idempotently, and the cursor must advance only after the batch content has been successfully validated.

For incremental data read from the content distribution platform, the shared protocol uses `ResourceCdnIndexResponse` to express `items`, `count`, `afterCursor`, `nextCursor`, and `hasMore`; each item carries the current cursor and `ResourcePackage`. Notifications sent by the Root platform to a Discovery node instead use information such as the notification batch, start and end sequences, and content distribution address; the two forms must not directly mix field names. A Discovery node should write to its local index and advance the next cursor only after the `ResourcePackage`, DID Document, digests, and Root proof association have all passed validation; a failed batch retains the original cursor and is retried after remediation.



15.8 Discovery Response Signatures and Source Proofs

A Discovery response should be able to distinguish at least the following information:

Information	Meaning
Query node DID	Who generated this response
Data source	Which distribution or index source supplied the data

Information	Meaning
Index version/time	Freshness of the read model used for the query
Match reason	Which fields or tags matched
Root proof reference	External evidence entry point for the publication fact
Response signature	Prevents the response from being replaced

A response signature proves that the node is responsible for the response content; it does not replace the resource control proof or Root proof.

The current `ResourceDiscoveryResponse` structure contains `discoveryDid`, `candidates`, `createdAt`, and an optional `proof`; the Discovery node query implementation may return an empty `proof`, so response signatures are a reserved protocol capability or a capability of deployments after enablement and should not be described as the default behavior of all current instances. If enabled, the signature should cover the query result, creation time, query node DID, and the necessary index or source digest; the caller must still independently check the DID Document, resource control proof, and Root proof of the candidate resource. The `score` in a candidate only indicates matching relevance; it is not a trust level or authorization conclusion.

15.9 Sequence Numbers, Request Identifiers, and Replay Protection

Field	Primary use	Reuse on retry
<code>requestId</code>	Associates one business request with its logs	Reuse for the same logical request
Idempotency key	Prevents repeated execution of submission side effects	Reuse for the same operation
<code>nonce</code>	Prevents replay of signed messages	Usually generate again
<code>sequence/cursor</code>	Ensures batch or event order	Use according to the synchronization protocol
Time window	Limits request freshness	Update when resigning

`requestId` connects one logical operation across the Registrar node, Root platform, content distribution platform, and Discovery node; `requestNonce` prevents the same signed envelope from being accepted again; `publicationCursor`, `sequenceFrom`, `sequenceTo`, `afterCursor`, and `nextCursor` propagate order or support incremental reads. Their numeric spaces and advancement rules differ: a request ID must not be treated as an event sequence, and pagination parameters from a Discovery query must not replace a publication synchronization cursor. When retrying the same logical request, `requestId` may be retained, but a new signed message should normally use a new nonce, and the receiver should determine whether it has already been processed according to idempotency rules.

15.10 Pagination, Filtering, and Compatibility Rules

Client page pagination, Discovery query pagination, and node synchronization cursors must not be mixed: page pagination serves presentation, Discovery pagination serves candidate results,

and synchronization cursors serve incremental propagation. When a cursor is invalid, return an identifiable error or require resynchronization; do not silently continue from an arbitrary position.

The current shared query structure has a default `limit` of 10. A Discovery query may also carry conditions such as resource type, capability tags, protocol, version, and version mode; this is only the default behavior of the Discovery query structure and does not represent the default page size of every management list interface. `hasMore` and `nextCursor` in a synchronization response serve incremental propagation, while `score`, version mode, and candidate ordering in query results serve user retrieval. During protocol evolution, new filter fields may be handled according to compatibility rules, but changes to cursor meaning, ordering stability, or version mode should be explicitly communicated to callers through version documentation.

15.11 Timeouts, Retries, and Idempotent Operations

Operation	Default action after a network timeout	Reason
Read-only query	Retry after limited backoff	Usually has no business side effect
Status query	Use the original request ID to query for confirmation	Distinguishes incomplete from not started
Resource submission	Do not blindly replay; query idempotent status first	May already have caused registration or publication side effects
Synchronization batch	Retry with the original cursor and batch	Preserves incremental order and idempotency

Retries must distinguish "no response received" from "explicit failure received": a network timeout only means that the result is unknown, so resource submission, publication, and state-change operations should first query with the original request identifier; explicit parameter, authorization, or signature errors should not be retried automatically. A background publisher may resume processing at the same publication cursor, and the receiver should identify duplicate messages using the combination of resource DID, version, digest, and cursor to avoid duplicate writes or incorrect progress. Specific timeouts, backoff, and concurrency are determined by deployment configuration; this chapter does not fix the values of a particular environment as protocol constants.

15.12 Protocol Version Negotiation

Version negotiation should declare at least the protocol version, supported capabilities, and incompatible constraints. When there is no common version, fail explicitly; do not infer from fields or silently downgrade to an insecure path.

```
{
  "protocolVersions": ["oan-resource-2026"],
  "capabilities": ["signed-response", "cursor-sync"],
  "required": ["request-signature"]
}
```

The example expresses negotiation information; it does not mean that all current nodes provide an independent capability-negotiation endpoint. The current shared protocol uses `oan-resource-2026` as the resource protocol version constant and uses `protocolVersion` in the envelope; `info.version` in `openapi.yaml` is the contract-file version, and the two must not be treated as equivalent. The negotiation result must at least fix field interpretations, signature coverage, cryptographic suite, cursor semantics, and error handling; if the other party supports only an insecure or uninterpretable downgrade path, refuse to establish that business link.

15.13 Message Size, Compression, and Transport Limits

Limits should account for the message before compression, the compressed transfer body, and the decompressed content at the same time; the receiver should apply size and format protections before resource writing or expensive validation. Compression changes only the transport method, not the logical content covered by the signature and digest.

The current shared protocol types define `ResourcePackage`, batch items, and envelope fields, but the common structure file does not uniformly declare message sizes, compression algorithms, or HTTP body limits for all deployments. Therefore, these limits should be jointly determined by the concrete service configuration, reverse proxy, and runtime environment, with verifiable values disclosed in inter-node capability or deployment documentation. Batch synchronization in particular should limit item size, batch item count, and cumulative decompressed size at the same time; when a limit is exceeded, return a clear error before persistence and retain the request ID, without truncating and continuing signature verification or indexing.

15.14 Correlation, Traceability, and Audit Identifiers

A resource propagation path should retain at least the following correlation fields:

```
requestId -> resourceDid -> resourceVersion -> publishSequence
          -> syncCursor -> discoveryIndexVersion -> errorCode
```

Nodes may add local trace identifiers, but must not lose the upstream `requestId`, resource DID, or version; logs must not record private keys or complete sensitive credentials.

For the publication path, the following searchable correlations should be retained at minimum:

Path stage	Key correlation fields	Purpose
Registration acceptance	<code>requestId</code> , resource DID, resource version	Locate one registration request
Root platform publication	<code>ResourcePackage</code> digest, Root proof reference, <code>publicationCursor</code>	Determine the publication fact and order
Incremental synchronization	<code>afterCursor</code> , <code>nextCursor</code> , batch sequence number	Determine whether synchronization was missed or duplicated
Discovery index	Discovery node DID, index time, error category	Explain the freshness of query results

Audit records should retain validation results and state transitions, rather than recording only

HTTP 200. For sensitive VCs, record their type, digest or reference, and verification conclusion, and avoid writing complete credential contents or any private key to ordinary business logs.

15.15 Protocol Compatibility and Deprecation Rules

Field or version deprecation should pass through the stages of "announcement -> compatibility period -> migration -> stop accepting," while retaining the error code and request ID. Changes involving signature coverage, authentication semantics, cursor meaning, or state semantics cannot be solved merely by adding optional fields; the protocol version should be upgraded or an explicit migration rule provided.

Compatibility tests should cover at least: whether an old-version envelope can be explicitly rejected or parsed according to compatibility rules, whether unknown optional fields leave signature semantics unchanged, whether an old cursor can never be mistaken for a new cursor, and whether the signing subject and `aud` are still verified under the same rules. During version migration, the sender should prefer the common version confirmed by both parties, and the receiver should return an associable request ID and the reason for incompatibility in an error response; before stopping acceptance of the old version, node configuration, background publication tasks, and historical cursors should first be migrated. Protocol files, shared structures, and service implementations must be updated together; changing only the version string in the documentation is insufficient.

15.16 References

Source	Type	Link
<code>oan-protocol-common</code>	Code repository: inter-node request, response, error, and event types	https://github.com/wolfbrother/oan-protocol-common
<code>oan-root-services</code>	Code repository: Root platform inter-node interfaces	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: Registrar node interfaces and upstream calls	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	Code repository: Discovery synchronization and query interfaces	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-trust-indexer</code>	Code repository: off-chain trust indexer governance-status query interfaces	https://github.com/OpenAgenet/oan-trust-indexer
Agentic Overlay Network Architecture	IETF draft: node interoperability architecture	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
OAN Resource Identity and Discovery	IETF draft: resource identity and discovery interoperability	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

Chapter 16 Public HTTP API

This chapter organizes the public HTTP APIs of OpenAgenet (OAN) into implementable and testable interface contracts. It covers the public and node-to-node calls of the official website backend, Registrar nodes, the Root platform, the content distribution platform, Discovery nodes, and the off-chain trust indexer. The paths, fields, and error semantics used by the browser, TypeScript SDK, and community skill package should remain consistent. Public APIs return only the materials required by public services; they do not return private keys, local identity backups, database administration capabilities, or access-statistics back-office data. An HTTP success response does not automatically mean that a resource has completed Root platform publication, network-wide distribution, or Discovery indexing.

16.1 API Conventions and Base URLs

Contract item	Unified requirement	Failure behavior
Method and path	Follow the actual route declaration of the service	Return an identifiable route or method error
Media type	Use an explicit Content-Type for JSON requests and responses	Reject a body that cannot be parsed
Request ID	The client may provide one; the server generates one when absent	Keep the value consistent in the response and logs
Time	RFC 3339 strings	Do not silently convert an incorrectly formatted value
Caching	Set it according to data freshness and status	Expired data must carry a degradation indicator

OAN interfaces transmit JSON using UTF-8. Time fields use RFC 3339 strings. DID, version, hash, and cursor values are handled as strings and should not be converted by clients into lossy numeric types. The base URLs for production environments are determined by deployment configuration. The protocol constrains paths and semantics, but does not make the current website domain a fixed condition for every deployment. Requests should carry a request ID that can run through one complete call chain. Responses should express results through stable machine-readable fields; presentation-layer text must not be used as the basis for program decisions.

16.1.1 Service Base Addresses

The current public backend address of the official website and the addresses of individual nodes are jointly determined by the frontend API configuration, node configuration, and deployment

environment. The SDK provides independent endpoint configuration for Registrar nodes, Discovery nodes, the Root platform, and the content distribution platform. Callers must not assume that all services are deployed on the same host or under the same domain. Cross-node requests should also record the target node DID, so that the communication peer is not identified solely by a network address.

16.1.2 Path and Version Conventions

The version in a path should be expressed through an explicit path, protocol field, or capability negotiation. It must not be inferred from a page title or service name. When fields are added compatibly, clients should ignore unknown non-required fields. When field meaning, signature coverage, or status semantics change, the protocol version must be increased or a migration period must be provided. Interface paths, methods, and media types are part of the contract; clients must not guess based on differences in case or trailing slashes.

16.1.3 Request Authentication

Public queries and health interfaces may be accessed anonymously according to the deployment policy. Resource registration, node-to-node submission, and governance operations require, respectively, a resource control proof, node authorization materials, or operations authentication. Browser CORS permission only determines whether a page can read a response; it does not grant resource control. Authentication failures should return stable errors and should not hide permission problems behind an ambiguous 404 or an empty successful result.

16.1.4 Response Format

Successful responses should use JSON objects to express data, status, version, freshness, or source. Error responses should contain at least a machine-readable error code, a request ID, and necessary field locations. A service must not return an HTML error page where JSON is expected. For asynchronous publication and indexing, the response should clearly distinguish accepted, processing, completed, and failed states. An HTTP 200 response alone must not represent completion of the full chain.

16.2 Health, Readiness, and Service Status Endpoints

Health checks should distinguish process liveness, service readiness, availability of critical dependencies, and successful completion of business smoke tests. The website backend's `/health` only indicates that the backend process can respond; it does not prove that the Registrar node, Discovery node, Root platform, or database is operating normally. After an operations deployment, node health, database connectivity, indexer synchronization status, and the minimum registration and Discovery business paths should be checked separately. Public status interfaces should not disclose internal connection strings, tokens, or stack traces.

16.3 Registrar Node Registration Endpoints

Request stage	Must be verified	A success response does not mean
Draft validation	Local identity, required fields, DID Document, and metadata	Submitted or published
Resource submission	Control proof, node authorization, and resource package digest	Root platform publication completed
Status query	Registration, publication, distribution, and indexing stages	All nodes have converged
VC retrieval	Issuer, subject, version, digest, and status	A guarantee of resource business quality
Same-DID update	Original control relationship, preceding version, and new digest	Automatic acquisition of update authority

The reference implementation accepts both `POST /resources/register` and the compatibility entry point `POST /resources/submit`; both enter the same resource registration handler. The browser or SDK generates the resource DID, DID Document, metadata, version, and digest locally, and submits them as a `ResourceRegistrationSubmission`. Private keys and local identity backups are not part of the request body. The Registrar node first checks the structure, resource type, authorization domain, and subject control proof, and then calls the Root platform's verification and publication endpoint with a signed upstream request.

The core fields of a successful response are as follows. The example values are provided only to explain the structure:

```
{
  "status": "submitted",
  "resourceDid": "did:oa:SKLG:example",
  "resourceType": "skill",
  "registrationCredential": {},
  "rootResponse": {}
}
```

`status: submitted` means that the Registrar node has completed its processing of this request and obtained a Root platform response. It does not mean that content distribution and indexing by all Discovery nodes have completed. The reference implementation issues and returns `registrationCredential`; the client should still allow the credential field to be empty or unparseable while preserving the submission result and original response, so that “registration failed” can be distinguished from “credential missing.” `GET /resources`, `GET /resources/{did}`, `GET /registrar/status`, and `GET /registrar/stats` are used to read registration records and node status, but the public scope is still determined by the deployed reverse proxy and access policy.

16.4 Root Platform Publication and Status Endpoints

The Root platform separates public read routes from management write routes. The resource publication entry point is `POST /root/resources/verify-and-publish`, called by a Registrar node with `upstreamAuth`. The Root platform re-verifies the source node identity, node authorization, resource

DID, DID Document, digest, and control proof. Resource queries use `GET /root/resources/{did}`. The historical version list and a specified version use `/root/resources/{did}/versions` and `/root/resources/{did}/versions/{version}`, respectively.

Interface category	Typical path	Main boundary
Root platform identity and status	<code>/root/did</code> , <code>/root/status</code>	Provides public identity or runtime summaries; does not expose management capabilities
Resource reads	<code>/root/resources/{did}</code> and version paths	Returns saved resource versions and publication materials
Publication writes	<code>/root/resources/verify-and-publish</code>	Requires a trusted upstream envelope from a Registrar node
Publication queues	<code>/root/queues/cdn-publish</code> , <code>/root/queues/discovery-notify</code>	Used to observe asynchronous propagation; whether they are public depends on the deployment policy
Governance management	Authorization, domain-update, and revocation paths	Management routes that require separate administrator authentication

A successful publication response means that the Root platform accepted and saved a compliant resource version. The content distribution publisher and Discovery notification tasks then continue processing it. Callers should confirm propagation progress through the resource version, publication queue, or subsequent status, and must not interpret one 2xx response from the Root platform as network-wide visibility.

16.5 Content Distribution Endpoints

The content distribution platform provides single-resource reads, batch reads, incremental indexing, and split-material reads. `GET /cdn/resources/{did}` returns a resource package. `POST /cdn/resources/batch-get` reads multiple resources by DID. `GET /cdn/documents/{did}` and `GET /cdn/metadata/{did}` return the DID Document and metadata, respectively. Catalog interfaces are located under `/cdn/catalog/...`. The Root platform publisher writes through the protected `POST /cdn/resources` or `POST /cdn/resources/batch`. `POST /cdn/purge` is a management operation and should not be exposed as a public write interface.

Incremental synchronization uses `GET /cdn/resources/index?afterCursor=<n>&limit=<n>`. In the current reference implementation, omitting parameters may return a compatibility format; supplying cursor or quantity parameters returns a structured page:

```
{
  "items": [],
  "count": 0,
  "afterCursor": 42,
  "nextCursor": 42,
  "hasMore": false
}
```

A Discovery node should save `nextCursor` only after fully validating the resource packages in the batch; it must not advance the cursor early merely because the HTTP request succeeded. The current service imposes an implementation-level body limit on batch publication requests. This is a deployment safeguard of the reference implementation, not a protocol value permanently fixed for every OAN node.

16.6 Discovery Query and Resource Detail Endpoints

Query result field	Purpose	Freshness or trust boundary
<code>resourceDid</code>	Precisely identifies the resource	Still requires resolution and control verification
<code>resourceType/version</code>	Filtering and version selection	Does not replace publication proof
<code>sourceNode</code>	Tracks the result source	Node authorization requires a separate check
<code>indexedAt/cursor</code>	Determines index freshness	Does not equal on-chain finality
<code>matchExplanation</code>	Explains a semantic match	Does not equal invocation authorization

The Discovery query entry point is `POST /discovery/resources/query`. The request may contain a natural-language description, resource type, capability tags, protocol, version, version mode, and quantity limit. If the query text can be recognized as a complete resource DID, the reference implementation prioritizes an exact-DID query. Otherwise, it prioritizes an available semantic index and falls back to a keyword-candidate flow when the semantic backend is unavailable or produces no result. `discoveryDid` in the response identifies the Discovery node that returned the results. `createdAt` identifies when the response was generated, and a candidate's `score` expresses only retrieval relevance.

```
{
  "query": "I need a tool that can search code repositories and summarize the project structure
  .",
  "resourceType": "skill",
  "capabilityTags": [],
  "versionMode": "latest",
  "limit": 10
}
```

Indexed-resource reads and diagnostic interfaces include `/discovery/index/resources`, `/discovery/index/resources/{did}`, `/discovery/index/stats`, and `/discovery/query/stats`. Whether these are exposed to the Internet should be determined by the deployment exposure policy. An empty candidate array is a valid query result and should not be converted into a network error. Semantic-backend fallback, index lag, or snapshot degradation should be reflected in observable information; duplicate candidates must not be manufactured to hide differences between data sources.

16.7 DID Resolution Endpoints

Infrastructure nodes return their own DID Documents through `GET /registrar/did`, `GET /root/did`, and `GET /discovery/did`, respectively. A resource DID can be resolved through the Root platform resource detail, the content distribution platform's `/cdn/documents/{did}`, or the detail of a resource already indexed by a Discovery node. The appropriate choice depends on whether the caller needs the authoritative saved version, a distributed copy, or a Discovery read model.

The `id` in a resolution response should exactly match the requested DID and preserve public verification methods, verification relationships, service endpoints, and OAN metadata. Clients need to distinguish three types of result: an invalid DID format is a request problem, a nonexistent resource is an absent record, and temporary unavailability of a dependency or node is a recoverable failure. No resolution path should return a browser local identity backup, private key, or server key file.

16.8 VC and Proof Retrieval Endpoints

Registration VCs, node authorization VCs, and Root proofs of trusted publication cover different facts. A Registrar node stores `registrationCredential` in the successful registration response and resource record. The Root platform may return the resource package and its Root proof through resource detail and version interfaces. The `../root-authorization` status interfaces of Registrar nodes and Discovery nodes expose the authorization materials or verification status observed by the respective node. The Root platform's infrastructure-authorization VC issuance entry point is a protected operation and should not be treated as an anonymous self-service retrieval interface.

Evidence	What it proves	What must still be checked after retrieval
Registration VC	A particular Registrar node processed a particular resource registration	Issuer, subject, resource digest, signature, and status
Node authorization VC	An infrastructure node was granted a role and authorization domain	Issuer, node DID, role, validity period, and revocation status
Root proof of trusted publication	A particular resource version and digest were accepted by the Root platform	Resource DID, version, digest, signature, and governance status

Returning a credential object from an interface does not mean that the credential has been verified. The caller should resolve the issuer's public key according to `proof.verificationMethod` and check credential time and governance status. When status cannot be obtained, the caller should explicitly return “indeterminate” or tighten high-risk operations rather than assuming validity.

16.9 Governance Status and Event Endpoints

The Root platform provides infrastructure node directories through `/root/registrars`, `/root/discovery-nodes`, and their DID detail interfaces. It provides a governance-event read model through

`/root/bulletin/events` and `/root/bulletin/events/{sequence}`. Registrar nodes and Discovery nodes expose the authorization status observed by each node through `/registrar/root-authorization` and `/discovery/root-authorization`, respectively. A Discovery node also expresses its current authorization domain through `/discovery/authorized-domains`.

The off-chain trust indexer's data is an off-chain projection of on-chain governance facts. Status responses should carry the latest sequence, observation time, or synchronization position so that callers can assess freshness. The Root platform's authorization, authorization-domain update, and revocation interfaces are management routes and require administrator authentication. If on-chain reads fail, the index sequence does not increase for an extended period, or local state cannot be confirmed, the Root platform and Discovery nodes should mark results as stale or indeterminate and tighten publication, synchronization, or Discovery-visibility decisions.

16.10 Metrics and Observability Endpoints

The website backend organizes multiple nodes and data sources through public read interfaces such as `/api/public/home/summary`, `/api/public/network/summary`, `/api/public/network/governance-events`, node directories, and node statistics. Node-side interfaces also include `/registrar/stats`, `/discovery/index/stats`, `/discovery/query/stats`, and `/cdn/catalog/resources/stats`. The website aggregation is a display read model and should not be treated as a replacement for the authoritative raw status of each node.

Metric fields should provide the generation time, statistical window, source, and missing-data status together. Windows such as `TODAY`, `THIS WEEK`, and `THIS MONTH` require a common time zone and explicit start and end boundaries. The cumulative resource count must not be confused with the number of additions within a window. Public interfaces return only necessary aggregate information. Access-statistics details, complete logs, database paths, management tokens, and internal error stacks remain operations data. The internal reporting entry point used by nodes to report statistics to the website should have authentication and exposure boundaries independent from public read interfaces.

16.11 Request and Response Models

Requests and responses should be able to express at least four results: success, processing, failure, and indeterminate:

```
status = accepted | processing | completed | failed | unknown
requestId = used to trace one call across nodes
```

Clients must determine asynchronous stages from machine-readable status and must not infer that publication, distribution, and Discovery indexing have completed from HTTP 200 or page text alone.

Request and response models are stable boundaries for interface fields, statuses, and errors. Field presence, types, enumerations, hash algorithms, time formats, DID syntax, and nesting relationships should be consistent with `oan-protocol-common` and the actual node types. Handling of

unknown fields should be determined by the protocol version. Missing required fields, duplicate fields, empty values, and incompatible versions should produce errors that can be located.

The actual type relationships of the core node interfaces are as follows. **Value** in the table means that the current handler returns dynamic JSON; it does not mean that every field forms a stable contract. Specific fields still need to be checked against the corresponding handler, shared structures, and error tests.

Service	Method and path	Input type	Output type or stable fields
Registrar node	POST /resources/register, POST /resources/submit	ResourceRegistrationSubmit	Value: status, resourceDid, resourceType, registrationCredential, rootResponse
Root platform	POST /root/resources/verify-and-publish	ResourceVerifyAndPublishRequest	Value: verification queue status, resource version, three hashes, lifecycle, and CDN queue status
Content distribution platform	POST /cdn/resources, POST /cdn/resources/batch	ResourceCdnPublishRequest, ResourceCdnBatchPublishRequest	Value: publication status, cursor, and batch requested/failed items
Content distribution platform	GET /cdn/resources/{did}	Path parameter did	ResourcePackage
Discovery node	POST /discovery/resources/query	ResourceDiscoveryQuery	ResourceDiscoveryResponse
Off-chain trust indexer	GET /v1/status, GET /v1/events	Optional limit query parameter	Dynamic status JSON; the event response contains an events array

The business stage of an interface cannot be inferred only from the HTTP status code. When a Registrar node returns **submitted**, the registration record has been written. When the Root platform returns **resource-verified-and-queued**, the resource has passed the Root platform's current verification and entered the distribution queue. When the content distribution platform returns **published**, the resource package has been written to the distribution platform. When a Discovery response returns candidates, it only means that the current Discovery node's index can see them. Callers still need to make subsequent judgments using the resource DID, version, hashes, Root proof, governance status, and freshness.

16.11.1 Registration Requests and Responses

A registration request must associate the resource DID, DID Document, resource type, meta-data, resource package, version, hashes, Registrar node, and control proof. Before signature verification, the server should confirm the request path, target, challenge, nonce, and content digest; only after verification should it perform duplicate checks and writes. The response should separately express registration acceptance, the registration VC, the Root platform submission result,

and retryable errors. It must not make the client infer completion of all stages from a single “success” message.

The reference implementation's public registration entry point directly accepts `ResourceRegistrationSubmission`. When a Registrar node forwards the request to the Root platform, it constructs a `ResourceVerifyAndPublishRequest` and binds the method, path, target, time, nonce, and business-body digest in `upstreamAuth`. The browser submits only resource materials and a control proof; it should not construct or hold the upstream signing identity of an infrastructure node.

16.11.2 Publication Requests and Responses

A publication request must associate the Registrar node source, resource version, resource package digest, DID Document hash, metadata hash, authorization materials, and request ID. The publication response should carry a Root proof of trusted publication or clearly state that it has not yet been produced, and should return a publication status, batch reference, or cursor reference. Hash mismatches, an unauthorized node, an invalid control proof, and temporary dependency unavailability should be distinguished as non-retryable and retryable failures.

When the Root platform submits to the content distribution platform, single-item and batch requests use `ResourceCdnPublishRequest` and `ResourceCdnBatchPublishRequest`, respectively. Batch items are associated with the Root platform publication order through `publicationCursor`. The accepted count, failed items, or completion cursor in the response is used to resume a publication task; it does not mean that a Discovery node has completed indexing.

16.11.3 Discovery Requests and Responses

A Discovery request may use a resource DID, resource type, capability tags, protocol, authorization domain, lifecycle, keywords, or a natural-language task description. The response should return candidate DIDs, versions, resource types, match explanations, the source Discovery node, index version, governance status, and proof references. A pagination cursor must be bound to the query conditions and the ordering snapshot. An empty result is not an interface failure; service unavailability, index lag, and snapshot degradation should be clearly marked.

The current `ResourceDiscoveryResponse` contains `discoveryDid`, `candidates`, `createdAt`, and an optional `proof`. A candidate contains the resource DID, resource type, score, version, lifecycle, capability tags, authorization domain, service endpoints, protocol bindings, resource-package information, and Root proof, among other fields. When an optional field is absent, the client must not use presentation-layer default text to fabricate a trusted fact.

16.11.4 Status and Error Responses

Status and error responses use HTTP status codes for coarse-grained results, JSON error codes for business reasons, and `requestId` for trace correlation. An error may contain a field path, retryability, recommended wait time, and current processing status, but must not echo private keys, complete control proofs, or internal exception stacks. The client should retain the original error for auditing while giving the user an explanation appropriate to its sensitivity.

The current error bodies of multiple Rust nodes use the concise structure `{"error": "machine_readable_reason"}`, with the HTTP status code distinguishing request errors, permission errors, or service errors. Not every existing response contains `requestId`, a field path, and retry guidance. These fields are therefore directions for interface consistency, and callers must not assume that every node currently returns a complete error envelope.

16.12 HTTP Status Codes and Common Errors

Status	Typical meaning	Client action
400	Invalid request format or field	Correct the input and retry
401/403	Identity or authorization requirements are not met	Do not retry blindly; check the credential and authorization domain
404	Resource or route does not exist	Distinguish an unknown resource from a path error
409	DID, version, or idempotency conflict	Query the current status before deciding whether to update
429	Rate or concurrency limit exceeded	Retry according to Retry-After or a backoff policy
5xx/timeout	Service or dependency failure	Retry a limited number of times while preserving the request ID

It is recommended that 2xx indicate that the request was processed according to the interface semantics, 4xx indicate that the request, authentication, authorization, or business preconditions were not satisfied, and 5xx indicate a server-side or dependency failure. **202 Accepted** can indicate only that an asynchronous task has been accepted; it cannot indicate that publication is complete. **409 Conflict** can be used for a same-DID version conflict or a duplicate state that requires client confirmation. **429 Too Many Requests** should be accompanied by rate-limit information. Services should avoid returning arbitrary different status codes for the same business reason.

The client should first inspect the HTTP status and then parse `error` or the business status in the JSON. A connection failure, TLS failure, or browser CORS rejection has no usable HTTP response and must not uniformly be displayed as “the server returned 500.” Sending POST to an endpoint that supports only GET normally produces **405 Method Not Allowed** at the routing layer; the method and reverse-proxy forwarding rules should be checked first.

16.13 Rate Limits, Caching, and Security Headers

Control	Public queries	Registration or node-to-node requests
Rate	May be limited by source and interface	Should consider subject, node, and idempotency key
Request size	Limit query descriptions and page sizes	Limit resource packages and batch sizes

Control	Public queries	Registration or node-to-node requests
Caching	Allow read-only caches with freshness information	Do not cache results that change authorization decisions
Security headers	TLS, CORS, and necessary security headers	Add authentication and source verification

Rate limiting should consider at least the source IP, authenticated subject, node DID, resource DID, and interface category, and should use different quotas for registration, Discovery, synchronization, and health requests. Read-only public queries may use short-lived caches, but DID Documents, VCs, Root proofs, governance status, and Discovery results must carry a version or freshness boundary. A conditional request's ETag must correspond to the response content version. Security headers, content-type checks, and response-body size limits should be enforced at the gateway or service layer. Cache testing should at least verify the following: the first request obtains data and freshness information; a conditional request does not return an incorrect version; after governance revocation, an old cache does not continue authorizing writes; and when the backend is unavailable, an old snapshot is clearly marked rather than disguised as a real-time result. Write interfaces and signed requests should not be stored by a shared cache.

Request-body limits, query-count boundaries, and CORS configuration in the current node code provide the first layer of protection. Production deployment also requires the reverse proxy to handle connection counts, rates, timeouts, and request-size limits.

16.14 CORS, Browser Clients, and Cross-Origin Policy

When a browser calls a node API, the server should allow only the necessary official website origins, methods, and request headers. A preflight request must not broaden business authorization. **Access-Control-Allow-Origin: *** must not be used together with credentials to transmit sensitive materials. Public cross-origin reads by the official website and node-to-node authentication are two separate controls: the former serves the browser security model, while the latter verifies the request subject and permissions. CORS changes should be a deployment check item to prevent the frontend from producing an opaque **Failed to fetch** error.

Registrar nodes, Discovery nodes, and the content distribution platform all construct their CORS layer from configuration. The website backend preferentially uses the configured frontend origin and falls back to a more permissive policy only when the configuration value cannot be parsed. Production environments should avoid triggering this fallback branch and should perform registration, Discovery, and preflight smoke tests from both Chinese and English pages after deployment. When a browser displays **Failed to fetch**, check the request URL, HTTPS mixed content, certificate, DNS, reverse proxy, preflight response, and allowed origins in sequence; do not inspect only the backend business logs.

16.15 Boundary Between Public Endpoints and Operations-Only Endpoints

The public boundary is divided into “public reads, resource-controller signatures, node-to-node authorization, and operations-only” categories. Public interfaces may provide health information, public catalogs, published resources, and limited metrics. Resource-controller interfaces receive a control proof but not a private key. Node-to-node interfaces require node identity, authorization VCs, and request signatures. Operations interfaces are exposed only within a controlled network or authentication boundary. Access-statistics data may be read only by back-office or operations-authorized subjects; ordinary visitors must not query details through the website or public APIs.

16.15.1 Public Query Interfaces

Public query interfaces include public catalogs, Discovery queries, published resource details, public DID Documents, public proofs, and health information. Returned data should be filtered by a field allowlist, hiding local file paths, private endpoints, database-management fields, and unpublished credentials. Public queries must still observe lifecycle, governance filtering, and authorization-domain visibility; anonymous access must not bypass these decisions.

The website's `/api/public/...` routes mainly provide aggregated display data and do not mean that all node routes are automatically public. Deployment should establish an allowlist of public paths and verify that anonymous access can read only filtered data. `POST /api/public/site-analytics/visit`, which records page visits, is a one-way collection entry point and does not provide an interface for reading statistical details.

16.15.2 Node-to-Node Interfaces

Node-to-node interfaces support cooperation among Registrar nodes, the Root platform, the content distribution platform, Discovery nodes, and the off-chain trust indexer. Requests should carry the source and target identities, authorization materials, protocol version, request ID, time window, nonce, content digest, and signature. Node-to-node interfaces should not be called directly by the website browser in place of a node; browser resource submissions should pass through the Registrar node's public entry point and server-side validation.

Typical node-to-node writes include a Registrar node calling `/root/resources/verify-and-publish`, the Root platform publisher calling `/cdn/resources/batch`, and the Root platform sending authorization-scoped synchronization notifications to Discovery nodes. Even when a path is reachable from the Internet, the receiver must still verify the trusted upstream envelope and node authorization; network reachability itself is not an authorization basis.

16.15.3 Operations Interfaces

Operations interfaces include deployment status, internal diagnostics, access-statistics details, database administration, key rotation, and recovery operations. They must not appear in the public API catalog or browser-side code, and should use separate authentication, least privilege, audit logs,

and network isolation. Errors returned by operations interfaces must not be fully passed through a public proxy to ordinary users.

The Root platform's authorization, revocation, and publication-management routes, as well as the website's internal node-statistics reporting entry point, should be separated from public read routes at the reverse-proxy layer. Combining routes in the same process in service code does not mean that they should share the same Internet exposure policy. Operations personnel should check application authentication and proxy path allowlists together.

16.15.4 Access Control and Exposure Boundaries

Access control should check the route, method, authenticated subject, node role, authorization domain, and resource scope together. Exposure boundaries should be enforced through ports, the reverse proxy, and CORS configuration. Before publication, four identities should be used to test allowed and denied paths separately: an anonymous browser, a resource controller, an unauthorized node, and an operations subject. The test should preserve status codes, response fields, and log-correlation evidence.

Test identity	Typical operations that should be allowed	Typical operations that should be denied
Anonymous visitor	Health, public catalogs, Discovery, and public resource reads	Authorization, revocation, database and statistical-detail reads
Resource controller	Submit a resource with a valid control proof	Submit a same-DID update not controlled by the subject
Authorized node	Perform node-to-node calls within its role and authorization domain	Access management operations belonging to another role
Operations subject	Perform management and diagnostics under separate authentication	Bypass auditing or obtain a business subject's private key

16.16 API Examples and Machine-Readable Interface Descriptions

```
openapi: 3.1.0
info:
  title: OAN Agent Contract
  version: 0.1.0
paths: {}
```

The example above is the actual skeleton of `oan-protocol-common/schemas/agent-contract/openapi.yaml`. It identifies the OpenAPI version and contract-file version, but `paths` is still empty. Therefore, it can serve only as a machine-readable contract entry point and must not be treated as a complete description of all public and node-to-node APIs. Interface verification should use the Router definitions of each service, shared protocol structures, configuration, and deployment proxy rules together.

The minimum executable example for a user should begin with a health or public query that requires no signature. The hostname in the following command is a placeholder:

```
curl --fail-with-body \
  --header "Accept: application/json" \
  "https://registrar.example/health"
```

Examples that write to the registration service, Root platform, or content distribution platform require a real DID, digest, control proof, or node-to-node signed envelope. A static fake signature must not be presented as an executable request. When the OpenAPI description is further developed, methods, request bodies, responses, authentication, errors, and examples should be completed route by route, and automated tests should check whether the description remains synchronized with the actual routes.

16.17 References

Source	Type	Link
oan-protocol-common	Code repository: HTTP payloads, error codes, and OpenAPI contract entry point	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	Code repository: Root platform public and internal APIs	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	Code repository: Registrar service APIs	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	Code repository: Discovery service APIs	https://github.com/OpenAgenet/oan-discovery-node
oan-sdk-ts	Code repository: Client HTTP calls and type wrappers	https://github.com/OpenAgenet/oan-sdk-ts
OAN Resource Identity and Discovery	IETF draft: resource identity and Discovery interface direction	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Efficient Agent Discovery Profile	IETF draft: Discovery query profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Chapter 17 Client SDK, Community Skill, and Developer Experience

This chapter explains how the client layer provides different entry points for resource providers, callers, and node operators: `oan-sdk-ts` supports programmatic integration, the community skill package supports agent-assisted resource organization, registration, discovery, and verification, and the official website provides browser-based interaction. All three reuse the OpenAget (OAN) protocol and do not change the semantics of DID, control proofs, registration VCs, Root proof, or governance status. Subject private keys, resource-control private keys, and Local Identity backups remain on the client side; servers receive only public materials and necessary proofs.

Entry point	Primary users	Core responsibilities	Responsibilities it does not assume
TypeScript SDK	Application developers and platform integrators	Types, request construction, registration and discovery calls, and verification assistance	Making trust decisions for users
Community skill package	Resource providers and agent operators	Organizing resources, assisting with registration and discovery, and explaining results	Holding governance keys or replacing node services
Official website	Browser users	Interactive registration, discovery, preview, and local download	Hosting user private keys or guaranteeing endpoint security
Operations assistance	Node operators	Building, configuration checks, deployment, and smoke testing	Automatically deciding production keys and rollback strategies

17.1 TypeScript SDK Scope

The TypeScript SDK consists of protocol types, an HTTP client, governance reads, and workflow assistance. It is suitable for Node.js, browser build environments, and runtimes compatible with `fetch`. Callers can configure the Registrar node, Discovery node, Root platform, and content distribution platform addresses separately. The SDK handles serialization, requests, and response parsing; it does not decide whether a resource is trustworthy on behalf of the caller.

In terms of code organization, `packages/protocol-types` provides shared types such as `ResourceRegistrationSubmission`, `ResourcePackage`, discovery queries, and governance responses;

`packages/client-ts` provides `0anClient` and HTTP calls to the various nodes; `packages/sdk-ts` provides DID Document drafts, resource-package binding, hash formatting, trust summaries, and lifecycle assistance; and `packages/governance-ts` targets governance reads from the off-chain trust indexer. This layering allows an application to depend only on the HTTP client or to combine resource validation and lifecycle assistance when needed, without duplicating protocol structures.

Capability area	Provided by the SDK	Still the caller's responsibility
DID and resource model	Types, parsing, and construction assistance	Confirming resource content and control relationships
Registration	Request construction, submission, and status reads	Safeguarding private keys and confirming the submission target
Discovery	Exact DID queries, semantic queries, and result parsing	Deciding whether relevance satisfies business requirements
Verification	Reading proof fields and verification assistance	Defining authorization policies and making final invocation decisions
Lifecycle	Aggregating registration, publication, distribution, and visibility status	Handling timeouts, retries, and business compensation

The SDK supports two common endpoint-selection patterns. For the official website and community users, the caller can provide a `baseUrl`, from which the client derives service addresses under the official gateway. For multi-node or third-party-node testing, the caller can provide `registrarEndpoint`, `discoveryEndpoint`, `rootEndpoint`, and `cdnEndpoint` separately. Endpoint configuration determines where requests are sent; it does not perform node authorization, resource-control proof generation, or governance decisions for the caller.

17.2 DID Identity and Key Operations in the SDK

Identity and key operations take place on the client. The SDK generates subject identities, resource DIDs, and verification methods, uses private keys to sign control challenges, resource versions, or requests, and writes public keys into the DID Document. A backup may contain subject and resource-control materials, but it must not enter logs, HTTP requests, or indexes. During import, the implementation should validate the structure, the relationship between the DID and public key, and signing capability.

In a Node.js scenario, `identity-store-node` can be used to save subject, agent, and node identities in separate buckets in a local directory. In a browser scenario, the current session is held by page state or browser storage and is exported through a user operation. The common boundary of both implementations is that the DID Document may be publicly distributed, while private keys may only be used for client-side signing; the server receives signed control proofs or node request envelopes, not the original private keys.

☒ Private-key generation, signing, and backup export occur on the client.

☒ Registration requests carry only the public DID Document, resource information, and necessary control proofs.

- ☒ Backup format, version, DID, public key, and signing capability are validated before import.
- ☐ Private keys must not be written to URLs, logs, telemetry, or server-side storage.

17.3 Registration Flow Example

A registration flow should first prepare the Local Identity, resource DID, DID Document, metadata, ResourcePackage, hashes, and control proof, and then call the Registrar node. The following example shows only the client-call boundary; `submission` should be constructed from data that has passed local validation and signing:

```
import { OanClient } from "@openagenet/oan-sdk-ts/client";

const client = new OanClient({
  registrarEndpoint: "https://registrar.example.org",
});
const result = await client.registerResource(submission);

console.log({
  resourceDid: result.resourceDid,
  registrationVc: result.registrationVc ?? null,
});
```

The registration VC may temporarily be absent. The client should preserve the resource DID, request ID, and status; a network timeout cannot by itself prove that registration did not occur. The client should first confirm an explicit success or failure, then query the status by request ID or resource DID, and distinguish the three stages of Registrar node acceptance, Root platform publication, and Discovery node visibility.

The minimum pre-submission checks can be summarized as “object consistency, digest consistency, valid control, and explicit target”: `didDocument.id` must equal `resourceDid`; the resource type must match the DID subject encoding; the digests of the DID Document, metadata, and ResourcePackage must correspond to the actual materials; the control proof must be generated by the corresponding resource-control private key; and the registration endpoint must be a Registrar node explicitly selected by the user. The skill package can assist in identifying missing items, but the final submission must still be confirmed by the client or user.

```
Registration accepted -> Root accepted -> Content distribution completed -> Visible in
discovery index
      |               |               |               |
      Registrar       Root           CDN           Discovery
```

17.4 Exact DID Discovery Example

Exact DID discovery uses the resource DID as the lookup key and does not depend on name similarity. After calling the discovery interface, the client should check the returned DID, version, ResourcePackage, and proof references, and then verify the DID Document, version hash, lifecycle, Root proof, and endpoint policy.


```
const resourceDid = "did:oon:SKDM:example";
const resource = await client.getDiscoveryResource(resourceDid);
if (resource.resourceDid !== resourceDid) {
  throw new Error("discovery_did_mismatch");
}
```

An exact hit proves only that the Discovery node returned a record for the corresponding identifier. It does not by itself prove that the version is still valid, that the node remains authorized, or that the business endpoint can be safely called.

After obtaining candidate results, an application can first perform a consistency check using `resourceDid`, then select a candidate version according to the requested version policy, and finally load the `ResourcePackage` or DID Document to complete evidence verification. If the detail interface returns 404, this usually means that a candidate summary exists in the discovery index but the corresponding detail is not yet readable. The client should preserve this distinction and report it to the caller instead of treating the candidate result as a complete callable object.

17.5 Semantic Discovery Example

Semantic discovery can combine a natural-language task, capability tags, and structured conditions. The English query used on the official website, “I need a tool that can search code repositories and summarize the project structure.”, can serve as an example input:

```
const discovery = await client.discoverResources({
  query: "I need a tool that can search code repositories and summarize the project structure",
  resourceType: "tool",
  limit: 10,
});

for (const candidate of discovery.candidates) {
  console.log(candidate.resourceDid, candidate.score);
}
```

The response should retain hit fields, tags, resource type, protocol, index version, source, and matching reasons. Semantic relevance answers “possibly applicable”; structured conditions answer “satisfies explicit filters”; and trusted evidence answers “whether the current version can be verified.” None of these can replace the others.

The SDK's HTTP-response wrapper should preserve protocol stages rather than compressing every interface into one Boolean success value. A registration call should at least preserve the Registrar node's `status`, resource DID, registration VC, and Root response. A discovery call should preserve `discoveryDid`, the candidate array, creation time, and any optional response proof. Dynamic status or explanation responses should preserve the original JSON so that the upper layer can process actual fields. This allows the SDK to distinguish “registration accepted,” “verified by the Root platform and queued,” “resource discovered,” and “evidence verification completed.”

Semantic-query result handling is best divided into two steps: first narrow candidates with query text, resource type, tags, or protocols; then check each candidate's lifecycle, authorization

domain, service binding, and proof status. A Discovery node may use a semantic index or fall back to keyword search when its backend is unavailable. Callers should focus on the returned candidates and explanation information, rather than hard-coding a particular search-backend name as a business dependency.

17.6 Verification and Trust Evidence Handling

Verification should first check the relationship between the DID Document and resource DID, the public key, and the control proof. It should then separately check the issuer, subject, version, signature, validity period, status, authorization domain, and capability scope of the registration VC, Root proof, and node authorization VC. When a critical proof is missing or expired, the client should mark the resource as unconfirmed or reject its use.

Evidence	Main fact it proves	Fact it cannot prove
Control proof	The submitter can use the corresponding control private key	The Registrar node accepted the registration or the resource was published
Registration VC	The Registrar node's statement about the registration fact	The Root platform published the resource or the endpoint is secure
Root proof	The specified resource version passed Root platform verification and entered the publication process	Every Discovery node has indexed the resource
Node authorization VC	The node has the corresponding qualification within the authorization domain and validity period	Every piece of business data returned by the node is correct
Governance status	A status fact recorded by the governance side at the current time	Resource functional quality or business suitability

A reproducible verification fixture should at least contain the resource DID, DID Document, ResourcePackage, corresponding digests, a control proof or registration VC, and the node DID Document and authorization materials used to validate node eligibility. The test should first confirm success with untampered materials, then separately modify the DID, version, digest, issuer, or lifecycle status and confirm that the SDK returns a clear verification error. Whether a failure blocks use depends on the evidence type: control-proof or digest failure should normally block registration; insufficient semantic matching affects only candidate ranking; and uncertain governance status should block high-risk invocation or require human confirmation.

17.7 Community Skill Package Responsibilities

The community skill package uses scripts and documentation to assist with resource organization, tag recommendation, DID generation, registration requests, discovery queries, evidence verification, and result interpretation. It is not a Registrar node, Root platform, or Discovery node, and it must not hold governance keys. User confirmation, private-key use, sensitive paths, and the final publication decision remain under the control of the user or operator.

The typical interaction follows the sequence “read materials → form a draft → user confirmation → local signing → call the service → explain the result.” The skill package can identify missing fields, recommend semantic tags, and call the SDK, but it must not silently replace resource endpoints, expand authorization domains, or present recommendations as statements made by the resource provider.

The public workflow entry points of `0anSkill` include `draftRegistrationFromResourceDescription()`, `registerFromResourceDescription()`, `registerBatchFromResourceDescriptions()`, `discover()`, `lifecycle()`, and read-only governance assistance methods. A resource-description workflow returns candidate information, missing inputs, quality issues, and recommended actions. This allows batch onboarding to repair items individually instead of losing the diagnostics for the entire batch because one resource failed. The skill package uses the official public gateway by default; third-party nodes participate only when explicitly configured by the user.

17.8 Resource Authoring and Registration Assistance

Resource-authoring assistance should uniformly handle agent services, skill packages, MCP servers, and tool APIs, and guide the user in filling in the name, description, use cases, endpoints, protocols, version, capability tags, authorization domains, and `ResourcePackage` information. Recommended tags are not final declarations; before generation, the implementation should check the consistency of the DID Document, metadata, and `ResourcePackage` hashes.

Input category	Minimum checks	Output purpose
Identity and control	Subject DID, resource DID, and verification methods	Generate the control proof
Capability semantics	Name, description, use cases, and capability tags	Semantic governance and discovery
Invocation information	Protocol, endpoint, and authentication hints	Filtering and negotiation before invocation
Version materials	Version, predecessor version, and <code>ResourcePackage</code> digest	Updates, idempotency, and integrity checks
Governance boundary	Resource type, authorization domain, and lifecycle	Admission and discovery filtering

When organizing resources, `authorizedDomains` and `capabilityTags` should be processed separately. The former is a machine-readable domain used by the Registrar node for authorization-scope decisions; the latter is a semantic-discovery signal. The skill package may propose tags from the description, but it must not infer an authorization domain from the DID subject encoding, endpoint hostname, or natural-language description. When the fact cannot be determined, the missing information should be returned to the user for completion.

17.9 Node Operators and Deployment Assistance

Node-operations assistance can perform dependency checks, configuration checks, Rust compilation, service startup, health checks, and post-deployment registration and discovery smoke tests.

The server, domain, keys, production configuration, and rollback plan require human confirmation.

- ☒ Check processes, listening ports, and health endpoints.
- ☒ Check database connectivity and required table structures.
- ☒ Check the off-chain trust indexer status and whether its synchronization sequence continues to advance.
- ☒ Verify the minimum path for structured registration responses, exact DID discovery, and semantic discovery.
- ☐ Do not treat the success of a single health endpoint as a substitute for a complete business smoke test.

The acceptance sequence for operations assistance should follow the business propagation sequence: first check processes and health endpoints, then check the database and node authorization status, next verify that a resource registration request receives a structured response, and finally confirm the discovery path using an exact DID query and a semantic query. When the Registrar node returns success but the Root platform, content distribution platform, or Discovery node is unavailable, the result should identify the corresponding stage rather than compressing the entire path into a single “deployment successful” conclusion.

17.10 Package Distribution and Compatibility Management

Published artifacts must include runtime entry points, built JavaScript, type declarations, scripts, necessary documentation, and dependencies. When the community skill package is published to npm, `SKILL.md`, scripts, or resource files must not be omitted. Before publishing the SDK, it should be installed from a clean directory and its imports, registration-request construction, and discovery-request construction should be verified.

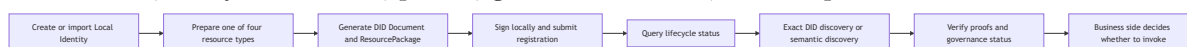
Release acceptance should include:

1. Preview the actual package file list and confirm that the presence of a file in the source repository does not mean that it is included in the published package.
2. Install the package to an empty directory and verify the module entry points it declares as supported.
3. Run minimum tests for registration, exact DID discovery, and semantic-discovery request construction.
4. Check that the README, type declarations, scripts, and resource files are accessible from the installed package.
5. Increase the major version for breaking interface or protocol-semantic changes and describe the migration boundary.

For the community skill package, the npm artifact must also retain `SKILL.md`, `references`, `agents`, `src`, and the built `dist`; checking only that TypeScript compilation succeeds is insufficient. For the SDK, verify that the root entry point and exports such as `./client`, `./governance`, `./protocol-types`, and `./identity-store-node` all point to actual JavaScript and declaration files. Release validation should run in an empty directory without using the source directory, so that omissions in `package.json` files, entry points, or dependency declarations can be detected.

17.11 Example Resources and Integration Templates

The minimum integration flow is: create or import a Local Identity; generate the resource DID, DID Document, metadata, ResourcePackage, and control proof; submit to a Registrar node; perform an exact DID query; perform natural-language, tag-based, and structured discovery; and, before invocation, verify the version, proofs, governance status, and endpoint.



The SDK is the programmatic entry point, the skill package is the operational assistant, and the official website is the interactive entry point. The four resource templates can share identity, version, semantic, and proof fields, but the protocol, endpoint, and ResourcePackage contents must reflect their individual differences.

The four resource types can reuse the same onboarding skeleton, but their invocation materials differ. An agent service normally needs a callable service endpoint; a skill package needs a manifest or download entry point; an MCP server needs an MCP protocol binding; and a tool API needs an API endpoint and input/output schema. The `protocolBindings`, service endpoints, resource descriptions, and package information in a template should be filled in according to the actual resource. The fact that all four types can be discovered does not justify copying one resource type's protocol fields into another.

17.12 Client Security and Secret Handling

Private keys and Local Identity backups are generated, used, and stored only on the client. DID Documents, registration VCs, and ResourcePackages should be handled according to their respective disclosure policies; logs should record only the DID, version, request ID, status, and error code, and must not record private keys, complete Local Identity data, or unsanitized signature material.

Client security boundary: A Local Identity file that can be downloaded or exported is control-rights material. It does not become public merely because it was generated by the official website. Registration VCs, DID Documents, and ResourcePackages should also be handled according to their respective disclosure policies. Their being downloaded in one archive does not give them the same disclosure level.

SDK and skill-package tests must not write real private keys into fixed fixtures or commit records. Automated tests can use temporary directories, temporary keys, and placeholder DIDs. After a test, identity files should be cleaned up, and the request body, exception text, and logs should be checked to ensure that they contain no private-key fields. If an application needs to

retain a Local Identity for a long period, the user should choose encrypted storage, an operating-system credential store, or an offline backup method; the SDK should not default to uploading it to a remote service.

17.13 Browser Registration and Local Identity Backup Behavior

The browser registration page treats the Local Identity as user-controlled material. The user can create and download a timestamped `oan-registration-identity.bundle.<timestamp>.json`, or import a backup; the browser sends only the public DID Document, resource data, and control proof to the Registrar node. The registration-result package must be generated only after submission succeeds and the resource DID has been confirmed.

User action	Browser-side change	Server-side effect
Create and back up	Generate the identity, update the current subject, and trigger a download	None
Import from local	Validate and load the backup selected by the user	None
Preview DID Document	Generate or reuse a preview based on the current form and identity	No registration submission
Submit registration	Validate current input and send public materials and proofs	Creates a registration request
Download after success	Package the DID Document, registration VC, and existing Local Identity	Does not create a second registration

17.13.1 Browser-Side Key Generation

The browser uses Web Crypto or the SDK to generate keys and establish the DID-to-public-key binding. The page must not place private keys in the DOM, URLs, public APIs, or error messages. Refreshing the page or clearing site data may lose an unbacked-up state, so the user should be prompted to back it up promptly.

17.13.2 Local Identity Import

The import operation reads the JSON file explicitly selected by the user, checks the structure of the version, subject configuration, resource configuration, and private-key materials, and verifies the relationship between the DID Document and public key. A failed import must not overwrite an existing valid identity. After a successful import, the preview and submission use the imported identity and recompute the control proof.

17.13.3 Local Backup Download

Local backup download uses the browser's standard download capability. The filename contains `oan-registration-identity.bundle` and the generation time to avoid overwriting an older backup. The

server does not participate in file generation; the user should keep the file as a sensitive key backup. Different browsers and mobile operating systems support automatic downloads differently. A failed download must not be marked as a valid backup, and an empty or incomplete file must not be treated as a valid backup.

17.13.4 Registration-Result Package Download

Registration-result packaging and download are triggered only after the server clearly returns success and the page confirms the resource DID. The archive name replaces the colons in the resource DID with hyphens. It contains the registration VC, DID Document, and existing Local Identity file. An empty VC may remain as an empty item, but no archive may be downloaded after failure, timeout, or a missing DID.

17.14 DID, VC, and ResourcePackage Download Formats

DID Documents, registration VCs, ResourcePackages, and Local Identity backups use recognizable JSON filenames and stable encodings. Public files must not contain private keys. Importers should verify the DID, version, and hashes from the content fields rather than trusting only the extension or filename.

File	Suggested contents	Sensitivity	Minimum validation
DID Document	DID, public keys, services, and OAN metadata	Public	DID, verification methods, and structure version
Registration VC	Issuer, subject, claims, signature, and status references	According to issuer and disclosure policy	Issuer, subject, signature, and validity period
ResourcePackage	Resource description, endpoint, capabilities, and version materials	According to resource policy	Resource DID, version, and digest
Local Identity backup	Subject and resource-control private-key materials	Highly sensitive	Package version, DID, and public/private-key correspondence

Unknown fields may be retained within a compatible version. Missing required fields, unrecognized structure versions, inconsistent digests, or private keys mixed into a public file must cause import or publication to be rejected.

The following is an **example ResourcePackage** used to explain field relationships. Its DID, digests, timestamps, and signature values are placeholders and do not represent an online resource. The example shows how a public ResourcePackage can reference the DID Document, metadata digest, ResourcePackage digest, and Root proof at the same time. An actual implementation may also retain extension fields allowed by the protocol type.

```
{
  "packageVersion": "1.0.0",
  "resourceDid": "did: oan: SKDM: ExampleResourceDid00000000000000000000",
  "resourceType": "skill",
```

```

"didDocument": {
  "@context": ["https://www.w3.org/ns/did/v1", "https://w3id.org/oan/v1"],
  "id": "did: oan: SKDM: ExampleResourceDid00000000000000000000",
  "verificationMethod": [
    {
      "id": "did: oan: SKDM: ExampleResourceDid00000000000000000000#key-1",
      "type": "Ed25519VerificationKey2020",
      "controller": "did: oan: SKDM: ExampleResourceDid00000000000000000000",
      "publicKeyMultibase": "zExamplePublicKey"
    }
  ],
  "authentication": [
    "did: oan: SKDM: ExampleResourceDid00000000000000000000#key-1"
  ],
  "oanMetadata": {
    "subjectType": "skill",
    "resourceType": "skill",
    "resourceDescription": {
      "name": "Example repository structure skill",
      "description": "Searches a code repository and summarizes its project structure.",
      "capabilityTags": ["code-repository", "project-structure"]
    },
    "capabilityTags": ["code-repository", "project-structure"]
  }
},
"didDocumentHash": "sha256: example-did-document-hash",
"metadataHash": "sha256: example-metadata-hash",
"packageHash": "sha256: example-package-hash",
"hashAlgorithm": "sha-256",
"metadata": {
  "resourceDid": "did: oan: SKDM: ExampleResourceDid00000000000000000000",
  "resourceType": "skill",
  "subjectType": "skill",
  "name": "Example repository structure skill",
  "description": "Searches a code repository and summarizes its project structure.",
  "capabilityTags": ["code-repository", "project-structure"],
  "lifecycleState": "active",
  "packageVersion": "1.0.0",
  "packageHash": "sha256: example-package-hash",
  "metadataHash": "sha256: example-metadata-hash",
  "hashAlgorithm": "sha-256",
  "updatedAt": "2026-01-01T00:00:00Z"
},
"rootProof": {
  "rootDid": "did: oan: INPT: ExampleRootDid00000000000000000000",
  "signature": "zExampleRootSignature",
  "hashAlgorithm": "sha-256"
},
"createdAt": "2026-01-01T00:00:00Z"
}

```


The ResourcePackage above can serve as input for public publication and discovery, but it cannot replace the Local Identity backup. The latter also contains subject or resource-control private keys and must be exported, imported, and stored separately. Putting the Local Identity and ResourcePackage into the same registration-result archive only consolidates the download operation; it does not change their sensitivity levels or verification methods. Compatibility decisions for a download format should be based on the content structure. The timestamp and DID in a filename are only for human recognition and avoiding overwrites; they cannot prove authenticity, issuer, or control rights.

17.15 References

Source	Type	Link
oan-sdk-ts	Code repository: DID, identity, ResourcePackage, registration, and discovery client capabilities	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	Code repository: community Skill, scripts, and developer workflows	https://github.com/OpenAgenet/oan-community-skill
oan-protocol-common	Code repository: protocol types and error contracts used by the SDK	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv yellow paper: client, resource-publication, and verification flows	https://arxiv.org/abs/2606.03163
OAN White Paper	arXiv white paper: developer ecosystem and open-infrastructure positioning	https://arxiv.org/abs/2606.03161

Chapter 18 Security, Privacy, and Abuse Prevention

This chapter explains how the security boundaries of OpenAgenet (OAN) cover resource control, node authorization, root-platform publication, discovery results, governance state, invocation endpoints, and users' local identities. An attacker may send public requests, submit forged or low-quality resources, issue repeated requests, observe public DIDs and VCs, or operate a compromised node. Security controls must state the protected object, attack preconditions, control point, failure result, verification evidence, and recovery action. The trustworthiness of discovery results does not imply the security of the business endpoint.

Security requirements uniformly use the following decision chain. The tables and checklists in the subsequent sections should all be traceable to this chain:

Attack preconditions -> Protected object -> Verification or control point -> Failure result -> Audit evidence -> Recovery action

Security object	Primary attack surface	Primary control point	Signals that must not be misinterpreted as security
Resource control	DID copying, document forgery, signature replay	Control key, challenge, version, and signature	Identical DID string, HTTP 200
Node authorization	Node impersonation, privilege escalation, expired authorization	Node DID, authorization VC, and governance state	Network reachability, historical success
Publication and distribution	ResourcePackage tampering, forged publication status	Hash, Root proof, and batch validation	Registration acceptance, cache hit
Discovery and invocation	Index poisoning, malicious endpoint, SSRF	Source, lifecycle, endpoint policy, and pre-invocation validation	High ranking, semantic relevance
Availability and privacy	Flooding, resource exhaustion, credential leakage	Quotas, isolation, redaction, and least privilege	Success of a single health endpoint

18.1 Threat Model and Trust Assumptions

The threat model treats the resource provider, Registrar node, Root platform, content distribution platform, Discovery node, off-chain trust indexer, browser, and invocation endpoint separately.

The resource-control private key is held by the controlling party, governance keys are used only for governance actions, and the indexer is a read-side projection of on-chain facts. The protocol cannot automatically repair a leaked key, a fully compromised host, or a vulnerability in a business endpoint.

Trust assumptions should be recorded separately by component and material:

Component or material	Trust boundary	Direct impact after compromise	Facts that must still be re-verified by other components
Resource-control private key	Held by the resource controller	Control of the resource may be transferred or abused	Node authorization, governance state, and publication result
Registrar node	Trusted only within its authorized domain	May receive, alter, or reject requests	Control proof, ResourcePackage hash, and root-platform publication
Root platform	Responsible for root verification, publication, and governance integration	Publication status or Root proof may be affected	Resource control and external governance facts
Discovery node	Provides query and index views	Results may be missing, stale, or poisoned	Source authorization, version, and Root proof
Off-chain trust indexer	Provides a read-side projection of on-chain governance facts	State may be delayed or queries may be incorrect	Original on-chain facts and synchronization sequence
Business endpoint	Not an OAN security trust root	The business invocation itself may be attacked	Protocol, authentication, and business policy of the caller

18.1.1 Resource-Control Threats

Resource-control threats include copying a public DID, DID Document, and VC to impersonate the controller, or using a new document to overwrite the same DID. The control points are the verification method, control challenge, signature, and version relationship. An incorrect public key, incorrect challenge, forged signature, or overwrite by an old version must be rejected.

18.1.2 Node-Trust Threats

Node-trust threats include an unauthorized node impersonating a Registrar node, an authorized node exceeding its privileges, and continued operation after suspension or revocation. The control points are the node DID, node authorization VC, governance events, authorized domain, and capability scope. Network reachability or historical success cannot replace current authorization.

18.1.3 Data-Poisoning and Abuse Threats

Data poisoning includes batches of fake resources, duplicate DIDs, low-quality descriptions, malicious tags, false endpoints, and forged ranking signals. Control points include control proofs,

idempotency rules, lifecycle, independent root-platform verification, tag boundaries, review, and rate limiting. Rejection records should distinguish specific reasons.

18.1.4 Availability Threats

Availability threats include registration flooding, complex semantic queries, oversized ResourcePackages, concurrent synchronization, database lock contention, and unavailable external dependencies. Control points are size limits, timeouts, concurrency quotas, pagination, query degradation, queue isolation, and backoff. Retries must not be unlimited.

18.1.5 Invocation-Endpoint Security Threats

Invocation-endpoint threats include malicious URLs, internal-network callbacks, insecure protocols, redirect bypasses, and compromised endpoints. Control points are protocol, domain, resolved address, port, redirect, and authentication validation. When validation fails, high-risk access must be blocked and the redacted reason recorded.

18.2 Resource Impersonation and DID Control Protection

The control relationship of a resource DID is established by the public key and control proof in the DID Document. A public DID, DID Document, registration VC, or Root proof cannot independently grant update authority. An update to the same DID must use the original control relationship or a control-transfer process permitted by the method; it must not be accepted merely because a new document is internally self-consistent.

The resource-control relationship can be checked item by item as follows:

Check item	Validation	Failure handling
DID and document	The DID declared by the document matches the resource DID in the request	Reject the request and do not enter the publication process
Public-key relationship	The verification method matches the control relationship	Mark the control relationship invalid
Signature challenge	The signature covers the current request, version, or challenge	Reject old signatures and signatures from another request
Version chain	The update declares the preceding version, digest, and control relationship	Do not allow an old version to overwrite the current record
State constraints	The resource has not been suspended, revoked, or restricted from updating	Reject or suspend according to governance state

“A signature with a valid format” alone is insufficient to prove control. The verifier must confirm that the signing public key is declared by the target DID Document and that the signature covers this specific operation.

18.3 Protection Against Forged Registration and Unauthorized Updates

Registration and update requests separately validate the source, node authorization, control proof, document hash, metadata hash, ResourcePackage hash, version, and idempotency state. An old version, old nonce, or incorrect hash must not overwrite the current record. Tests cover missing proofs, invalid signatures, invalid versions, incorrect hashes, and duplicate submissions.

Rejection conditions for registration and updates should be distinguishable to clients:

Situation	Control failure point	Permitted result
Missing control proof	Resource control has not been established	Reject the registration or update
Invalid node authorization	The request source is not eligible for access	Reject and record the authorization reason
Invalid version or preceding version	The update chain is discontinuous	Reject the overwrite without changing the current version
Inconsistent hash	The document, metadata, or ResourcePackage has been tampered with	Reject publication
Retry of a valid request already processed	The idempotency key or request ID already exists	Return a stable result without producing a duplicate side effect

18.4 Malicious or Compromised Registrar Nodes

The Root platform must not trust only the conclusion forwarded by a Registrar node. It should re-verify the node identity, authorization, control proof, DID Document, ResourcePackage, and request signature. If the node exceeds its authority or the materials have been altered, publication is rejected. A Registrar node's historical record does not automatically become a root-platform publication fact.

18.5 Malicious or Compromised Discovery Nodes

Callers should verify the source of the discovery response, the identity of the query node, index version, resource DID, DID Document, version, Root proof, lifecycle, authorized domain, and governance state. High relevance or HTTP success cannot replace verification. If a Discovery node is compromised, its authorization can be revoked and its index can be stopped from being used.

18.6 Root-Platform Compromise and Key Protection

A compromised Root platform may alter materials pending publication, forge Root proof, block propagation, or expose configuration. Publication keys and databases should be isolated, with least privilege, controlled signing, hashes, auditing, backup validation, and node resynchronization. Recovery must not change resource-control private keys or on-chain facts.

18.7 Spam Registration and Resource-Catalog Poisoning

Catalog-poisoning governance focuses on resource authenticity, availability, and semantic quality. The registration entry retains the control proof and subject, the Root platform independently verifies and isolates suspicious resources, and Discovery nodes filter by lifecycle, governance state, and publication evidence. Cleanup must distinguish rejection, suspension, revocation, and node-authorization revocation.

Spam-registration governance should not rely on a single score. Before processing, it should combine the control proof, resource type, tag namespace, endpoint verifiability, duplication, source node, and governance state. After processing, it should record the action and propagation scope.

- ☐ Apply multidimensional rate limits by subject, node, interface, and time window to batch requests.
- ☐ Distinguish network retries, idempotent submissions, and actual version updates for duplicate DIDs.
- ☐ Preserve the distinctions among normative tags, aliases, and inferred matches for low-quality or malicious tags.
- ☐ Distinguish suspension, revocation, taking offline, and node-authorization revocation for published resources.
- ☐ After cleanup, verify convergence of the Root platform, content distribution platform, and Discovery nodes.

18.7.1 Fake Resource Registration

Fake registrations are identified by signals such as missing control proofs, inconsistent DID Documents, unverifiable endpoints, or forged metadata, and are rejected before entering publication. Public errors must not echo private keys or complete signature materials.

18.7.2 Duplicate Registration and Version Poisoning

Duplicate registration may be a network retry, an update to the same DID, or a malicious submission. The system distinguishes these cases using the resource DID, version, document and ResourcePackage hashes, and request ID. A duplicate valid request returns a stable result, while a real update follows control and version rules.

18.7.3 Semantic-Tag Poisoning

Tag poisoning includes occupying popular tags, disguising non-normative tags as normative tags, cross-type mismatches, and misleading keywords. Registration validates tag IDs, vocabulary versions, type relationships, and namespaces. Discovery distinguishes normative tags, aliases, original text, and semantically inferred matches.

18.7.4 Rate Limiting, Review, and Cleanup

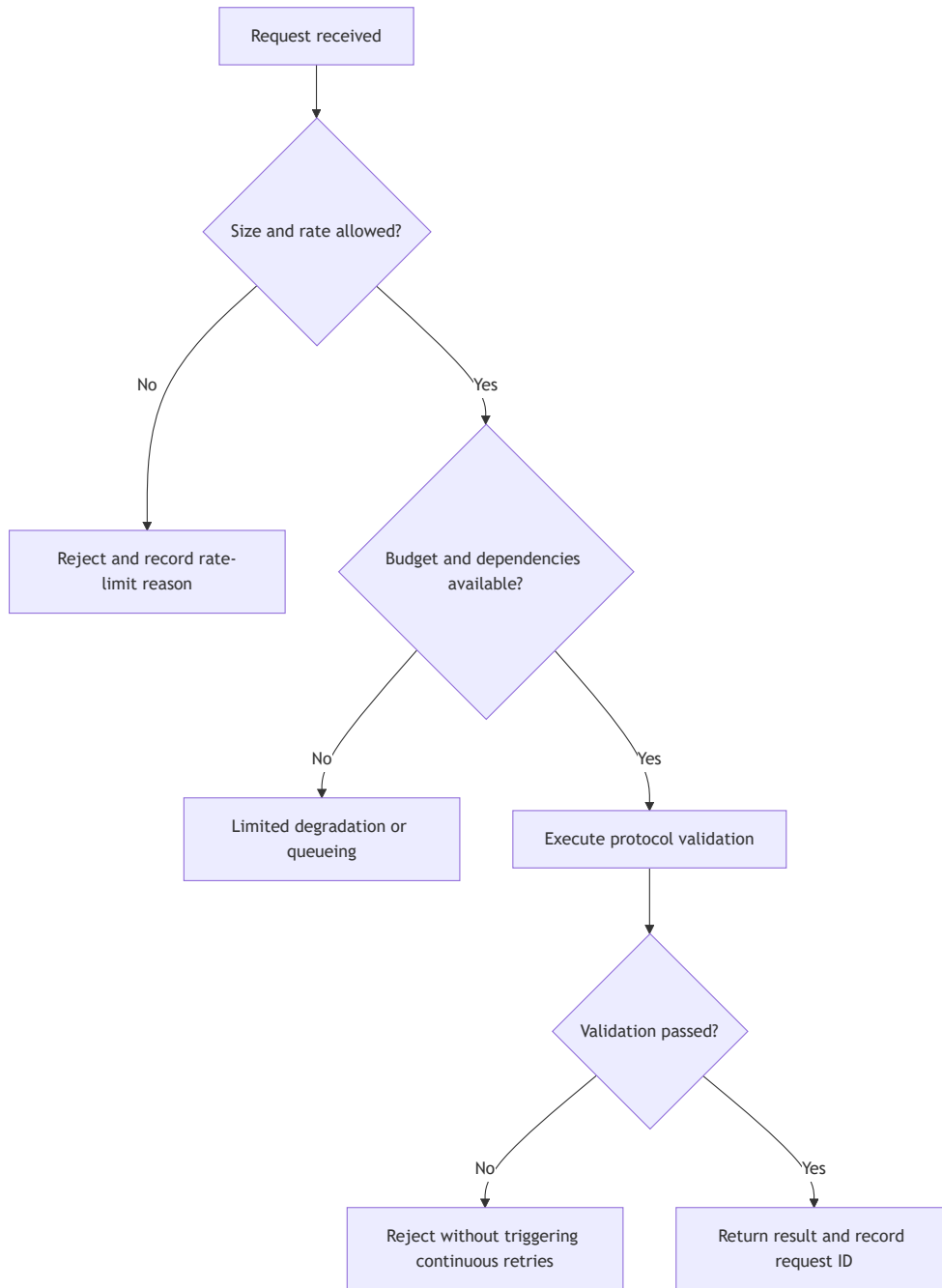
Rate limiting, review, and cleanup are applied by source, subject, node DID, resource DID, interface, and risk level, while retaining the reason, operator, and impact scope. After cleanup, convergence of the Root platform, content distribution platform, and discovery index should be verified.

18.8 Denial of Service and Resource Exhaustion

Registration, discovery, synchronization, health, and governance queries are configured with separate resource budgets. Limits apply to request bodies, batches, decompression expansion, query depth, concurrency, database connections, and external-call time. A semantic backend failure may fall back to a constrained path, but must not relax trusted filtering. Degradation must be marked as incomplete or unconfirmed.

Resource budgets should be isolated by path so that high-cost discovery queries do not bring down registration and health checks:

Path	Primary budget	Action on excess	Security baseline
Registration	Request body, signature-verification count, concurrency, and external-dependency time	Rate-limit, queue, or reject	Do not skip control proof
Discovery	Query length, page size, candidate count, and execution time	Truncate, cache, or degrade	Do not bypass governance filtering
Node synchronization	Batch size, cursor, parallelism, and retry count	Back off, pause, or reset cursor	Do not repeatedly advance a processed sequence
Database/index	Connections, locks, disk, and index growth	Isolate, alert, or enter protective read-only mode	Prioritize verified data that has already been accepted



18.8.1 Registration-Interface Attacks

Registration attacks include high-frequency submissions, oversized JSON, duplicate DIDs, expensive signature verification, and dependency calls. The entry point first performs size, rate, idempotency, and format checks, and then performs high-cost validation. Failed requests must not trigger continuous publication retries.

18.8.2 Discovery-Query Attacks

Discovery attacks include unbounded broad queries, complex semantic retrieval, duplicate queries, and result amplification. Limit input length, page size, candidate count, execution time, and concurrency, and use stable cursors and caching. Degradation must not bypass governance

filtering.

18.8.3 Inter-Node Synchronization Pressure

Synchronization pressure includes large batches, cursor replay, excessive parallelism, retry storms, and unavailable distribution. The synchronizing party uses batch and concurrency limits, exponential backoff, and cursor idempotency. A duplicate batch must not be written or advance state again.

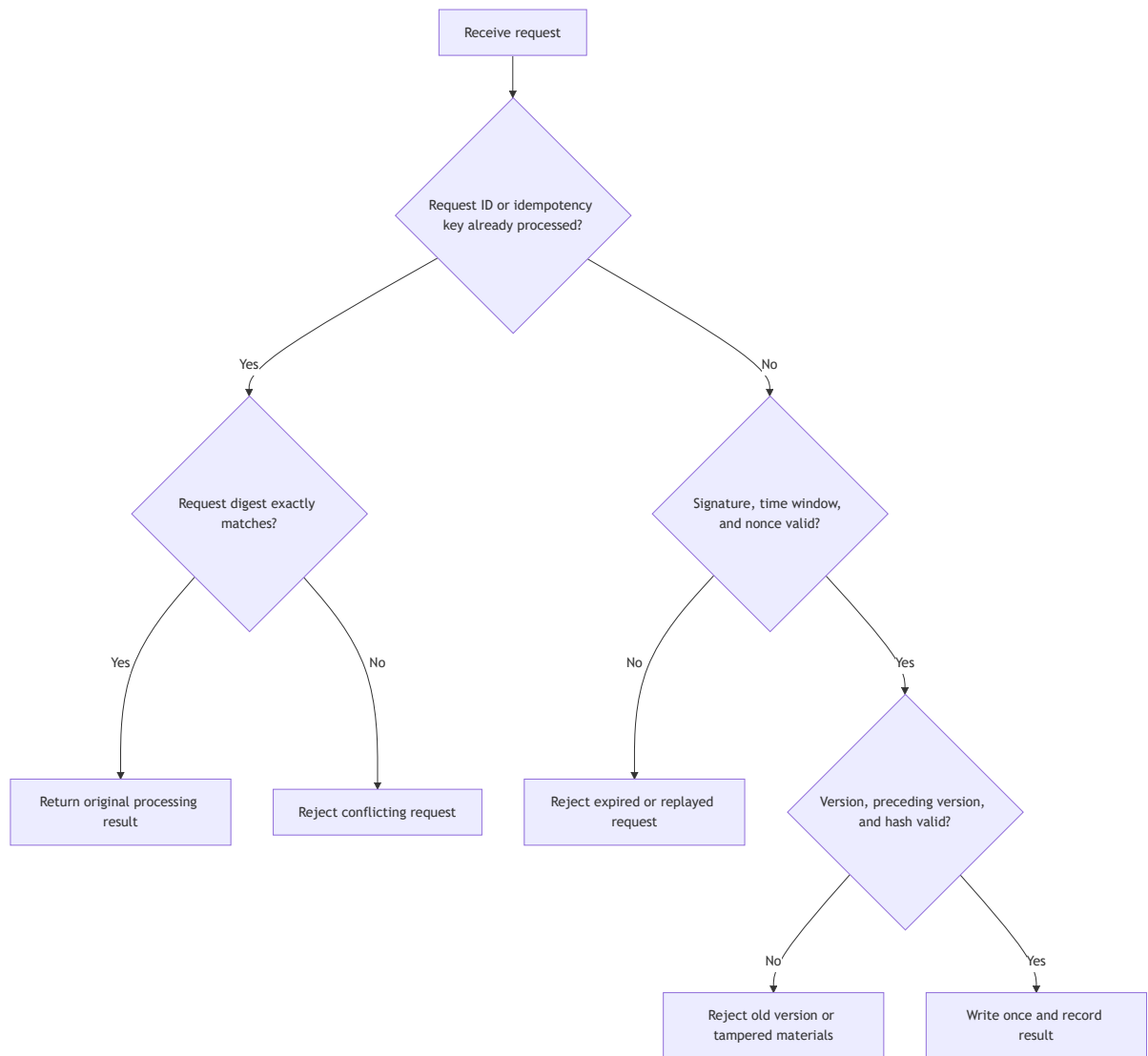
18.8.4 Database and Index Resource Exhaustion

Database and index exhaustion includes connection-pool depletion, locks, disk exhaustion, and index bloat. Field and batch limits, index maintenance, timeouts, isolated queues, retention policies, and capacity alerts should be provided. When resources are insufficient, valid data and health queries should be protected first.

18.9 Replay, Duplicate Submission, and Expired Publication

Replay protection distinguishes requests, registrations, publications, synchronization batches, and old caches. Request IDs, nonces, time windows, and signatures prevent a request from being accepted again. DID, version, preceding version, and hashes determine business updates, while cursors and batch sequences ensure idempotent synchronization.

The decision order for replay and duplicate submissions should prioritize idempotency and handle state updates cautiously:



18.10 Endpoint Abuse and Unsafe Resource Invocation

Before invocation, check the endpoint protocol, certificate, domain, resolved address, authentication, version, lifecycle, governance state, and publication evidence. Where necessary, negotiate business permissions, sign requests, and validate responses. OAN does not guarantee that a business endpoint is free of vulnerabilities or malicious behavior.

Before using discovery results, the caller should complete at least the following checks:

- ☐ The resource DID, DID Document, public key, and version digest are consistent.
- ☐ The Root proof, lifecycle, and governance state remain verifiable.
- ☐ The endpoint protocol, domain, certificate, resolved address, port, and redirects comply with local policy.
- ☐ The authentication, authorized domain, request signature, and response validation required for invocation are defined.

- The freshness and source of the Discovery node's result are not mistaken for a guarantee of the business service.

18.11 Sensitive Metadata and Credential Exposure

Private keys, Local Identity backups, internal endpoints, database paths, administrative tokens, unpublished resources, complete signature originals, and log fields that can be associated with individuals are sensitive information. Public interfaces must not return Local Identity backups. Server logs require allowlisting, redaction, and access control.

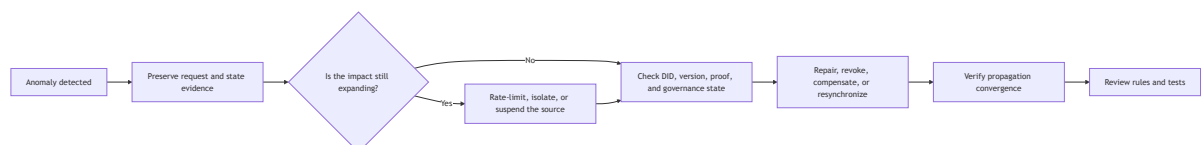
Information category	May appear in a public response	Log handling	Action after failure
DID, version, and public metadata	According to the resource disclosure policy	May be recorded with a retained correlation ID	Process discovery or publication according to state
Registration VC and Root proof	According to the disclosure policy	Record digest and status; the complete sensitive material need not be recorded	Reject use if validation fails
Private key and Local Identity backup	No	Must not be recorded	Immediately block and investigate the leakage path
Administrative token, database path, and internal endpoint	No	Record only after redaction or allowlisting	Rotate credentials and limit the exposure surface
Complete signature original and personally correlated logs	Normally should not be public	Redact, minimize, and restrict access	Delete unnecessary copies and review access

18.12 Logging, Auditing, and Incident Response

Security logs correlate the request ID, resource DID, node DID, version, event or cursor, source, result, error code, and time, and distinguish on-chain evidence, service logs, and manual records. Log-write permissions should be restricted. Alerts should locate isolation, revocation, rate limiting, compensation, and recovery verification actions.

Incident response should retain the minimum evidence that remains correlatable:

```
request_id / resource_did / node_did / version / event_or_cursor
source / result / error_code / observed_at / operator_action
```



18.13 Security Requirements for Third-Party Nodes

Third-party nodes protect their own hosts, node private keys, authorization VCs, configurations, databases, dependencies, and ports, and connect through the node DID, DID Document, node authorization, and request signature. In an abnormal situation, authorization may be suspended or revoked. A resource provider must not be required to surrender a subject or resource-control private key.

Before a third-party node connects, an auditable minimum inspection record should be created containing the node DID, DID Document, node authorization VC, authorized domain, validity period, public endpoint, software version, key custody method, and revocation contact. A resource provider submits only the resource control proof and should not surrender a subject or resource-control private key.

18.14 Endpoint Validation, Callback Security, and SSRF Considerations

Before reading or invoking an endpoint, callback, or synchronization address, validate the protocol, host, port, resolved address, redirect, and authentication. Private, loopback, link-local, and cloud-metadata addresses are blocked according to policy. DNS rebinding must be prevented between resolution and connection. Rejection reasons must be auditable without exposing topology.

SSRF protection should cover four points: before resolution, after resolution, before connection, and after each redirect. Checking a URL only at the string level is not sufficient. When rejecting a request, record the policy-match category and request ID, but do not echo internal addresses, network topology, or resolution details.

Stage	Required checks	Failure result
Before resolution	Scheme, host, port, credentials, and encoding	Reject invalid URL
After resolution	IPv4/IPv6, loopback, private, link-local, and cloud-metadata addresses	Reject restricted address
Before connection	Actual connection target, certificate, timeout, and policy	Fail closed
After redirect	Protocol, host, resolved address, and port at every hop	Reject out-of-policy redirect

18.14.1 Endpoint Format and Protocol Validation

Allow only supported protocols and permitted port ranges. Reject endpoints with a missing host, invalid encoding, embedded credentials, or an incompatible type. HTTPS certificates and host names are validated according to risk. HTTP in a development environment must not become the default public security configuration.

18.14.2 Internal and Loopback Address Restrictions

Before every connection, check IPv4, IPv6, loopback, private, link-local, reserved, and cloud-metadata addresses. Use standard resolution and network policies rather than relying only on a string blacklist.

18.14.3 Callback Request Security

Bind callbacks to a one-time request ID, source context, and target. Limit methods, request headers, response size, and timeout. A callback must not change authorization or governance state. Failed retries must be limited.

18.14.4 Redirect and DNS-Resolution Risks

DNS resolution results, connection targets, and the redirect chain are evaluated as one decision. Revalidate after a redirect to a different host, protocol, or restricted address. Timeouts, certificate changes, or inconsistent resolution should fail closed.

18.15 Package and Dependency Supply-Chain Integrity

Source code, Cargo/npm dependencies, lock files, build artifacts, and ResourcePackages form the supply-chain boundary. Builds use locked dependencies, controlled sources, hash or signature verification, and traceable commits. ResourcePackages are hashed again before entering the Root platform or a Discovery node.

Supply-chain verification covers the “source-lock-build-artifact-deployment” chain:

1. Check the source commit, dependency lock files, and controlled package sources.
2. Build Rust, TypeScript, and community Skill artifacts in a reproducible or controlled environment.
3. Save a digest or signature for binaries, npm packages, ResourcePackages, and distribution materials.
4. Compare artifact digests before deployment, and record the version and rollback materials after deployment.
5. If a dependency or artifact is abnormal, stop publication, preserve evidence, and rebuild.

18.16 Availability Objectives and Graceful Degradation

Security state and availability are expressed separately. An old snapshot may support a read-only result with a freshness marker, but must not approve registration, publication, authorization, or high-risk invocation. Dependency failure, index lag, unknown governance state, and resource exhaustion should have different states and recovery actions.

State	Content that may be returned	Actions that must be prohibited	Facts that must be marked
Normal and fresh	Current query, state, and proof	No additional restriction	Source and time
Old snapshot available	Read-only result with freshness	Registration, publication, authorization, and high-risk invocation	Snapshot time and lag
Index lagging	Confirmed partial facts	Treating unsynchronized data as nonexistent or current	Current sequence and target sequence
Governance unknown	Limited query or error state	Approving invocation through an unknown state	Reason for unknown state
Dependency unavailable	Health error or restricted cache	Unlimited retries or relaxed trusted filtering	Dependency and recovery state

The acceptance focus for graceful degradation is not merely that “the page still has content.” Users and callers must be able to see the freshness, source, and uncertainty of the content, and old data must not automatically execute registration, publication, authorization, or high-risk business invocation.

18.17 References

Source	Type	Link
<code>oan-protocol-common</code>	Code repository: signatures, hashes, credentials, and secure request structures	https://github.com/wolfbrother/oan-protocol-common
<code>oan-root-services</code>	Code repository: root-platform verification, publication, and node authorization	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: registration entry point, control proofs, and abuse controls	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	Code repository: queries, indexing, and result security boundaries	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-trust-indexer</code>	Code repository: governance state and revocation-state projections	https://github.com/OpenAgenet/oan-trust-indexer
OAN Yellow Paper	arXiv yellow paper: security, privacy, and trust boundaries	https://arxiv.org/abs/2606.03163

Chapter 19 Resource Lifecycle, Governance, and Version Management

This chapter specifies the temporal, state, and evidence relationships of resources in the OpenAgent (OAN) network. Resources such as Agent Services, Skill packages, MCP services, and Tool APIs use DIDs to identify subjects, use DID Documents to describe control relationships, capabilities, and service endpoints, and use resource packages to carry publishable content. A registration VC proves that a Registrar node has accepted a registration, while a Root proof proves that a specific resource package has passed Root platform verification and entered the publication pipeline. “Registered”, “published”, “indexed”, and “currently callable” are different facts and must be proved separately.

The state relationships in this chapter can be understood by using a resource version as the association key:

```
Resource DID
-> DID Document version
-> Resource package version + document/metadata/package hashes
-> Registration VC
-> Root proof
-> Distribution cursor and discovery index version
```

The chain above represents the association between pieces of evidence. It does not mean that all steps complete synchronously, nor does it mean that a response from a Registrar node means that the resource is already visible to a Discovery node.

19.1 Resource Lifecycle State Machine

State	Factual meaning	Default discovery behavior
Draft	Formed only in the publisher's local environment	Not returned
Registered	Accepted and recorded by a Registrar node	Does not yet represent network-wide visibility
Pending publication	Waiting for Root platform verification or a propagation task	Processing status may be queried
Published	Root platform verification has completed and publication evidence has been produced	Eligible for indexing

State	Factual meaning	Default discovery behavior
Discoverable	Verified and indexed by a Discovery node	May be returned to the querying party
Suspended, revoked, deactivated	Temporarily unavailable, explicitly withdrawn, or terminated by the subject	Not returned as a valid resource by default

States must be triggered by events, proofs, or persistent records. They cannot be inferred from a page button or a single successful HTTP response. Registration success indicates, at most, that the Registrar node has completed acceptance. The resource can enter the Published state only after Root platform verification, authorization checks, resource package digest verification, and completion of the publication task. It can enter the Discoverable state only after a Discovery node has completed verification and indexing. Failures must preserve the reason, associated identifiers, and retry boundaries; they must not all be described simply as completion of network-wide registration.

19.2 Initial Registration

The publisher creates or imports a local identity in the browser and uses a local private key to sign the DID Document and registration request; the private key should not be uploaded to the Registrar node. The Registrar node verifies the format, control relationship, resource type, required metadata, and node authorization. After successful acceptance, it records the registration and returns a registration VC according to the interface contract. The Root platform then independently verifies the resource package, Registrar node authorization, and publication conditions. The content distribution platform and Discovery nodes subsequently receive and index the content asynchronously. Missing local identity, invalid signatures, missing fields, network failures, an empty VC, and propagation delays must be reported separately and must not be summarized as completion of network-wide registration.

Stage	Publisher-side local action	Network-side fact	Evidence that can be verified
Generation	Create or import a local identity and generate a DID Document	Not yet registered	Local identity backup, DID Document, and control signature
Acceptance	Send the signed request	Registrar node records the registration	Response, request identifier, and registration VC (if any)
Publication	Wait for asynchronous processing	Root platform forms a publication fact	Root proof
Discoverability	Query the resource	Discovery node completes indexing	Query result, source, and indexing time

19.3 Updating a Resource with the Same DID

The basis for an update with the same DID is continuity of the control relationship, not equality of the DID string. An update request should include the preceding version, resource DID, version, and content digest, and should be signed by the existing control public key or an explicitly authorized update key. The Registrar node verifies control authority and the version relationship. The Root platform re-verifies the resource package and publication authorization. A Discovery node accepts only versions that have passed through the Root platform publication process. Repeated submission of identical content and the same preceding version should be handled idempotently; a content change must advance the version and digest. Requests without authorization, with a version rollback, or with a fork must be rejected. Before the new version has completed propagation, the old valid version may continue to be returned.

Update scenario	Control proof	Version relationship	Processing result
Same DID, same digest, same preceding version	Valid	Duplicate	Return idempotently without creating a new valid version
Same DID, new digest	Valid	Advancing	Re-verify, publish, and establish a new version
Same DID, new digest	Invalid or unauthorized	Any	Reject without overwriting the old version
Same DID, old version	Valid	Rollback	Reject or mark as a version conflict

19.4 DID Document Version Management

DID Document versions express changes to the identity document, control keys, service endpoints, or capability metadata. They are not equivalent to the `did:own` method version or the resource package version. The current version should be determined jointly by the stable DID, the predecessor relationship, a sequence or equivalent version rule, and the controller's signature; timestamps are for audit purposes only. A resolver must not resolve a fork at the same predecessor version solely according to arrival time. By default, a Discovery node returns the highest-sequence version that is published and has a valid state, and supports querying historical versions by version or content digest.

The order of version determination should remain deterministic:

Lifecycle state filtering -> Proof verification -> Publication sequence comparison -> Version comparison -> Query ordering

Resource package versioning and interface compatibility are different dimensions. A Skill package may update only its documentation or examples, while a Tool API may change its input/output schema and endpoint at the same time. An MCP service may also change its protocol binding or exposed capabilities. Therefore, a version number cannot replace a change description. The `packageVersion`, `packageHash`, `metadataHash`, and `lifecycleState` in a resource package should be consistent with the corresponding values in the DID Document metadata and the Root proof.

A minimal binding record for audit purposes can be represented as follows:

```
{
  "resourceDid": "did:oon:SKDM:example-resource-did",
  "packageVersion": "1.2.0",
  "didDocumentHash": "sha256:example-did-document-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "packageHash": "sha256:example-package-hash",
  "lifecycleState": "active",
  "previousVersion": "1.1.0"
}
```

The DID, version, and digest values above are illustrative placeholders. They express the binding relationship between fields, not an official resource or fixed version.

19.5 Resource Package Version Management

A resource package is the publication unit verified by the Root platform, stored by the content distribution platform, and indexed by Discovery nodes. A version must point to a set of normalized metadata, entry points, protocol bindings, dependencies, and content digests. The Root proof should bind the resource DID, resource type, resource package version, and package digest. A resource DID may remain unchanged during a normal upgrade. An old resource package may be marked as superseded while its audit evidence is retained. By default, a Discovery node returns the latest valid package; when a caller requests a specific version, its publication proof and credential status must still be verified.

The update target can be selected according to the scope of the change. If only the published package content changes, at least the resource package version and package hash must be updated. If service endpoints, public keys, or resource metadata in the DID Document change, the DID Document and its digest should be updated together. If capability tags, authorization domains, or lifecycle state change, semantic governance and visibility must also be checked again. Any change that affects a field bound by the Root proof cannot be made only by modifying a local index at a Discovery node.

19.6 Metadata and Endpoint Changes

Changes to capability tags, descriptions, use cases, protocol bindings, endpoints, authentication requirements, and dependencies that affect discovery or invocation decisions should produce a new DID Document version, a new resource package version, or a combination of both, followed by re-signing, verification, and indexing. An endpoint change must also check TLS, authentication, protocol compatibility, and actual capabilities. Taking an old endpoint offline does not mean that the resource DID has been revoked; the caller should decide whether to invoke the resource according to the new version and state.

19.7 Registration VC Lifecycle

A registration VC proves that a Registrar node accepted a registration. It is not a guarantee of resource quality and does not replace the Root proof. A verifier should check the issuer's authorization, subject DID, resource type, version or content digest, issuance time, validity period, and revocation status. When the VC is missing or invalid, the issuer is unauthorized, or the status is unknown, the resource must not be marked as a trusted registration. Credential revocation affects only the fact proved by that credential and does not automatically rewrite the DID Document. Deactivation of a DID does not mean that historical credentials disappear.

When processing credentials, the querying party should record “credential exists” and “credential is currently valid” separately. Even if the registration VC for resource version `v1` has a valid signature, it cannot directly prove that `v2` has been registered. Likewise, the Root proof for `v2` cannot replace the Registrar node's evidence for the registration fact. After a resource update, credential revocation, or subject deactivation, a Discovery node should filter results according to the latest verifiable state while retaining historical credentials as audit material.

19.8 Resource Suspension and Recovery

Suspension is a reversible state. Its causes should distinguish controller action, governance restrictions, temporarily unverifiable evidence, endpoint maintenance, and node synchronization failures. A network timeout or missing metrics must not directly change the resource lifecycle state. Suspension should have a control signature, governance event, or other verifiable evidence. Before recovery, the control proof, VC, Root proof, endpoint, and governance conditions must be checked again, and the Discovery node must resynchronize according to the publication sequence rather than merely changing its local state.

19.9 Resource Revocation and Deactivation

Resource revocation targets publication eligibility or a version. DID deactivation targets the resource subject. Credential revocation targets a particular proof. Endpoint withdrawal targets a service entry point. Revocation and deactivation should record the initiator, basis, target DID or version, reason, effective time, and propagation scope. Historical materials may be retained for audit, but must not be returned as currently callable resources. Before propagation converges, a node must label the state and freshness instead of claiming that revocation has taken effect across the entire network immediately.

The handling of node authorization changes and resource states can be checked using the following relationships:

Observed fact	What it can indicate	What it cannot indicate directly
Registrar node authorization revoked	The node no longer has authorization to accept new trusted publication inputs	Already published resources are automatically revoked
Discovery node suspended	Queries or indexes from the node require degraded handling	Resource control has become invalid

Observed fact	What it can indicate	What it cannot indicate directly
Resource version revoked	The specified resource or version should no longer be treated as a valid result	Other versions are necessarily revoked
Node recovered and synchronization completed	The node has regained the corresponding operating conditions	All historical missing data has been restored automatically

19.10 Node Suspension and Resource Visibility

Node authorization is a governance object, while resource control belongs to the resource DID. After a Registrar node loses authorization, the Root platform should stop accepting new trusted publication inputs from it and re-evaluate existing publications according to governance rules. After a Discovery node loses authorization, it should not continue publishing results as a normally authorized node, although its data may be retained for recovery and audit. Node recovery must first restore identity and authorization and then use the cursor to fill in missing data. Clients should distinguish node failure, synchronization in progress, and resource revocation.

Version selection is not simply “last write wins”. For example, if a Discovery node receives 1.1.0 and 1.2.0 for the same resource at the same time, it should first confirm that both have verifiable publication sources and valid states, and then select the current result according to the publication sequence, predecessor relationship, and version rules. If the digest or proof for 1.2.0 fails verification, it cannot overwrite 1.1.0 merely because its version number is larger. The selection result should also retain the adopted version and the reasons for excluding other versions, so that propagation delays and version conflicts can be diagnosed.

19.11 Version Selection by a Discovery Node

A Discovery node first filters by state, signatures, registration VC, Root proof, version validity, and index freshness, and then performs semantic and structured ranking. By default, for the same DID it returns the version that is published, has a valid state, and has the highest publication sequence. When there is a version conflict, a sequence gap, or unknown proof status, it should report an anomaly instead of selecting randomly. Both exact DID queries and semantic queries should return the version, digest, source, evidence status, and indexing time.

19.12 Historical Versions and Auditability

Audit records should associate the DID, document version, resource package version, digest, control signature, registration VC, Root proof, publication sequence, node receipt time, and indexing time. Historical queries should distinguish current valid, superseded, revoked, archived, and not-yet-synchronized-by-the-node states. Historical evidence is used to reconstruct publication facts and must not bypass current-state checks. Node reconstruction or database migration must not silently discard evidence that the specification requires to be retained.

Historical records should be able to answer at least three questions: who controlled and signed

a particular version; whether the Root platform accepted and published that version; and when and in what state a specified Discovery node observed that version. If only a resource package file can be found, without a control signature, publication proof, or index observation record, it proves only that the material was once stored. It does not prove that the material was once a valid version in the network.

19.13 Resource VC and Publication Evidence Lifecycle

Registration VCs and Root proofs have different issuers and factual scopes. The former proves registration acceptance, while the latter proves that a specific resource package passed Root platform verification and entered the publication pipeline. Both must bind an explicit DID, version, and digest. When an update produces a new digest, the corresponding evidence should be produced again. Revocation, deactivation, expiration, and authorization loss must be checked separately. A proof file downloaded to the publisher's local environment does not change the server-side validity judgment.

The evidence association can be checked item by item by version:

Same resource DID

- > Same resource package version
- > Corresponding DID Document/metadata/package digest
- > Registration fact in the registration VC
- > Root proof
- > Receipt and indexing observation by a Discovery node

If any link points to a different DID, version, or digest, it should be treated as a binding inconsistency. File names or download times must not be used to infer that the evidence belongs to the same publication.

19.14 Update Propagation and Stale Index Convergence

Registrar node -> Root platform verification and publication -> Content distribution platform

- > Discovery node receipt -> Verification, indexing, and visibility

Each segment is associated using the DID, version, digest, publication sequence, or event cursor. Repeated delivery must be idempotent, and interrupted retries must continue from the confirmed cursor. When there is a sequence gap, digest mismatch, invalid proof, or unknown governance state, a Discovery node must not update its valid projection. Convergence of an old index is determined by completion of verification and projection of the new version, not by the fact that a request has been sent.

Convergence check	Passing condition
Content	The resource package can be obtained and its digest is consistent
Evidence	The target DID, version, and digest in the publication proof are consistent

Convergence check	Passing condition
Ordering	The publication sequence or synchronization cursor has no unexplained gap
State	The lifecycle state of the new version has been re-evaluated
Query	Exact and semantic queries no longer return the old version as valid

Propagation checks should record the state observed by each node rather than comparing only resource counts. Equal counts may mean that an old version is still indexed, or that a new version has replaced the old one. The resource DID, version, digest, lifecycle state, publication sequence, and observation time must be compared. When an anomaly occurs, retain the last successful cursor and the original response so that missing or out-of-order events are not concealed by resetting the index.

19.15 Resource Ownership Transfer Strategy

An ownership transfer must be authorized by the former controller, confirmed by the new controller, and made effective after verification by the Root platform or a governance process. Resubmitting the same DID or presenting a new registration VC is not sufficient to prove a transfer. Operational maintenance authority and publication delegation may be authorized separately, but their scope, duration, and revocation method should be recorded separately from control authority. If the original private key is lost, control must not be forcibly taken over by bypassing signatures; an auditable governance recovery process should be used. Where the current implementation does not provide a complete transfer interface, this section is a protocol design requirement.

Before the control transfer is completed, the former controller remains the key source of evidence for resource updates and revocation decisions. A statement from the new controller, operational authorization, or resource package signature cannot independently override the old control relationship. After the transfer is completed, maintenance records should retain the former controller's authorization, the new controller's confirmation, the effective fact established by the Root platform or governance process, and the observation that the Discovery node has completed the update. This distinguishes “transfer proposed”, “transfer approved”, and “nodes converged”.

19.16 References

Source	Type	Link
oan-root-services	Code repository: publication status, versions, and Root proofs	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	Code repository: registration, updates, and registration credentials	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	Code repository: index versions, lifecycle, and discovery status	https://github.com/OpenAgenet/oan-discovery-node

Source	Type	Link
<code>oan-trust-indexer</code>	Code repository: governance, suspension, and revocation status	https://github.com/OpenAgenet/oan-trust-indexer
<code>oan-protocol-common</code>	Code repository: shared structures for versions, lifecycle, and status	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv yellow paper: resource lifecycle, versioning, and governance mechanisms	https://arxiv.org/abs/2606.03163
<code>did:oan</code> DID Method Specification	Standard/method specification: resource identity and update control	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md

Chapter 20 Observability, Metrics, and Operations

This chapter explains how observability answers, at the same time, whether a service is alive, whether business functions are available, whether data is fresh, and whether an anomaly is converging. The Root platform, Registrar nodes, Discovery nodes, content distribution platform, off-chain trust indexer, database, and official website backend each produce operational and business evidence. The website may use an old snapshot to improve the user experience, but an old snapshot cannot replace the actual backend health status. Metrics must include their source, time, status, and latency; unknown values must not be treated as zero.

Observational evidence is divided into four layers: raw facts defined by service interfaces or source code, operational metrics aggregated by the backend, display data used by official website pages, and smoke tests and alert records produced by operational procedures. When recording a metric, at least `source`, `observedAt`, `dataStatus`, and `lag` should be retained together so that “the interface has no data,” “the data is zero,” and “an old snapshot is being displayed” can be distinguished. A display result on the official website only shows that the frontend obtained a response at a certain time; it cannot replace independent verification of a node, database, or off-chain trust indexer.

Observation layer	Core question	Typical evidence
Liveness	Can the process respond?	liveness response, process status
Readiness	Are dependencies and configuration satisfied?	readiness, database and upstream status
Business	Are registration, publication, and discovery available?	smoke test request, response, and correlation identifiers
Data	Are metrics and indexes fresh?	time window, cursor, latest sequence
Operations	Has the failure been detected and converged?	logs, alerts, recovery records

20.1 Service Health and Readiness Model

The checking sequence should progress from “can it respond?” to “can it complete the business operation?” First verify the process and `/health`, then verify readiness and the database connection, and finally check the dependencies among the Root platform, Registrar nodes, Discovery nodes, content distribution platform, and off-chain trust indexer. A 200 response from an interface can

only serve as evidence for that checkpoint. A service may be marked as business-available only after dependency checks, data status, and registration/discovery smoke tests have all passed. If `summary` continues to fail, the backend should remain in a failed state even if the page can still display an old snapshot.

Liveness only proves that the process can respond; readiness should also check necessary dependencies and configuration; business smoke tests verify real paths such as registration, publication, and discovery. A health check should record the target, response code, duration, dependency status, and time. A connectable database does not mean that the indexer has caught up with upstream events, and a responsive summary interface does not mean that the statistics are complete. Recovery confirmation requires at least consecutive successful health checks together with the corresponding business smoke tests.

Status	What it proves	What it does not prove
liveness	The process can respond to the check	Dependencies are normal or business functions are available
readiness	Configuration and necessary dependencies are ready	Network-wide publication and indexing are complete
Business smoke test passed	The specified path is available under the test conditions	All resources and all loads are normal
Data expired	Recent data was not updated in time	The resource has been revoked
Metric missing	The metric cannot currently be obtained	The metric value is zero

20.2 Registration Metrics

Registration metrics should establish a traceable chain around each resource registration. It is recommended that `requestId` be bound to the registration request, `resourceDid` and the resource version or content digest be bound to the registration fact, and the publication sequence, distribution batch, and index version be bound to subsequent stages. Client-side repeated clicks or network retries may create multiple request records, but must not cause the same resource version to be counted as multiple successful registrations. When the VC is empty, the returned result should be recorded truthfully; an empty value must not be interpreted as a valid credential. Periodic statistics should be calculated in the aggregation layer using a unified time zone; the frontend must not accumulate counts based on page refreshes.

Registration metrics should distinguish request receipt, validation rejection, successful registration, VC return, Root platform acceptance, publication completion, and discovery visibility. Deduplication should use a request or resource-version correlation identifier rather than performing rough deduplication by IP address. Production statistics should exclude smoke tests that are explicitly marked as such. `TODAY`, `THIS WEEK`, and `THIS MONTH` must use a unified time zone and half-open time intervals; periodic values must not be confused with cumulative values because of caching or degradation.

Metric	Counting unit	Correlation key	Statistical considerations
Registration requests	Request or registration attempt	requestId	Distinguish retries from duplicate submissions
Successful registrations	Resource-version registration	resourceDid + version	Does not equal Root platform publication
VC returned	Returned credential result	resourceDid + credentialId	An empty VC is not counted as a valid credential
Root platform publication	Publication fact or publication batch	resourceDid + sequence	Asynchronous status is counted separately
Discoverable	Version queryable in the index	resourceDid + indexVersion	Must be combined with freshness

20.3 Publication and Propagation Metrics

Stage	Start time	Completion time	Main evidence
Registration accepted	Registrar node receives the request	Registration record is formed	Request identifier, resource DID, response status
Root platform publication	Publication task is created	Root proof is formed	Publication sequence, resource package digest
Content distribution	Distribution batch is sent	Content can be retrieved and verified	Batch, cursor, content digest
Index visibility	Discovery node receives the content	The query can return it	Index time, source, version

Propagation metrics use the resource DID, version, content digest, and publication sequence throughout all stages, and record the event, receipt, verification, distribution, and indexing times separately. End-to-end latency is the last queryable time minus the initial acceptance time; it must not be replaced by the duration of one individual segment. Retries and duplicate batches should be counted idempotently. A stalled cursor, sequence gap, or offline node should enter backlog or failure metrics.

The propagation chain should at least answer “at which stage did the resource stop?”: the Registrar node accepted it but the Root platform has not published it; the Root platform published it but content distribution has not completed; distribution completed but the Discovery node has not indexed it; or indexing completed but the query conditions did not match. The following correlation can be used for verification, rather than treating the resource count on a page as a substitute for propagation facts:

```
resourceDid + version
-> publicationSequence
-> distributionBatch / contentDigest
-> discoveryIndexVersion / indexedAt
-> queryRequestId / visibleAt
```

Retries of the same resource version should reuse an idempotency key or be marked as retries. If a record exists for only one stage, the operational result should explicitly state “accepted,” “propagating,” or “not yet queryable,” rather than simply reporting success.

20.4 Discovery Query Metrics

Query type	Must record	Interpretation boundary
Exact DID query	DID, duration, result count, source	A hit does not mean that it is currently trusted
Structured query	Conditions, duration, hits, and filtered results	A condition match does not mean that it is callable
Semantic query	Original description, converted conditions, and ranking results	Relevance does not replace control and governance verification
Failed query	Status code, error type, requestId	An empty result cannot be equated with a service failure

Discovery metrics distinguish exact DID queries, structured-condition queries, and semantic queries, and record request duration, result count, empty results, rejections, timeouts, and server errors. Relevance is not the same as trustworthiness. Results still need status, credential, publication proof, and freshness checks. Error responses retain correlation identifiers so that a request from the official website can be traced to Discovery node logs.

Query metrics should record the query mode, a digest of the normalized input, request identifier, service node, hit count, duration, response status, and data status. The original task description of a semantic query may be used for troubleshooting and quality analysis, but logs and statistics should not retain unnecessary personal or sensitive content; a digest, length, and hash can be used instead of the full text. An empty result must be evaluated together with whether the request was valid, whether the index is fresh, and whether the resource is within the node's coverage; it cannot be directly classified as a Discovery node failure. After an exact DID query hits, the resource status, publication evidence, and version must still be checked.

20.5 Index Freshness and Synchronization Metrics

```
Synchronization status = upstream latest sequence - local cursor + latest successful time +
    latest error
Uncertain freshness -> mark as unknown; do not fill it with 0
```

Index rebuilding, incremental synchronization, synchronization pauses, and synchronization failures should use different status values. Operations personnel can then determine whether there are no new events, processing is delayed, or the synchronization path is interrupted. Synchronization observation includes the current event cursor, upstream latest sequence, lag distance, latest successful time, latest failure reason, index update time, and pending item count. A continuously increasing `latest_sequence` only shows that new observable events exist upstream; it does not by itself prove that this node has completed indexing. When the lag distance is unknown, display

unknown rather than zero. Index rebuilding and incremental synchronization should be marked separately.

Freshness assessment should observe the upstream latest sequence, the locally processed cursor, the latest successful synchronization time, and the latest error together. If the `latest_sequence` of the off-chain trust indexer continues to increase, this only proves that observable upstream events still exist; the local cursor must also be checked for progress, along with any sequence or digest gaps and whether the Discovery node has completed the corresponding index. If there is no upstream sequence, the interface fails, or the node has not yet produced data, the status should be `unknown` or `unavailable`; it must not be disguised as a lag distance of zero.

20.6 Governance and Authorization Metrics

Governance metrics record the event source, on-chain location, event type, index cursor, node status changes, and latest synchronization time. Authorization eligibility, resource control, and publication results are observed separately. The indexing of a governance event does not mean that all resource statuses have been updated, and a normal node does not mean that every authorization artifact is valid. Verification failures should retain their evidence and reasons. Governance observation should also retain the original event digest and separately record the receipt, decoding, projection, and external-serving times of the off-chain trust indexer. Resource control is proven by the resource's own signature and DID document; node authorization is proven by the governance path. These must remain separate in metrics and failure analysis.

20.7 Network-Level and Node-Level Statistics

The Network page may display both network-level aggregates and node-level details, but the aggregation source, update time, and missing nodes must be traceable. All periodic statistics use the same time zone, boundaries, and deduplication rules. When an old snapshot is used, it carries the snapshot time and degradation marker. Aggregated values must not simply add cumulative values with different definitions.

Window	Start	End	Applicable value
TODAY	Midnight of the current day	Midnight of the following day	Current-day periodic value
THIS WEEK	Agreed start of the week	Start of the next period	Periodic weekly value
THIS MONTH	First day of the current month	First day of the next month	Monthly periodic value
ALL TIME	Defined statistical start point	Current query time	Cumulative value

All windows use the same time zone and half-open interval `[start, end)`. Missing data, request failure, and an actual zero value must be encoded separately; missing values must not be filled with zero.

20.7.1 Network-Level Metrics

Network-level metrics describe nodes and resources within the statistical scope, including node count, registration, publication, discovery, governance synchronization, and network status. Each metric should state its inclusion conditions, deduplication method, source interface, and update time. If a node lacks data, the aggregate result should state its coverage and missing data; the node must not be counted as zero merely because it did not report.

20.7.2 Node-Level Metrics

Node-level metrics record health status, authorization status, resource processing volume, synchronization cursor, index freshness, and latest error by node DID or stable node identifier. Offline, unauthorized, not-yet-synchronized, and statistics-interface failure states use different values, avoiding the concealment of real conditions with zeros or empty strings.

20.7.3 Registration, Publication, and Discovery Metrics

Registration, publication, and discovery metrics respectively represent acceptance by a Registrar node, trusted publication by the Root platform, and index visibility at a Discovery node. They must not be merged into one “success count.” The same resource version is correlated by DID, version, and publication sequence. Retries, duplicate indexing, and old snapshots must not create artificial growth.

20.7.4 Metric Time Windows

Time windows use a unified time zone and explicit boundaries. Periodic values use fixed start and end times, while cumulative values are calculated from the defined statistical start point. Interfaces, the official website Network page, and backend reports must use the same definition to avoid abnormal periodic values caused by differences in server time zones, request times, or cache generation times.

20.7.5 Missing Data and Degraded Display

Missing metrics, interface failures, data that has not yet been produced, and old-snapshot display are marked separately. A page may display old data with its time, but must show the degraded state. The backend continues to record the actual failure. Missing data must not be converted into zero, and a snapshot must not be treated as a real-time health conclusion.

20.8 Visibility of On-Chain Governance Events

Governance-event display should distinguish the on-chain event time, receipt and indexing time of the off-chain trust indexer, the time when the official website interface was generated, and the time when the page displayed the event. The event sequence or on-chain location is used for ordering and traceability; bilingual descriptions are display-layer mappings and cannot replace the original event. When the chain or indexing service is temporarily unavailable, the page should show an unavailable or delayed state.

20.9 Logs, Traces, and Correlation Identifiers

Cross-service records should at least correlate the request identifier, resource DID, version, publication sequence, event cursor, and node identifier, and should record the time, level, result, and error digest. Logs must not write private keys or sensitive request content. Error information exposed by public interfaces should be sufficient to locate a problem without revealing internal configuration.

Correlation fields at the interface layer should correspond to the protocol types: node-to-node requests use `requestId`, `requestNonce`, `bodyHash`, and `upstreamAuth`; resource propagation uses `publicationCursor`, batch sequence, and resource package version; discovery queries use the request identifier, Discovery node DID, candidate count, retrieval backend, and duration; the off-chain trust indexer should also record the last processed checkpoint, governance sequence, and synchronization time. These fields should be organized as structured logs rather than merely concatenated into a free-form message.

```
{
  "requestId": "req-example",
  "resourceDid": "did: oan:SKDM:example-resource",
  "packageVersion": "1.0.0",
  "publicationCursor": 42,
  "nodeDid": "did: oan:INDS:example-node",
  "operation": "discovery_query",
  "result": "completed",
  "elapsedMs": 18
}
```

The example only shows public or low-sensitivity fields that can be used to correlate a processing flow. Production logs must still filter resource descriptions, endpoint credentials, signature material, and internal error stacks according to the deployment policy. `requestId` supports cross-service tracing, but is not an authorization credential and cannot replace resource control proof or the governance-event sequence.

20.10 Snapshot, Cache, and Degraded-Mode Behavior

Status	Frontend behavior	Backend meaning
live	Display the latest response	Interface and data are normal
stale	Display the snapshot and mark its time	Backend refresh failed or data expired
unavailable	Display the reason for unavailability	Interface is unreachable or returned an error
unknown	Make no trust judgment	There is insufficient evidence

Snapshot replacement must satisfy four conditions: “complete response, structural validation, source update, and time update.” A partial response cannot replace a complete old snapshot. An old snapshot is a user-experience mechanism, not health proof. The frontend may first display a

browser-side snapshot and refresh in the background. After success, it replaces the snapshot with complete new data and updates the source and time. After failure, it keeps the snapshot but displays the degraded state; continued failure still triggers a backend alert. A loading state is displayed only on a cache-free first visit; loading must not permanently cover data that is already available.

A snapshot record should contain at least the data source, generation time, schema version, and data status, for example:

```
{
  "source": "official-discovery-summary",
  "capturedAt": "2026-09-10T10:00:00Z",
  "dataStatus": "stale",
  "schemaVersion": 1,
  "coverage": "official-nodes",
  "payload": { "resourceCount": 12 }
}
```

Here, `resourceCount: 12` is a business value from one particular snapshot and does not represent the current real-time count. The frontend may continue to display it when a refresh fails, but the backend must still record the refresh failure and its duration. When the interface recovers, only a complete response that passes structural validation may replace the snapshot.

20.10.1 Loading an Old Snapshot

When the page opens, it first reads an available browser-side snapshot and marks its generation time and source. If snapshot parsing fails, the schema version is incompatible, or the content integrity check fails, the snapshot should be discarded and the page should enter the first-load flow; damaged data must not be rendered as normal statistics.

20.10.2 Background Data Refresh

After displaying an old snapshot, the frontend requests the latest interface in the background without blocking the user's view of the existing content. The request should have timeout and failure handling. The refresh must not change the resource details or query results currently being viewed unless the new response passes complete structural validation.

20.10.3 New Data Replacement

Only a complete and structurally valid new response may replace the old snapshot, while the cache time, source, and data status are updated together. If the new response is partially missing, retain the usable old data and mark the degraded state; an empty response or zero value must not replace valid content.

20.10.4 Long-Term Failure and Alerts

Consecutive failures, long-term absence of data, and a synchronization cursor that has not advanced for an extended period should enter backend alerting. Record the interface, first failure

time, latest successful time, and evidence of the error. Continuing to display a snapshot must not suppress an alert. Recovery must be confirmed by consecutive successes and a business check.

20.10.5 First Load Without a Cache

When no snapshot exists, display the first-load state and explicitly finish it after success, failure, or timeout. A failure state should explain that the data is temporarily unavailable. It must not display loading text indefinitely, and it must not fabricate a zero value or an empty health conclusion.

20.11 Backup, Recovery, and Disaster Recovery

Backup object	Check after recovery	Handling when the check fails
Node identity and authorization	DID, public key, authorization status	Stop external service
Database	Critical records, migrations, and indexes	Do not start write tasks
Publication queue	Incomplete tasks and idempotency keys	Pause duplicate delivery
Synchronization cursor	Cursor continuity and upstream sequence	Resume synchronization from a trusted position
Website artifacts	Correspondence among submission, hash, and configuration	Roll back or rebuild

Backups cover node identity and authorization material, databases, resource packages, publication queues, synchronization cursors, configuration versions, and website deployment artifacts. Private keys and credentials are encrypted separately and access to them is restricted. After recovery, first verify integrity, sequence continuity, and key correspondence; then start write tasks, and confirm recovery through health checks, registration/discovery smoke tests, and index catch-up. The recovery order should first restore configuration and node identity, then the database and its migrations, resource packages, and publication queue. Next verify the synchronization cursor, governance authorization status, and website build artifacts, and finally restore external writes. The recovery process must prevent old queue items from being published repeatedly and prevent cursor rollback from causing duplicate projections. The service may first start in read-only mode or with writes paused; open writes only after checking `resourceDid`, version, publication sequence, and authorization status. A passing `/health` only means that the process has recovered; it cannot by itself prove that historical indexing, governance projection, and incomplete publication tasks have been restored.

20.12 Deployment Smoke Tests and Functional Verification

Minimum smoke-test loop:

1. Record the deployment commit, artifact digest, and environment configuration.
2. Check service health, database connection, and index cursor.

3. Use an isolated identity to complete registration and an expected-failure request.
4. Check Root platform publication, distribution, and exact/semantic discovery queries.
5. Save responses, logs, the timeline, and the cleanup result for test resources. Smoke tests use isolated test identities and test resources, and save the submission time, correlation identifiers, responses, logs, and cleanup results. Successful registration cannot replace discovery verification. Root platform publication, content distribution, index convergence, and database status must also be checked. If any key step fails, the response is uncertain, or production data remains uncleared, the deployment must not be declared successful. Smoke-test results should form a reviewable timeline rather than output only a “passed” string. Each step records the target address or service role, commit or artifact digest, request identifier, response status, duration, key response fields, and side-effect cleanup result. Registration acceptance, Root platform publication, and a discovery query hit are assessed separately. When writing a real resource to a production environment is inappropriate, use a controlled failure request to verify validation logic and an existing isolated test resource to verify the discovery path. If any key step is uncertain, the deployment status should remain pending verification.

20.12.1 Registration Function Smoke Test

Use an isolated identity and a controlled resource to verify request validation, control signatures, Registrar node acceptance, duplicate submission, and error responses. If the interface returns a registration VC, check its structure and its association with the resource DID, version, or digest. Clean up the test data after the test and save the evidence.

20.12.2 Discovery Function Smoke Test

Use a published isolated test resource to execute an exact DID query and a semantic query, and check the resource DID, version, status, source, publication evidence, and index time. If the result is empty, an old version is returned, or the interface fails, use the publication record and synchronization cursor to determine whether this is propagation delay or a functional failure.

20.12.3 Node Health Check

Check the liveness, readiness, and dependency status of the Root platform, Registrar nodes, Discovery nodes, content distribution platform, off-chain trust indexer, and official website backend one by one. Record the response code, duration, version, and check time; do not check only whether the process exists.

20.12.4 Database and Indexer Check

Check the database connection, migration status, critical records, and indexer cursor. Verify the relationship and continuous change between the upstream latest sequence, local cursor, and `latest_sequence`, and distinguish the absence of new events from a stopped synchronization.

20.12.5 Deployment Handling after Failure

When a key smoke test fails, stop publication confirmation and retain the logs, commit, artifact hash, and test-resource information, then handle the issue through the repair or rollback process. Only after a retry succeeds, data side effects have been cleaned up, and evidence is complete may an acceptance result be issued again.

20.13 Data Retention, Deletion, and Privacy Operations

Public metrics return only necessary aggregate data. Access statistics, request correlation identifiers, and operational logs should be controlled by backend permissions. Browser-side local identity private keys, backup contents, and sensitive material submitted by users must not be written to access statistics. Deletion operations should explain the different retention rules for online data, backup copies, and immutable audit records.

20.14 Operational Alerts and Long-Term Failure Detection

An alert record should contain at least the service, interface, first failure time, latest successful time, consecutive failure count, error digest, impact scope, and current status.

Status	Trigger basis	Recovery condition
Intermittent failure	A small number of requests fail within the window	Subsequent requests succeed consecutively
Consecutive failure	Consecutive requests fail beyond the threshold	Health and business checks succeed consecutively
Long-term stagnation	A metric or cursor has not updated beyond the allowed duration	Data recovers and the sequence advances again
Dependency failure	An upstream service or database is unavailable	Dependency recovers and regression smoke tests complete

Use a stable fingerprint to deduplicate the same failure. A recovery alert must have both consecutive successful health checks and the corresponding business smoke test; it must not rely only on restarting the process. Long-term failure detection covers **summary**, governance synchronization, publication queue, and index-cursor stagnation. Alerts record the service, interface, consecutive failure count, first failure time, latest successful time, error digest, and impact scope, and deduplicate the same failure. Recovery requires consecutive success and business verification. When the website continues to display an old snapshot, backend alerts must not be suppressed. Periodic checks such as the **summary** interface should retain two facts: “the latest success” and “the latest failure,” rather than only retaining the current status. An alert event should at least correlate the service role, interface, time window, consecutive failure count, latest successful sequence or time, error digest, and impact scope. Short-lived network jitter for the same interface may be merged into one failure fingerprint, while continued failure should be escalated. During recovery, first confirm consecutive interface success, then confirm that the data time or cursor has advanced again, and finally execute the affected registration/discovery business smoke test. Continuing to display an old snapshot in the browser must not pause or clear this alert.

20.15 Public Service Status and Maintenance Communication

Public status information should state the affected service, start time, current status, data delay, and next update, without exposing keys, internal addresses, or user data. During maintenance, read-only queries or old snapshots may be retained, but the time point and unavailable functions should be marked. A recovery announcement should be based on actual checks and business smoke tests.

20.16 References

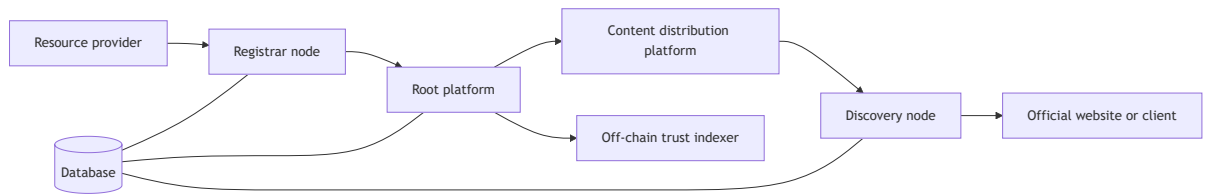
Source	Type	Link
<code>oan-root-services</code>	Code repository: Root platform status, publication, and distribution metrics	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: Registrar node status, registration statistics, and health interfaces	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	Code repository: synchronization, indexing, querying, and freshness metrics	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-trust-indexer</code>	Code repository: synchronization cursor, projection status, and governance metrics	https://github.com/OpenAgenet/oan-trust-indexer
OAN Yellow Paper	arXiv Yellow Paper: operational status and trust-evidence boundaries	https://arxiv.org/abs/2606.03163

Chapter 21 Deployment and Conformance Requirements

This chapter explains how deployment conformance demonstrates what was deployed, how it was built, whether its dependencies are ready, whether the business functions are available, and how failure is handled through rollback. The node services, website backend, and frontend static resources of OpenAgenet (OAN) should separately record the source commit, build artifacts, configuration version, and acceptance evidence. The official deployment describes a runnable instance, while the protocol requirements apply to every implementation that claims conformance to the specification.

21.1 Reference Deployment Topology

The reference topology consists of the Root platform, Registrar nodes, Discovery nodes, the content distribution platform, the off-chain trust indexer, databases, and the official website. Only necessary endpoints protected by TLS and a reverse proxy are exposed externally. Nodes communicate using configured service addresses and authorization materials. Third-party nodes may be deployed independently, but they must pass conformance checks for node identity, authorization, interfaces, and synchronization before joining.



The deployment topology should separately identify public entry points, internal services, databases, and governance data sources. Public addresses, ports, and node counts are environment configuration and should not be fixed in the diagram as protocol requirements.

During deployment, checks can be organized around four boundaries: public entry points, business services, state storage, and governance and synchronization. The official website and reverse proxy belong to the access boundary; node services carry registration, publication, distribution, or discovery workflows; databases store the runtime state of their respective services; and the off-chain trust indexer reads governance events and builds a read model. The specific hosts and ports at each boundary may change between environments, but every cross-boundary connection should identify its initiator, target, purpose, authentication materials, and the impact of failure.

21.2 Configuration and Secret Management

Configuration category	Source	Deployment check	Exposure boundary
Node identity	Controlled key file or key service	DID, public key, and permissions match	Must not enter public interfaces
Database	Environment configuration	Connection, migration, and permissions are valid	Must not be exposed through the website proxy
Upstream endpoint	Environment profile	Address is reachable and protocol is compatible	Accessed only by necessary services
Governance configuration	Authorization configuration and version record	Package, object, and cursor are consistent	Must not return secrets

Configuration is injected per environment. The code package must not carry production private keys, database credentials, or external service tokens. The node DID private key, Root platform authorization materials, database connection information, on-chain package or object configuration, and discovery and distribution endpoints should have restricted file permissions and separate backups. Required fields should be checked before startup; during rotation, the audit record for the old and new configurations should be retained. A failed pre-start check should prevent the service from entering the ready state. Configuration rotation should record the expiration time of the old configuration, the activation time of the new configuration, and the verification result.

The following minimal record can be used as release evidence for configuration checks. The values indicate only whether a value exists and whether it passed validation; secrets must not be written into the report:

```
profile=production
service=oan-homepage-backend
OAN_HOME_PAGE_LISTEN_ADDR=present:valid
OAN_OFFICIAL_ROOT_ENDPOINT=present:https
OAN_OFFICIAL_REGISTRAR_ENDPOINT=present:https
OAN_OFFICIAL_DISCOVERY_ENDPOINT=present:https
database=present:connected
node_identity=present:did-public-key-match
secrets=not-printed
```

If a required endpoint, database, node identity, or governance configuration cannot be validated, the deployment should remain in the configuration stage. A successful process start or accessible page must not be used to conceal a missing item.

21.3 Build and Release Artifacts

A release record should include the source code version, lockfile, Rust and frontend build environments, build steps, artifact hashes, configuration version, and release time. Deployment may synchronize a versioned source archive or an equivalent package containing the confirmed version, then generate the Rust backend and frontend static resources in the target build environment. The source archive, binaries, static resources, and configuration should be mutually traceable. Before

release, a traceable chain of “source version → build → artifact → deployment” should be established. Temporary source changes or unrecorded build directories in the target environment must not serve as the official source.

Artifact	Source	Verification information
Source archive	Explicit Git commit	Commit, file list, and archive digest
Rust binary	Locked dependencies and build environment	Artifact hash, version, and target platform
Frontend static resources	Official website build script	Build commit, resource list, and digest
Configuration	Environment-specific material	Configuration version or fingerprint, without exposing secrets

A release record can describe the correspondence between artifacts with a summary that contains no secrets:

```
{
  "sourceCommit": "<git-commit>",
  "lockfile": "<lockfile-sha256>",
  "backendArtifact": "<backend-sha256>",
  "frontendArtifact": "<frontend-sha256>",
  "configFingerprint": "<config-fingerprint>",
  "builtAt": "<utc-time>",
  "environment": "<profile>"
}
```

The values enclosed in angle brackets are record templates, not fixed official OAN values. During an actual release, they should be replaced with real digests, and the source archive, build log, artifact hashes, and server deployment record should be kept in the same acceptance record.

21.4 Database and Storage Requirements

Before deployment, confirm the database version, connection, migration, and index status. Migrations must be idempotent, traceable, and preceded by a backup. Storage should cover resource registration, versions, credential associations, publication tasks, synchronization cursors, governance events, and the data required for statistics. After recovery, check key records, sequence continuity, queue status, and index consistency; checking only whether the process starts is insufficient.

Database acceptance should separately record that the database can be connected to, migrations are complete, data is available, and consistency remains after recovery. Important checks include the uniqueness of the resource DID and version, idempotency keys for publication tasks, records required by the discovery index, and the continuity of governance events and synchronization cursors. If migrations are complete but a required index is missing, the service should not enter a writable ready state. Database checks must not return internal table structures through public website interfaces. Reports should retain only the necessary version, count, digest, and verification results.

21.5 TLS, Domains, Reverse Proxy, and Network Exposure

The domain and TLS termination point should match the actual service entry point. The reverse proxy should forward only necessary paths while preserving the correct Host, protocol, and client association information. Public ports, cross-origin sources, firewalls, and node-to-node access rules should be configured per environment. Databases, management interfaces, and internal queues must not become exposed to ordinary Internet users because of website proxy configuration. During acceptance, check from both the public network and the internal network the allowed paths, certificate hostname, HTTP-to-HTTPS redirect, CORS scope, and rejection results for non-public paths.

21.6 Deployment Order and Dependency Checks

The recommended order is to record the baseline, back up and synchronize the code, build the artifacts, prepare the database, start the Root platform and its dependent services, then start the Registrar nodes, distribution, Discovery nodes, and indexer, and finally deploy the official website and perform business smoke tests. If any required dependency, migration, health check, or critical smoke test fails, stop at the current step, retain the evidence, and enter repair or rollback. The pass condition for each step should be defined by the dependency actually required by the next step; a started process alone is not a release gate.

21.6.1 Code Synchronization and Build

Generate an archive or equivalent release package from an explicit source code version. After transferring it to the target build environment, verify the files and version information. Compile the node services using the Rust dependencies locked by the project, and generate static resources and backend artifacts using the frontend build script. Do not release from temporary workspace files or unrecorded local build directories.

21.6.2 Database Preparation

Confirm that the backup can be restored, the database is reachable, the migration version is correct, and required indexes are present before running an idempotent migration or startup check. A failed migration must not be followed by starting a service that writes new data. If manual handling is required, retain the failure log, database version, and recovery point.

21.6.3 Node Startup Order

Start the Root platform and its database and publication dependencies first, followed by the Registrar nodes, content distribution platform, Discovery nodes, and off-chain trust indexer. The actual order may be adjusted according to the deployment topology, but before a downstream service starts, confirm that the upstream address, identity, authorization, and interfaces are ready. The existence of a process must not be treated as a substitute for dependency availability.

21.6.4 Service Health Checks

Check process liveness, readiness interfaces, critical dependencies, and configuration loading results one by one, and record the response code, latency, version, and time. A passing health check does not mean that the business path has passed. In particular, separately verify Root platform publication, registration acceptance, discovery queries, distribution, and index synchronization.

21.6.5 Post-deployment Functional Verification

Post-deployment functional verification should cover at least:

1. Health and readiness interfaces.
2. Valid requests and expected failure requests for the Registrar nodes.
3. Root platform publication, content distribution, and synchronization to the Discovery nodes.
4. Exact DID queries and semantic queries.
5. Database migration status, cursors, logs, and alerts.

When any critical item fails, retain the submission, artifact, configuration, and response evidence, and stop the release completion confirmation.

Interface verification should record the request method, actual URL, HTTP status, response type, key business fields, and correlation identifiers. For dynamic JSON interfaces, checking only for a 2xx response is insufficient. Also check that the registration response contains the resource DID and registration status, that the Root platform response is `resource-verified-and-queued`, that the content distribution response contains a publication cursor, that the discovery response can be deserialized as `ResourceDiscoveryResponse`, and that the indexer status identifies the current synchronization position. When accessed through the official website proxy, additionally verify that the proxy-path mapping to the original node path has not changed the request body or error semantics.

21.7 Registrar Node Functional Smoke Tests

Use an isolated test identity and controlled resource to verify form or request validation, control signatures, Registrar node acceptance, duplicate submission, and error responses. If the interface returns a registration VC, check its association with the resource DID, version, or digest. Subsequent publication and cleanup of the test resource must not pollute normal resource statistics. Successful registration proves only that the Registrar node accepted the request; it does not automatically prove that the Root platform has published it or that the Discovery nodes have indexed it.

Acceptance step	Result	Failure handling
Valid registration	Acceptance result and correlation identifier are verifiable	Stop confirmation and retain request evidence
Duplicate registration	Idempotent; no incorrect new registration fact is added	Check deduplication and database changes

Acceptance step	Result	Failure handling
Invalid signature	Clearly rejected and the original state remains unchanged	Check for side effects
VC result	If returned, consistent with the DID, version, or digest	Mark as missing or invalid; do not fabricate success
Cleanup	Test resource can be identified and cleaned up	Enter manual review

21.8 Discovery Node Functional Smoke Tests

Perform exact DID queries and semantic queries against an isolated test resource that has been published. Check the DID, resource type, version, status, source, Root proof status, and index time. If a query returns no result, an old version, or an interface failure, use the cursor and publication record to determine whether the condition is propagation delay or a functional fault. The pass condition for a semantic query is an explainable candidate and source, not a fixed ranking or an undefined relevance score.

21.9 Root Platform, Distribution, and Indexer Verification

Independently check the publication status of the Root platform, resource or cursor records of the content distribution platform, synchronization status of the off-chain trust indexer, and index results of the Discovery nodes. Each check must state its source, time, and normal criterion. Observing success at the Root platform does not prove that downstream services are already discoverable. The same isolated resource version should be used throughout the verification, and the Root platform publication sequence, distribution digest, index version, and query result should be recorded. Equality of total counts across interfaces must not substitute for a content-consistency check.

21.10 Governance Synchronization Verification

Select a governance event that can be confirmed, and check its on-chain position, event type, trust indexer cursor, node-state read model, and update time. Across two or more consecutive checks, the relationship between the cursor and `latest_sequence` should match the synchronization state of the deployment environment. The absence of new events must be distinguished from a stopped synchronization process. If governance events have been read but the projection has not been updated, classify the synchronization path as incomplete rather than allowing it to pass solely because the indexer interface responded successfully.

21.11 Upgrade, Rollback, and Compatibility Checks

Before and after an upgrade, check database migrations, protocol versions, resource schemas, historical versions, node interoperability, and rollback paths. Continue an upgrade only when compatibility and backup conditions are satisfied. If the new artifact fails health or business verification,

roll back according to the recorded version and configuration, then rerun the key acceptance checks. Upgrade verification should use representative resource DIDs, resource packages, registration requests, and discovery queries rather than checking only whether the service starts.

21.11.1 Pre-upgrade Checks

Record the current software version, binary hash, configuration, database state, and cursor state. Confirm that the backup is readable, the target version can be built, the migration can be rolled back or recovered, and the old version remains available.

21.11.2 Data and Configuration Compatibility

Check newly added fields, resource package and credential structures, protocol routes, environment variables, node authorization materials, and frontend interface expectations. Compatible conversions should be repeatable. Historical data that cannot be converted must stop the release before deployment rather than be silently discarded during runtime.

21.11.3 Rollback Conditions

Evaluate rollback when a database migration fails, a critical health check fails, registration or discovery smoke tests fail, the publication queue becomes abnormal, indexing remains stalled, or a security configuration is incorrect. For irreversible data migrations, first pause writes and process the recovery plan; replacing only the binary is insufficient.

21.11.4 Post-rollback Verification

After rollback, check the service version, configuration, database records, node identity, publication sequence, and synchronization cursor again. Then perform health, registration, discovery, Root platform, distribution, and index verification. Rollback may be closed only when the old version has returned to a demonstrable working state.

21.12 Third-party Node Onboarding and Conformance Testing

Third-party nodes should submit node identity, authorization materials, public endpoints, supported interfaces, and environment information, and pass tests for signature verification, authorization, protocol responses, resource publication, synchronization, querying, security boundaries, and failure recovery. Passing the tests grants onboarding eligibility only within the explicitly defined scope; it does not replace subsequent operational monitoring and governance management. The admission report should specify the node role, authorization domain, interface version, test environment, and validity period. A role, domain, or interface that was not tested must not be inferred to be compliant from another passing item.

The minimum acceptance evidence for a third-party node can be organized in the following order:

```
node identity -> authorization -> protocol response
              -> controlled registration/publication
              -> synchronization/discovery
              -> failure recovery and evidence review
```

Testing should first be completed in an independent database and isolated endpoints. A node's ability to call an interface does not mean that it has authorization, and its ability to synchronize data does not mean that it may publish resources. Every conclusion should be bound to the actual test scope and evidence.

21.13 Reproducible Builds and Release Provenance

Build records should trace artifacts back to the commit and lockfile, and should retain at least the build environment, commands, dependency versions, artifact hashes, and operator. The running binaries, frontend static resources, and source archive on the server should be mutually verifiable. Configuration and secrets are not included in public artifact hashes, but their version or fingerprint should be recorded. For the Rust backend and the official website frontend, build results should be retained separately, and the deployment record should explain how they are combined into the same release batch.

21.14 Environment-specific Configuration Files

Development, test, staging, and production environments should use separate profiles, endpoints, databases, and keys. Test authorization materials and production private keys must not be reused across environments. Configuration changes should have a commit, approval, or operation record. Differences between environments running the same code should be concentrated in configuration rather than implemented through unrecorded manual source edits. During acceptance, verify that the current profile matches the endpoints, database, node identity, and governance objects, so that a passing result in a test environment is not mistakenly recorded as a production pass.

21.15 Post-deployment Functional and Security Verification

The acceptance record summarizes versions, environment, configuration, health interfaces, business smoke tests, database, governance synchronization, permissions, TLS, public ports, and log checks. It should also confirm that private keys have not entered logs or public interfaces, management data is not exposed to ordinary users, and error responses do not disclose secrets. Retain reproducible command output or response summaries. The final conclusion should state the scope, evidence time, uncovered items, and whether the service is permitted to provide services externally. If any critical item is **FAIL** or uncertain, the overall result must not be written as passed.

21.16 References

Source	Type	Link
oan-root-services	Code repository: Root platform services and deployment configuration	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	Code repository: Registrar node and deployment configuration	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	Code repository: Discovery node and deployment configuration	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	Code repository: Trust indexer and chain endpoint configuration	https://github.com/OpenAgenet/oan-trust-indexer
oan-protocol-common	Code repository: Cross-service protocols, configuration-related types, and error contracts	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv yellow paper: Deployment, trust, and verification boundaries	https://arxiv.org/abs/2606.03163

Chapter 22 External Protocols and Ecosystem Mapping

This chapter treats external protocol integration as a traceable transformation process: first identify the external object and its version; then map verifiable fields to OpenAgetnet (OAN) resource metadata or a resource package; next verify resource control, registration credentials, the Root platform's Root proof, and governance status separately; and finally explain to the caller which information can be used for discovery and which information must still be rechecked before invocation. An adapter must preserve the external original, source, summary, and transformation version, and must not treat a transformed summary directly as an OAN trusted publication fact.

External material	OAN carrying location	Content available for discovery	Content requiring independent verification
DID Core / DID Document	Resource DID Document and verification methods	id, verification methods, service endpoints	Control relationship, signatures, status, and publication evidence
A2A Agent Card	Resource metadata, protocol binding, or associated description	Capabilities, skills, use cases, inputs and outputs, and interaction entry points	Card publisher's relationship to OAN control, registration credentials, and the Root platform's Root proof
MCP service metadata	MCP service resource package and its protocol binding	Server capabilities, tool inventory, transport, and authentication declarations	Service endpoint, version, resource package hash, and lifecycle
OpenAPI or tool API description	Tool API resource package and its protocol binding	Operations, parameters, schemas, errors, and endpoints	Document digest, control signature, business authorization, and actual availability

A mapping result answers “how the external system describes and connects to this resource”; it does not directly answer “whether this resource has been authorized by the OAN network.” The latter determination must return to the OAN DID, signatures, registration credentials, Root proof, and governance status.

22.1 W3C DID Core and did:oan

An adapter should store common DID Core fields and `did:oan` method fields in separate layers. Common fields allow DID tools to understand the document, while OAN fields express the resource type, capabilities, version, resource package, and publication relationships. The following object is only an example of a mapping record, not a new DID Document:

```
{
  "source": {"type": "did-document", "version": "did-core-compatible"},
  "oan": {"method": "did:oan", "resourceDid": "did:oan:SKDM:<identifier>"},
  "mapping": {
    "preserved": ["id", "verificationMethod", "service"],
    "mapped": ["resourceType", "capabilityTags", "protocolBindings"]
  }
}
```

This record only describes the transformation process. `did:oan` uses the common identity and service expression mechanisms of DID Core, and supplements the information required for resource discovery and publication through OAN resource metadata, resource packages, and associated proofs.

22.1.1 DID Core Data Structure Correspondence

The `id`, `verificationMethod`, control relationships, and `service` in a DID Document correspond to the common data model of DID Core. OAN resource types, capability tags, use cases, versions, resource package references, and publication-proof associations belong to the OAN resource model or method extensions. An adapter should preserve the original fields and original paths, and must not disguise OAN extension fields as DID Core standard fields.

DID Core field	OAN corresponding content	Verification focus
<code>id</code>	Resource or node DID	Do not rewrite it because of a name, URL, or display format
<code>verificationMethod</code>	Verification method of the controller or node	Public key, control relationship, and signed object are consistent
<code>authentication / assertionMethod</code>	Authentication and assertion purposes	Do not add an arbitrary public key to an authorization relationship
<code>service</code>	Resource endpoint and protocol entry point	Readable does not mean callable

22.1.2 did:oan Method Extensions

The `did:oan` method extensions express the resource subject, resource type, capability set, endpoints, protocol bindings, version references, and publication-evidence associations. Method extensions must specify field types, required conditions, normalization rules, and signature coverage. Changes to extension fields should enter DID Document or resource package version management.

The method-extension checks proceed in the following order:

1. Parse the method name and method-specific identifier.

2. Check the resource type or node role against the subject code.
3. Check metadata, services, versions, and resource package references.
4. Independently verify the control proof, hash, registration VC, and Root proof.

An extension that affects signature coverage, resource type, version selection, or governance decisions should be treated as a breaking change and entered into the change record in Chapter 25.

22.1.3 Resolution Behavior

A resolver receives the complete `did: oan`, first validates the method name and method-specific identifier, then reads the corresponding DID Document and verifies its structure, control relationship, status, and version. Not found, method unsupported, invalid document, missing proof, and unknown status should be returned separately; errors must not be concealed by an empty document or the most recent cache.

Resolution status	Meaning	Follow-up handling
resolved	The document is readable and structurally valid	Continue verifying control, status, and publication evidence
not-found	No corresponding document was found	Do not substitute a file with the same name
method-unsupported	The resolver does not support this method	Stop high-risk invocation
invalid-document	The structure or binding is invalid	Reject and retain error evidence
status-unknown	The status source is stale or unavailable	Degrade service or require manual review

22.1.4 Interoperability Boundaries

DID Core interoperability ensures that the common identity structure can be understood; it does not ensure that an external resolver understands OAN resource semantics, the Root platform's Root proof, or governance status. An external system can read the DID Document and obtain entry points, but before invocation it must still verify the resource version, publication evidence, and endpoint according to OAN rules.

An interoperability report should record format interoperability and OAN semantic interoperability separately. The former proves that an external resolver can read common fields; the latter proves that the caller can understand the resource type, capabilities, version, resource package, and publication evidence. They must not be merged into a single “compatible” conclusion.

22.2 A2A Agent Card Mapping

The relationship between OAN and an A2A Agent Card can be distinguished as “network-level resource discovery” and “service-level invocation description”: OAN provides stable identity, source, version, and governance association through the DID Document, resource metadata, and resource package; an A2A Agent Card describes the agent service provider, supported interaction entry

points, capabilities, skills, input and output modes, and security declarations. The two can reference each other, but a mapping does not change the identity, signature, or authorization semantics of either side.

Mapping direction	Content preserved	Conclusions that cannot be derived
OAN → A2A	Resource DID, name, description, capability tags, use cases, service endpoints, protocol bindings, and version	The mapping cannot automatically obtain A2A session authorization or business permissions
A2A → OAN	Service provider, capabilities, skills, input and output modes, security declarations, and original digest	The Card cannot automatically obtain OAN registration eligibility or the Root platform's Root proof

The capability descriptions, use cases, endpoints, and protocol bindings in an OAN resource profile can provide a trusted discovery entry point for an A2A Agent Card. For reverse integration, an Agent Card can only serve as a source for a registration draft or supplementary description; the resource controller must still generate and sign the resource materials through the OAN process. Detailed invocation flows are determined by A2A, MCP, a tool API, or the resource's own protocol.

22.2.1 Capability Description Mapping

OAN's `capabilityTags`, `capabilityDescription`, and use-case information can be mapped to the `skills`, skill names, descriptions, and example tasks of an A2A Agent Card. During mapping, preserve the tag-tree path, original text, resource package version, and source. The semantic matching result from an OAN Discovery node can help locate candidate resources, but it cannot prove that a resource is authorized or necessarily possesses the declared capability.

It is recommended to preserve both normalized tags and the original description:

```
{
  "tag": "code.repository",
  "sourcePath": "skills[0].name",
  "sourceVersion": "1.0",
  "resourcePackageVersion": "1.0.0"
}
```

Normalized tags serve filtering and semantic governance; the original text serves explanation and review.

22.2.2 Use-Case and Skill Mapping

OAN use-case fields and skill package information can be used to generate the skill summary, example tasks, and input and output descriptions of an Agent Card. The generated result should retain the resource package version and field sources. If an external Card provides only a summary, reverse registration can produce only a resource description requiring verification; it cannot fill in control, version, endpoint, or governance information that OAN does not contain.

Use-case field	Fact it should express	Handling when missing
Task	The work the user wants to complete	Mark the description as incomplete
Input	Type, source, and prerequisites	Do not invent content
Output	Result form or deliverable	Do not treat a tag as a promise
Constraints	Permission, cost, timeliness, and data boundaries	Recheck before invocation

22.2.3 Service Endpoint Mapping

Service endpoints, protocol types, transport methods, and authentication requirements in an OAN DID Document or resource package can be mapped to the `supportedInterfaces`, protocol bindings, and security declarations of an Agent Card. Endpoint mapping only provides connection-preparation information and does not mean that the endpoint is available; the caller must still perform TLS, authentication, version, status, and business-authorization checks. Endpoint mappings can record “entry point, protocol, transport, authentication, version, and status” item by item. When any item comes from an external description, retain the original path or digest so that it can be checked again before invocation.

Endpoint information	Use during discovery	Recheck before invocation
Entry-point URL	Locate the service	TLS, domain name, and reachability
Protocol/transport	Select a client	Version and negotiation result
Authentication requirements	Prepare credentials	User authorization and permission scope
Version	Select a compatible implementation	Whether it is still supported
Status	Indicate availability	Latest status and governance evidence

22.2.4 Identity, Signature, and Trusted-Evidence Mapping

The `provider`, signature, or security declaration of an Agent Card can be stored as external input, but the OAN resource identity is determined by `did:oan` and its control relationship. The overall signature of an Agent Card proves the Card's content and its publisher's statement; a registration VC, the Root platform's Root proof, and governance status prove different facts in the OAN process and must be retained separately rather than replacing one another.

Identity and evidence mappings should be stored in layers:

- The resource control relationship answers “who can control this DID.”
- The Agent Card or external declaration answers “how does the external system describe this resource.”
- The registration VC answers “what registration fact did the Registrar node declare.”
- The Root proof answers “what content did the Root platform verify and publish.”

22.3 MCP Resource and Server Metadata Mapping

OAN treats an MCP service as a discoverable resource. Mapping should record three layers: “server, tool, and invocation.” The server layer describes identity, protocol version, transport, and endpoint; the tool layer describes the tool or resource inventory, names, descriptions, and input schemas; the invocation layer describes sessions, authentication, and error constraints. The resource package should preserve the inventory original or a stable reference, version, and hash; the discovery index may store only a retrieval summary. A successful MCP initialization only means that the service completed one protocol interaction; it does not mean that the Root platform has verified and published the resource.

22.3.1 MCP Server Mapping

The name, description, protocol version, transport, endpoint, and server capabilities of an MCP Server are mapped to the metadata of an OAN MCP service resource. The server DID, control proof, resource version, resource package hash, and Root proof are stored independently to distinguish “server self-description” from “network-verified publication.” Each tool record should also retain its server and resource package version to avoid deduplicating across services solely by tool name.

22.3.2 Tool Definition Mapping

MCP tool names, descriptions, input schemas, and available capabilities can be mapped to OAN tool entries, capability descriptions, and tags. A tool entry should include its server, resource package version, content digest, and location in the original inventory. Identical tool names do not mean that the tools belong to the same controller. A discovery index may use summary fields, but the caller should be able to return to the complete inventory to verify the tool boundary.

22.3.3 Input and Output Constraint Mapping

MCP input schemas, result structures, and error constraints are mapped to OAN input and output descriptions, while preserving the original schema, media type, version, and hash. If mapping fails or fields are incomplete, the resource may be registered as incompletely described, but the caller must not be told that the constraints have been verified. Input and output mappings can use a schema version and digest to establish a stable reference. When field types, required relationships, enumeration values, or error structures change, treat the change as a new description version rather than merely updating display text.

22.3.4 Invocation and Authentication Information Mapping

MCP transport, session establishment, authentication requirements, and pre-invocation conditions can form part of OAN service information and support connection preparation after discovery. OAN publication proof proves only the declared materials and their source; it cannot grant business permissions to an MCP server or guarantee session success. Invocation and authentication mappings should distinguish “access conditions visible during discovery” from “permissions granted by

the service at runtime.” OAN registers and verifies declarations and their sources, but does not replace the business authentication of an MCP service or tool API.

22.4 Tool and API Description Mapping

OpenAPI mapping should establish stable entries at the path-and-operation level:

```
operationId: searchRepository
source: openapi.yaml#/paths/~1repositories~1search/get
oanResourceDid: did:oan:TLDM:<identifier>
schemaHash: sha256:<value>
```

A normalized summary can be used for discovery and deduplication; the original document is used for pre-invocation review.

A tool API should use the path and operation as the minimum audit unit and retain at least the operationId, original document path, server endpoint, schema version, authentication requirements, and digest. The normalized summary is used for discovery and deduplication; the original document is used for pre-invocation review. A readable document does not mean that the interface is callable.

22.5 VC and VP Ecosystem Interoperability

VC/VP processing should follow the sequence “parse → cryptographic verification → status check → trust-policy decision.” Each failure must produce a clear intermediate status; “valid JSON” must not be displayed directly as “trusted.”

Stage	Output	Is it sufficient for acceptance?
Parsing	Subject, issuer, type, and claims	No
Signature verification	Signature matches the public key	No; the relationship must also be checked
Status check	Validity period and suspension/revocation status	No; the authorization scope must also be checked
OAN trust policy	Decision based on source and governance	Yes / No / Review required

An adapter can parse the subject, issuer, credential type, validity period, status, and proof materials of an external VC/VP, and retain the original object and verification result. Being parseable only means that the format is readable; acceptance also depends on issuer authorization, control relationship, governance status, and the OAN trust policy. For input whose status information cannot be obtained, whose signature verification fails, or whose issuer relationship is unclear, retain the failure reason and route it to rejection or manual review.

22.6 External Organization and Platform Identity Mapping

An external organization, platform account, domain name, or its DID can be stored as associated information for a resource provider, operator, delegate, or endpoint host, but the relationship

type, evidence source, responsibility scope, and validity period should be recorded. A domain name, enterprise account, platform authentication record, and resource-control DID are not the same field. An external account name cannot replace control of the resource DID, and domain or platform authentication cannot directly create eligibility for Root platform publication.

22.7 External Credential and Trust-Evidence Mapping

An external evidence record is recommended to include `issuer`, `subject`, `credentialType`, `issuedAt`, `expiresAt`, `statusSource`, `sourceHash`, `verifiedAt`, and `verifier`. When no status endpoint is available, record “status not checked” rather than assuming validity. OAN treats it as supplementary evidence, while node authorization, resource control, and Root platform publication are still determined by their respective OAN evidence chains.

22.8 Field Source, Version, and Hash Requirements

A field source can be represented by the following audit record. `agent-card.json`, the digest, and the time in the example are placeholder values:

```
{
  "field": "capabilityTags[0]",
  "source": "agent-card.json#/skills/0",
  "sourceHash": "sha256:<value>",
  "mappingVersion": "oan-a2a-v1",
  "verifiedAt": "<timestamp>"
}
```

Each mapped field should record the source object, source path, original version, normalized version, signing subject, and content digest. The normalization operation should be reproducible. When sorting, case conversion, aliases, or field truncation changes the digest, the digest and mapping version must be updated, and the old result must be retained for auditing.

22.9 Compatibility Profiles and Migration Paths

An interoperability profile should specify the supported DID, A2A Agent Card, MCP, and OpenAPI versions; required and optional fields; signature algorithms; transport; error policy; and migration method. During migration, first retain the original evidence, read the old and new profiles in parallel, check field digests and signature boundaries, and then switch to single-write operation. If migration fails, return to the old profile without overwriting the sole source. The profile itself should have a name and version so that callers know which set of mapping rules is being used.

22.10 External Protocol Capability Negotiation

Negotiation results can be classified as `compatible`, `partially-compatible`, or `incompatible`, while also reporting the common protocols, versions, transports, and security capabilities together with unmet capabilities:

```
{
  "result": "partially-compatible",
  "commonProtocols": ["https"],
  "missing": ["required-auth-scheme"]
}
```

The client or adapter declares the protocols, versions, transports, and security capabilities it supports, and selects an available profile based on the intersection of both sides. When no common capability exists, return an explicit incompatibility result. Successful negotiation only means that the format and interaction capabilities are available; resource identity, publication evidence, governance status, and business permissions must still be verified separately.

22.11 External Signature and Evidence Retention

An adapter's output should store the original text, normalized digest, external signature, and OAN signature separately, and assign a source identifier to each item of material. When repackaging or forwarding, do not retain only the transformed object; it should at least be possible to answer “what was the original object, who signed it, what did they sign, did the transformation change the content, and which OAN party signed again.” During adaptation, retain external signatures, credential references, original objects, versions, digests, verification time, verifier, and failure reasons. When re-encoding or generating an OAN resource package, do not lose the boundary between external materials and the OAN control signature; the two types of signature separately describe the facts they cover.

The adapter output should store the original object, normalized object, external signature, resource-control signature, registration VC, and Root proof separately. An audit should be able to answer: what was the original object, who signed it, what did they sign, did the transformation change the content, which party signed again, and when did verification occur.

22.12 References

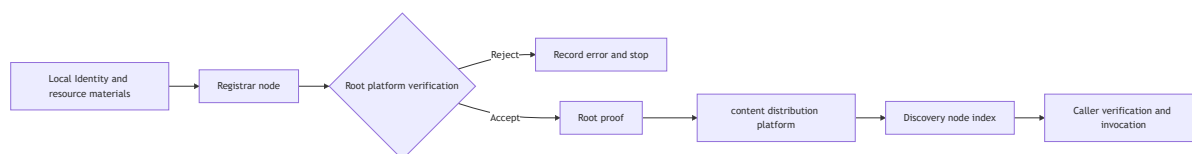
Source	Type	Link
oan-protocol-common	Code repository: resource metadata, protocol types, and external-description bindings	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	Code repository: client-side resource materials and protocol-mapping helpers	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	Code repository: community integration scripts and external-resource registration flows	https://github.com/OpenAgenet/oan-community-skill
OAN White Paper	arXiv white paper: ecosystem interoperability and open-infrastructure positioning	https://arxiv.org/abs/2606.03161

Source	Type	Link
OAN Yellow Paper	arXiv yellow paper: resources, trust, and verification mechanisms	https://arxiv.org/abs/2606.03163
Agentic Overlay Network Architecture	IETF draft: agent network architecture mapping	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
Efficient Agent Discovery Profile	IETF draft: discovery metadata and query profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Chapter 23 End-to-End Reference Flows

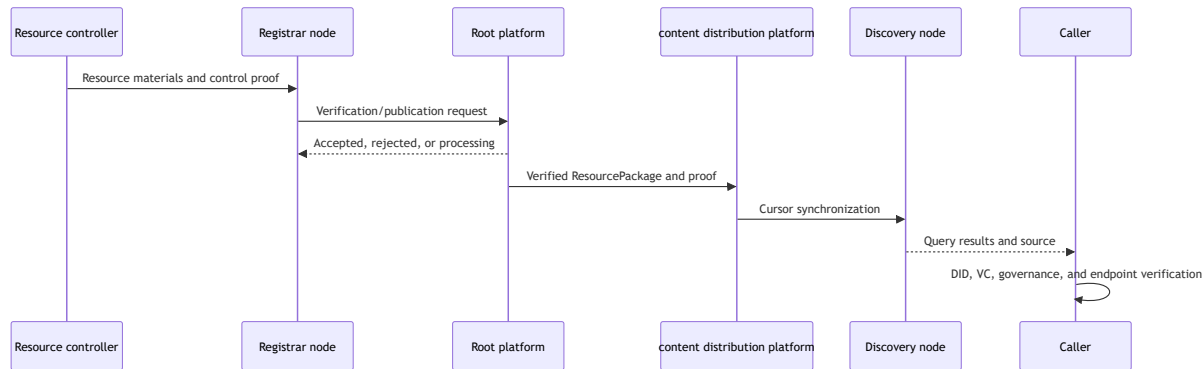
The flows in this chapter consistently use the state chain of “preparation, submission, verification, propagation, discovery, invocation, update or recovery.” Interface names, website buttons, and deployment addresses in the reference implementation belong to the example environment; the specification does not require every implementation to use the same interface, but it does require key facts to be independently verifiable. An end-to-end record must be associated with at least the resource DID, resource type, version, request identifier, source node, observation time, and evidence summary. “Completion” in a flow must be determined by the corresponding facts and evidence: the Registrar node returning an acceptance result, the Root platform forming a publication fact, the content distribution platform completing material transfer, and the Discovery node establishing an index are different stages and must not be merged into one success state.

Stage	Primary fact	Qualified output
Preparation	Local Identity, DID Document, resource metadata, and ResourcePackage	Verifiable control materials
Registration	Registrar node acceptance and upstream submission	Registration result, VC (if any), and associated ID
Publication	Root platform verification and formation of a publication fact	Root proof
Propagation	content distribution platform transfer and cursor recording	Verifiable ResourcePackage and synchronization result
Discovery	Discovery node verification and index establishment	Candidate results with source and freshness
Invocation	Caller re-verification and execution of local policy	Decision to allow or reject invocation



This chapter assumes that the Local Identity and test resources are ready, node endpoints are accessible, and the test and production environments are isolated. It provides a reference path from resource creation, registration, trusted publication, discovery, and verification to update and fault handling. The examples show reproducible interaction relationships; they do not elevate a particular website button, current deployment address, or one successful response into a mandatory interface for all implementations. During execution, test resources, test subjects,

and request identifiers should be isolated from production resources, and the associations among request materials, response summaries, DID Documents, registration credentials, Root proofs, synchronization cursors, and query results should be preserved.



Every step should record the input source, fact producer, verifier, state change, and evidence; a page prompt or a single successful HTTP response cannot represent end-to-end completion.

23.1 Create a Local Identity

The client generates the subject keys and subject DID locally, establishes a Local Identity record, and exports an identity backup file with a time identifier. When registering an agent service, skill package, MCP service, or tool API, the subject then generates or associates the resource identity and resource control proof locally. The private key remains within the publisher’s control and is not sent with an ordinary registration request; during recovery, the file structure, correspondence between the public and private keys, consistency of the subject DID, and the ability to sign again must be re-verified. Acceptance evidence includes the DID, public key, backup summary, and equivalence after recovery, rather than treating successful private-key upload as the result.

Stage	Local action	Acceptance evidence
Generate	Generate the subject DID, public key, and private key	The public key is consistent with the subject DID Document
Establish record	Save the subject and the resource identity records under its control	The record structure and version are valid, and the control relationship is clear
Back up	Export the identity package from the browser or client	The file is parseable, and the private key was not sent to a node
Recover	Import and verify the key relationship	The DID, public key, and signing capability are equivalent before and after recovery

23.2 Register an Agent Service

Take as an example an agent service that declares “code repository search and project structure summarization.” The publisher first prepares the resource DID, capability tags, capability description, use cases, endpoint, protocol declaration, authentication conditions, and version, and then signs the DID Document or ResourcePackage materials with the local control key. The client

submits the public registration materials and control proof to the Registrar node; the Registrar node validates the request and submits it to the Root platform, returning a registration result and a registration VC if one is available. The publisher must then wait for Root platform publication, content distribution, and Discovery node indexing, and obtain the corresponding states and evidence separately.

```
{
  "resourceType": "agent_service",
  "capabilityTags": ["code.repository", "analysis.summary"],
  "version": "1.0.0",
  "authorizedDomains": ["technology.software_engineering"]
}
```

Registration materials, request summaries, the registration VC (if any), Root platform status, ResourcePackage summary, and discovery results should be retained separately. “Submission succeeded” must not be written as “discovered across the network.” When submitting the same resource again, first determine whether the resource DID, version, or materials have changed; repeated requests for the same version should be judged by the actual response of the Registrar node as to whether they are idempotent, and a new version must not be inferred from a page prompt.

23.3 Register a Skill Package

A skill package example should provide its name, capabilities, inputs and outputs, use cases, dependencies, version, installation or invocation entry point, tags, subject DID, and resource DID, and calculate the content digest of the ResourcePackage. The publisher first normalizes and signs the materials locally, then submits them to the Registrar node; after submission, it checks the acceptance result, resource type, version, digest, and registration credential if any, and then checks Root platform publication and the Discovery node index in the order specified in 22.6. A skill package being installable or runnable locally does not mean that it has received Root platform publication or a Discovery node index.

Skill package acceptance checks two lines at the same time: the **product line** checks the entry point, dependencies, inputs and outputs, and use cases; the **network line** checks the DID, ResourcePackage version, digest, control signature, registration, publication, and index status. The results of the two lines are recorded separately. Failure of the product line means that the resource itself cannot be used as declared; failure of the network line means that the resource has not formed a verifiable network publication chain. The pass result of one line cannot replace the other.

23.4 Register an MCP Service

An MCP service example should prepare the server identity, tool or resource inventory, transport method, endpoint, authentication method, protocol version, and compatibility declaration. The publisher includes the inventory digest, service endpoint, and resource version in the resource materials and signs them locally. The Registrar node verifies the fields and control proof; after discovery, the caller must still establish a session according to the MCP flow and check the DID,

ResourcePackage, publication evidence, and governance status according to OpenAget (OAN) rules.

Check result	Fact represented	Fact not represented
Field validation passed	The Registrar node can understand the materials	The Root platform has published them
Registration VC returned	The registration fact has a credential	An MCP session will necessarily succeed
Root platform publication	A network publication fact has formed for a specific resource version	Business permission is automatically granted
MCP session succeeded	One connection interaction succeeded	The resource will remain available or its governance status is valid

23.5 Register a Tool API

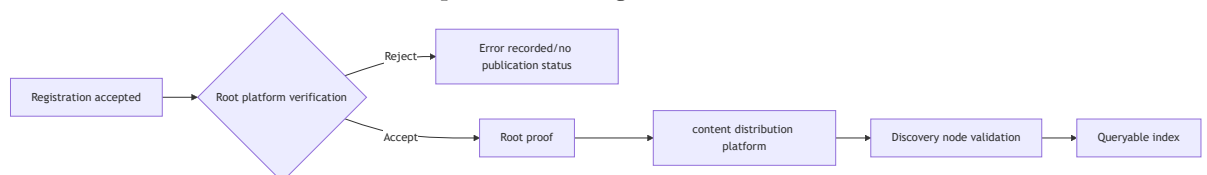
A tool API takes an OpenAPI or equivalent description as input and specifies the operations, parameters, request body, responses, errors, authentication, server endpoint, and version. The publisher first determines the source and version of the description file, then binds the normalized description digest to the ResourcePackage and signs it; the Registrar node verifies the materials, and the caller can confirm through the digest that the obtained API contract has not been replaced. Digest verification cannot prove that the interface is currently available; actual business permissions are still determined by the API server.

```
operationId: searchRepository
source: openapi.yaml#/paths/~1repositories~1search/get
version: 1.0.0
schemaHash: sha256:<value>
```

The original description, normalized digest, and OAN control signature should be saved separately; being readable as a document does not mean that the interface is callable.

23.6 Publish and Distribute Resources

After a resource is accepted by the Registrar node, the Root platform verifies the control proof, node authorization, resource type, version, content digest, and publication conditions, and forms a Root proof. The content distribution platform saves and transfers the verified materials according to the publication list or cursor. The Discovery node receives, verifies signatures, validates, and establishes structured and semantic indexes according to the cursor. Each step records its result independently: successful registration must not substitute for discoverability, and material distribution must not substitute for completed indexing.



23.6.1 Registration Result

The registration result must contain at least the resource DID, resource type, version or digest, acceptance status, registration VC if the interface returns one, and an associated identifier. When the VC is empty, it can only be truthfully recorded as empty; a credential must not be fabricated. When the Registrar node rejects acceptance, the error reason should be retained, and the client may retry according to the request identifier or an idempotent method allowed by the protocol. The client should also record which node returned the registration result and the observation time, so that later asynchronous publication status is not mixed into the registration result.

```
{
  "resourceDid": "did:oon:AGDM:<identifier>",
  "status": "submitted",
  "registrationCredential": null,
  "observedAt": "<timestamp>"
}
```

`submitted` only represents the registration-stage result; it cannot be interpreted as Root platform publication or discovery visibility.

23.6.2 Root Platform Publication

The Root platform checks the Registrar node authorization, resource control signature, ResourcePackage integrity, version relationship, and governance conditions, then records the publication sequence and forms a Root proof after acceptance. Rejection, duplication, and asynchronous processing should each have an explicit status. When the Root platform has not accepted the resource, content distribution or local caching must not be marked as trusted publication. Publication evidence must be bound at least to the resource DID, version, and each digest; repeated processing must not produce mutually conflicting current versions.

The Root platform publication record must be associated at least with the resource DID, version, DID Document hash, metadata hash, package hash, Registrar node DID, publication sequence, and processing status; repeated processing should remain idempotent.

23.6.3 Content Distribution Platform

The content distribution platform provides the DID Document, ResourcePackage, digests, and proof materials according to the Root platform publication list or cursor. The recipient validates integrity and then records the receipt cursor. Resumable transfer and repeated delivery should be idempotent; successful distribution only means that the materials can be obtained, not that the Discovery node has established a queryable index. If the recipient finds that the digest, version, or proof is inconsistent, it should reject writing to the visible index and retain diagnostic information for the failed materials.

The distribution validation order is “list, content, digest, proof, cursor.” The sender records the batch and cursor; the recipient first verifies each version and digest and then commits the local state. Successful transfer does not mean that indexing is complete.

23.6.4 Discovery Node Indexing

After receiving the materials, the Discovery node verifies the Root proof, digest, version, governance status, and authorized domain, and then writes to the structured and semantic indexes. Completed indexing should have the resource DID, version, source, indexing time, and cursor evidence. A semantic index hit cannot override invalid proof or a revoked resource. The querying party should judge whether the result is fresh based on the indexing time and synchronization cursor, and return to the verifiable source for re-checking when the latest status is required.

The index record must contain at least `resourceDid`, version, source, authorized domain, lifecycle, `indexedAt`, and the synchronization cursor. Only verified materials enter the visible index; a semantic hit only provides a retrieval entry point.

23.7 Discover by DID

The caller submits the complete resource DID to the Discovery node as an exact query condition, for example by placing the DID in the query field when requesting `/discovery/resources/query`. The Discovery node returns the resource version, digest, source, lifecycle status, indexing time, and Root proof status in its current index. Pre-invocation verification must still be performed after obtaining the result. Not found, node not synchronized, stale index, and revoked resource should be handled separately; an empty result must not be directly interpreted as proof that the resource does not exist.

```
{
  "resourceDid": "did:oon:SKDM:<identifier>",
  "limit": 1
}
```

Acceptance should distinguish an exact hit, not found, not yet synchronized, stale index, suspended/revoked, and query error, and retain the querying node, request identifier, observation time, and response summary.

23.8 Discover by Structured Conditions

The caller queries by combining conditions such as resource type, subject type, tags, protocol, version, status, and authorized domain, and specifies field encoding, intersection/union relationships, filter order, pagination, and sorting. The Discovery node first applies hard conditions such as authorized domain and lifecycle, and then performs tag, protocol, or text-relevance processing; the actual filter order for each interface is subject to its protocol description. A tag match in the result only means that the retrieval condition was satisfied; the caller must still check the DID, signature, credential, publication evidence, version, and endpoint.

Condition	Function	Example
<code>resourceType</code>	Restrict the resource type	<code>skill</code>
<code>protocol</code>	Restrict the protocol	<code>mcp</code>
<code>capabilityTags</code>	Capability matching	<code>code.repository</code>

Condition	Function	Example
authorizedDomains	Query visibility scope	<code>technology.</code> <code>software_engineering</code>
limit	Quantity boundary	Range allowed by the implementation

23.9 Discover by Natural-Language Task Description

The caller submits a task description, for example, “I need a tool that can search code repositories and summarize the project structure.” The Discovery node or its semantic component extracts capabilities, use cases, resource types, protocols, tags, and status conditions from the description, and then performs hard-condition filtering and semantic ranking. The current implementation may use a lexical, structured, vector, or hybrid path, depending on the deployment configuration. When returning results, it should retain the matched resource DID, resource version, source, and explanation fields when provided by the interface. Semantic relevance is not control or security authorization.

Task description -> capability/use-case extraction -> tags and structured conditions -> discovery query -> match rationale

The explanation result should distinguish the sources of text matching, tag matching, structured filtering, and semantic relevance.

23.10 Verify a Discovered Resource Before Invocation

The caller obtains the DID Document, ResourcePackage, Root proof, registration VC, and status information from the discovery result, verifies DID resolution, control relationship, credential, publication evidence, governance authorization, version, and endpoint in sequence, and then decides whether to invoke according to local policy. A discovery result is a candidate resource and an evidence entry point, not invocation authorization. When verification fails, status is unknown, evidence is stale, or endpoint authentication conditions are not met, high-risk invocation should be stopped and the reason recorded.

- ☐ DID and DID Document verification passed.
- ☐ ResourcePackage, version, and digest are consistent.
- ☐ The registration VC and Root proof are verifiable.
- ☐ Governance status and freshness satisfy the policy.
- ☐ Endpoint, authentication, and business permissions satisfy the invocation conditions.

The caller should not use the candidate object returned by the discovery interface directly as the final invocation object. A candidate is usually an indexed representation of `ResourceDiscoveryCandidate`; the complete DID Document and ResourcePackage must be obtained through the corresponding read interfaces. The content distribution platform’s resource package

read returns `ResourcePackage`, the DID Document read returns `DidDocument`, and the metadata read returns `ResourceMetadata`. After successful reads, the caller should verify the resource DID, version, three types of hashes, registration VC, Root proof, and governance status in the same verification context.

23.10.1 DID Resolution

Resolve the complete resource DID, confirm that the method is `did:oon`, read the DID Document, and validate consistency among `id`, verification methods, control relationship, service endpoints, and resource metadata. Resolution not found, unsupported method, invalid document structure, and abnormal status should be handled separately; arbitrary same-name JSON must not be used as a substitute.

The resolution result should provide at least the following fields for continued verification in subsequent steps:

Field	Function	Handling when missing
<code>did / id</code>	Confirm that the request identifier and document identifier are consistent	Stop
<code>verificationMethod</code>	Find the control relationship and public-key materials	Stop control-rights verification
<code>service</code>	Find the resource access or invocation entry point	Mark endpoint as incomplete
<code>resourceMetadata</code>	Compare the resource type, version, and capabilities	Mark description as incomplete
Resolution status	Distinguish success, not found, and method error	Must not default to success

23.10.2 Control-Rights Verification

Verify the resource signature according to the verification methods and control relationships in the DID Document, confirming the normalized content covered by the signature, the resource DID, version, and predecessor relationship. Mathematical consistency between the public key and signature does not mean that the publisher is authorized to control another DID; verification must also bind the target DID and the existing control relationship. Verification inputs should come from the currently obtained DID Document and `ResourcePackage`, rather than only using truncated display fields in the discovery result.

Control-rights verification should be completed in the following order:

1. Obtain the verification method and public-key reference from the DID Document.
2. Confirm that the public key belongs to the control relationship of the target DID.
3. Reconstruct the signature input according to the declared normalization rules.
4. Verify that the signature covers the DID, version, `ResourcePackage` digest, and predecessor relationship.

5. Associate the verification result with the source information returned by the Registrar node, Root platform, and Discovery node.

23.10.3 Registration VC Verification

Verify the issuer, subject DID, resource type, version or digest, signature, validity period, and revocation/suspension status of the registration VC. When the VC is missing or the interface does not return one, handle it as unknown or incomplete as defined by the protocol; the page text “submission succeeded” must not be treated as a credential. A registration VC only proves the registration fact issued by the Registrar node; it does not automatically prove that Root platform publication and Discovery node indexing are complete.

Check object	Facts to verify	Meaning of failure
Issuer	Whether it is an allowed Registrar node or an issuer specified by the protocol	The credential source is untrusted
Subject	Whether it points to the current resource DID	The credential does not match the resource
Claims	Type, version, digest, and registration fact	The content cannot support the current version
Proof	Signature, public key, and signature coverage	The credential cannot be verified
Status	Validity period, suspension, or revocation information	Can only be marked unknown or rejected

23.10.4 Root Proof Verification

Verify the issuer, target resource DID, ResourcePackage version, package digest, metadata digest, publication sequence, and signature of the Root proof, and confirm that the Root platform’s own authorization status is valid. The proof only covers the facts it declares; it does not guarantee endpoint business quality.

The Root proof verification result should be recorded separately from the registration VC. The following is an example of a verification-result record; the status values describe the verification stage and are not fixed interface responses:

```
{
  "resourceDid": "did:oa:<method-specific-id>",
  "registrationCredential": "verified|missing|invalid",
  "rootPublicationProof": "verified|missing|invalid",
  "governanceStatus": "active|suspended|revoked|unknown"
}
```

Only when the target DID, version, and digests in the proofs are mutually consistent may the caller use “registered” and “published by the Root platform” as consecutive evidence; a missing item cannot be replaced by the other. After Root proof verification passes, the caller must also use the Discovery node’s indexing time and cursor to determine whether the resource is queryable.

23.10.5 Governance Status Check

Check node authorization, authorized domain, suspension or revocation events, and their freshness from a trusted governance-status source, and record the on-chain position, index cursor, and check time. When governance data is stale or the source is unavailable, sensitive invocations should be rejected or sent for manual review rather than being allowed by default.

The governance check must output at least the following decisions rather than only a Boolean:

Decision	Meaning
Authorization eligibility	Whether the current node has the corresponding role and authorization scope
Resource status	Whether the target resource is suspended, revoked, or still valid
Data freshness	Whether the event sequence, index cursor, and most recent successful time are acceptable
Evidence location	The event, transaction, or index record corresponding to the status
Invocation decision	Allow, reject, downgrade to display-only, or send for manual review

23.11 Update a Resource While Retaining Its DID

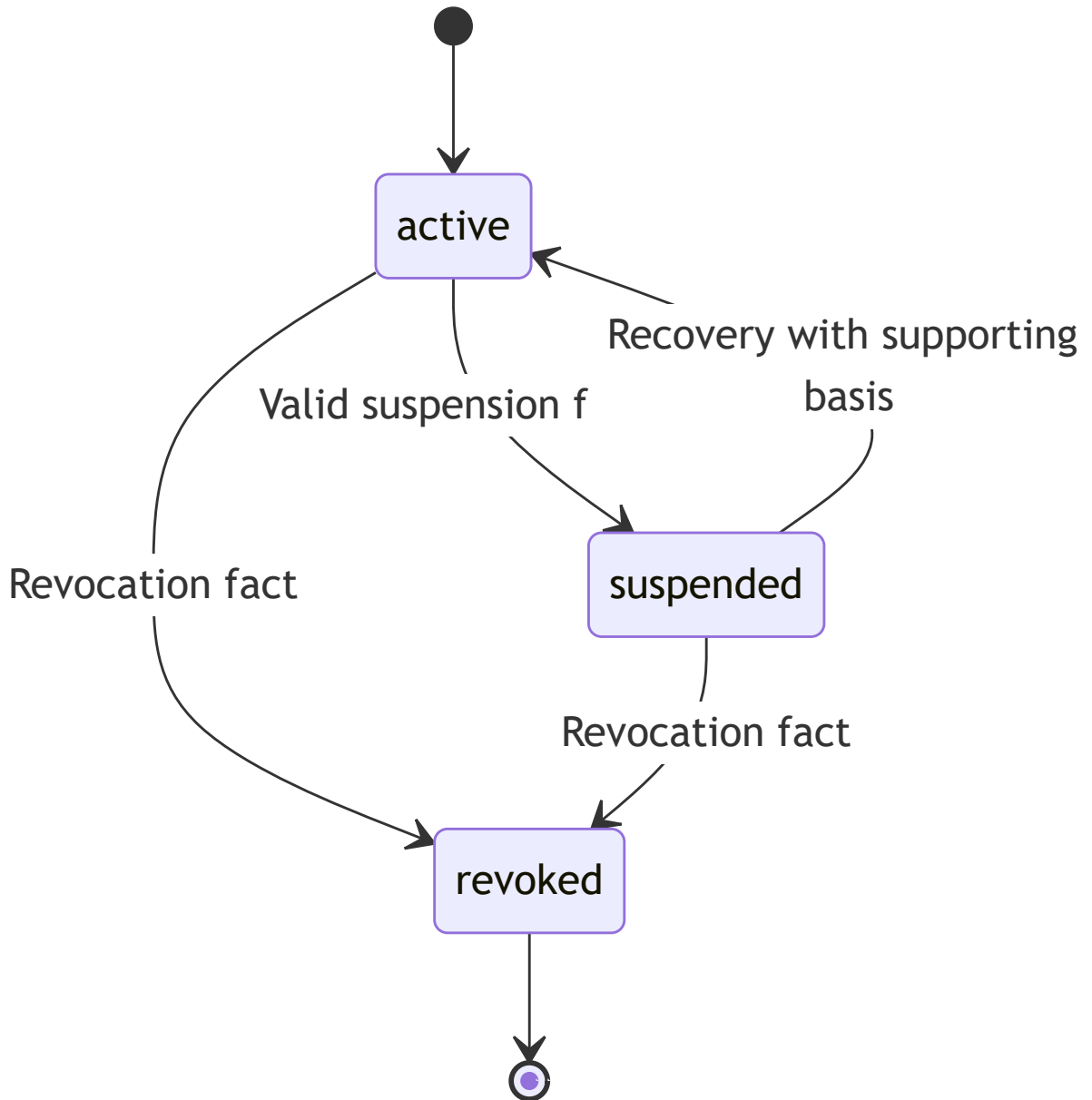
The original controller generates a new DID Document or ResourcePackage version with the same DID, carries the predecessor version and content digest, and signs it using the existing control relationship. The Registrar node and Root platform re-verify it and produce new registration or publication evidence; the distribution and discovery nodes converge according to the publication sequence. Repeated submission of identical content should be idempotent, and an unauthorized new document must not overwrite the old version. Completion of an update must be determined from the new publication proof, downstream synchronization status, and query result together, rather than only from another successful registration-interface response.

Check item	Requirement before and after update
resourceDid	Remains the same
Resource version	Increases according to the rules and never rolls back
Control proof	Covers the new normalized materials
Publication proof	Binds the new version and digests
Discovery result	Prioritizes the current version while keeping history auditable

23.12 Suspend, Revoke, and Recover a Resource

Resource suspension is triggered by the controller, governance, or a valid evidence status; revocation targets a resource or version; recovery requires rechecking the control proof, credential, publication proof, and endpoint. Each node should record the triggering fact, processing time, and propagation status according to its responsibility. The status in a discovery result is the status to

which the node has synchronized and does not necessarily equal the time when the operation just occurred. A network timeout or temporarily offline node does not mean that the resource has been revoked. When propagation to the Discovery node is incomplete, the status and freshness should be displayed and historical evidence retained.



23.13 Connect a Third-Party Registrar Node

The third party submits the node DID Document, authorization request, authorized domain, public endpoint, and runtime configuration, and completes identity, signature, interface, resource-validation, security-boundary, and upstream-submission tests. After passing, it obtains registration eligibility within the explicitly defined scope only; while the node is running, its authorization status, versions, and failed-request retries must still be checked, and one acceptance test cannot permanently replace governance status. Connection tests should use isolated resources and retain test requests, responses, error branches, authorization scope, and validity period, so that test resources are not

mistaken for formal network resources.

- ☐ The node DID Document, role, endpoint, and protocol version are consistent.
- ☐ The authorized domain is consistent with the governance authorization materials.
- ☐ Evidence exists for positive, negative, retry, and recovery cases.
- ☐ The private key and governance secrets are not included in public materials.

23.14 Connect a Third-Party Discovery Node

The third-party Discovery node obtains the node identity, authorization materials, and publication-source configuration, first completes a full synchronization to establish a baseline, and then performs incremental synchronization, digest validation, version selection, DID/structured/semantic queries, deduplication, freshness, and degradation tests. When providing results externally, it retains the source, version, indexing time, and authorized domain; local sorting must not be rewritten as a Root platform publication fact. A query interface being available does not mean that synchronized data covers the latest publication sequence.

The connection order is “establish a full baseline → incremental synchronization → inject duplicates/out-of-order/missing data → query and deduplicate → freshness and recovery verification.” Local sorting only affects display order.

23.15 Observe Governance Events Through the Network

After a governance event is produced on-chain, the off-chain trust indexer reads it in order and forms a node-authorization status read model. The Root platform and Discovery node then execute policies according to their synchronization status. An observer should record the event type, on-chain position, index cursor, projection time, and page display time, distinguishing on-chain facts from website display. Continued growth of `latest_sequence` only proves that the indexer is reading new events; whether the authorization status of a specific node has changed must also be checked against whether the corresponding event has entered the read model.

On-chain event → indexer cursor → authorization-status read model → Root platform/Discovery
node decision → page observation

23.16 Resolve and Verify a Resource DID

The client first resolves the resource DID and DID Document, and then verifies the control relationship, resource metadata, service endpoint, version, and related publication evidence. The resolution result may be encapsulated by an SDK or local verifier, but the caller should still be able to obtain the original document, digest, and error status for auditing and review. Successful resolution only proves that the document can be read and passes structural checks; it cannot bypass checks of the ResourcePackage, credential, Root proof, and governance status.

It is recommended to output distinguishable statuses such as `resolved`, `invalid`, `not-found`, and `status-unknown`, rather than compressing every failure into an empty object.

23.17 Retrieve Historical Resource Versions

The caller requests historical ResourcePackages and their DID Documents, digests, and proofs by DID, version, or publication sequence, and explicitly identifies whether each has been superseded, revoked, or is for audit only. Historical content must not automatically become the currently discoverable or callable version merely because it can be read. When a historical version lacks the corresponding publication proof or status evidence, it should be marked as incompletely verifiable rather than supplemented with the current version’s proof.

Historical records must be associated at least with the DID, version/sequence, digest, publication proof, current status, and query time. “Readable” and “currently callable” must be two separate fields.

23.18 Handle an Unavailable or Stale Discovery Node

The caller determines whether discovery results are usable based on response errors, node health, indexing time, cursor, and declared freshness. An existing old snapshot may be displayed first to improve the experience, but its time and degraded status must be marked. For actions requiring the latest authorization or version, the caller should re-verify from a trusted source and must not rely only on an old cache.

Situation	Read-only display	Authorization/invoke action
Node timeout	An expired snapshot may be displayed with an explicit marker	Stop or switch and re-verify
Stale index	Display the expiration time	Retrieve the current evidence again
Unknown source	Do not display a trusted indicator	Reject
Recovery completed	Update the snapshot	Perform complete verification again

23.18.1 Discovery Node Unavailable

When a discovery request times out, the connection fails, or an explicit service error is returned, record the node and associated identifier, and retry or switch to another authorized Discovery node according to policy. Unavailability represents infrastructure status; it does not automatically mean resource revocation, and an empty result must not be treated as proof that the resource does not exist on the network.

A retry record must contain at least the endpoint, request identifier, error type, count, backoff time, and last known indexing time. A connection failure must not be confused with a valid empty result.

23.18.2 Return an Old Snapshot

An old snapshot may be used for limited read-only display, but it must carry the generation time, source, and degraded-status marker. It may display the resource name, type, historical match results, and last indexing time, but it cannot by itself be used to determine current authorization, revocation status, endpoint availability, or invocation eligibility. If a resource in the snapshot concerns invocation, authorization, or a sensitive decision, the current DID Document, credential,

publication proof, and governance status must be obtained and verified again before the action. After online evidence is obtained again, the new result should supersede the snapshot while retaining the temporal relationship between the old and new sources.

```
{
  "source": "discovery-node",
  "snapshotAt": "<timestamp>",
  "stale": true,
  "usableFor": "read-only-display"
}
```

23.18.3 Determine Data Freshness

Determine freshness from the indexing time, event cursor, latest upstream sequence, and allowed delay. A `latest_sequence` that does not increase may mean that there are no new events, or that synchronization has stopped; the most recent successful time and error evidence must also be considered. Unknown freshness must not default to latest.

A freshness report should provide `indexedAt`, `lastSuccessfulSyncAt`, `latestSequence`, `cursor`, and error status together; it must not display only one sequence number.

23.18.4 Switch to Another Discovery Node

Before switching, confirm the candidate node's authorization status, endpoint, and supported query profiles, and retain the source node in the result. Results from different nodes need to be deduplicated and cross-validated by DID, version, digest, and publication proof; they must not simply be concatenated as equally trusted results.

Switching checklist:

- ☐ The candidate node DID, role, and authorization status are valid.
- ☐ The query profile and authorized domain meet the caller's requirements.
- ☐ Results are deduplicated by DID, version, and digest.
- ☐ Conflicting results are checked against Root platform publication evidence.

23.18.5 Re-synchronize After Recovery

After the node recovers, fill gaps from the persisted cursor or trusted publication sequence, re-verify affected versions and revocation status, and then update the index projection. Recovery is complete when the cursor catches up within the allowed range, key resource queries are consistent, and error alerts have recovered; process restart alone is not sufficient.

Recovery verification should sample at least one newly added resource, one updated resource, one revoked or suspended resource, and one resource that did not change during the outage, confirming that recovery did not only restore the count while missing lifecycle states.

The recovery completion criterion can be written as:

```
Cursor caught up + key resource consistency + lifecycle re-filtering + error alerts recovered
```

23.19 References

Source	Type	Link
<code>oan-protocol-common</code>	Code repository: shared objects and protocol types for end-to-end flows	https://github.com/wolfbrother/oan-protocol-common
<code>oan-root-services</code>	Code repository: Root platform verification, publication, and governance interfaces	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: Registrar node flows	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	Code repository: Discovery synchronization, indexing, and query flows	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-trust-indexer</code>	Code repository: governance-status synchronization and query flows	https://github.com/OpenAgenet/oan-trust-indexer
<code>oan-sdk-ts</code>	Code repository: client-side end-to-end invocation helpers	https://github.com/OpenAgenet/oan-sdk-ts
<code>oan-community-skill</code>	Code repository: community end-to-end operation examples	https://github.com/OpenAgenet/oan-community-skill
OAN White Paper	arXiv white paper: overall vision and ecosystem flows	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv yellow paper: protocol flows and verification mechanisms	https://arxiv.org/abs/2606.03163
GRAIL Semantic Discovery Paper	arXiv paper: semantic discovery flows	https://arxiv.org/abs/2605.02489
<code>did:oan</code> DID Method Specification	Standard/method specification: resource identity in end-to-end flows	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Resource Identity and Discovery	IETF draft: identity, registration, and discovery flows	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Agentic Overlay Network Architecture	IETF draft: node collaboration flows	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
Efficient Agent Discovery Profile	IETF draft: discovery query flows	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Chapter 24 Conformance Profiles and Acceptance Criteria

This chapter explains how conformance profiles turn each requirement into a "test object, input, steps, criteria, evidence, and conclusion," and do not label untested capabilities as conformant. The offline materials that can currently be reused directly are in `oan-third-party-node-admission-test-kit/fixtures/v1-alpha`, where the manifest, case matrix, expected fixtures, and requirement checks form an auditable baseline; the real Root platform, content distribution platform, on-chain governance, and production recovery should be identified separately.

Conclusion	Meaning	Follow-up action
PASS	All criteria are satisfied within the declared scope	Archive the evidence
FAIL	At least one mandatory criterion is not satisfied	Block release or admission
NOT TESTED	No reproducible evidence is available	Must not claim conformance
CONDITIONAL	Valid only under explicitly stated limitations	Record the limitations and deadline

Conformance review presupposes an isolated test environment, fixed test vectors, a clearly specified specification version, and results that can be retained as evidence. A profile's "pass" means only that the corresponding role has completed the prescribed capability and rejection-behavior tests; it does not automatically prove that the entire OpenAgenet (OAN) network or the quality of a resource business has passed. Test records should reproduce the inputs, steps, responses, logs, versions, and determinations.

The applicable scope of a profile should be stated clearly at the beginning of the report, for example, "resource provider - ResourcePackage generation," "Registrar node - registration interface," and "Discovery node - query and indexing." The same test resource may run through multiple profiles, but each role is responsible only for the facts it produces or verifies; the state of a test resource during registration, publication, distribution, and discovery should be correlated through the DID, version, digest, and request identifier, rather than by name alone.

24.1 Resource Provider Conformance Profile

A resource provider should be able to generate or import a Local Identity, create a resource DID, form a valid DID Document, resource metadata, and ResourcePackage, and sign them with the local control key. Acceptance tests cover the required fields, capability tags, use cases, endpoints, versions, and digests of four resource types; qualifying evidence consists of verifiable signature materials, structured inputs, and rejection results for invalid inputs. Private keys must not appear in upload requests or logs.

Acceptance object	Test input	Pass criterion	Evidence to retain
Local Identity	Create a new identity, import a valid backup, import a corrupted backup	A subject control relationship can be generated or restored, and the corrupted file is rejected	DID, public key, import result; do not retain the private key
Resource DID	One instance of each of the four resource types	The DID is consistent with the control relationship in the DID Document	DID Document, normalized digest, signature verification result
Resource metadata	Required fields, missing fields, and fields with incorrect types	Valid data can generate a ResourcePackage, and invalid data is explicitly rejected	Input samples, error responses, validation logs
ResourcePackage	Content, version, hash, and endpoint	The digest can be recomputed, and verification fails after tampering	ResourcePackage digest, recomputation result, version information

The acceptance order should be fixed as "identity available -> DID Document generation -> metadata validation -> ResourcePackage digest and signature -> submission-material check." If any step fails, the materials must not continue to be marked as publishable; test records should be associated with the resource DID, test vector, and specification version.

The resource provider's pass condition is not "a webpage can download a file," but the ability to reconstruct the same materials locally and verify the control relationship. Test personnel should retain at least one valid set of materials, one set missing a required field, and one set with a modified signature or digest, and check the generation result, explicit error, and no-side-effect result respectively.

24.2 Registrar Node Conformance Profile

Using an isolated test identity, submit valid materials, materials with missing fields, materials with invalid signatures, materials from an unauthorized node, duplicate materials, and expired materials, and check the Registrar node's status codes, error structures, registration records, idempotent results, VC (if any), and upstream submission. The qualifying condition is that only valid requests within the authorized scope are accepted, rejected requests do not change the state of valid resources, and the processing path can be reviewed using logs and correlation identifiers.

Registrar node acceptance should cover at least the following request sequence:

1. Submit valid materials and confirm the response structure, registration state, and correlation request identifier.
2. Resubmit the same DID, version, and digest and confirm an idempotent result that does not produce a second valid registration fact.
3. Replace the control signature, node authorization, or resource digest and confirm that the request is rejected and the original registration remains unchanged.
4. Submit expired materials or materials outside the authorized scope and confirm that the rejection reasons are distinguishable.
5. Check the failure state, retry boundary, and pending record when the upstream is temporarily unavailable.

For every request, retain the normalized request digest from the sending side and the correlation identifier returned by the Registrar node; for asynchronous upstream calls, record "accepted by the Registrar node" and "processed by the Root platform" as two independent records. If no registration VC is present in a successful response, record that the field is missing or was not returned rather than fabricating a credential in the acceptance materials.

Checkpoint	What to observe	Qualifying result
Request validation	Required fields, version, digest, and signature	Invalid requests are rejected before a registration is created
Control	Relationship between the resource DID and the control signature	A signature from another subject cannot overwrite the resource
Authorization scope	Registrar node identity and permitted resource scope	An out-of-scope submission does not change the valid state
VC result	Issuer, subject, type, and status	The VC, if returned, is verifiable and corresponds to the registration fact
Upstream submission	Root platform or subsequent publication task	Failure has a clear state and does not impersonate network-wide visibility

24.3 Root Platform Conformance Profile

Validate valid and tampered ResourcePackages, control proofs, Registrar node authorization, version sequences, and digests. The Root platform forms a Root proof for a trusted publication only when the control relationship, node authorization, content integrity, and governance conditions are all satisfied; repeated publication should be idempotent, and rejection or asynchronous failure must not produce a publication fact that a Discovery node could treat as a valid version.

Root platform acceptance uses a four-step check: "validate inputs, determine eligibility, form the publication fact, and read the result again":

- The control signature binds the resource DID, version, and content to be published.

- ☐ Registrar node authorization is valid, and the submission scope matches the authorization scope.
- ☐ The ResourcePackage, DID Document, and digest validate against one another.
- ☐ The version sequence has no rollback, unexplained gap, or unexplainable fork.
- ☐ The issuer, subject, covered content, and status of the Root proof are verifiable.
- ☐ Rejection, duplication, and asynchronous failure do not generate a valid publication record.

Acceptance evidence should retain both the initial processing result and the publication status obtained by a subsequent query, so that "request received," "validation passed," "publication fact formed," and "indexed by a Discovery node" can be distinguished as separate stages.

The key criterion for the Root platform profile is that "rejected input must not form a valid publication fact that can propagate." Therefore, in addition to checking interface responses, test personnel should reread the publication record, content distribution manifest, or publication sequence, and confirm that an incorrect digest, incorrect control signature, or out-of-scope Registrar node has not entered a subsequent queue; these checks should use independent test resources and request identifiers.

24.4 Content Distribution Service Conformance Profile

Deliver complete packages, incomplete packages, incorrect digests, duplicate batches, checkpoint cursors, and old versions to the content distribution platform, and check the manifest, response, receipt cursor, integrity, and audit records. The qualifying condition is that only materials permitted by the Root platform are delivered, duplication and checkpointing do not lose content or repeatedly change facts, and the receiver can verify the content using the digest and proof.

Test step	Constructed condition	Pass criterion
Complete delivery	Manifest, content, digest, and publication proof are all present	The platform accepts the content and returns a traceable batch
Integrity failure	Delete content or modify the digest	Reject the content and do not update the version available for discovery
Duplicate delivery	Submit the same batch and cursor again	Idempotent; do not count the publication fact twice
Checkpoint recovery	Continue from an intermediate cursor	Fill only the gap, with a consistent final digest
Old-version delivery	Below the currently valid version	Do not overwrite newer valid content

At the end of testing, recheck the receiver's content against the manifest and digest, and record the delivery batch, cursor, response time, retry count, and final state. Successful distribution means only that the content can be retrieved and verified; it does not replace the Root proof.

Checkpoint and duplicate tests should be run again after restarting the receiver or interrupting the network, to check whether the persisted cursor still points to the last confirmed batch. If the

receiver has saved the content but not the corresponding proof or cursor, treat it as an incomplete distribution state and do not directly hand it to a Discovery node for establishing a trusted index.

24.5 Discovery Node Conformance Profile

First perform full and incremental synchronization, then use published test resources for DID, structured, and semantic queries, while adding duplicate materials, old versions, revoked states, sequence gaps, and expired data. The qualifying condition is that indexing occurs only after validation, the latest valid version is selected by default for the same DID, results are not duplicated, provenance and freshness are returned, and relevance does not override control or governance checks.

The Discovery node test sequence is as follows:

1. Check the resource count, versions, and digests after full synchronization.
2. Inject an incremental publication and confirm that only the corresponding index record is added or updated.
3. Perform an exact DID query, a structured-condition query, and a semantic query separately.
4. Add duplicate materials, an old version, a revoked resource, and an expired snapshot, and check filtering and deduplication.
5. Verify the source node, version, freshness, and publication-evidence reference in the results.

Query type	Must verify	Check that it cannot replace
Exact DID query	DID, current version, and source	Control and governance status
Structured query	Type, tags, protocol, and status conditions	Actual resource availability
Semantic query	Conversion from the input description to tag/text conditions	Authorization and signature verification

Multiple records for the same DID should be selected deterministically according to version and publication status; when freshness, governance status, or publication evidence cannot be confirmed, mark the result as uncertain and hand it to the caller for review.

Query acceptance for the Discovery node profile should use the same set of published resources to cover six result categories: "exact hit, structured hit, semantic hit, duplicate record, old version, and invisible resource." The report should record the query input, filter conditions, returned resource DID, version, source, index time, and duplicate count, rather than recording only the total number of results.

24.6 Off-Chain Trust Indexer Conformance Profile

Provide sequential governance events, a checkpoint, duplicate events, missing events, and temporarily unavailable upstream data, and check the authorization-state read model, event cursor, latency, and error state. The qualifying condition is correct ordering and idempotency, recovery

from the cursor after restart, and no treatment of unread events or manual displays as on-chain facts; evidence includes event positions, cursors, and projection records.

Input scenario	Expected projection	Evidence fields
Continuous events	Authorization state updates in order	Event position, cursor, projection time
Duplicate events	State does not change repeatedly	Event unique identifier, idempotent result
Sequence gap	Pause or mark as incomplete	Gap range, error state, alert
Restart recovery	Continue from the persisted position	Cursors before and after restart, completion records
Upstream unavailable	Retain the last trusted state and mark it as stale	Most recent successful time, failure reason

The "state updated" status of the off-chain trust indexer must point back to a governance event or an explicit upstream proof; the existence of a projection record in the database alone cannot serve as a governance fact.

When test events arrive continuously, check event order, duplicate events, and sequence gaps separately; when `latest_sequence` temporarily stops increasing, use the most recent successful read time and error state to determine whether there are "no new events" or synchronization has "stalled." A test can be recorded as passed only when the cursor, event position, and authorization-state read model correspond to one another.

24.7 Parser and Client Conformance Profile

Use inputs consisting of a valid DID, an unknown DID, an unsupported method, an invalid structure, an incorrect control signature, an abnormal status, a historical version, and a missing proof, and check the result types and stop behavior. The qualifying condition is distinguishable errors, no silent acceptance of invalid materials, no presentation of a historical version as the current version, and availability of the original document, digest, and error information for review.

Input state	What the client should display or return	Follow-up action
Valid and resolved	Current DID Document and resolution status	Continue control, publication, and governance verification
Not found	Explicit not-found	Do not substitute a same-name cache
Method unsupported	Explicit method-unsupported	Stop high-risk calls
Document or signature invalid	invalid-document or verification failure	Reject and retain the original evidence
Status unknown or expired	status-unknown	Degrade the display or request re-verification
Historical version	Explicit version and time	Do not use it as the current version

Acceptance should also check that error paths do not mislabel partial resolution results as trusted resources, and do not bypass control, credential-status, or governance-status checks through

automatic retries.

24.8 SDK Conformance Profile

The SDK should cover Local Identity, resource DID, metadata, control signatures, registration, exact DID discovery, semantic discovery, VC verification, and Root proof verification. Use the test inputs for `oan-sdk-ts` to check serialization, error mapping, and cross-environment endpoint configuration; the qualifying condition is that the SDK does not upload private keys, does not lose version or evidence fields, and enables callers to distinguish registration success from discoverability.

SDK acceptance checklist:

- ☐ The results of creating and importing a Local Identity can be reviewed, and the private key does not enter network requests.
- ☐ When generating a resource DID, DID Document, and signature, the fields are consistent with the Rust protocol types.
- ☐ Registration success, registration failure, and "not yet discoverable" map to different results.
- ☐ Exact DID queries and semantic queries retain source, version, and freshness information.
- ☐ Verification results for the VC, Root proof, and governance status are not merged into a single Boolean.
- ☐ Network errors, server errors, and format errors can be distinguished by the caller.

Cross-environment tests should use fixed node endpoint configuration and test vectors; one online address or one successful request cannot serve as proof of the SDK's permanent compatibility. The test report should also record the SDK unit tests, node integration tests, and actual requests to the official website separately, so that local object-test results are not mistakenly written as end-to-end interoperability results. The acceptance conclusion should be associated with at least the SDK commit, protocol public type version, test vector, target endpoint, and response digest.

24.9 Security and Key Custody Acceptance Criteria

Test the generation, storage, backup, recovery, rotation, and permission boundaries of subject, resource, and node keys, and check the verification of signatures, VCs, and Root proofs. The qualifying condition is that private keys are used only within the controller's boundary, backups are recoverable and protected, and incorrect keys and unauthorized signatures are rejected; evidence should be redacted and must not submit the private key in plaintext.

Security acceptance task checklist:

- ☐ Establish the usage boundaries of the subject private key, resource control private key, and node runtime credentials.
- ☐ Check the generation, import, permissions, and recovery result of backup files, without writing private keys to logs or request bodies.

- ☐ Perform rejection tests using an incorrect public key, an incorrect signature, and a signature from another subject.
- ☐ Verify the issuer, subject, signature coverage, and status of the VC and Root proof.
- ☐ Check whether sensitive fields in exceptions, debug logs, and evidence packages have been redacted.

The test report should make clear that "the controller can sign" and "the service node can verify" are two different facts; the latter does not mean that the node has obtained the former's private key. Security acceptance should treat the client, Registrar node, Root platform, Discovery node, and evidence storage as separate observation boundaries, and record item by item whether private keys appear in request bodies, logs, temporary files, error stacks, or test archives. Control signatures, registration VCs, and Root proofs should each be separately checked for issuer, subject, signature coverage, and status.

24.10 Interoperability and Compatibility Acceptance Criteria

Use the same set of vectors for Rust protocol types, the TypeScript SDK, the official website API, the Registrar node, the Discovery node, and A2A/MCP mappings, and compare DID Documents, ResourcePackages, digests, signatures, errors, and version results. The qualifying condition is consistent normalization and field semantics across languages, with mappings retaining provenance and without weakening OAN control and governance verification through adaptation.

Comparison dimension	Rust implementation	TypeScript/HTTP implementation	Qualifying condition
Serialization	Public protocol types	SDK and official website request bodies	Fields, types, and defaulting rules are consistent
Digest	Digest of normalized content	Digest generated/read by the frontend or SDK	The same input produces the same digest
Signature	Control signature and proof verification	Verification result returned by the SDK/node	Signature coverage is consistent
Error	Node error structure	SDK error mapping	The failure reason is not lost
Mapping	A2A Agent Card, MCP metadata	Adaptation or display result	Preserve the original text and provenance

Cross-language comparison must use the same version, input, and normalization rules; comparing only the final page text cannot prove protocol interoperability. The interoperability conclusion should also distinguish three levels: "the same input produces the same object," "different implementations can parse the object," and "the cross-node end-to-end flow succeeds." Record the corresponding evidence separately; do not determine interoperability passed merely because the final page displays the same text.

Cross-language acceptance may use the method of "one fixture, three comparisons": first compare the normalized results of the Rust protocol types and TypeScript objects, then compare the HTTP request sent by the SDK or official website with the result actually parsed by the node, and finally compare the resource DID, version, and digest throughout the registration, publication, and discovery paths. Any difference at any level should record the differing field and applicable version, and must not be attributed to a network problem alone.

24.11 Performance and Availability Acceptance Criteria

Measure the success rate, latency, backlog, recovery time, and resource consumption of registration, publication, synchronization, query, and governance reads in a test environment with clearly defined concurrency, resource scale, network conditions, and statistical windows. Thresholds should be defined in advance in the profile or deployment acceptance record; without measurement data, the result can only be marked as not tested, and the performance of one demonstration must not be written as a general performance commitment.

Path	Fields recommended for recording	Boundary of interpretation
Registration	Request count, success rate, P50/P95 latency, error classification	Not equivalent to network-wide discovery time
Publication	Publication batches, queue time, failed retries	Not equivalent to completed distribution
Synchronization	Cursor, backlog, time per batch, recovery time	Not equivalent to query quality
Query	Query type, hit rate, P50/P95, duplicate rate	Semantic relevance must be explained separately
Governance read	Latest sequence, latency, abnormal duration	Do not interpret a stalled sequence as no events

The report should distinguish at least the cold-start, steady-state, and failure-recovery windows, and should provide the environment, data scale, concurrency, and statistical method at the same time.

Performance results should state whether they include network round trips, database writes, semantic retrieval, and waiting for asynchronous propagation; registration interface latency cannot directly represent network-wide resource visibility latency, and query hit rate cannot directly represent semantic quality. When timeouts, retries, or partial failures occur, also report the request count, failure classification, and final state after recovery. The minimum directory for an evidence package may be fixed as:

```
evidence/
  manifest.json
  spec-and-profile/
  fixtures/
  requests-and-responses/
  logs-and-metrics/
  deployment/
```

```
review-and-result/
```

`manifest.json` should list the digest, source, generation time, test environment, and corresponding clauses for each material; the directory structure specifies only how evidence is organized and does not require production configuration or sensitive data to be made public.

The evidence manifest may use the following record format that contains no secrets:

```
{
  "profile": "discovery-node",
  "profileVersion": "v1-alpha",
  "implementationCommit": "<git-commit>",
  "fixtureId": "valid-resource-v1",
  "environment": "isolated-test",
  "result": "PASS",
  "artifacts": [
    {"path": "requests-and-responses/query.json", "sha256": "<digest>"},
    {"path": "logs-and-metrics/discovery.log", "sha256": "<digest>"}
  ],
  "reviewedAt": "<timestamp>"
}
```

The angle-bracketed content consists of placeholder values in the evidence-manifest template; a formal report should replace them with the actual commit, digest, and time, but must not write private keys, database credentials, or unredacted internal endpoints.

24.12 Evidence Required for Conformance Implementation

Review materials should include at least the specification and protocol versions, code commit, configuration digest, test vectors, interface requests and responses, logs, metrics, deployment records, and failure handling. Submit sensitive content as digests, redacted excerpts, or through controlled viewing; evidence must identify the test time, environment, operator, tool version, and covered clauses.

24.12.1 Protocol and Data-Structure Evidence

Provide examples of DIDs, DID Documents, resource metadata, ResourcePackages, VCs, Root proofs, and API responses, identify the schema, normalization, version, and content digest, and explain the expected results of valid, missing, and invalid examples. Each data sample should identify at least `fixtureId`, object type, schema/profile version, normalization method, content digest, expected conclusion, and covered clause; valid samples and negative samples should be retained in pairs.

Protocol evidence should also cover HTTP methods and paths, the deserialization type of request JSON, response types or dynamic fields, error status, and key validation results. For registration, Root platform validation and publication, content distribution, and discovery queries, retain at least one valid request and one key failed request with its response for each. Shared types should be traceable to `oan-protocol-common`, and node-local types and dynamic JSON should be traceable to

the corresponding handler. Retaining only an interface address or one successful screenshot cannot prove that the input/output contract is complete.

24.12.2 Cryptographic and Identity Evidence

Provide public keys, signatures, verification methods, control relationships, credential issuers, and verification results; provide private keys only as evidence of existence or a digest in a controlled test environment. The materials should prove the object covered by the signature and the target DID, rather than merely displaying a signature string. Cryptographic evidence should allow recomputation of the normalized object covered by the signature and the source of the public key used for verification, and should record the verification results for the control signature, registration VC, and Root proof separately.

24.12.3 Node-Behavior Evidence

Provide requests and responses, events, cursors, logs, and timelines for registration, Root platform publication, distribution, discovery, governance synchronization, and error handling, associated with the resource DID, version, and request identifier. For asynchronous flows, also record stages such as "received, verified, published, and indexed"; do not retain only the last successful response.

24.12.4 Deployment and Smoke-Test Evidence

Provide the source commit, build artifacts, configuration digest, service health, database status, registration and discovery smoke-test results, and failure-handling records. Public materials should be redacted, and production keys and internal management endpoints must not be included in the evidence. Deployment evidence should at least link the source commit, build artifacts, runtime configuration, and smoke-test results; the smoke test should include one success path and one expected failure path.

24.12.5 Version and Provenance Evidence

Provide the code commit, lock files, schema/protocol version, ResourcePackage and proof digests, artifact hashes, and publication records, so that reviewers can confirm that the test object and deployment object came from the same source. Provenance checking should at least compare the code commit, dependency lock files, build-artifact digest, and runtime version identifier; when the objects do not match, they must not be considered the same test conclusion.

24.13 Test Vectors and Interoperability Fixtures

Test fixtures should fix valid DIDs, DID Documents, resource metadata, ResourcePackages, hashes, control signatures, registration VCs, Root proofs, governance events, and invalid inputs, and provide forms readable by Rust, TypeScript, or JSON. Each vector should be marked with the expected success/failure, version, digest, and covered clause, avoiding dependence on live public-network data.

The recommended fixed-fixture record form is as follows:


```
{
  "fixtureId": "valid-resource-v1",
  "input": "resource-package.json",
  "expected": "PASS",
  "schemaVersion": "<version>",
  "contentHash": "sha256:<value>",
  "covers": ["23.1", "23.3", "23.10"]
}
```

The same `fixtureId` should point to the same original input in Rust, TypeScript, and HTTP tests; if the adaptation layer must convert it, retain the digests before and after conversion and the conversion rules at the same time.

24.14 Negative Tests and Rejection Behavior

Negative tests cover invalid DIDs, missing fields, incorrect signatures, unauthorized nodes, expired or revoked credentials, old versions, duplicate requests, incorrect digests, malicious endpoints, and sequence gaps. Acceptance checks not only the returned error, but also confirms that there are no side effects such as writing an invalid index, generating an incorrect publication proof, advancing a cursor, or leaking secrets.

Negative scenario	Rejection result that must be observed	Side effect that must not occur
Invalid DID or missing structure	Distinguishable format/field error	No registration or index write
Incorrect or unauthorized signature	Control/authorization failure	Do not overwrite the valid resource
Expired or revoked credential	Invalid status	Do not generate a valid publication proof
Incorrect digest or malicious endpoint	Integrity/policy failure	Do not advance the publication or synchronization cursor
Duplicate or old version	Idempotency or version conflict	Do not create an erroneous new version

Each negative test must check the response, database or index changes, event cursor, sensitive information in logs, and subsequent query results at the same time.

24.15 Conformance Evidence Retention and Review

Archive evidence by specification version, code commit, environment, time, operator, and test fixture. When a change affects the protocol, schema, signature, governance, node behavior, or deployment path, the relevant profile must be retested. The review conclusion should state pass, fail, not tested, or conditional pass, and must not conceal a key failure behind a single overall score.

Conclusion	Review requirement	Retest condition
PASS	All mandatory criteria and rejection behaviors have evidence	Retest when the specification, code, or deployment object changes
FAIL	State the failed input, impact scope, and blocking action	Rerun the original and regression vectors after the fix
NOT TESTED	List the missing environment or evidence	Do not reuse the old conclusion before the prerequisites are completed
CONDITIONAL	State the limitation, deadline, and responsible person	Review before the limitation is removed or the deadline expires

The review conclusion should also distinguish "node profile passed," "end-to-end path passed," and "production deployment passed," so that a local test result is not overgeneralized as conformance of the entire network.

24.16 References

Source	Type	Link
oan-protocol-common	Code repository: protocol types, schemas, and error contracts	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	Code repository: Root platform conformance and publication test objects	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	Code repository: registration flow and server-side validation	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	Code repository: discovery queries, indexing, and semantic test objects	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	Code repository: governance state and synchronization test objects	https://github.com/OpenAgenet/oan-trust-indexer
oan-sdk-ts	Code repository: client conformance and cross-language invocation verification	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	Code repository: community Skill smoke tests and operational verification	https://github.com/OpenAgenet/oan-community-skill
OAN Yellow Paper	arXiv yellow paper: protocol security properties and conformance boundaries	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF draft: resource identity and discovery test directions	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Efficient Agent Discovery Profile	IETF draft: discovery profile test directions	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Chapter 25 Versioning, Compatibility, and Change Management

This chapter explains that the version object of OpenAget (OAN) is not a single number, but a set of coordinates across multiple layers:

Layer	Example	Purpose
Specification	Unified technical specification major/minor/patch versions	Defines semantics and requirements
Protocol	Node API, error, and event versions	Defines interaction contracts
Data schema	DID Document, ResourcePackage, VC, and governance event schema/profile	Defines parseable structures
Method	<code>did:oan</code> method rules	Defines identifier and resolution semantics
Implementation	Rust crate, npm package, and website build commit	Provides concrete code
Deployment	Configuration, binary, and service commit	Defines the runtime environment

Release records must associate the layers above and must not use Cargo or npm version numbers as substitutes for protocol, schema, or deployment versions. One implementation version may support multiple protocol or schema versions, and one resource DID may correspond to multiple historical DID Documents and ResourcePackage versions; therefore, releases, compatibility, migration, and rollback must all record the actual mappings. Version records must also distinguish "code committed," "artifact built," "service deployed," and "change effective," so that repository state is not mistaken for production state.

The relationships among version layers can be checked as follows:

Change object	Direct impact	Must not be inferred directly
Rust crate or npm package	A specific implementation or client capability	The protocol, DID method, or production deployment has been upgraded
Protocol/API	Requests, responses, and error handling between nodes	Historical ResourcePackages have already been migrated
Resource schema/profile	Parsing of DID Documents, ResourcePackages, VCs, or events	Control rights or the resource DID have changed

Change object	Direct impact	Must not be inferred directly
<code>did:oon</code> method	Identifier resolution and control semantics	Ordinary resource versions must use a new DID
Deployment artifacts and configuration	Runtime behavior in a particular environment	All nodes or third-party implementations have been upgraded

25.1 Specification Version Management

The unified technical specification uses major, minor, and patch versions to record structural, semantic, and errata changes. A major version change means that existing implementations may no longer interoperate directly; a minor version is used for backward-compatible extensions; and a patch version is used for clarifications or corrections that do not change specification semantics. Each release retains the affected sections, release date, status, and implementation mapping.

Level	Applicable change	Must retain
Major	Breaks existing interoperability or semantics	Impact scope, migration, and approval
Minor	Backward-compatible extension	New fields, compatibility tests, and implementation mapping
Patch	Clarification or erratum	Clause, original intent, and release date

25.2 Protocol Version Management

Node protocols and public APIs should explicitly define version identifiers, supported ranges, request/response schemas, error semantics, and negotiation methods. During the transition period, old clients may use only profiles that the server still declares as supported. The server must not silently change signatures, versions, publication sequences, or governance decisions merely to accommodate legacy requests; when compatibility is not possible, it must return an identifiable incompatibility error.

```

client-supported profiles intersection server-supported profiles
empty intersection -> explicit incompatible response

```

Protocol compatibility therefore requires more than checking whether a request can be deserialized. It must also verify whether the same input still produces consistent results for signature verification, publication sequencing, governance decisions, error codes, and retry behavior. The server must not silently change these semantics to accommodate legacy requests. If an old version can still be read but cannot be safely written, read, write, and query capabilities must be declared separately.

Capability	Compatibility issue	Recording method
Read	Whether an old client can parse the response and unknown fields	Supported response profiles and examples
Write	Whether the request will generate a different signature, version, or publication event	Supported write versions and rejection conditions
Query	Whether pagination cursors, sorting, and semantic filters retain their meaning	Query version, errors, and boundary examples
Retry	Whether a retry causes duplicate registration or publication	Idempotency keys, sequence numbers, and duplicate-request tests

25.3 Resource Schema Version Management

Resource metadata, ResourcePackages, VCs, governance events, and discovery responses record schema or profile versions separately. They are not one shared version: `packageVersion` in a ResourcePackage describes the resource content version; the schema version of a DID Document describes the document structure; the schema version of a VC describes the credential claims and proof structure; and the event version of a governance event describes its payload and event semantics. A reader should identify each version first, then select the parser and validation rules. It must not use the ResourcePackage version as a substitute for the credential or event schema version.

The handling boundaries for field changes are as follows:

Change	Default handling	Additional requirements
Add an optional field	Compatibility candidate	Old readers should ignore or retain it; new readers provide fallback behavior
Add a required field	Potentially breaking	Provide default semantics, upgrade the profile, or migrate old data
Rename a field, change its type, or change its unit	Breaking	New version, conversion rules, and negative examples
Change the digest algorithm or signature coverage	Breaking	New validation rules, test vectors, and republishing
Add an unknown status or governance event	Requires explicit handling	Old implementations must not treat an unknown status as valid or completed

```
{
  "schemaVersion": "<resource-package-schema>",
  "packageVersion": "<resource-version>",
  "resourceDid": "did:oa:<method-specific-id>",
  "metadataHash": "<hash>",
  "verificationProfile": "<verification-profile>"
}
```

The example above illustrates field relationships only; the angle-bracketed values are not fixed OAN values. For historical materials, readers should retain the original version, digest, and verification result. Required fields, digest algorithms, signature structures, or status values that cannot be understood must be rejected or placed into an explicitly isolated state; they must not be silently treated as currently valid data.

Readers should identify the schema/profile first, then validate required fields, the digest algorithm, the signature structure, and the status value. When deleting or renaming a field, changing its type, or changing signature coverage, a new profile, migration notes, and negative examples should be produced.

25.4 DID Method Compatibility

Changes to the `did:oa` method must assess the DID Core data structures, method-specific identifiers, resolver, DID Documents, historical resources, and Root proofs together. A normal resource version upgrade usually keeps the resource DID unchanged and expresses the version change through a new DID Document, ResourcePackage, content digest, control proof, or Root proof. This is a different layer of change from the resolver implementation version or node software version. If method resolution rules, identifier normalization, or control semantics change incompatibly, a strategy must be defined for continued resolution, migration, or deactivation of old DIDs; updating only the implementation version is insufficient.

Object	Must check
Identifier	Syntax, case, semantic code, and normalization
DID Document	id, verification methods, services, and extension fields
Control semantics	Public keys, signature coverage, and update authorization
Historical resources	Old DIDs, versions, VCs, and Root proofs
Resolver	Success, error, unknown method, and migration behavior

An unchanged resource DID does not mean that the document content is immutable. The updater must still prove control over the DID and ensure that the new document, ResourcePackage version, VC, and Root proof can validate one another. A Discovery node should select the currently available version according to its supported versions and freshness rules, while retaining traceable relationships to historical versions. Conversely, simply changing a server, SDK, or resolver does not automatically create a new resource version.

25.5 Backward-Compatible and Breaking Changes

Compatibility is determined by verifiable results for old clients, old nodes, historical data, and signature verification behavior, not merely by whether fields can still be deserialized. The assessment should cover at least registration submission, Root platform publication, content distribution platform processing, discovery indexing, governance validation, SDK/Skill invocation, and failure

recovery. If any stage interprets the same input as a different resource, version, status, or trust result, it must not be deemed compatible merely because it is parseable.

Change type	Default determination	Required action
Add an optional field	Compatibility candidate	Regression tests with old clients
Clarify an error	Requires assessment	Check whether rejection behavior changes
Remove a required field	Breaking	Increase the version and migrate
Change signature or governance semantics	Breaking	Governance approval, republishing, and retesting

25.5.1 Compatible Changes

Compatible changes preserve existing required fields, signature semantics, status meanings, and interface success/failure rules. New content should be safely ignorable by old implementations or identifiable through negotiation. Even if the code compiles, results must be verified with old clients, old nodes, and historical data.

- ☐ Existing required fields and error semantics remain valid.
- ☐ Existing signature, status, and version-selection results remain unchanged.
- ☐ Old clients can read or safely ignore the new content.
- ☐ Historical resources, DID Documents, and credentials remain verifiable.

25.5.2 Breaking Changes

Breaking changes include removing or renaming required fields, changing DID or signature semantics, changing resource status, protocol paths, version selection, or governance authorization decisions. Such changes must increase the version, publish the impact, schedule a migration window, and prevent nodes that have not completed migration from mistakenly mixing new and old materials. The following checks should be completed before releasing a breaking change:

- ☐ List affected nodes, clients, SDKs/Skills, DID Documents, ResourcePackages, VCs, Root proofs, and indexes.
- ☐ Specify the read, write, and discovery behavior of the old version during the transition period.
- ☐ Provide before-and-after migration examples, negative examples, rollback data, and retest records.
- ☐ Prevent unmigrated components from interpreting new and old materials as the same valid version.

Before release, freeze the impact scope and identify affected nodes, clients, DID Documents, ResourcePackages, VCs, Root proofs, governance status, and databases. Unmigrated objects must not mix new and old materials. For historical objects that cannot be migrated in one step, explicitly state whether they remain readable, verifiable only, prohibited from updates, or placed in an isolation

area, and ensure that discovery results do not display expired or unverified objects as the currently valid version.

25.5.3 Version Identifiers

A version identifier should state its layer, format, source, and comparison rules. For example, specification, protocol, ResourcePackage, schema, Rust crate, npm package, and deployment commit versions cannot substitute for one another. Record the ResourcePackage version separately from resource DID stability, and store both the version and content digest in the evidence.

```
{
  "specVersion": "<spec>",
  "protocolVersion": "<protocol>",
  "schemaVersion": "<schema>",
  "resourceVersion": "<resource>",
  "implementationCommit": "<commit>",
  "deploymentId": "<deployment>"
}
```

25.5.4 Migration and Transition Period

The transition period should specify the parallel support range for old and new versions, start and end times, the dual-read/single-write data strategy, node upgrade order, failure rollback, and the conditions for stopping the old version. During the transition period, different versions must not create unexplainable forks of valid versions for the same DID. At the end, retest the registration, Root platform publication, content distribution platform, discovery query, and governance paths. If the old version is read-only, the server must state this clearly in its protocol capability declaration and error responses; callers must not have to infer migration status through trial and error. The migration plan should record at least:

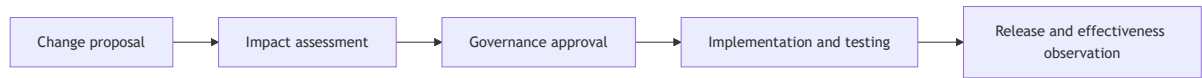
Phase	Permitted operations	Must observe
Dual read	Read old and new formats	Parsing and digest consistency
Single-write switch	Generate only the new format	Whether old-client errors are identifiable
Convergence	Complete historical data and indexes	Consistency of DIDs, versions, and Root proofs
Completion	Close the old path	Rollback entry point and residual data

It is recommended to extend read capabilities first, then switch writers, and finally close the old read path. Throughout the process, retain evidence of database migration, rollback, and discovery/publication retesting.

25.6 Specification Changes for Governance Controls

Changes involving node authorization, governance events, resource status, Root proofs, trust decisions, or node protocols must go through a formal proposal, impact assessment, governance

approval, release, and effectiveness record. The proposal, approval, code implementation, build artifact, deployment status, and production effectiveness are different types of evidence and must be recorded separately. Signers, verifiers, and affected nodes must be explicit; code being merged or a website being updated cannot substitute for governance effectiveness.



25.6.1 Change Proposal

The proposal should describe the motivation, scope, affected sections and objects, compatibility level, migration plan, test evidence, risks, rollback method, and intended effective time. Proposals involving signatures, authorization, or DID resolution should also list the impact on historical data and third-party nodes.

Item	Content to provide
Motivation and scope	Reasons, roles, and objects
Compatibility	Major/minor/patch level and basis
Migration	Data, nodes, clients, and evidence
Verification	Test vectors, integration tests, and rollback drills
Effectiveness	Approving authority, conditions, and time

25.6.2 Governance Approval

Approval should produce a verifiable governance record describing the decision-making authority, approved content, version, scope, conditions, and basis for effectiveness. An ordinary implementation commit, an operational verbal confirmation, or a local configuration on a single node cannot replace a fact that requires governance authorization. The governance approval record should answer at least "who approved it, what was approved, when it takes effect, who verifies it, and which objects are affected":

```

approver: <governance-authority>
approvedScope: [protocol, schema, node-policy]
effectiveCondition: <event-or-time>
verificationEvidence: <record>
  
```

25.6.3 Release and Effectiveness

The release record associates the specification version, protocol/schema version, code commit, artifact hash, and configuration. Effectiveness should be executed according to the declared time, governance event, or node upgrade condition. Backups, compatibility checks, and conformance retesting must be completed before effectiveness; updating a page does not mean that the protocol has taken effect.

After effectiveness, observe cross-node status, client errors, publication cursors, indexing latency, version distribution, and rollback feasibility to confirm that implementation status matches the specification declaration. If deployment is complete but a Discovery node is still using an old

protocol, old schema, or old governance configuration, record it as “deployed but not fully effective” rather than marking it complete directly.

25.6.4 Rollback and Emergency Changes

Emergency changes are limited to fixing security, data integrity, or major availability risks, and must record the reason, authorization, impact scope, and temporary validity period. Rollback must consider the database, publication sequence, governance status, and historical evidence; it is not enough to replace the service binary with an older version. Complete the formal review record afterward. Rollback checks for an emergency change should be performed by object:

Object	Rollback requirement
Binary and configuration	Restore verifiable build artifacts and configuration digests
Database	Retain the pre-migration backup and recovery validation
Synchronization cursor	Prevent duplicate processing or skipped events
Publication status	Do not leave a failed publication recorded as a valid fact
Cache and index	Define the cleanup and resynchronization scope

Rollback checks should also cover synchronization cursors, caches, and unfinished publication tasks. After an emergency change ends, complete the formal approval and retesting.

25.7 Deprecation and Migration Strategy

Deprecated objects may include interface paths, fields, digest algorithms, protocol versions, node binaries, SDK/Skill packages, or configuration items. A notice should state the object, replacement, support period, affected versions, migration tools, verification method, and service-stop conditions. It should separately specify whether the old object is readable, writable, discoverable, or callable during the transition period. Old interfaces or fields may return explicit deprecation information during the transition period, but must not silently change results. After nodes and clients complete migration, stop the old version according to the recorded conditions and retain audit evidence.

The notice should provide at least before-and-after migration examples, verification commands, the support end time, and a failure rollback entry point.

25.8 Change Records and Release Notes

Each change must record the affected sections, data structures, interfaces, code repositories, database migrations, tests, deployment, rollback, operators, and known impacts. It must also identify the responsible person and deadline when specification clauses and implementation status are inconsistent. Release notes should tell maintainers “what changed, why it changed, who needs to act, and how to verify it,” and should allow them to trace the change identifier to approval, commit, artifact, deployment, and effectiveness checks.

```

changeId: CHANGE-<id>
scope: [spec, protocol, schema, implementation, deployment]
affectedChapters: []
repositories: []
compatibility: compatible|breaking|deprecated
tests: []
deployment: <commit-or-artifact>
rollback: <procedure>
owners: []
knownImpact: <summary>

```

The YAML above is an example of a release record, not a fixed OAN configuration. Status checks can proceed in the following order: after code is committed, record the commit identifier; after the build is complete, record the artifact digest; after deployment is complete, record the target environment and service status; after the smoke test passes, record the test result; and only after the protocol, data, and governance effectiveness conditions are satisfied, record **effective**. If any stage fails, retain the failure reason and rollback action instead of overwriting the status as successful.

25.8.1 Specification Change Records

Record the sections, clauses, version, change type, original text and new intent, compatibility conclusion, approval record, and effective time. Mark errata, clarifications, and specification semantic changes separately so that substantive changes are not presented as textual revisions. Specification change records can be checked using the following fields:

Field	Content
affectedChapters	Affected sections and clauses
changeType	Erratum, clarification, compatible extension, or semantic change
approval	Approval record
effectiveAt	Effectiveness condition or time
implementationMap	Corresponding code, schema, and deployment versions

25.8.2 Protocol Change Records

Record changes to routes, requests/responses, status codes, errors, signatures, cursors, version negotiation, and node interoperability, and associate them with protocol tests and the support period for old versions. At minimum, state whether old requests can still register, publish, query, and retry, and what stable error semantics are returned when they are incompatible. Protocol change records should list routes, requests/responses, status codes, errors, signatures, cursors, and negotiation behavior separately, and should include at least one successful old-version example, one successful new-version example, and one incompatible example.

25.8.3 Implementation Change Records

Record the affected repositories, commits, dependencies, Cargo/package versions, database migrations, build artifacts, and deployment environments. If an implementation fix has not yet been synchronized with the specification, clearly state its temporary status and who is responsible for the subsequent specification update. Rust crates, npm packages, website builds, and node binaries must be recorded separately; none of their version numbers may be treated as the version number of the entire OAN implementation. Implementation change records should distinguish the four statuses "code merged," "build completed," "deployed," and "smoke test passed," so that the existence of a commit is not interpreted as production effectiveness.

25.8.4 Migration Notices and Known Impacts

List the nodes, clients, SDKs/Skills, configurations, and databases that need upgrades. Explain the impact on old resources, historical DIDs, VCs, Root proofs, indexes, and caches, as well as known limitations after migration failure and rollback. Migration notices should list actions and impacts by role and distinguish "must upgrade" from "can continue reading":

Role	What to watch
Resource provider	ResourcePackage, DID Document, and signature-generation methods
Node operator	Configuration, database, cursor, and service upgrade order
Client and SDK	Version negotiation, error handling, and cache invalidation
Reviewer	Old resources, historical evidence, and retesting scope

25.9 References

Source	Type	Link
<code>oan-protocol-common</code>	Code repository: protocol, schema, error codes, and version-related structures	https://github.com/wolfbrother/oan-protocol-common
<code>oan-root-services</code>	Code repository: Root platform publication and compatibility changes	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: registration protocol and version migration	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	Code repository: indexing, querying, and synchronization compatibility	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-trust-indexer</code>	Code repository: governance events and projection version migration	https://github.com/OpenAgenet/oan-trust-indexer
<code>oan-sdk-ts</code>	Code repository: client versions and compatibility	https://github.com/OpenAgenet/oan-sdk-ts

Source	Type	Link
oan-community-skill	Code repository: scripts, examples, and release compatibility	https://github.com/OpenAgenet/oan-community-skill
OAN White Paper	arXiv white paper: ecosystem evolution and the positioning of open infrastructure	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv yellow paper: protocol versions, trust, and change boundaries	https://arxiv.org/abs/2606.03163
did:oan DID Method Specification	Standard/method specification: DID method compatibility	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Resource Identity and Discovery	IETF draft: resource identity and discovery version direction	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Agentic Overlay Network Architecture	IETF draft: network architecture evolution direction	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
Efficient Agent Discovery Profile	IETF draft: discovery profile evolution direction	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Appendix A Normative Data Schemas

The schemas in this appendix are the common basis for protocol implementations, test vectors, and interface validation. Field names are governed by the actual JSON serialization result. Rust types, TypeScript types, and HTTP examples must not independently invent synonymous fields. If a field exists only in a reference implementation or a specific resource type, it should be explicitly marked as optional or as a profile extension. Schema versions, content digests, signature coverage, and unknown-field handling rules must remain consistent with Chapters 5 through 8, Chapter 16, and Chapter 25 of the main text.

A.1 DID Document Schema

The public Rust type for a `did:oa`n DID Document contains `@context`, `id`, `verificationMethod`, `authentication`, `assertionMethod`, `service`, and the optional `oanMetadata`. The `id` must exactly match the DID being resolved. The `id` and `controller` of verification methods, their public-key material, and their reference relationships must support signature verification. Resource types, capability tags, use cases, protocol bindings, implementation links, and resource package information in `oanMetadata` are OpenAget (OAN) extensions; the specific fields are governed by the serialization definition in `oan-core`. Public keys, digests, and DIDs in the example are placeholder values. Registration and index validation should fail when `id` is missing, a verification-method reference cannot be resolved, a service endpoint is not a valid URI, or the resource type does not match the DID semantic code. The current public type does not include `controller` as a top-level DID Document field. If a DID profile uses this field, its compatibility scope must be documented separately; it must not be confused with `verificationMethod.controller`.

```
{
  "@context": ["https://www.w3.org/ns/did/v1", "https://w3id.org/oa/v1"],
  "id": "did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "verificationMethod": [{
    "id": "did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
    "type": "Ed25519VerificationKey2020",
    "controller": "did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "publicKeyMultibase": "zExamplePublicKey"
  }],
  "authentication": ["did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"],
  "assertionMethod": ["did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"],
  "service": [],
  "oanMetadata": {
    "subjectType": "skill",
```

```

    "resourceType": "skill",
    "resourceDescription": {
      "name": "Example repository structure skill",
      "description": "Searches a repository and summarizes its project structure.",
      "capabilityTags": ["code-repository", "project-structure"],
      "useCaseExamples": ["Summarize the layout of a source repository."]
    },
    "capabilityTags": ["code-repository", "project-structure"],
    "authorizedDomains": ["software-development"]
  }
}

```

Validation item	Valid condition	Failure handling
id	Exactly matches the requested target DID and conforms to did:oan syntax	Reject registration or indexing
Verification method	Public-key encoding, controller , and reference relationships support signature verification	Return a key or signature-structure error
oanMetadata	subjectType and resourceType use the public resource-type enumeration	Return a resource-type mismatch
Unknown extensions	Do not change the semantics of existing fields	Ignore or reject according to the schema version

A.2 Resource Metadata Schema

The four resource types share information such as the resource DID, resource type, name, description, version, capability tags, use cases, endpoints, protocol bindings, and implementation links. However, the current public types carry this information at different levels: **ResourceMetadata** uses **resourceDid**, **resourceType**, **subjectType**, **name**, **description**, **capabilityTags**, **authorizedDomains**, **protocolBindings**, **services**, **lifecycleState**, **packageVersion**, **hashes**, and **updatedAt**; the DID Document expresses the corresponding information through **oanMetadata.resourceDescription**, **capabilityTags**, **protocolBindings**, and **packageInfo**. The enumeration values for the four resource types are **agent_service**, **skill**, **mcp_server**, and **tool_api**; the display name **agent** must not be used as a protocol value. Length, duplicate-item, and format restrictions for strings, arrays, and URIs should be supplemented by the specific profile or node validation rules. Unknown resource types, an empty DID, missing digests, or unresolvable endpoints must not be treated as complete metadata. New optional fields may be added compatibly; breaking changes should be handled through version management.

Field	Type	Required	Constraint
resourceDid	string	Yes	Must be the DID of the resource itself

Field	Type	Required	Constraint
resourceType	enum	Yes	Public enumerations such as agent_service , skill , mcp_server , and tool_api
subjectType	enum	Yes	Consistent with the resource subject type
name	string	Yes	A direct field of ResourceMetadata ; multilingual expressions are carried by the profile
description	string	No	Does not replace structured capability fields
capabilityTags	array[string]	No	Used for semantic governance and discovery; elements should be comparable
authorizedDomains	array[string]	No	Used for authorization-domain decisions; not equivalent to capability tags
services	array[ServiceEndpoint]	No	Each item contains id , type , and serviceEndpoint
packageVersion	string	Yes	Must remain consistent with the resource package version
packageHash /metadataHash	string	Yes	Digest references represented according to hashAlgorithm

```
{
  "resourceDid": "did:oa:SKDM:ExampleResourceDid",
  "resourceType": "skill",
  "subjectType": "skill",
  "name": "Repository analysis",
  "description": "Searches a repository and summarizes its project structure.",
  "capabilityTags": ["software-development", "code-analysis"],
  "authorizedDomains": ["software-development"],
  "packageVersion": "1.2.0",
  "packageHash": "sha256:example-package-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "hashAlgorithm": "sha256",
  "lifecycleState": "active",
  "updatedAt": "2026-09-07T01:02:08Z"
}
```


A.3 ResourcePackage Schema

A ResourcePackage is a verifiable, distributable versioned object defined by the current public oan-package types. Its fields include packageVersion, resourceDid, resourceType, the complete didDocument, didDocumentHash, metadataHash, packageHash, hashAlgorithm, metadata, rootProof, and createdAt. A resource package has no generic top-level manifest, contentDigest, or resourceMetadata fields. If a specific resource requires a file manifest, it should be placed in an agreed extension field or external resource material, with the profile made explicit. Package digests currently cover the DID Document, resource metadata, and package declaration separately. The ResourcePackage version cannot replace the DID Document or resource version. Validation must be rejected when the resource DID, version, digest, metadata, or Root proof from the Root platform is missing; when digests do not match; or when resource types do not match.

```
{
  "packageVersion": "1.2.0",
  "resourceDid": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "resourceType": "skill",
  "didDocument": {
    "@context": ["https://www.w3.org/ns/did/v1", "https://w3id.org/oan/v1"],
    "id": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "verificationMethod": [{
      "id": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
      "type": "Ed25519VerificationKey2020",
      "controller": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
      "publicKeyMultibase": "zExamplePublicKey"
    }],
    "authentication": ["did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"],
    "assertionMethod": ["did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"],
    "service": []
  },
  "didDocumentHash": "sha256:example-did-document-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "packageHash": "sha256:example-package-hash",
  "hashAlgorithm": "sha256",
  "metadata": {
    "resourceDid": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "resourceType": "skill",
    "subjectType": "skill",
    "name": "Repository analysis",
    "description": "Searches a repository and summarizes its project structure.",
    "capabilityTags": ["software-development", "code-analysis"],
    "authorizedDomains": ["software-development"],
    "lifecycleState": "active",
    "packageVersion": "1.2.0",
    "packageHash": "sha256:example-package-hash",
    "metadataHash": "sha256:example-metadata-hash",
    "hashAlgorithm": "sha256",
    "updatedAt": "2026-09-07T01:02:08Z"
  },
}
```

```
"rootProof": {
  "rootDid": "did: oan: INRT: 7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "packageClaims": {
    "resourceDid": "did: oan: SKLG: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "resourceType": "skill",
    "version": "1.2.0",
    "didDocumentHash": "sha256: example- did- document- hash",
    "metadataHash": "sha256: example- metadata- hash",
    "packageHash": "sha256: example- package- hash",
    "hashAlgorithm": "sha256",
    "lifecycleState": "active",
    "authorizedDomains": ["software- development"]
  }
},
"createdAt": "2026-09-07T01:02:08Z"
}
```

Publication validation is performed in the following order:

- 1. Parse the package version, resource type, and required structure.
- 2. Validate consistency among the resource DID, DID Document, metadata, and package declaration.
- 3. Recalculate content digests according to the normalization rules.
- 4. Verify the Root proof before entering the distribution or discovery process.

A.4 Root proof Schema

The Root platform trusted publication proof contains at least `rootDid` in the current public `RootProof` type and may contain `bulletinEventHash`, `signature`, `packageClaims`, `proof`, `cryptoSuite`, and `hashAlgorithm`. The resource DID, resource type, version, DID Document digest, metadata digest, package digest, lifecycle state, and authorized domains are primarily located in `packageClaims`, rather than being required as flat fields at the top level of the proof object. The issuer must be a Root platform identity currently authorized to perform Root platform publication. Verifiers should check the Root platform signature, claim scope, hash bindings, version relationships, bulletin event reference, and governance status, rather than checking only that JSON fields exist. When the proof is invalid, digests do not match, the issuer is unauthorized, or the target version has been revoked, the Discovery node must not treat the corresponding package as a currently valid resource.

Field	Fact proven	Verification focus
<code>rootDid</code>	Which Root platform identity formed the proof	Root platform DID Document and authorization status
<code>resourceDid</code>	The corresponding resource	Consistency with the package and DID Document
<code>packageClaims.packageHash</code>	The specific published content	Equal to the resource package digest

Field	Fact proven	Verification focus
<code>packageClaims.version</code>	The resource package version	Consistent with <code>packageVersion</code>
<code>bulletinEventHash</code>	The corresponding Root platform bulletin event	Bulletin chain and event order are verifiable
<code>proof/signature</code>	Claim integrity	Suite, signature input, and issuer public key

A.5 Registration VC Schema

The public response for a registration VC uses the W3C-style `type`, `issuer`, `issuanceDate`, `credentialSubject`, `credentialStatus`, and `proof` to express the registration fact issued by the Registrar node. The public Rust credential model also provides internal structures in the forms of `subject`, `status`, `issuedAt`, `expiresAt`, and `claims`; the two forms must not be mixed within the same profile. The registration VC currently returned by the official Registrar node uses claims such as `credentialSubject.resourceDid`, `resourceType`, `didDocumentHash`, `metadataHash`, `packageHash`, `packageVersion`, `hashAlgorithm`, `authorizedDomains`, and `lifecycleState`. It proves that the Registrar node accepted the registration request and issued a registration credential; it does not automatically prove that the Root platform has published the resource or that the resource endpoint is available. Verification fails when the issuer, target DID, claim bindings, or proof is missing. When a VC is empty, expired, revoked, or has an unknown status, callers should distinguish among incomplete, invalid, and temporarily indeterminate states.

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1", "https://openagenet.org/credentials/v1"],
  "type": ["VerifiableCredential", "OANResourceRegistrationCredential"],
  "issuer": "did:oa:INRG:RegistrarExample",
  "issuanceDate": "2026-09-07T00:00:00Z",
  "credentialSubject": {
    "id": "did:oa:SKDM:ExampleResourceDid",
    "resourceDid": "did:oa:SKDM:ExampleResourceDid",
    "resourceType": "skill",
    "didDocumentHash": "sha256:example-did-document-hash",
    "metadataHash": "sha256:example-metadata-hash",
    "packageHash": "sha256:example-package-hash",
    "packageVersion": "1.2.0",
    "hashAlgorithm": "sha256",
    "authorizedDomains": ["software-development"],
    "lifecycleState": "active"
  },
  "credentialStatus": {
    "type": "OANResourceRegistrationStatus",
    "status": "active"
  },
  "proof": { "type": "DataIntegrityProof", "proofValue": "..." }
}
```

A registration VC proves that the Registrar node accepted the registration and issued a registration credential. A resource can be used as a network-wide discoverable resource only when both the Root platform publication proof and the Discovery node index status meet the requirements.

A.6 Node Authorization VC Schema

In the public credential model, a node authorization VC expresses node eligibility through `credential_type`, `issuer`, `subject`, `role`, `status`, `issuedAt`, optional `expiresAt`, `claims`, and `proof`. After JSON serialization, the field names are respectively `type`, `issuer`, `subject`, `role`, `status`, `issuedAt`, `expiresAt`, `claims`, and `proof`. Authorized domains and capability scope are normally located in `claims`, rather than being fixed flat fields. Role values should be consistent with the node-role enumeration and deployment profile. Authorized domains and capability scope should use comparable structured values. Verifiers must check the credential signature, governance status, and current validity period together; the existence of a historical VC must not cause suspension or revocation events to be ignored. The `credentialSubject`, `validFrom`, and other fields below are provided only as display-layer compatibility examples; when using the public credential model, the fields above should be used.

```
{
  "type": "NodeAuthorizationCredential",
  "issuer": "did:oa:INRT:RootPlatformExample",
  "subject": "did:oa:INDS:DiscoveryNodeExample",
  "role": "discovery",
  "status": "active",
  "issuedAt": "2026-09-01T00:00:00Z",
  "expiresAt": "2027-09-01T00:00:00Z",
  "claims": {
    "authorizedDomains": ["software-development"],
    "capabilities": ["query", "index", "serve_public_results"]
  },
  "proof": { "type": "DataIntegrityProof", "proofValue": "..." }
}
```

A.7 Governance Event Schema

The public bulletin model for governance events consists of `Bulletin`, `BulletinEvent`, and `BulletinEventCore`. The event core fields are `sequence`, `previousHash`, `eventType`, `subjectDid`, `actorDid`, `payload`, and `createdAt`. The event body additionally contains `eventHash` and `signature`, as well as the optional `proof`, `cryptoSuite`, and `hashAlgorithm`. Current event types include Root platform initialization, content distribution platform information updates, authorization/revocation of Registrar nodes, authorization/domain updates/revocation of Discovery nodes, node revocation, and capability-tag tree updates. An off-chain projection may additionally record receipt time, projection time, a cursor, and errors, but these are not public core fields of `BulletinEvent`. The original on-chain or bulletin-chain event is the source of truth; page display text and projection status are derived information. When an event lacks a sequence, previous hash, signature, or

target subject, or when projection order is discontinuous, authorization status must not be silently updated. The anomaly should be retained until the missing events are filled.

```
{
  "sequence": 1842,
  "previousHash": "sha256:example-previous-event-hash",
  "eventType": "DISCOVERY_NODE_AUTHORIZED",
  "subjectDid": "did:oan:INDS:DiscoveryNodeExample",
  "actorDid": "did:oan:INRT:RootPlatformExample",
  "payload": { "authorizedDomains": ["software-development"] },
  "createdAt": "2026-09-07T01:02:03Z",
  "eventHash": "sha256:example-event-hash",
  "signature": "zExampleSignature",
  "cryptoSuite": "ed25519-sha256",
  "hashAlgorithm": "sha256"
}
```

A.8 Registration Request and Response Schema

The current official Registrar node browser/client entry point is `POST /resources/register`. The request is deserialized as `ResourceRegistrationSubmission` and contains `resourceDid`, `resourceType`, `didDocument`, `didDocumentHash`, `metadata`, `packageVersion`, `packageHash`, `metadataHash`, `hashAlgorithm`, nullable `registrationCredential`, and `subjectControlProof`. Only when forwarding to the Root platform internally does the node wrap the request with `registrarDid` and `upstreamAuth`, using `POST /root/resources/verify-and-publish`. Therefore, the internal forwarding package must not be treated as a public registration request. The response usually contains `status`, `resourceDid`, `resourceType`, `registrationCredential`, and `rootResponse`; different deployments may add status or correlation identifiers. Whether retries with the same DID, version, and digests are fully idempotent depends on the node storage and Root platform implementation. A valid registration must not be written when the control proof is missing, fields are incomplete, or the previous version does not match.

```
POST /resources/register
Content-Type: application/json

{
  "resourceDid": "did:oan:SKDM:ExampleResourceDid",
  "resourceType": "skill",
  "didDocument": {},
  "didDocumentHash": "sha256:example-did-document-hash",
  "metadata": {
    "resourceType": "skill"
  },
  "packageVersion": "1.2.0",
  "packageHash": "sha256:example-package-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "hashAlgorithm": "sha256",
  "registrationCredential": null,
  "subjectControlProof": {
```

```

    "challenge": {},
    "proof": {}
  }
}

```

Situation	Response status	Handling
Validation passes and upstream processing completes	2xx	Return the registration result, registration VC, and Root platform response
Request body cannot be deserialized or shape validation fails	400	Correct the DID, digests, control proof, or required fields
Upstream Root platform rejects the request	Node-specific mapping	Handle according to the returned error code; do not treat failure as registered
Network failure or upstream temporarily unavailable	5xx or connection error	Query by request DID, version, and digests before deciding whether to retry

A.9 Discovery Request and Response Schema

The fields of the public `ResourceDiscoveryQuery` are optional `query`, `resourceType`, `capabilityTags`, `protocol`, and `version`, together with `versionMode` and `limit`, which have default values. Exact queries by complete DID, structured-condition queries, tag queries, and natural-language queries are ways for the node to use these fields; they are not separate core `queryType` fields. Pagination and freshness fields may appear in a node read model or an extended profile. The public `ResourceDiscoveryResponse` contains the Discovery node DID, a candidate array, a creation time, and an optional response proof. A candidate contains the resource DID, resource type, score, version, lifecycle, capability tags, authorized domains, services, protocol bindings, resource package information, and a reference to the Root proof. Index time, source descriptions, and query explanations are node- or official-website-derived display information. Semantic scores cannot replace trusted evidence. Unknown conditions, invalid DIDs, expired cursors, or unavailable query services should return identifiable errors rather than hiding failure behind an empty result.

```

{
  "query": "I need a tool that can search code repositories and summarize the project structure .",
  "resourceType": "tool_api",
  "capabilityTags": ["code-analysis", "repository-search"],
  "protocol": "https",
  "versionMode": "latest",
  "limit": 20
}

```

```

{
  "discoveryDid": "did:oon:INDS:DiscoveryNodeExample",
  "candidates": [{

```

```

    "resourceDid": "did:oon:SKDM:ExampleResourceDid",
    "resourceType": "skill",
    "score": 0.94,
    "version": "1.2.0",
    "lifecycleState": "active",
    "capabilityTags": ["code-analysis", "repository-search"],
    "authorizedDomains": ["software-development"],
    "services": [],
    "protocolBindings": [],
    "packageInfo": { "packageHash": "sha256:example-package-hash" },
    "rootProof": { "rootDid": "did:oon:INRT:ExampleRootDid" }
  }],
  "createdAt": "2026-09-07T01:02:08Z",
  "proof": null
}

```

A.10 Error Response Schema

The current public error structure of nodes is based on { "error": "machine_readable_error_code" }; a trusted-call rejection may also include `reason`. Future or specific gateways may extend this to an object containing a message, a correlation request identifier, and retry guidance, but these are not fields currently guaranteed by all nodes. Error codes should distinguish input errors, signature/credential errors, unauthorized access, not found, duplicates, version conflicts, unavailable dependencies, synchronization lag, and internal errors. Public-facing messages must not disclose private keys, databases, or internal configuration. Clients receiving an unknown error code should handle it as a failure rather than defaulting to success.

```

{
  "error": "sync_stale",
  "reason": "discovery_index_behind_publication"
}

```

If a gateway uses an extended error object, the client should first determine whether `error` is a string or an object, and then read optional fields. It must not assume an object shape or treat the HTTP status code alone as the business error code. The Registrar node currently maps some upstream errors to its own HTTP error responses; callers should retain both the HTTP status and the response body.

A.11 Node Authorization and Governance Status Schema

Node authorization status and governance status are usually formed as read-side projections by an off-chain trust indexer or a node status interface. They are not equivalent to the `NodeAuthorizationCredential` or the bulletin event body. A projection may contain the node DID, node role, authorized domains, current status, status reason, validity period, most recent governance event, event sequence, synchronization cursor, data time, and freshness. The specific fields are governed by the corresponding interface. Status values should at minimum distinguish

active, suspended, revoked, expired, unknown, and syncing. When the governance source is missing or the indexer is behind, an active status must not be fabricated. For sensitive write actions, use a reject or manual-confirmation policy when the status is unknown or exceeds the freshness permitted by the deployment. A display interface may separately mark the delay.

Status	Permitted read-only uses	Effect on write or authorization operations
active	May be displayed and enter normal validation	Allowed within the capability scope
suspended	May display historical status	Reject new sensitive operations
revoked	Retain only for audit and historical references	Reject all operations requiring current authorization
expired	May indicate that authorization has expired	Must not be used as the basis for current authorization
unknown/syncing	May display only the uncertain status	Wait for confirmation; do not allow sensitive operations

A.12 Distribution Manifest and Synchronization Cursor Schema

The current public structures on the distribution side fall into two categories. A content distribution platform index response uses `items`, `count`, `afterCursor`, `nextCursor`, and `hasMore`, with each item containing `cursor` and a complete `ResourcePackage`. A batch notification sent by the Root platform to a Discovery node uses a batch identifier, Root platform DID, target Discovery node DID, authorized domains, a sequence range, `items`, a content distribution platform manifest address, an update address, and proof. An item additionally carries the resource DID, operation, resource package version, DID Document digest, metadata digest, package digest, hash algorithm, lifecycle state, bulletin sequence, and bulletin event hash. Manifest version, generation time, error, or confirmation fields may be added by a specific interface; fields from one interface must not be applied directly to another. Cursors must advance monotonically, repeated batches must be safely replayable, and processing of an item must be paused when there is a sequence gap or digest mismatch. A cursor is transmission progress; it does not mean that the resource has already been indexed by the Discovery node.

```
{
  "items": [{
    "cursor": 1842,
    "package": {
      "packageVersion": "1.2.0",
      "resourceDid": "did:oon:SKDM:ExampleResourceDid",
      "resourceType": "skill",
      "didDocument": { "id": "did:oon:SKDM:ExampleResourceDid" },
      "didDocumentHash": "sha256:example-did-document-hash",
      "metadataHash": "sha256:example-metadata-hash",
      "packageHash": "sha256:example-package-hash",
      "hashAlgorithm": "sha256",
```



```

    "metadata": {
      "resourceDid": "did:oon:SKDM:ExampleResourceDid",
      "resourceType": "skill",
      "subjectType": "skill",
      "name": "Repository analysis",
      "lifecycleState": "active",
      "packageVersion": "1.2.0",
      "packageHash": "sha256:example-package-hash",
      "metadataHash": "sha256:example-metadata-hash",
      "hashAlgorithm": "sha256",
      "updatedAt": "2026-09-07T01:02:08Z"
    },
    "rootProof": { "rootDid": "did:oon:INRT:ExampleRootDid" },
    "createdAt": "2026-09-07T01:02:08Z"
  }
}],
"count": 1,
"afterCursor": 1800,
"nextCursor": 1842,
"hasMore": false
}

```

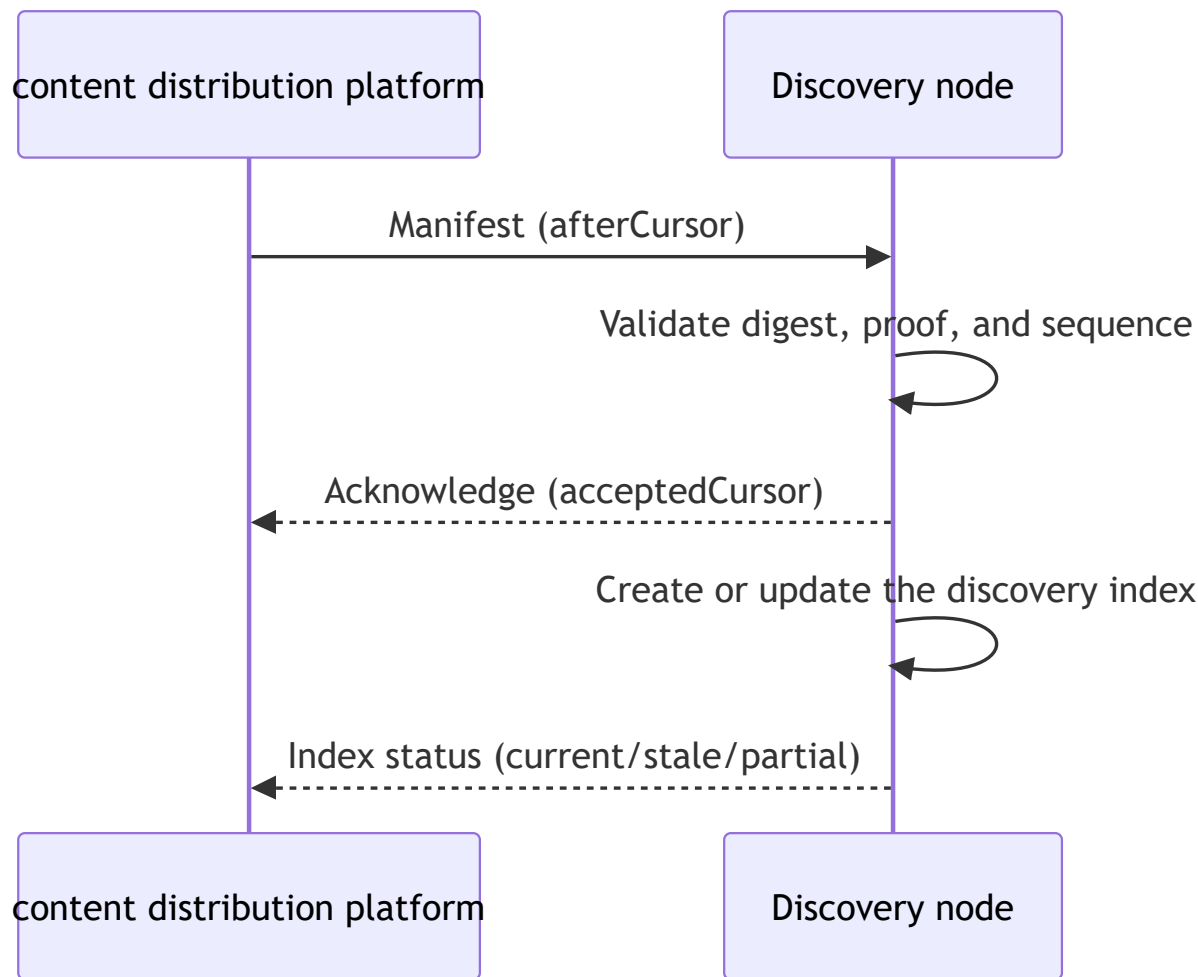
The minimum-shape example for a Root platform notification item is:

```

{
  "resourceDid": "did:oon:SKDM:ExampleResourceDid",
  "resourceType": "skill",
  "operation": "upsert",
  "packageVersion": "1.2.0",
  "didDocumentHash": "sha256:example-did-document-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "packageHash": "sha256:example-package-hash",
  "hashAlgorithm": "sha256",
  "lifecycleState": "active",
  "bulletinSequence": 1842,
  "bulletinEventHash": "sha256:example-event-hash",
  "capabilityTags": ["software-development"],
  "authorizedDomains": ["software-development"]
}

```

Manifest/notification -> validate digest, proof, sequence, and authorization domain -> confirm
transfer progress -> create or update the discovery index



A.13 Search Explanation and Evidence Schema

Search explanations should state the query type, parsed conditions, matched capability tags or use cases, structured filters, relevance score, result source, version, resource status, index time, Root proof, and freshness. An explanation is derived information that helps callers understand the result; it cannot serve as signature or authorization evidence. When fields are missing, the semantic-model version is incompatible, or the source cannot be verified, the result should be explicitly marked as unknown or incomplete rather than supplemented with speculative reasons.

Explanation layer	Content that may be expressed	Evidence it cannot replace
Query parsing	Query type, normalized conditions, tags, and use-case matches	Resource control
Relevance	Score, matched fields, and reason for ordering	Root platform publication proof
Source	Discovery node, index time, and index version	Current node authorization status
Trust status	VC, Root proof, governance status, and freshness	Guarantee of actual invocation results

```
{
  "queryExplanation": {
    "queryType": "semantic",
    "normalizedIntent": "repository structure summarization",
    "matchedTags": ["code-analysis", "repository-search"],
    "filtersApplied": { "resourceTypes": ["skill", "tool_api"] },
    "modelVersion": "semantic-index-v1"
  },
  "evidence": {
    "resourceDid": "did:oa:SKDM:ExampleResourceDid",
    "rootPublication": "verified",
    "governance": "active",
    "indexedAt": "2026-09-07T01:02:08Z",
    "freshness": "current"
  }
}
```

A.14 References

Source	Type	Link
oan-protocol-common	Code repository: normative data structures and schemas	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	Code repository: TypeScript data types and client validation	https://github.com/OpenAgenet/oan-sdk-ts
did:oa DID Method Specification	Standard/method specification: DID Documents and identifier data structures	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Resource Identity and Discovery	IETF draft: resource identity data structures	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

Appendix B Protocol and API Reference

This appendix provides rules for reading and implementing the interfaces. Paths, fields, and statuses are cross-checked against each node's service entry points, `oan-protocol-common/schemas/agent-contract/openapi.yaml`, the frontend and backend API wrappers on the official website, and actual deployment configuration; capabilities declared only in the common contract must not be written as if they were exposed by all nodes. Interface changes must be associated with the protocol version, error pattern, authentication method, idempotency rules, and the change record in Chapter 25.

B.1 Endpoint Catalog

The endpoint catalog lists health, DID, resource registration, Root platform publication, content distribution, discovery, governance status, and statistics interfaces by service role, and distinguishes public entry points, node-to-node entry points, internal-only entry points, and deployment prefixes. The official website currently forwards some registration and discovery requests through a same-origin backend; the nodes themselves also expose independent service endpoints, so website paths cannot automatically be inferred as protocol paths for all nodes. Example base URLs use placeholders and do not hard-code the website's current domain or a particular server address as a protocol requirement; each row records at least the call direction, HTTP method, authentication, request/response pattern, failure behavior, and version. The table below lists only representative entry points that can be verified in the current code; management, statistics, and debugging interfaces not listed here remain subject to the routes of their corresponding nodes.

Service role	Call direction, method, and path	Authentication boundary	Main returns
Registrar node	Client -> Registrar node: <code>POST /resources/register</code> ; Client -> Registrar node: <code>GET /registrar/status</code> , <code>GET /registrar/did</code>	The registration endpoint receives a resource control proof; node-to-node forwarding to the Root platform uses node-to-node signatures	Registration VC, acceptance status, Root platform response

Service role	Call direction, method, and path	Authentication boundary	Main returns
Root platform	Registrar node -> Root platform: POST /root/resources/verify-and-publish; Client or node -> Root platform: GET /root/resources/{did}, GET /root/status	Publication writes are limited to authorized nodes or administrative calls; public reads depend on deployment	Root proof, resource version, status, and cursor
content distribution platform	Root platform or publisher -> content distribution platform: POST /cdn/resources, POST /cdn/resources/batch; Client or node -> content distribution platform: GET /cdn/resources/{did}, GET /cdn/resources/index	Publication uses node-to-node authentication; resource reads and index reads depend on deployment	ResourcePackage, index entries, synchronization status
Discovery node	Client -> Discovery node: POST /discovery/resources/query; Node or operations personnel -> Discovery node: GET /discovery/index/resources/{did}, GET /discovery/status	Queries may be public; management and synchronization interfaces are restricted	Resource results, source, and proof references
Operations API	Operations personnel -> node: GET /health and node status, metrics, and audit interfaces	Operations authentication and network boundary	Status summary, without secrets

The correspondence between core payload types and endpoints is as follows. When shared types come from `oan-protocol` or `oan-package`, field names follow the JSON mapping of `serde`; `Value` represents a dynamic response in the current implementation and cannot be used to infer a fixed schema that does not exist.

Endpoint	Request JSON type	Response type or main fields	Notes
POST /resources/register	ResourceRegistrationSubmit	Dynamic JSON: registration status, resource DID, registration VC, Root platform response	/resources/submit is bound to the same handler

Endpoint	Request JSON type	Response type or main fields	Notes
POST /root/resources/verify-and-publish	ResourceVerifyAndPublishRequest	Dynamic JSON: resource -verified-and-queued , version, three categories of hashes, and CDN queue status	Node-to-node call limited to authorized Registrar nodes
POST /cdn/resources	ResourceCdnPublishRequest	Dynamic JSON: published , version, and publication cursor	ResourcePackage must pass Root platform source and hash validation
POST /discovery/resources/query	ResourceDiscoveryQuery	ResourceDiscoveryResponse	limit defaults to 10, versionMode defaults to latest
POST /discovery/query/explain	ResourceDiscoveryQuery	Dynamic explanation JSON	Not equivalent to a normal discovery response
GET /cdn/resources/{did }	Path parameter did	ResourcePackage	Reads a ResourcePackage directly
POST /cdn/resources/batch-get	Local ResourceBatchGetRequest	requestedCount, foundCount, items	The request body contains a resourceDids array

B.2 Request Header Fields and Signatures

Request headers and signature materials are defined by the interface profile. The current node-to-node `SignedRequestEnvelope` uses JSON fields such as `requestId`, `protocolVersion`, `purpose`, `method`, `path`, `aud`, `requestTimestamp`, `requestNonce`, `bodyHash`, and `proof` to carry signature semantics; the browser-facing public registration and discovery entry points on the official website must not arbitrarily add a set of required `X-OAN-*` headers that are not defined at the service entry point. If the interface uses a protocol envelope to carry this information, the JSON fields take precedence; if the deployment layer additionally uses HTTP headers, they may only be documented as deployment-profile details. The server must validate the normalized request covered by the signature, the target DID, the time window, and the nonce; it must reject missing, expired, replayed, or misdirected requests.

The roles of node-to-node protocol envelope fields can be checked against the table below. `method` and `path` bind the protected HTTP request, `aud` binds the recipient, and `bodyHash` binds the request body; together, these fields prevent a valid proof issued by one node from being moved to another request. When a browser submits a resource, the resource control proof is located in the request body's `subjectControlProof`; it must not be mistakenly written as a node-to-node HTTP header.

Field	Purpose	Validation focus
<code>requestId</code>	Identifies one node-to-node request	Used for log correlation; cannot replace replay protection
<code>protocolVersion</code> , <code>purpose</code>	Identifies the protocol version and call purpose	Must match the profile allowed by the receiving interface

Field	Purpose	Validation focus
<code>method</code> , <code>path</code> , <code>aud</code>	Binds the HTTP method, path, and recipient	Must not differ from the actual request or target node
<code>requestTimestamp</code> , <code>requestNonce</code>	Restricts the time range and prevents reuse	Check the clock window and nonce replay cache
<code>bodyHash</code> , <code>proof</code>	Binds the request body and provides an integrity proof	Recalculate the request-body digest and validate the proof

```
POST /resources/register
Host: registrar.example
Content-Type: application/json
```

The `upstreamAuth.proof` of a node-to-node request should cover the method, normalized path, recipient, timestamp/nonce, content digest, and actual request body; the server should perform the write only after completing the time-window, replay-cache, signer-authorization, and resource-control checks. In a browser registration request, the resource control proof is located in the request body's `subjectControlProof`; it is not the same type of credential as node-to-node `upstreamAuth`.

B.3 Response Header Fields and Cache Directives

A response may provide the source, protocol/mode version, data time, freshness, ETag, cache control, correlation identifier, and necessary security headers when applicable; these fields are not guaranteed to be uniform public response headers across all current node interfaces, and the `X-*` items below may only serve as deployment-layer or interface-profile extension conventions. An ETag or cache hit only indicates that the content version has not changed; it does not prove that the resource status or governance authorization remains valid. When returning an old snapshot, the response or body should include identifiable time and degradation indicators; caching must not be used to conceal an upstream failure.

Field or directive (when applicable)	Purpose	Usage notes
<code>ETag</code>	Identifies the response content version	Does not mean that the governance status is valid
<code>Cache-Control</code>	Specifies caching and revalidation policy	An old snapshot must also identify its freshness
<code>X-Request-Id</code>	Correlates the client, nodes, and logs	Must not contain private information
<code>X-OAN-Data-Time</code>	Indicates the data generation or collection time	Distinguish it from the server response time
<code>X-OAN-Freshness</code>	Indicates statuses such as current and stale	Must not conceal an upstream failure

B.4 Status Codes and Error Codes

HTTP status codes express transport-layer and generic processing results, while OpenAgent (OAN) error codes express specific semantics such as field errors, invalid signatures, unauthorized

access, not found, duplication, version conflicts, dependency failures, synchronization lag, and internal errors. The current public error-code vocabulary is defined by `oan-protocol-common/docs/error-codes.md`, including `invalid_did_document_structure`, `invalid_registration_credential`, `invalid_request_signature`, `unauthorized_domains`, `invalid_root_proof`, and `package_decode_failed`; that file defines error semantics but does not bind every error code to a unique HTTP status. Different nodes may map these errors through an HTTP status or response body. Clients should determine whether to retry from the actual error structure; they must not treat every 2xx response as publication success, nor conflate 404, empty results, and synchronization incomplete.

HTTP status	OAN error code examples	Client action
400	<code>invalid_did_document_structure</code> , <code>invalid_authorized_domains</code> , <code>package_decode_failed</code>	Correct the input or ResourcePackage and retry; the specific status depends on the interface implementation
401/403	<code>invalid_request_signature</code> , <code>unauthorized_domains</code>	Re-authenticate, use an authorized node, or stop the request
404	Resource-not-found error returned by the node	Distinguish not found from an unsynchronized index
409	Duplicate or version-conflict error returned by the node	Obtain the current version and process it according to the update rules
429	Deployment-layer rate-limit error	Follow the retry indication and back off
5xx	Dependency unavailable or internal error	Back off and retry only when the interface marks the request as retryable

B.5 Pagination and Cursor Rules

List interfaces use explicit `limit`, `cursor`, and sorting rules. Cursors may only be generated and interpreted by the server; clients must not assume that they are readable or modifiable. The current public `ResourceDiscoveryQuery` type always provides `limit`, with a default value of 10, but the request model of `POST /discovery/resources/query` has no `cursor` field; the Discovery node truncates candidate results according to `limit`. The interface that actually provides `afterCursor`, `nextCursor`, and `hasMore` is the content distribution platform's `GET /cdn/resources/index`; its cursor is not the same type as the discovery query's `limit`. Page size should have a deployment limit, and the sort key must remain stable during pagination; expired, invalid, or filter-mismatched cursors should return an error. Publication cursors, synchronization cursors, event cursors, and pagination cursors must not be used interchangeably.

```
GET /cdn/resources/index?limit=20&afterCursor=1800
Accept: application/json

{
  "items": [],
  "count": 0,
```



```

"afterCursor": 1800,
"nextCursor": 1800,
"hasMore": false
}

```

A pagination cursor only represents the reading position in the content distribution platform's list; the `limit` in a discovery query only controls the number of candidate results for that request and does not produce a resumable discovery pagination cursor. Publication sequences, distribution synchronization cursors, and governance event cursors must use the fields and validation rules defined by their respective interfaces.

B.6 Version and Compatibility Header Fields

Protocol versions, resource schema versions, capability declarations, and compatibility policies are expressed through defined protocol envelope fields, request-body fields, or deployment profiles, and remain traceable in requests and responses. Current public protocol constants include `oan-resource-2026`, while the default value of `ResourceDiscoveryQuery.versionMode` is `latest`; such protocol fields must not be directly rewritten as uniform HTTP headers without code support. When the server encounters an unsupported version, it should return an explicit incompatibility result; compatible extensions must not change the semantics of DID, signatures, status, or publication sequences. See Chapter 25 for version selection rules.

```

{
  "query": "a tool that can search code repositories and summarize project structure",
  "capabilityTags": ["code-search", "project-structure-summary"],
  "versionMode": "latest",
  "limit": 10
}

```

The example above is a discovery query request for `POST /discovery/resources/query`; all fields correspond to the current `ResourceDiscoveryQuery`. `protocolVersion` belongs to the node-to-node `SignedRequestEnvelope` and must not be mixed into this query object. The current query model has no unified `capabilities` field; capability conditions should be expressed through `capabilityTags` or an extension explicitly supported by the target interface. When the server does not support the requested version, it should return an explicit version-incompatibility error and the range of acceptable versions; clients must not ignore the response-mode version merely because they received an HTTP 2xx response.

B.7 Health, Readiness, and Liveness Response Formats

The common minimum structure of the public `/health` response across current nodes is `HealthResponse`: `status`, `nodeType`, and an optional node DID. A node status interface may additionally provide database, upstream, synchronization cursor, or error summaries. Operations systems can use this information to distinguish process liveness, dependency readiness, business availability, and data synchronization status, but must not treat every field as a uniform schema present on

every node. Passing liveness does not imply readiness, and passing readiness does not imply that registration or discovery smoke tests have passed; sensitive configuration and tokens must not appear in the response.

```
{
  "status": "ok",
  "nodeType": "discovery",
  "did": "did:oan:INDS:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz"
}
```

If the deployment provides an extended status interface, it may add check time, dependency status, and synchronization cursors beyond the minimum response above; extension fields should identify their source and version, and the extended response must not be treated in reverse as the mandatory response structure of `/health` on every node.

Check type	What it proves	What it does not represent
liveness	The process can still respond	Dependency and business availability
readiness	The service is ready to receive requests	Successful registration or discovery smoke tests
dependency	Database, upstream, or off-chain trust indexer status	All business paths are functioning normally
smoke test	A critical business flow can be completed	Long-term performance and full correctness

B.8 Public, Authenticated, and Operations-Only API Matrix

The interface matrix must at least distinguish public read-only interfaces, authenticated publication/registration interfaces, node-to-node interfaces, and operations-only interfaces. Public discovery, DID, status, and necessary statistics may be accessible from the Internet; write, management, database, full-log, detailed access-statistics, and key-related interfaces must be protected by authentication, network boundaries, and auditing. The relay interface of the official website's backend must not expand downstream service permissions, and ordinary users must not obtain backend statistics or secrets through public interfaces.

Access level	Examples	Permitted data	Protection requirements
Public read-only	DID, discovery, necessary status	Published ResourcePackages and public proofs	Rate limiting, input validation, auditing
Authenticated write	Registration, updates, node-to-node synchronization	Objects within the authorized scope	Signatures, nonce, idempotency, and authorization checks
Operations only	Metrics, full logs, administrative operations	Internal operational data	Strong authentication, network isolation, and least privilege

Access level	Examples	Permitted data	Protection requirements
Prohibited from public access	Database, private keys, detailed access statistics	Not open to Internet users	Must not be leaked through the official website's relay

B.9 Reference Sources

Source	Type	Link
<code>oan-protocol-common</code>	Code repository: protocol types, OpenAPI contract, and error codes	https://github.com/wolfbrother/oan-protocol-common
<code>oan-root-services</code>	Code repository: Root platform API	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: registration API	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	Code repository: discovery API	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-sdk-ts</code>	Code repository: client API wrappers	https://github.com/OpenAgenet/oan-sdk-ts
OAN Resource Identity and Discovery	IETF Internet-Draft: resource identity and discovery API	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Efficient Agent Discovery Profile	IETF Internet-Draft: discovery API profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Appendix C Cryptography and Encoding

This appendix defines the cryptographic and encoding boundaries used by OpenAget (OAN) for identity, credentials, resource publication, and cross-node verification. The current shared Rust implementation is primarily based on `oan-crypto`, `oan-credentials`, and `oan-did-oan`; the browser-side TypeScript SDK uses the Web Cryptography API to generate Ed25519 Local Identity. Any cross-language implementation must recompute the same digest from the same canonicalized input and must handle algorithms, encodings, and error states explicitly.

C.1 Supported Algorithms

The current `CryptoSuite` includes `Ed25519Sha256Legacy`, `Ed25519Sha256`, and `Sm2Sm3`: the first two use Ed25519 signatures and SHA-256 hashing, while the latter uses SM2 signatures and SM3 hashing. The corresponding verification method types are `Ed25519VerificationKey2020` or `SM2VerificationKey2020`, and the proof types are `Ed25519Signature2020` or `SM2Signature2020`. The algorithm suite must be consistent with the public key, proof, and hash algorithm; unknown suites are rejected, and legacy suites may only be retained under a compatibility policy and must not be silently downgraded.

Algorithm Suite	Signature Algorithm	Hash Algorithm	Typical Use
<code>Ed25519Sha256Legacy</code>	Ed25519	SHA-256	Compatibility with existing credential or digest flows
<code>Ed25519Sha256</code>	Ed25519	SHA-256	Current Ed25519 proof flow
<code>Sm2Sm3</code>	SM2	SM3	Identity and proof flows supporting SM2

The algorithm suite, verification method, public key encoding, and `proof` type must match as a set; unknown or deprecated suites must not silently switch to another algorithm suite.

In implementation terms, `Ed25519Sha256Legacy` and `Ed25519Sha256` use the same Ed25519 key and SHA-256 hash, but their signing inputs differ: the former first computes SHA-256 over the canonicalized JSON, then signs the UTF-8 bytes of the resulting 64 lowercase hexadecimal characters; the latter directly signs the UTF-8 bytes of the canonicalized JSON. `Sm2Sm3` directly uses SM2 to sign the UTF-8 bytes of the canonicalized JSON, and its digest function uses SM3. A cross-language

implementation must not determine the input solely from the signature algorithm name; it must read both `cryptoSuite` and `hashAlgorithm`.

Selection	Decision in the Current Implementation	Failure Handling
<code>cryptoSuite</code>	Determines the signature algorithm, hash algorithm, and signature input form	Reject when unrecognized
<code>proof.type</code>	The <code>Ed25519Signature2020</code> or <code>SM2Signature2020</code> corresponding to the suite	Reject when the type does not match
Verification method type	Derives the suite from <code>Ed25519VerificationKey2020</code> or <code>SM2VerificationKey2020</code>	Reject when it cannot be derived
Public key carrier	Use either <code>publicKeyMultibase</code> or <code>publicKeyJwk</code>	Reject when both are missing or cannot be parsed

C.2 Canonicalized JSON and Hashing

The shared implementation first converts the serializable object to JSON, then recursively orders object keys lexicographically; arrays retain their original order, strings use JSON escaping, numbers use their serialized representation, and the rule for omitting empty fields is determined by the Rust/TypeScript data model. `Ed25519Sha256Legacy` signs the SHA-256 hexadecimal result of the canonicalized JSON, while `Ed25519Sha256` and `Sm2Sm3` sign the UTF-8 bytes of the canonicalized JSON. Cross-language implementations must fix the rules for UTF-8, key ordering, numbers, and empty fields; any difference will cause digest or signature verification to fail.

```
{
  "b": 2,
  "a": {
    "y": 1,
    "x": "OAN"
  },
  "items": ["one", "two"]
}
```

During canonicalization, keys are recursively ordered lexicographically, arrays retain their original order, and the input uses UTF-8. The corresponding canonicalized result is:

```
{
  "a": {
    "x": "OAN",
    "y": 1
  },
  "b": 2,
  "items": ["one", "two"]
}
```

Cross-language tests should fix the rules for null values, number representations, string escaping, and omission of object fields at the same time, rather than relying solely on the default serialization behavior of a particular language runtime.

For example, the SHA-256 result of the canonicalized string above is `03332fb018e2f0376e263dabf4b2231e3bd70767`. This value corresponds only to the example object, example key order, and example UTF-8 encoding; it is not a protocol constant. On the Rust side, `oan-crypto::canonical_json` and `hash_json_with_suite` can be used to produce a baseline; on the TypeScript side, the object should first be recursively sorted according to the same rules, and `crypto.subtle.digest("SHA-256", ...)` should then be used to compute the digest over the UTF-8 bytes.

Implementation must distinguish three changes in particular: changing object member order should not change the result after canonicalization, changing array element order will change the result, and whether changing a number from 1 to 1.0 is considered equivalent depends on the JSON number representation before the input enters the canonicalization function. Production code must not first convert JSON to indented text and then sign it directly, nor may it rely on the order of fields returned by a database.

C.3 JWS and Detached Signatures

The current OAN shared implementation uses `DataIntegrityProof` to express detached proofs, containing `type`, `creator`, `created`, `proofPurpose`, `proofValue`, `cryptoSuite`, `hashAlgorithm`, and `verificationMethod`. `proofValue` is an unpadded Base64url-encoded signature value; the signature input is determined by the canonicalized result of the object to be signed and the algorithm suite. The verifier must first determine the suite, reconstruct the signature input, and verify the public key and signature, then check the creator, verification method, target DID, validity period, and status; it must not merely compare `proofValue` strings.

```
{
  "type": "DataIntegrityProof",
  "creator": "did:oon:SKDM:ExampleResourceDid#key-1",
  "created": "2026-09-07T01:02:03Z",
  "proofPurpose": "assertionMethod",
  "verificationMethod": "did:oon:SKDM:ExampleResourceDid#key-1",
  "cryptoSuite": "Ed25519Sha256",
  "hashAlgorithm": "SHA-256",
  "proofValue": "base64url-without-padding"
}
```

Signature verification failures should distinguish inconsistent canonicalized input, a mismatched public key, an invalid signature, an incorrect proof purpose, and an unacceptable status, so that the caller can decide whether to correct the request, retrieve the data again, or stop processing.

The minimum signature verification sequence is as follows:

1. Determine the suite from `proof.cryptoSuite` or a compatible `proof.type`, and confirm that it is consistent with the verification method and public key format.
2. Locate the public key in the DID Document by `verificationMethod`, and check the relationship among the method identifier, controller, and target DID.

3. Remove `proof` from the object being verified and reconstruct the canonicalized signature input according to the suite; legacy suites must also first compute the SHA-256 hexadecimal digest.
4. Decode `proofValue` as unpadded Base64url and perform signature verification, then check `created`, `purpose`, issuer authorization, and credential status.

```
payload without proof -> canonical JSON -> suite-specific signature input
                                -> Base64url proofValue -> verify with DID public key
```

The fact that `proofValue` can be decoded only shows that its encoding is readable; it does not show that the signature is correct. A correct signature also does not automatically show that the issuer has authorization from the Root platform or that the resource remains available.

C.4 DID and Key Encoding

The current syntax for a resource DID is `did: oan: <four uppercase semantic code characters>: <32-character Base58 suffix>`; the suffix uses the Bitcoin Base58 character set and excludes 0, O, I, and l. The `did: oan` parser also validates the correspondence between the resource type and the semantic-code subject portion. Depending on the algorithm, public keys may use a `z`-prefixed Base58 multibase or a JWK: Ed25519 uses `kty=OKP`, `crv=Ed25519`, and an unpadded Base64url `x`; SM2 uses `kty=EC`, `crv=SM2`, and `x` and `y`. Private keys must not enter a DID Document, ordinary protocol request, or service log.

```
did: oan: <four uppercase semantic code characters>: <32-character Base58 suffix>
```

Material	Recommended Representation	Must Not Appear
Resource DID	<code>did: oan</code> string	Incorrect semantic code, illegal Base58 character
Ed25519 public key	multibase or Ed25519 JWK	Private key, raw string with no declared algorithm
SM2 public key	SM2 JWK or a public key representation supported by the implementation	Incomplete <code>x/y</code> coordinates
Private key	Client-protected storage	DID Document, ordinary request, service log

`publicKeyMultibase` currently consists of a `z` prefix followed by the Base58 encoding of the public key bytes; Ed25519 uses a 32-byte public key, while SM2 uses uncompressed SEC1 public key bytes. JWK uses unpadded Base64url-encoded fields; after decoding, Ed25519 `x` should be 32 bytes, and SM2 `x` and `y` should each be 32 bytes. `oan-crypto` parses these carriers according to the verification method or explicit `cryptoSuite`, and performs signature verification only after parsing succeeds.

The semantic code of an example DID expresses only the subject or resource type and the application domain; the public key is not carried directly by the suffix. A randomly generated DID and its key pair are generated in two independent steps. If repeatable derivation is required by the business logic, `DidOan::derive` control material and a nonce must be used explicitly, and the

derivation rule must be treated as an implementation-specific capability; a randomly generated DID must not be treated as an identifier that can be reverse-derived from a public key.

C.5 VC and VP Encoding

Registration VCs and node authorization VCs use JSON objects to express the issuer, subject, status, issuance/expiration times, claims, and proof; the shared credential implementation is based on `AgentRegistrationCredential`, `NodeAuthorizationCredential`, and `DataIntegrityProof`. At issuance, a signature is generated over the object without `proof`; at verification, `proof` is removed to reconstruct the original payload, which is then verified according to the suite specified by the proof. If a VP is presented by an external system, its original structure, credential references, and verification results should be retained; parseability does not mean acceptance under the OAN trust policy, and expired, revoked, subject-mismatched, or unauthorized issuers must be rejected or marked unacceptable.

```
{
  "type": ["VerifiableCredential", "AgentRegistrationCredential"],
  "issuer": "did:oon:REGS:RegistrarExample",
  "credentialSubject": {
    "id": "did:oon:SKDM:ExampleResourceDid"
  },
  "proof": { "type": "DataIntegrityProof", "proofValue": "..." }
}
```

Successful VC parsing only means that the structure is readable; the caller must still verify the signature, issuer authorization, subject, validity period, and credential status. When a VP serves as an external presentation container, the credential source and verification result should be retained; the act of presentation itself must not be treated as authorization.

The verification object for a registration VC is the `AgentRegistrationCredential` with `proof` removed; the verification object for a node authorization VC is the `NodeAuthorizationCredential` with `proof` removed. The following table can be used as the implementation order for the verification chain:

Check	Object of Attention	Meaning When Not Passed
Structure	type , issuer , subject , time, claims , proof	The credential cannot be parsed or proceed to subsequent verification
Signature	proof , public key in the issuer's DID Document	The credential content is not proven by the corresponding private key
Binding	issuer , subject , resource DID, DID Document hash, and ResourcePackage hash	The credential does not correspond to the current resource
Authorization	Issuer identity, node role, authorization domain, and governance status	The issuer cannot make this statement within the current boundary
Status	status , issuedAt , expiresAt , and available revocation facts	The credential is invalid or should no longer be used

The verifier should retain enough of the original credential, DID Document, verification time, and error code to support re-verification; it should not store only a boolean value of “verification succeeded”.

C.6 Key Backup and Recovery Format

Local Identity backups are sensitive client files and may contain the version, update time, default subject/agent identifiers, subject/agent/node records, DID Documents, public and private key JWKs, and resource profiles. The TypeScript SDK currently uses an identity storage snapshot with version 1; identity backup files downloaded through the website registration flow include a time identifier in the filename, and after a successful resource submission, the registration VC, DID Document, and Local Identity file may also be packaged for storage. Backup files should be kept privately by the user. On import, the JSON structure, version, public/private key correspondence, and DID consistency must be checked; unsupported versions, missing keys, or parse failures must not result in a partial load.

```
{
  "version": 1,
  "updatedAt": "2026-09-07T01:02:03Z",
  "defaultSubjectDid": "did:oon:SUBJ:ExampleSubjectDid",
  "subjects": [],
  "agents": [],
  "importedNodes": []
}
```

The import flow should complete the integrity and version checks first, then load subject, agent, and node records as one operation. If a public and private key do not match, the DID is inconsistent, or the version is unsupported, the entire backup should be rejected; a partially usable identity must not be generated.

The private key JWK in a backup file is highly sensitive material. Browser download, import, and re-export should all be actively initiated by the user; the page, Registrar node, Root platform, and Discovery node do not need to receive this private key. After import, the client may sign a test message with the private key and verify it with the public key from the backup to confirm that the key pair matches; the private key or test signature must not be written to logs. The time identifier in the backup filename is used only to distinguish downloaded files and does not participate in identity or signature computation.

```
{
  "version": 1,
  "subjects": [
    {
      "did": "did:oon:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
      "verificationMethodId": "did:oon:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz#key-1",
      "publicKeyJwk": { "kty": "OKP", "crv": "Ed25519", "x": "example-public-x" },
      "privateKeyJwk": { "kty": "OKP", "crv": "Ed25519", "x": "example-public-x", "d": "example-private-d" }
    }
  ]
}
```

```

],
"agents": [],
"nodes": []
}

```

The key values in the example above are unusable placeholders intended only to show field positions; a real backup must not use this example content, and production private keys must not be placed in public documentation or test output.

C.7 DID and Resource Identifier Test Vectors

Test vectors should cover valid DIDs for the four resource types, infrastructure node DIDs, organization and developer DIDs, as well as semantic-code case errors, partial omissions, incorrect separators, incorrect suffix lengths, prohibited Base58 characters, extra segments, and resource-type mismatches. Each vector fixes the input string and the parse result or error type; generated DIDs validate only the format and type relationship, and a random suffix must not be required to be identical across runs. Cross-language tests should use the same JSON encoding and algorithm suite, and should store the expected digest or signature verification result.

Vector Category	Input	Passing Criterion
Valid DID	Four resource types and infrastructure DIDs	Parsing succeeds and the type relationship is correct
Format error	Incorrect case, separator, or length	A parse error is returned consistently
Prohibited character	0, O, I, L, etc.	Reject without silently correcting
Type mismatch	Semantic code does not match the resource type	Reject or return an explicit error

Test vectors should also record the input, expected error, and side-effect criteria. For example, `did: oan:SKFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz` should pass syntax parsing and the `skill` type check; submitting the same DID as an `mcp_server` resource should produce a type mismatch error. Replacing a suffix character with 0, O, I, or L should fail during parsing, and the client must not rewrite it automatically before submission.

Test Field	Example Value	Expected Result
Valid semantic code	SKFI	Parsed as the SK subject code and FI application-domain code
Valid suffix	7 YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz	Length is 32 and contains only permitted Base58 characters
Resource type match	SKFI + skill	Type check passes
Resource type conflict	SKFI + mcp_server	Returns <code>ResourceTypeMismatch</code> or the corresponding error code

C.8 Hash and Signature Verification Fixtures

Fixtures should include at least the original canonicalization input, canonicalized JSON, algorithm suite, hash algorithm, digest, signature input, public key, proof, and expected verification result, and should separately cover the correct payload, field-order changes, number/string changes, public key changes, truncated signatures, and proof-suite mismatches. The Rust implementation may use `oan-crypto` tests to produce the baseline, and the TypeScript implementation should read the same fixture to recompute it. Fixtures must not contain production private keys; dedicated test keys should be used when generating fixtures, with the public keys and signature results made available.

A fixture is recommended to use one JSON file that can be read by both Rust and TypeScript, with the input object and expected result stored separately. An example structure is:

```
{
  "suite": "Ed25519Sha256",
  "payload": { "a": 1, "b": "OAN" },
  "canonicalJson": "{ \"a\":1, \"b\": \"OAN\" }",
  "signatureInputEncoding": "utf-8-canonical-json",
  "publicKeyJwk": { "kty": "OKP", "crv": "Ed25519", "x": "test-public-key" },
  "proofValue": "test-signature-base64url-no-padding",
  "expected": { "canonicalJson": true, "signatureValid": true }
}
```

The public key, signature, and digest in a fixture must be generated from the same dedicated test key set; the `x` and `proofValue` in the example remain structural placeholders and cannot be used directly as inputs for a passing test. At least the following negative variants should be run: change only JSON member order, change array order, change one string, replace the public key, truncate the signature, and cross-replace `cryptoSuite` and `proof.type`. A correct implementation should accept only the original payload paired with the matching public key, and should produce stable failures for the other variants.

C.9 References

Source	Type	Link
<code>oan-protocol-common</code>	Code repository: cryptographic suites, keys, signatures, and hash implementations	https://github.com/wolfbrother/oan-protocol-common
<code>oan-sdk-ts</code>	Code repository: client cryptographic materials and DID operations	https://github.com/OpenAgenet/oan-sdk-ts
did:oan DID Method Specification	Standard/method specification: DID identifiers and verification methods	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Yellow Paper	arXiv yellow paper: cryptographic proofs and trust model	https://arxiv.org/abs/2606.03163

Appendix D Conformance Test Matrix

This appendix defines “conformance” as reproducible input, rule execution, and evidence judgment, rather than treating successful access to an online service on one occasion as complete proof. The test executor should record the test suite version, fixture version, version of the implementation under test, configuration summary, execution time, environment type, and redacted output; every case must produce a stable case ID, a **PASS** or **FAIL** conclusion, the actual error cause, and locatable evidence files. For rejection cases, the pass condition is not merely that the request fails. It must also show that the rejection occurred at the correct boundary, that no publication or index state that should not exist was written, and that the error code or reason conforms to the agreed contract. The currently directly reproducible offline evaluation is centered on `oan-third-party-node-admission-test-kit/fixtures/v1-alpha`; its `case-matrix.md` and `requirements-traceability-matrix.md` define the fixed fixtures, expected outputs, and checks for 49 requirements. This suite verifies the evaluation package and protocol judgment materials; it does not automatically amount to end-to-end proof against the official online Root, chain, content distribution platform, or off-chain trust indexer.

Every test record should contain at least the following fields:

Field	Recording requirement
Case identifier	Use the stable case ID in the manifest; do not use a temporary line number in a test file as the unique identifier.
Preconditions	Record the fixture, protocol version, authorization-domain set, governance snapshot, simulated responses, and necessary service status.
Operation and input	Record the called interface, HTTP method, request-body summary, query conditions, and signature or hash inputs; private keys, tokens, and database connection strings must be redacted.
Actual result	Record the status code, normalized result, error code, lifecycle, version, cursor, and necessary response summary.
Pass criterion	Compare the actual result item by item with the <code>outcome</code> , <code>reasonCode</code> , and normative clauses in the expected fixture.
Evidence artifacts	Save the JSON report, Markdown report, references to the input and expected fixtures, version, and redaction rules.

Test failures fall into three categories: protocol or security failures that block publication,

identity or authorization failures that block node admission, and operational or freshness failures that do not block publication but must trigger alerts and leave an audit trail. The test suite must not silently retry, automatically correct input, or ignore unknown fields merely to obtain a “pass.” If the implementation has an explicit compatibility mode, it should be marked separately in the report. On-chain events, real content distribution, and cross-node network timing that are not covered by the current V1-alpha fixture should be marked as uncovered, rather than treating offline pass results as evidence that they are covered.

Test execution results should distinguish “the test result is a pass” from “the object under test accepts the input.” For an input that is expected to be rejected, the service returning a rejection is the correct behavior of the object under test, so the test case itself should be recorded as **PASS** rather than counting the business rejection directly as a failure. The minimum record in a report may use the following structure:

```
{
  "caseId": "fixture.registrar.rejects-did-mismatch",
  "expectedOutcome": "invalid",
  "actualOutcome": "invalid",
  "verdict": "PASS",
  "reasonCode": "DID_ID_MISMATCH",
  "sideEffect": "no_resource_record_written",
  "evidence": ["evidence/fixture.registrar.rejects-did-mismatch.json"]
}
```

`expectedOutcome` describes the fixture expectation, `actualOutcome` describes the implementation's actual judgment, and `verdict` describes whether the test passed; these three meanings must not be merged into an ambiguous `status` field. Write tests should also compare the number of resource records, publication-queue entries, and index entries before and after the request to prove that the rejection produced no subsequent side effects.

Different test levels have different conclusion scopes: offline fixtures check rules and report generation, node integration tests check HTTP routes, persistence, and calls between nodes, and end-to-end tests check the propagation of publication, distribution, discovery, and governance state. The report should record the level and must not merge offline rule checks, service health checks, and complete business acceptance into a single “system passed” conclusion.

D.1 Resource Registration Tests

Registration tests should use `agent_service`, `skill`, `mcp_server`, and `tool_api` as the primary four resource categories. Positive samples should simultaneously satisfy consistency among `resourceDid`, the DID Document `id`, DID Document metadata, ResourcePackage metadata, the VC subject, and hash fields, and should contain a verifiable subject-control proof, resource type, and authorization domain. Registrar interface-layer tests should cover `/resources/register` and its compatibility entry point `/resources/submit`, verifying shape validation, authorization-domain validation, submission to the Root platform, and the boundary at which the registration credential and Root platform response are returned. Negative samples should replace one fact at a time, such as a DID mismatch, VC subject mismatch, package-hash mismatch, missing `packageProof`, missing credential proof, in-

consistent control challenge, incomplete metadata, or a suspended resource, so that one fixture does not contain multiple indistinguishable errors.

Test item	Prerequisite data and steps	Pass and rejection criteria	Evidence and impact
Minimal positive registration	Load registrar-resource.valid.json and submit a fixed ResourcePackage to the test Registrar.	Accept; resource type, DID, VC, authorization domain, and hashes are all bound; the returned result must not claim a publication stage beyond the one actually reached.	Request summary, response summary, resource record, and expected fixture; after passing, allow entry into subsequent publication tests.
Field and type consistency	Use did-mismatch , credential-subject-mismatch , and metadata-incomplete separately.	Reject with a stable reason; no valid resource record readable by a Discovery node may be written.	Error response and before/after storage comparison; failure blocks registration publication.
Proof and hash binding	Use tampered-proof , credential-hash-mismatch , package-hash-mismatch , and missing-proof fixtures.	Reject; validation must cover the proof subject, issuer, DID Document hash, metadata hash, and package hash.	Validation path and redacted proof summary; failure blocks publication.
Duplicate submission and update	Submit the same DID and version twice, then submit an incremented version with a controller signature.	Duplicate submission must not create two conflicting current versions; an update must pass control and version rules; historical and current versions must have distinguishable meanings.	Two responses, resource records, version list, and retry log; an abnormal duplicate write blocks publication.

The current fixed fixtures directly cover positive registration and multiple rejection conditions. Real Registrar database idempotency, concurrent writes, and propagation of updates for the same DID still need to be executed in a running-service scenario and marked in the report as live integration coverage; offline fixtures cannot substitute for them.

The four resource categories should use the same set of binding checks while using their respective valid **resourceType** and DID semantic codes. Test data may be organized as follows, making it possible to reuse the same assertions without applying fields from one resource category incorrectly to another:

Resource type	Example DID subject code	Minimal business materials	Key assertion
Agent service	AG	DID Document, service endpoint, capability tags	Service endpoint and resource type are consistent with the DID Document

Resource type	Example DID subject code	Minimal business materials	Key assertion
Skill package	SK	DID Document, package information, download or manifest address	Package hash, version, and resource metadata are consistent
MCP service	MC	DID Document, MCP service endpoint, protocol binding	Protocol binding and endpoint reference are consistent
Tool API	TL	DID Document, API endpoint, input/output description	Tool/API type and interface description are consistent

The same negative case should change only one fact under test, such as replacing only the DID Document id while keeping the VC, hashes, authorization domain, and lifecycle unchanged. This allows the failure reason to point accurately to the corresponding clause and prevents the service from masking the target defect with an earlier validation failure.

D.2 Root Platform Publication Tests

Root platform publication tests should separate Registrar acceptance from Root platform acceptance, validation, and publication. The input contains the registration service node's identity materials, the resource DID Document, the ResourcePackage, registration credential, subject-control proof, version, and hashes. The Root platform first validates node authorization and the request signature, then validates resource binding; only after all checks pass does it create a trusted Root platform publication proof and a publication fact available for distribution. Tests should check that a failure leaves no intermediate “publish first, validate later” state and that there is no path by which a Discovery node accepts a result merely because the Registrar returned success. For duplicate delivery, reprocessing must remain idempotent: the same publication fact must not generate a conflicting version, while changed content or proof must be judged again under version and control rules.

The current third-party admission test suite does not include the Root, chain, or real publication queue within the V1-alpha evaluation scope. Therefore, the offline part of this section can validate only publication input materials and expected rules; it cannot claim complete coverage of real Root platform publication. When running services, record the Root platform interface response, publication job ID, resource version, trusted Root platform publication proof summary, rejection reason, and reprocessing result. Logs containing private governance materials must not enter the public report.

The key assertion for Root platform publication tests is “validate first, then create a publishable fact.” They may be executed in the following order:

1. Prepare the identity materials for an authorized Registrar node and a resource registration submission.
2. Separately tamper with the node-to-node signature, resource control proof, DID Document

hash, ResourcePackage hash, and authorization domain, and call `POST /root/resources/verify-and-publish` for each case.

3. Check the HTTP result, machine-readable error, resource version, publication queue, and trusted Root platform publication proof for each rejection.
4. Send the original valid request again and confirm that duplicate publication does not create a conflicting current version.

The storage and queue comparison in step 3 is necessary evidence; saving only the HTTP response cannot prove that the Root platform left no partial publication state for a failed request.

D.3 Distribution and Synchronization Tests

Distribution tests center on “the publication fact reaches an authorized node in a verifiable manner.” First establish a baseline with full synchronization, then perform incremental synchronization using a cursor, injecting duplicate events, out-of-order events, missing events, checkpoint recovery, and content-hash errors separately. A receiving node must not overwrite data based only on arrival time; it should decide whether to accept it according to the resource DID, version, publication proof, and cursor rules. Materials that are missing or have an incorrect hash should remain pending or failed and must not enter a visible index. Recovery tests should record the last successful cursor, retry count, backoff result, and final convergence state, distinguishing “transfer complete,” “local validation complete,” and “visible to discovery.” A content distribution platform cache hit or HTTP success can prove only a transport-layer result; it cannot replace the receiver's validation of the trusted Root platform publication proof.

The boundary between this section and the current V1-alpha fixture should be written explicitly into the report: the existing test suite validates authorization domains, resource visibility, provenance, and fixed evidence rules, but its manifest explicitly excludes a real CDN, Root queue, and on-chain synchronization. After the implementation is connected to complete distribution tests, use repeatable resource versions and manually constructed cursors, and save four categories of evidence from the sender, receiver, validator, and indexer. Any failure that could cause an old version to become visible again, a revocation state to be lost, or an unvalidated resource to enter the index should be treated as a publication-blocking issue.

Distribution tests must at least verify that cursor advancement and resource acceptance are related but distinct results: a receiving node may successfully read one page of data while rejecting one resource in that page because of a proof or hash error. Whether the cursor advances and whether the failed item enters a retry queue must be recorded according to the established interface and implementation rules. Re-sending the same publication fact should be safely replayable; sending different content while reusing the same resource DID and version should enter a conflict-handling path and must not silently overwrite accepted content.

```
{
  "afterCursor": 1800,
  "nextCursor": 1812,
  "hasMore": true,
  "accepted": 11,
```



```

"rejected": 1,
"rejectedReasons": ["invalid_root_proof"]
}

```

The example above is an illustrative result-summary structure for a test report; the fields should follow the actual content distribution interface response. Advancement of `nextCursor` does not mean that every item has been validated and accepted. The report must also save the DID and reason for each rejected item.

D.4 Discovery and Semantic Search Tests

Discovery tests should cover exact DID queries for `/discovery/resources/query`, resource type, protocol, capability tags, authorization domain, and lifecycle filtering, as well as natural-language queries and the query-explanation interface, if enabled by the implementation. Test input first passes syntax, scope, and authorization-domain checks, then is converted into structured conditions, tag conditions, or semantic-retrieval conditions. Results must retain the resource DID, version, source, governance status, and necessary proof summary. Relevance scores, keyword hits, tag matches, and semantic similarity may be used only for ranking or explanation; they must not bypass authorization-domain checks, suspension/revocation filtering, DID Document validation, or trusted Root platform publication proof checks.

Query category	Representative input	Behavior that must be validated
Exact DID query	A fixed resource DID or a nonexistent DID.	Hit the unique resource or return a stable empty result; do not return another resource because of textual similarity.
Structured compound query	A combination of <code>resourceType</code> , <code>protocol</code> , <code>capabilityTags</code> , <code>limit</code> , and authorization domain.	Parse fields according to protocol rules; reject invalid enumerations, negative values, or values over the limit; pagination must neither duplicate nor omit accepted results.
Semantic query	For example, <code>Find an MCP server for security audit.</code>	Convert query text into explainable candidate conditions; returned results must identify semantic, tag, text, or other sources, and must not treat similarity as a trust proof.
Visibility boundary	In-domain, out-of-domain, suspended, revoked, protocol-mismatched, and type-mismatched resources.	Return only resources within the authorization domain and with an available status; an empty result is valid and must not be filled with recommendations.
Deduplication and explanation	Multiple versions of the same DID, duplicate index records, and an explain request.	Deduplicate according to DID and version rules, and explain candidate sources and filtering reasons.

`fixtures/v1-alpha/discovery` already provides fixed cases for visibility, empty results, arbitrary tag matching, invalid filters, invalid tags, invalid resource types, invalid protocols, `limit` boundaries,

and overly short queries; these cases should be used as the regression baseline. The specific quality of semantic models, vector stores, and natural-language ranking must not be inferred from fixed-rule fixtures; record the model version, index build time, and explanation result separately.

Discovery-result tests should record candidate generation, filtering, ranking, and proof checking separately. Even if a resource is highly matched by text or capability tags, it must not appear in callable results if the authorization domain does not cover it, its lifecycle is unavailable, its DID Document is inconsistent with the ResourcePackage, or its trusted Root platform publication proof is invalid. Conversely, an empty result does not mean that the discovery service is faulty. It constitutes a recall or indexing defect only when a valid resource satisfying all filter conditions exists but the response is still empty.

Test stage	Observed value	Pass condition
Input parsing	Query text, DID, resource type, tags, protocol, and <code>limit</code>	Valid input enters the corresponding retrieval path; invalid input fails in a controlled manner
Candidate generation	Candidate DIDs, match sources, and original scores	Candidates come from locally indexed data; resources are not invented
Governance filtering	Authorization domain, lifecycle, and publication-proof status	Candidates outside the boundary are excluded
Result output	DID, version, source, status, and proof summary	Ranking does not change the trust conclusion; results are reviewable

D.5 Governance Authorization Tests

Governance tests must treat node authorization, resource control, and resource publication results as three independent judgments. A node may perform actions allowed by its role only when its node DID Document, authorization credential, authorization domain, protocol version, and lifecycle status are consistent. The resource owner's control proof establishes only the right to operate the resource and cannot replace node authorization. Use `active`, `suspended`, `revoked`, `unknown`, and non-canonical case variants separately, and verify that sensitive operations reject or fail closed when the status cannot be confirmed or has become invalid. For authorization domains, cover the empty set, wildcard, parent and child domains, sibling domains, duplicates, case variations, and unknown domains, confirming that a tag tree or resource description does not enlarge the node's authorization scope.

The `governance`, `node`, `domains`, and `lifecycle` fixtures provide stable inputs for offline judgment. The source, event sequence, or projection time in the governance snapshot must be saved together with the test report. Projection latency of the off-chain trust indexer, on-chain finality, and real revocation propagation belong to the running integration scope and cannot be claimed complete based only on snapshot tests.

Governance authorization tests should preferably use a closed-loop assertion of “authorization fact -> node action -> resource visibility.” For a revoked Discovery node, do not merely verify that the authorization snapshot status is `revoked`; also verify that its synchronization or query action is

rejected, that its existing authorization scope does not continue to expand, and that already indexed resources do not become valid results again because of an old cache. For an authorization-domain change, check both the new and old domains to avoid validating only the added domain while missing whether the old permission was correctly withdrawn.

D.6 Version Update and Lifecycle Tests

Lifecycle tests center on state transitions and version relationships rather than using timestamp order as the sole basis. Establish an active resource, publish a new version with a valid proof from the original controller, and then verify the relationship among the current version, historical versions, and same-DID query results. Next apply suspension, revocation, and restoration facts separately, and check whether the statuses seen by the Registrar, Root platform, content distribution platform, Discovery node, and caller change within their allowed propagation boundaries. Updates without controller authorization, version rollback, same-version content conflicts, and restoration without a basis should all be rejected. Discovery queries should return only the available current version by default. If the interface supports historical queries, it must state the status of historical records clearly and must not allow an old version to bypass current governance status and become callable again.

The existing `lifecycle` fixture can validate the normative form of status values and active/inactive semantics. Same-DID updates, real historical persistence, and cross-node convergence should use service integration tests, retaining the DID Document hash, ResourcePackage hash, version, control proof, publication proof, and index time for every submission.

Records for a same-DID update should contain at least the following relationships:

```
same resource DID
-> controller proof for the update
-> strictly selected package version
-> new DID document/package hashes
-> Root publication evidence
-> Discovery current-version result
```

Even when only the description or endpoint is updated, recompute the affected DID Document hash, metadata hash, or ResourcePackage hash and validate the corresponding proof; incrementing the version number cannot replace content binding. Historical-query tests should also confirm that historical versions are traceable but do not bypass the current suspension, revocation, or authorization status.

D.7 Security and Abuse-Prevention Tests

Security tests should validate “rejection with no side effects.” Fixed signature samples confirm that a valid Ed25519 signature can be verified and that the same payload after tampering cannot pass. At the service level, also replace the request nonce, request ID, audience, path, protocol version, and signer, and check replay, cross-interface reuse, and unauthorized calls. Request size, query limit, text length, invalid JSON, incorrect Content-Type, malicious endpoints, and abnormal authorization domains should be limited before expensive semantic retrieval, network requests, or

persistence are entered. Rate-limit and timeout tests should record response status, wait time, resource consumption, and recovery, and must not use unlimited retries to hide rejection.

Sensitive-evidence tests must ensure that private keys, access tokens, database connection strings, internal file paths, and unredacted control materials do not enter reports, public website interfaces, or ordinary node responses. `security/sensitive-evidence.json` and report-redaction checks are the currently reproducible baseline. Real services should additionally sample application logs, reverse-proxy logs, and error responses to confirm that exceptional branches do not disclose internal stacks or secrets.

Side-effect checks in security tests should cover three locations: input buffers or temporary files before request processing, server-side persistent records, and downstream publication or index queues. For oversized requests, malicious endpoints, and invalid signatures, a passing result should include request limiting, no resource-record write, no node-to-node publication call, and no public error stack. For rate-limit tests, distinguish “the service rejects the request” from “the service process is unavailable”: the former is a policy result, while the latter is a stability defect.

D.8 Third-Party Node Admission Tests

Third-party node tests begin with node identity consistency, requiring no contradiction among configuration, DID Document, status response, authorization credential, role, endpoint, protocol version, lifecycle, and authorization domain. After admission, verify separately whether a Registrar node can accept and submit within its authorization domain, and whether a Discovery node can synchronize and query only Root platform publication content within its authorized scope. Also verify that invalid nodes, out-of-domain resources, protocol-version mismatches, and revoked status do not continue to receive permission because of configuration caches or historical credentials. Logging and failure-recovery tests should check that the request ID, cursor, rejection reason, and retry record are sufficient to reconstruct the event without containing private keys or internal secrets.

The current suite's `node-identity.*`, `authorization-domain`, `governance-snapshot`, and `discovery-visibility` fixtures are suitable as offline admission gates for third-party nodes. Real node admission must additionally use isolated endpoints and an independent database; test nodes must not be connected directly to the official production data plane. The report should state the node under test, authorization domain, configuration version, and revocation/restoration times.

The admission conclusion for a third-party node should be split into five observation areas: identity, protocol, authorization, business, and operations. Consistent identity materials establish “who the node is”; successful protocol calls establish “whether the interface can communicate”; only a passing authorization and business loop establishes “whether the node can complete registration, synchronization, or discovery within its authorization scope”; logging, rate-limiting, and recovery evidence are used to judge whether the node can operate continuously. Any failed observation area should retain its specific failure reason in the report and must not be offset by another area passing.

D.9 DID Resolution and Method Conformance Tests

Resolution tests should validate the method prefix, semantic code, subject-type mapping, identifier character set, and normalization rules of `did:oon`; case correction, string parseability, or a server

returning JSON must not be treated as proof of control. DID Document tests must check consistency among `id`, controllers, public keys, authentication/assertion references, service endpoints, resource type, and resource metadata, and must provide deterministic failure for an incorrect method, incorrect segment count, invalid characters, unknown subject code, or incorrect service reference. If the implementation provides historical-version resolution, also record the version, hash, and status used for resolution, distinguishing the current resolution result from a historical record.

Test evidence should contain identifiers before and after normalization, parsed fields, the verification method, and the error reason. Private keys may be used for signing at test runtime but must not be written to fixture reports.

Method conformance tests should also distinguish the syntax, structure, and control layers: the syntax layer confirms that the string has the `did:oan` form; the structure layer confirms that the DID Document's `id`, verification methods, and service references are mutually consistent; the control layer requires generating a proof with the corresponding private key and verifying it with the public key. Passing only the first two layers cannot establish that a particular subject controls the resource. Fixed suffixes used in test vectors can reproduce resolution results but should not be interpreted as key-derivation results in a production environment.

D.10 API Compatibility and Error-Contract Tests

API tests should check successful and failed contracts at the same time. For each interface, record at least the method, path, Content-Type, required fields, handling of unknown fields, status code, error-body structure, error reason, pagination fields, and retry guidance. A client must not treat any 2xx response as business success, nor automatically interpret a network timeout as a resource rejection. For compatibility versions, distinguish ignorable extension fields, protocol versions that must be rejected, and differences in field order or JSON representation that must not change semantics. Retries apply only to explicitly retryable transport or temporary dependency failures. Requests carrying signatures and write side effects must use request ID, nonce, or server-side idempotency rules to prevent duplicate submission.

`oan-protocol-common/docs/error-codes.md`, protocol types, and the `requests` fixtures in the test suite are the main sources for checking the error contract. The current fixtures cover an empty request body, invalid method, malformed JSON, missing fields, oversized request, incorrect Content-Type, and incorrect query type. When a service adds an error code, it should update the common error documentation, client mapping, and expected fixture together.

Compatibility tests should assert request-delivery results and business-semantic results separately. An unknown ignorable field should not change the meaning of existing fields; an unknown protocol version should produce a clear incompatibility result; a missing required field should fail before business write; and a network timeout can be recorded only as an unknown transport result, not directly judged as registration failure. For interfaces with write side effects, retry tests must use a fixed `requestId`, nonce, or equivalent idempotency condition, and check whether the server generated duplicate VCs, duplicate versions, or duplicate publication tasks.

Scenario	Expected test conclusion	Evidence to save
Valid request and known version	Business processing succeeds	Status code, response summary, and resource status
Add an unknown extension field	Process according to interface compatibility rules	Original request and field-processing result
Missing required field	Controlled rejection with no write side effect	Error code and before/after storage counts
Retry after network timeout	No duplicate business fact is created	Two request identifiers, responses, and final record

D.11 Freshness, Convergence, and Recovery Tests

Freshness tests should record observation time, fact-production time, synchronization cursor, index-update time, and permitted maximum delay separately. The test flow may first publish a resource with a unique version and hash, confirm Root platform acceptance, observe the states of the content distribution platform, Discovery node, and off-chain trust indexer, and finally compare whether all layers converge on the same DID, version, status, and proof. After deliberately interrupting the network, restarting a service, deleting temporary transfer state, or injecting duplicate events, recovery must continue from a verifiable cursor or a full rebuild; it must not infer missing facts from “the latest response.” An old snapshot may be used for non-authoritative display or limited degradation, but authorization, revocation, and new-publication decisions must follow explicit freshness boundaries.

The report should list each layer's `observedAt`, `source`, `cursor`, `version`, `lifecycleState`, validation result, and latency at a minimum. For results exceeding the threshold, mark whether the outcome blocks sensitive operations, degrades to read-only, or only produces an operational alert. The current V1-alpha suite does not provide a complete real-network convergence evaluation, so this part should be published separately from integration reports for the Root, CDN, Discovery node, and off-chain trust indexer.

Convergence tests should use a traceable resource version as the correlation key rather than comparing only the number of resources returned by each interface. The following minimum evidence chain may be retained in the report:

```
{
  "resourceDid": "did:oan:SKFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "version": "1.0.1",
  "rootPublishedAt": "2026-09-07T01:02:03Z",
  "cdnObservedAt": "2026-09-07T01:02:05Z",
  "discoveryObservedAt": "2026-09-07T01:02:07Z",
  "trustIndexObservedAt": "2026-09-07T01:02:08Z",
  "didDocumentHash": "sha256:example",
  "packageHash": "sha256:example",
  "converged": true
}
```

The times, DID, and digests in the example are for illustrating report fields only. Actual acceptance should judge `converged` according to the threshold configured for the test environment,

while retaining raw responses, log correlation identifiers, and cursors from every layer. Only then can propagation delay, index failure, lagging governance state, and version-binding errors be distinguished.

D.12 References

Source	Type	Link
oan-protocol-common	Code repository: test data structures, schemas, and error contract	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	Code repository: Root platform validation and publication test objects	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	Code repository: registration protocol and validation test objects	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	Code repository: discovery, indexing, and semantic-query test objects	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	Code repository: governance-state and synchronization test objects	https://github.com/OpenAgenet/oan-trust-indexer
OAN Yellow Paper	arXiv yellow paper: security properties and verification requirements	https://arxiv.org/abs/2606.03163
Efficient Agent Discovery Profile	IETF draft: testing directions for the discovery profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

Appendix E Reference Deployments and Examples

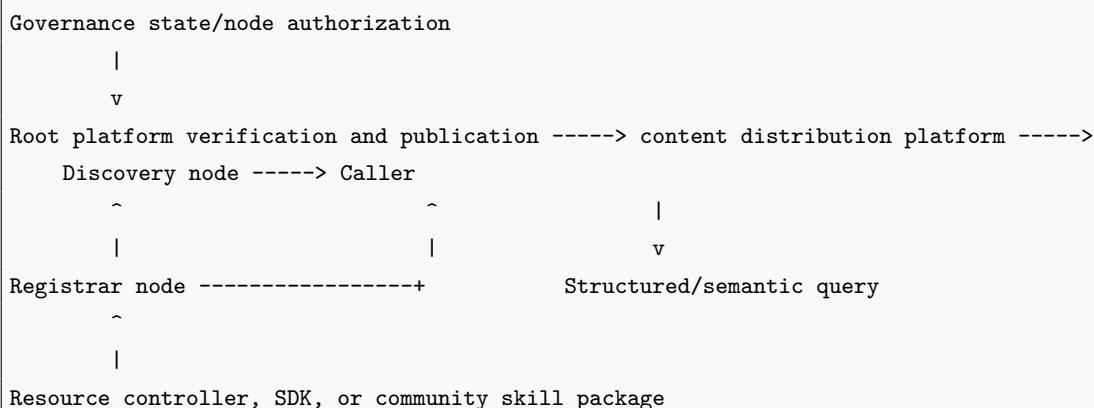
This appendix provides reference paths that can be used for development, verification, and operations handover. The specification requires attention to role boundaries, evidence binding, state propagation, and interface semantics; the reference implementation uses Rust nodes, a TypeScript SDK, community skill packages, PostgreSQL/NATS, and a website gateway; specific domain names, ports, keys, database URLs, model directories, and system service names are all environment configuration. Deployment personnel should first confirm the trust root, authorized domains, node DIDs, and protocol version of the target network before writing configuration, and should not copy private keys, tokens, or online addresses from the examples. Server-side build artifacts, logs, caches, and database data are kept separate from the source archive, and secrets are injected only through protected runtime configuration.

When synchronizing versioned source archives, the archived content should come from a confirmed commit or release version, rather than unreviewed files in the working tree. The following command represents one general method for creating a source archive:

```
git archive --format=tar --prefix=release/ <COMMIT_OR_TAG> | gzip -c > <RELEASE_ARCHIVE>.tar.gz
```

The archive represents source deliverables only; build directories, dependency caches, database files, runtime keys, and logs should not be mixed into the source package. Source code, build artifacts, configuration, runtime data, and website static files should be stored separately in controlled directories defined by the deployment environment, with their versions and digests associated through release records. Directory names, filesystem layout, and transfer methods are environment implementation details, not protocol requirements.

The recommended general startup chain is as follows:



The startup order should first satisfy dependencies such as the database, message system, and off-chain trust indexer, then start the Root platform, Registrar node, content distribution components, and Discovery node, and finally start the website backend, static frontend, and reverse proxy. During shutdown or upgrade, new writes should be stopped first, and the publication and distribution queues should be allowed to reach an acceptable state before the program is replaced, while the database, cursors, version records, and redacted logs are retained. If a dependency is temporarily unavailable, the service should perform limited retries and backoff according to its own configuration, and report `unknown`, `unavailable`, or `pending` synchronization externally, rather than generating false publication or authorization facts.

E.1 Official Network Deployment

The reference layering of the official network is as follows: the Registrar node handles resource onboarding, the Root platform performs network-level verification, publication, and coordination, the content distribution platform transfers Root platform-verified `ResourcePackages` and proofs, the Discovery node establishes structured and semantic indexes within the authorized domain, the off-chain trust indexer projects governance events into queryable state, and the website backend provides the frontend with display and user-operation entry points through a public gateway. The website is not a trust authority; a website failure should not change the protocol state of a resource or node.

Production deployment should verify at least the following chain:

1. The database, message system, and listening addresses of all nodes are ready, and the public Internet exposes only the public HTTPS entry protected by the reverse proxy.
2. The Root platform, Registrar node, Discovery node, content distribution platform, and off-chain trust indexer each return an interpretable status through `/health` or their role-status interface.
3. The Registrar node can complete controlled registration-request validation and submission to the Root platform; registration smoke tests should use dedicated test resources or controlled failure inputs and must not pollute the production resource directory.
4. Root platform publication results can be consumed by the content distribution platform, validated and indexed by the Discovery node, and then observed through exact and semantic queries.
5. The website frontend's `VITE_*` endpoints point to the current public gateway, and build artifacts contain no residual `localhost` or `127.0.0.1`; after website deployment, perform smoke tests for registration, discovery, Network, routing, and the browser console.

Deployment records should retain the build version, configuration summary, service-switch time, health-check results, registration and discovery smoke-test results, index cursor, and failed-retry information. Public domains, node addresses, and certificates belong to the target environment configuration; when migrating to another environment, endpoints and certificates authorized by that environment must be used.

Each service should be released through a reversible replacement order: first verify the new binary and static artifacts, then switch the corresponding service through the target environment's service manager, and finally check routing through the public entry point. Website static files may first be uploaded to a controlled temporary release directory; after checking the file count, entry file, and endpoint references, switch to the current version. For node services, retain the old binary, configuration, and runtime state before replacing the new version. During rollback, restore only a verified old version; do not reset the database, publication cursor, or governance state.

Layer	Typical observations	Handling boundary on failure
Process layer	Service-manager status, listening ports, and startup logs	Fix the configuration or binary and restart; process liveness is not business availability
Interface layer	<code>/health</code> , node status, and public-route status codes	Distinguish reverse-proxy, upstream-service, and request-format problems
Business layer	Registration acceptance, Root platform publication, and discovery queries	Use controlled test data; block publication or roll back on failure
Data layer	Database connection, cursor, index statistics, and version records	Retain the scene and backups first; do not directly delete or rebuild business data

E.2 Private and Alliance Network Deployment

Private or alliance networks may deploy the Root platform, Registrar node, Discovery node, and content distribution platform within an organization's internal or isolated network, but they should retain the same protocol responsibilities: the Root platform is the hub for network-level verification, trust authorization, data distribution, and semantic governance; the Registrar node is responsible for resource onboarding; and the Discovery node establishes indexes only for published materials. Network isolation changes reachability, not the rules for judging DIDs, control proofs, Root proofs, authorized domains, or lifecycle states.

Typical configuration boundaries are as follows:

Configuration item	Handling in private/alliance networks	Facts that must not change
Trust root	Use the root DID, root keys, and governance process agreed by the organization or alliance, and inject them through secure configuration.	Node authorization must be verifiable; it cannot rely only on network segments or IP allowlists.
Resource visibility	Control the visible scope through authorized domains, Discovery node partitions, and the reverse proxy.	Visibility filtering must not disguise an out-of-domain resource as an in-domain resource.
Data distribution	An internal CDN, object storage, or node-to-node synchronization may be used.	The recipient must still verify the hash, version, and Root proof.

Configuration item	Handling in private/alliance networks	Facts that must not change
Operations boundary	Place the database, off-chain trust indexer, and management interfaces in a controlled network.	An ordinary caller must not obtain private keys or governance write access merely because the network is reachable.

When an alliance member joins, it should first submit its node identity, role, endpoint, authorized domain, and protocol-version materials, and then obtain authorization from the governance authority; revocation or suspension should be able to propagate to the Root platform and Discovery node. During integration, use an independent database and test domain to avoid accidentally synchronizing private-network test resources to the official public network.

The boundary of an alliance network should also appear in operations accounts and data backups: node operators may manage processes, configuration, and logs, but should not read the resource controller's private key from application configuration; resource publishers submit only public DID Documents, proofs, and ResourcePackages and do not gain governance write access because the network is internal. During cross-member integration, record each member's node DID, authorized domain, configuration version, and revocation status; after testing, clean up temporary credentials and test-resource indexes while retaining necessary audit summaries.

E.3 Minimal Registrar Node Deployment

The minimal Registrar node needs only its own DID Document and runtime key, resource-record storage, Root platform address, Root platform DID or audience, listening address, CORS/network policy, and necessary database configuration. The reference Rust implementation provides `/health`, `/registrar/did`, `/resources/register`, a compatible submission entry point, registration status, resource records, the capability-tag tree, tag suggestions, and authorization queries; the specific available interfaces should be determined by the source and configuration of the target version. The registration flow is: receive a ResourcePackage, check the DID and resource type, metadata, authorized domain, control proof, and hash, generate a signed request for the Root platform, wait for Root platform verification and publication, then save the registration record and return the registration credential and Root platform response.

Minimum verification should include:

```
curl -fsS "${REGISTRAR_BASE_URL}/health"
curl -fsS "${REGISTRAR_BASE_URL}/registrar/did"
curl -X POST "${REGISTRAR_BASE_URL}/resources/register" \
  -H 'content-type: application/json' \
  --data-binary @registration-request.json
```

`${REGISTRAR_BASE_URL}` is an example variable injected by the deployment environment, not a fixed address or port requirement. The Registrar node must not receive the resource controller's private key; the private key is used in the resource controller's SDK, browser, or local Skill, while the node receives only the public key, signature, and public proof. When the Root platform is

unreachable, the node must return a distinguishable upstream failure and must not display local storage or acceptance as Root platform publication.

E.4 Minimal Discovery Node Deployment

The minimal Discovery node needs its own identity and authorization materials, a synchronization endpoint for the Root platform or content distribution platform, authorized domains, a database, index configuration, and a listening address. It first obtains or receives Root platform publication materials, verifies the Root proof, ResourcePackage hash, DID Document, version, and lifecycle, and then writes them into the local structured index; semantic retrieval may use the tag tree, text fields, and vector index on this basis, but must not skip publication and authorization verification. The reference implementation provides role interfaces such as `/discovery/did`, `/discovery/resources/query`, query explanations, query suggestions, the capability-tag tree, synchronization history, and index statistics.

The criterion for completed initial synchronization is not “the HTTP request succeeded,” but rather: the synchronization cursor has been recorded, the ResourcePackage's source and hash checks have passed, resources within the authorized domain have entered the index, out-of-domain, suspended, revoked, or protocol-mismatched resources have been filtered, and index statistics are consistent with the synchronization record. It is recommended to use a fixed resource set to perform the following query:

```
{
  "query": "Find an MCP server for security audit",
  "resourceType": "mcp_server",
  "limit": 10
}
```

Returned results should distinguish matching sources such as text, tags, structured conditions, and semantic relevance, and retain the resource DID, version, status, source, and freshness fields. If the Root platform or content distribution platform is temporarily unavailable, a known old snapshot may be displayed, but it must not be used to produce a new trusted publication or authorization conclusion.

The synchronization status of the Discovery node should be distinguished according to the following table for shared use by the Network page, operations interfaces, and alerting rules:

Status	Meaning	Permitted action
ready	The cursor is advancing, the index is queryable, and the most recent verification succeeded	Allow normal queries
syncing	Full or incremental synchronization is in progress	Query confirmed data and mark the synchronization status
stale	An old index remains, but it has exceeded the freshness boundary configured for the environment	Degrade the display or block sensitive invocations

Status	Meaning	Permitted action
unknown	The upstream, cursor, or governance status cannot be confirmed	Must not be used to confirm that a resource is trusted or callable

These statuses are observations at the operations and presentation layers, not substitutes for resource lifecycle states. The specific freshness threshold should be provided by the deployment environment and operations policy; it cannot be inferred as a fixed protocol delay from one query response.

The minimal Registrar node configuration can be abstracted as the following environment-configuration example; all values are placeholders and cannot be used directly in production:

```
REGISTRAR_LISTEN_ADDR=<LISTEN_ADDRESS>
ROOT_ENDPOINT=<AUTHORIZED_ROOT_ENDPOINT>
DATABASE_URL=<DATABASE_CONNECTION_REFERENCE>
NODE_DID=<NODE_DID>
AUTHORIZED_DOMAINS=<AUTHORIZED_DOMAIN_SET>
```

After startup, at least three results should be recorded separately: “node reachable,” “request validated locally,” and “request submitted to the Root platform.” The latter two must not be ambiguously represented by the same HTTP 200 status code; if the Root platform times out or returns an unknown result, the Registrar node should retain retryable request-correlation information and mark the result as unknown or pending.

E.5 SDK Registration Example

The SDK example should show that the private key is used only locally by the caller, that the DID Document is bound to the ResourcePackage, that the registration request is accepted by the designated Registrar node, and that the registration result requires continued observation of publication and discovery status. The following code shows the interface relationship; endpoints, domain names, and resource content must be replaced according to the environment:

```
import { createAgentIdentity, createRegistrationSubmissionFromIdentity } from "@openagenet/oan-sdk-ts";
import { OanClient } from "@openagenet/oan-sdk-ts/client";

const identity = await createAgentIdentity(
  "Repository structure summarizer",
  "skill",
  undefined,
  {
    domainCode: "DM",
    authorizedDomains: ["technology.software_engineering"],
    manifestUrl: "https://example.invalid/skill/manifest.json",
    description: "Searches code repositories and summarizes project structure.",
    capabilityTags: ["code.repository", "analysis.summary"],
  },
);
```

```

);

const submission = createRegistrationSubmissionFromIdentity(identity, {
  endpoint: "https://example.invalid/skill/manifest.json",
  packageVersion: "1.0.0",
  packageHash: "sha256:replace-with-real-hash",
  metadataHash: "sha256:replace-with-real-hash",
});

const client = new OanClient({
  registrarEndpoint: process.env.OAN_REGISTRAR_ENDPOINT!,
});

const result = await client.registerResource(submission);
console.log(result.status, result.resourceDid, result.registrationCredential);

```

This is an SDK flow illustration. Before real publication, actual package hashes, a complete resource description, an accessible endpoint, and an authorized domain accepted by the Registrar must be used; `example.invalid` or placeholder hashes cannot be used as publishable materials. The VC, Root platform response, and resource DID returned by the SDK should be saved by the caller and continuously verified according to status; the SDK does not hold private keys for the caller or automatically grant business invocation permissions.

When processing registration results, network errors, Registrar rejection, Registrar acceptance with an unknown Root platform result, and Root platform publication should be recorded separately. The example code shows only result branches and does not treat a particular field of `result` as a fixed field present in every version:

```

try {
  const result = await client.registerResource(submission);
  console.log("registrar response received", result);
  // The caller continues to read and verify the VC, Root platform publication status, and
  // discovery visibility.
} catch (error) {
  console.error("registration transport or client error", error);
  // Do not automatically generate a new DID or resubmit unconditionally when the result cannot
  // be confirmed.
}

```

Real integration should securely retain the request-correlation identifier, resource DID, and Local Identity backup; error logs may contain only redacted summaries and must not output signing private keys, the complete Local Identity backup, or database connection strings.

E.6 DID and Semantic Discovery Examples

DID exact queries are used to confirm whether a known resource is visible in the authorized view of a specified Discovery node; semantic queries are used to find candidate resources from a task description. Both should return the candidate resource's DID, resource type, version, status,

capability tags, authorized domain, endpoint or endpoint reference, source, and verification summary. A caller must not skip DID Document, Root proof, VC, lifecycle, or endpoint verification because of a high semantic-match score, a tag hit, or a recommendation displayed by the website.

```
const exact = await client.discoverResources({
  resourceDid: "did:oon:SKDM:replace-with-a-real-identifier",
  limit: 1,
});

const semantic = await client.suggestDiscoveryQuery({
  query: "Find an MCP server for security audit",
});

const candidates = await client.discoverResources({
  query: "Find an MCP server for security audit",
  resourceType: "mcp_server",
  capabilityTags: semantic.capabilityTags,
  limit: 10,
});
```

If the implementation supports query explanations, the caller should further read the explain result to describe how the query was converted into structured conditions, tags, or semantic conditions, and why candidates were retained or filtered. When there is no match, return a stable empty result; overly short queries, out-of-domain conditions, or invalid filter conditions should return controlled errors. Pagination and deduplication should follow DID and version rules; multiple indexed copies of the same resource must not be displayed as multiple independent resources.

Exact and semantic queries may use the same discovery entry point, but their acceptance assertions differ: exact queries focus on uniqueness, complete DID matching, and version selection; semantic queries focus on candidate sources, filtering reasons, and ranking explanations. A minimal manual acceptance record may be written as follows:

```
queryType=semantic
query=Find an MCP server for security audit
resourceDid=did:oon:MC:example-identifier
matchedBy=capabilityTags,text
governanceState=active
source=authorized-discovery-index
verification=root-proof-and-package-hash
```

Here, `example-identifier` is only a placeholder. If the result contains only text similarity and no verifiable source, governance state, or proof summary, it should be treated as a candidate recommendation and not directly as a trusted callable object.

E.7 Resource Verification Example

After receiving a discovery result, the caller should verify it in the order of “identity, binding, source, governance, endpoint”: first parse `did:oon` and check the public keys, controllers, and service references in the DID Document; then check the resource DID, type, version, DID Document hash, metadata hash, and package hash in the `ResourcePackage`; next verify the Root proof and its

lifecycle status; then check the issuer, subject, authorized domain, hash bindings, and proof of the registration VC; finally verify the endpoint, protocol, certificate, and invocation permissions according to business policy. Any failure should block high-risk invocation and retain a redacted failure reason.

```
import { verifyResourcePackageShape, assertUsableLifecycle } from "@openagenet/oan-sdk-ts";

verifyResourcePackageShape(resourcePackage);
assertUsableLifecycle(resourcePackage);
// Continue with the caller's own VC, node-authorization, and endpoint-verification policies.
```

The SDK's structural validation functions cannot replace governance-state reads, network-freshness judgments, or business authorization. Verification results should record the version, source URL, hash, issuer, status time, and verification-component version on which they rely; private keys, tokens, and complete sensitive requests must not be recorded.

E.8 Operations Smoke-Test Example

Post-deployment smoke tests should run in dependency order and record “service reachable,” “protocol action succeeded,” and “state converged” separately. The recommended check sequence is: health interfaces and ports; database connection and migration status; Root platform status and governance/authorization reads; Registrar node health and controlled registration request; content distribution queue and publication status; Discovery node synchronization cursor, index statistics, exact query, and semantic query; latest cursor of the off-chain trust indexer; website backend health, frontend routes, registration/discovery transport, and browser console. The deployment process should be considered complete only after the public endpoint, registration transport, discovery query, Network summary, and browser-route checks pass.

A smoke-test record may use the following structure:

```
{
  "deploymentId": "2026-09-07T12:00:00Z-<commit>",
  "environment": "<environment>",
  "checks": [
    {"name": "root-health", "status": "pass", "observedAt": "..."},
    {"name": "registration-transport", "status": "pass", "resourceDid": "..."},
    {"name": "discovery-query", "status": "pass", "candidateCount": 1},
    {"name": "trust-indexer-freshness", "status": "unknown", "reason": "..."}
  ]
}
```

The DIDs, times, and statuses in the example illustrate a record format. When a smoke test fails, first determine whether the cause is network transport, build artifacts, configuration, database, governance state, synchronization delay, or a functional error, and then decide whether to retry, roll back, or block public release; do not obtain superficial success by deleting logs, resetting cursors, or skipping registration/discovery checks.

E.9 Same-DID Update Example

A same-DID update must be generated by the original resource controller or an explicitly authorized controller using a new control proof; the DID itself remains unchanged, while updated content in the ResourcePackage or DID Document forms a new version and hash. The process is: read the Local Identity and current version, modify metadata or the ResourcePackage, increment the version according to the applicable versioning scheme, recalculate the metadata and package hashes, sign the update materials, submit them to the Registrar node, have the Root platform re-verify them and form a new Root proof, and then synchronize through the content distribution platform and Discovery node. The Discovery node responds with the latest available version by default, but must retain auditable version and status information.

Update tests should cover a missing control proof, a control proof corresponding to old materials, version rollback, same-version content conflicts, hash inconsistency after an update, and republication of an old version. A failed update must not overwrite the current valid version, and a Discovery node must not treat a VC returned by the Registrar node as a new version; the registration credential, Root proof, and discovery status should be recorded separately.

The comparison record after an update is complete should contain at least the old and new resource DIDs, DID Document hashes, ResourcePackage hashes, control-proof summary, Root proof summary, discovery-index version, and query time. If the content has not changed, the client may reuse an existing preview or idempotent result; if any field affecting a binding relationship changes, the hashes must be recalculated and re-verified, and the display-only version string must not be changed by itself.

E.10 Resource Suspension and Recovery Example

Suspension, revocation, and recovery are lifecycle facts; they are not equivalent to deleting a DID or erasing history. The governance authority or resource controller generates a state change through the permitted governance process, the Root platform and off-chain trust indexer read and project the fact, the Discovery node filters unavailable resources after synchronization, and the caller rechecks the status during verification. Suspension usually means temporarily unavailable, while revocation means that the current credential or resource status is no longer available; recovery must have a clear basis and the Discovery node must not independently change the resource back to active.

```
State-change fact -> governance event/authorization materials -> Root platform and indexer read
                  -> Discovery node updates filtering state -> caller re-verifies before
                  deciding whether to invoke
```

Audit records should include the resource DID, original status, new status, source of the fact, event or request identifier, observation time, propagation time, and execution result. When governance status has not synchronized or has exceeded the permitted freshness, sensitive invocation should fail closed; the website or an old snapshot may display “pending synchronization,” but must not display the resource as currently callable.

E.11 Cross-Protocol A2A and MCP Mapping Example

The goal of cross-protocol mapping is to reduce the cost for existing Agent platforms, MCP clients, and tool callers to integrate with OpenAgent (OAN), not to force objects from different protocols to be treated as the same object. During mapping, the names, capability tags, use cases, inputs, outputs, service endpoints, protocol bindings, and versions in the OAN DID Document and resource description may be mapped to an A2A Agent Card, MCP service metadata, or tool API description; during reverse conversion, the original DID, original fields, version, source, and hash must be retained. An A2A Agent Card or MCP metadata describes capabilities and connection methods, but does not replace OAN control proofs, VCs, Root proofs, governance status, or authorized domains.

OAN DID Document/ResourcePackage	
-	id, resourceType, capabilityTags, useCases
-	protocolBindings, service endpoint, version, hashes
+++>	A2A Agent Card: identity reference, skills, interfaces
+++>	MCP metadata: server capabilities, tools, transport
+++>	Tool/API description: operation, schema, endpoint

The mapper should make the source and conversion direction of each field explicit: capability names may be used for display and retrieval, DIDs and hashes are used for identity and integrity traceability, signatures and VCs are retained as independent verification materials, and endpoint mapping represents only a connection entry point and does not automatically authorize invocation. If the target protocol has no equivalent field, use an extension field or an additional reference, and indicate lost information in the result; semantic similarity, protocol compatibility, or successful metadata conversion must not be interpreted as proof that the resource is trusted or usable for business purposes.

The mapping relationship below only describes how information may be carried; it does not require A2A Agent Cards, MCP metadata, or tool API descriptions to use OAN field names:

OAN information	Possible location in the external description	Retention requirement
Resource DID, DID Document source	Identity reference or extension field	Retain the original DID and source; do not rewrite it as an ordinary name
Capability tags, use cases, and description	Agent Card skills, MCP capabilities, or tool description	May be used for display and retrieval, but cannot replace proofs
Protocol bindings, endpoints, and inputs/outputs	Interface, transport, and schema descriptions	Retain the version and source; endpoint availability requires separate verification
VC, hashes, and Root proof	Verification attachment or reference in the external object	Do not delete or overwrite the original proofs because conversion succeeded

If a mapped object re-enters the OAN registration or discovery flow, DID, hash, control-proof, authorized-domain, and lifecycle checks must be performed again; an external protocol object may

serve as onboarding material, but cannot independently become an OAN publication fact.

E.12 References

Source	Type	Link
<code>oan-root-services</code>	Code repository: Root platform deployment and publication examples	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: Registrar node deployment and registration examples	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	Code repository: Discovery node deployment, synchronization, and query examples	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-trust-indexer</code>	Code repository: trust indexer deployment and governance synchronization examples	https://github.com/OpenAgenet/oan-trust-indexer
<code>oan-sdk-ts</code>	Code repository: client integration examples	https://github.com/OpenAgenet/oan-sdk-ts
<code>oan-community-skill</code>	Code repository: community registration and discovery operation examples	https://github.com/OpenAgenet/oan-community-skill
OAN Yellow Paper	arXiv yellow paper: reference deployment and end-to-end mechanisms	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF draft: boundaries of resource identity and discovery examples	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

Appendix F Repository and Implementation Mapping

This appendix describes implementation through the chain “specification clause -> data structure/interface -> implementation file -> test evidence -> deployment configuration -> maintenance responsibility”, rather than treating the repository name itself as proof of functionality. The current core runtime implementation consists of the shared protocol library, Root platform, Registrar node, Discovery node, content distribution platform, and off-chain trust indexer; the SDK, community skill package, and official website are client and public entry points; the third-party node admission test kit provides independent evaluation support and is not a production runtime node. `oan-onboard-tests` and the root-level `genesis` are not independently verifiable directories in the current workspace. Therefore, historical references involving them can only be treated as source-material conventions and cannot serve as evidence of the current implementation.

Mapping statuses have the following meanings:

Status	Meaning
Implemented	The corresponding logic can be located in the current code, tests, or configuration, and can be reproduced or checked through the specified method.
Partially implemented	Data structures, interfaces, or partial workflows exist, but end-to-end, production-grade, or complete failure-scenario coverage is still missing.
Evaluation support	Provided by fixed fixtures, report generators, or verification scripts; does not mean that the runtime node has implemented the capability.
Reference / to be verified	Required by the specification or proposed by the deployment design, but the current workspace lacks sufficient code or test evidence for confirmation.

Each implementation fact should record both the source version and the verification time. For inputs, explain how they are converted into validation conditions, authorization-domain filters, or query conditions; for outputs, explain whether they are raw facts, verification results, cached projections, search rankings, or display summaries; for Root proof, VCs, signatures, and governance status, the issuer, verifier, bound content, and status source must be retained. Capability labels, semantic relevance, A2A Agent Card, or MCP metadata only describe or assist capability discovery and cannot replace control and governance verification. Mapping records must distinguish at least three types of evidence: source-code evidence explains where the logic and fields are defined, test

evidence explains what result the logic produces for fixed inputs, and runtime evidence explains what the service, configuration, and dependencies can actually complete in the specified environment. They answer “what was written”, “what was tested”, and “what was run”, respectively, and cannot replace one another with a README, a single successful compilation, or one health endpoint. For dynamic data, also record the commit, configuration summary, database/index state, and request correlation identifier at the time the evidence was generated.

F.1 oan-protocol-common

This repository provides shared Rust workspace-level foundations and primarily maps to Chapters 5 through 9, Chapter 16, and Appendices A through C. `crates/oan-core` carries DID Document, resource types, and core domain objects; `oan-crypto` handles cryptographic suites, keys, and hash-related logic; `oan-credentials` handles credential construction or verification; `oan-package` handles `ResourcePackage` and its bindings; `oan-protocol` handles cross-node requests, responses, and protocol constants; `oan-publication-events` handles publication events; `oan-storage` provides JSON, SQLite, or PostgreSQL storage abstractions; `oan-service-security` provides shared logic for request envelopes, nonce, request ID, and signatures; `oan-semantic-recommender` and its `data` directory provide capability-label trees, authorization-domain classification, and recommendation/normalization assistance.

Mapping object	Main location	Corresponding specification facts	Current verification method and boundary
DID Document and resource types	<code>crates/oan-core/src/lib.rs</code>	<code>did:oan</code> identifiers, subject types, and core fields	Rust unit tests and calls from each node; complete coverage still requires confirmation through specification-pattern and method tests.
Hashes, signatures, and cryptographic suites	<code>crates/oan-crypto/src/lib.rs</code>	Control proofs, request signatures, package/metadata hashes	Valid/tampered-signature fixtures in <code>oan-third-party-node-admission-test-kit</code> ; not equivalent to key custody or a production HSM guarantee.
Credentials and <code>ResourcePackage</code>	<code>crates/oan-credentials</code> , <code>crates/oan-package</code>	Structural bindings for registration VC, <code>ResourcePackage</code> , and Root proof	Subject, hash, and proof consistency checks in registration/admission fixtures; the real path requires service integration verification.
Protocol types and errors	<code>crates/oan-protocol</code> , <code>docs/error-codes.md</code> , <code>schemas/agent-contract</code>	HTTP payloads, protocol versions, error contracts, and external descriptions	Node compilation, client tests, and fixed request fixtures; version compatibility must be retested by release version.

Mapping object	Main location	Corresponding specification facts	Current verification method and boundary
Storage and events	<code>crates/oan-storage</code> , <code>crates/oan-publication-events</code>	Resource records, cursors, publication events, and database boundaries	Tests for each service and local integration; transaction, backup, and recovery strategies cannot be proved by the shared library alone.
Semantic and authorization-domain assistance	<code>crates/oan-semantic-recommender/data/*</code>	Capability-label trees, semantic governance vocabulary, and authorization-domain classification	Recommender tests and Discovery/Registrar consumer tests; recommendation output is not an authorization decision.

The shared library’s maintenance responsibility is to keep data structures, serialization, error codes, and cryptographic bindings consistent across nodes. Any field change should trigger synchronized checks of the Root platform, Registrar node, Discovery node, SDK, community skill package, and fixed fixtures. The shared library does not have authority to decide node authorization, resource publication, or business calls on behalf of upper-layer services.

When tracing downward from an HTTP interface, the contract can be located through the implementation chain: `services/registrar-node/src/main.rs` in the Registrar node binds `/resources/register` and `/resources/submit` to `register_resource`; `services/root-node/src/main.rs` in the Root platform binds `/root/resources/verify-and-publish` to `verify_resource_and_publish`; the Discovery node’s service directory of the same name binds `/discovery/resources/query` to `resource_query`; `services/cdn-node/src/main.rs` in the content distribution platform handles `/cdn/resources`, batch publication, and resource reads; and `src/api.rs` in the off-chain trust indexer handles read interfaces such as `/v1/status` and `/v1/events`. A handler signature is direct evidence for confirming input and output types, while a route table is direct evidence for confirming HTTP methods and paths.

When maintaining interfaces, record “shared DTO changes” separately from “dynamic response-field changes”: the former requires checking `oan-protocol`, `oan-package`, all deserialization callers, and cross-language SDKs; the latter requires checking the handler that generates the JSON, error-code tests, public proxies, and the endpoint table in Appendix B. Successful Rust compilation only proves that the type layer compiles; it does not prove that JSON fields, HTTP paths, authentication boundaries, and asynchronous state semantics still match the external documentation. When locating a shared field, maintainers should verify it in the order “domain structure -> serialization implementation -> callers -> fixture”. For example, first confirm the `ResourcePackage` structure and validation methods in `oan-package`, then check the request/response types in `oan-protocol` and the deserialization paths in the Registrar node and Discovery node, and finally use resource fixtures to verify behavior for missing, tampered, and version-conflicting inputs. The fact that a single node compiles after modification does not show that the cross-repository protocol remains compatible; at minimum, the corresponding SDK exports and the community skill package’s inputs and outputs must also be checked. When a field in the shared library changes, maintainers should first determine whether the change is a serialization-compatibility change, a

validation-semantics change, or a purely internal implementation change. Serialized fields, enum values, error codes, and hash calculation methods belong to the cross-repository impact surface; even if the Rust workspace compiles, node HTTP payloads, TypeScript types, community skill package scripts, and admission-test fixtures must still be checked. If only an internal helper function changes and external payloads and error behavior remain unchanged, the relevant unit tests should be retained as boundary evidence, without expanding the protocol-version scope.

F.2 oan-root-services

This repository corresponds to Chapter 4, Chapters 10 through 11, Chapters 14 through 16, and the related lifecycle, distribution, and operations chapters. The main Root platform service is located at `services/root-node/src/main.rs`. Its routes include the root DID Document, status, Registrar node and Discovery node directories, resource details and versions, capability-label trees, announcements/events, authorization, revocation, and internal interfaces for the content distribution platform and Discovery node. `services/cdn-publisher` consumes publication tasks, prepares ResourcePackage from the Root platform, publishes them to the CDN in batches, and writes back publication results; `services/cdn-node` provides the data-plane service for the content distribution platform. Configuration examples are located in each service directory's `config.example.toml`.

The Root platform implementation boundary can be checked as follows:

Root platform action	Input	Output or persisted fact	Evidence and deviation
Node authorization and status reads	Node DID Document, authorization materials, governance/status data	Authorized-node directory, status, and authorization domain	Root service routes and authorization-handling logic; on-chain finality depends on the off-chain trust indexer and runtime configuration.
Resource verification and publication	Registrar node submission, resource DID Document, ResourcePackage, control proof, and hash	Accept/reject result, resource version, Root proof, or publication task	Root service tests and shared types; service integration tests should confirm that failures do not produce an erroneous published state.
Content distribution coordination	Verified resource version, distribution task, and cursor	CDN publication queue, batch response, and published marker	Queue, batching, and write-back logic in <code>cdn-publisher/src/main.rs</code> ; not equivalent to external reachability of the CDN.
Semantic governance	Capability-label tree, aliases, authorization-domain classification	Semantic materials that Registrar and Discovery nodes can synchronize	<code>oan-semantic-recommender/data</code> and Root platform interfaces; vocabulary evolution requires versioning.

Root platform maintainers must also monitor operational boundaries involving idempotency, version conflicts, publication retries, queue backlogs, database backups, and loss of governance-

status connectivity. The Root platform issues Root proof, which proves only facts within its verification and publication scope; it does not prove the business quality of resource endpoints and does not hold resource control private keys. Root platform code review should record public query routes separately from internal write routes. `/root/resources/{did}`, version queries, and capability-label queries are read paths; Registrar node submissions, node authorization, revocation, content distribution queues, and Discovery node notifications are protected write or coordination paths. When modifying `services/root-node/src/main.rs`, maintainers should also check publication-task consumption in `services/cdn-publisher/src/main.rs`, ResourcePackage reads in `services/cdn-node/src/main.rs`, and the queues, database, and upstream endpoints in Root platform configuration; a Root platform HTTP response alone cannot establish that content distribution has completed. The key verification point for Root platform changes is when the publication fact is created. Maintainers should record the Registrar node submission time, Root platform verification result, resource version and hash, Root proof, content distribution task status, and Discovery node notification status separately. Only after Root platform verification has produced a publication fact do the subsequent distribution and discovery paths have a citable root source. Root platform query interfaces, content distribution platform caches, and Discovery node index entries should be retained as separate evidence.

F.3 oan-registrar-node

This repository primarily corresponds to Chapters 6 through 9, Chapter 16, and the resource lifecycle section. Its implementation entry point is `services/registrar-node/src/main.rs`. Current routes include `/health`, `/registrar/did`, `/resources/register`, the compatible `/resources/submit`, resource lists and details, status, statistics, Root platform authorization, capability-label trees, label suggestions, label normalization, registration-domain directories, and registration recommendations. The configuration example is located at `services/registrar-node/config.example.toml`; data directories, key directories, and optional SQLite/PostgreSQL databases are determined after configuration parsing.

The Registrar node processing chain is:

```
Resource controller submits public materials and control proof
-> validate shape, DID, resource type, metadata, and authorization domain
-> construct a signed request for the Root platform
-> receive the Root platform verification/publication response
-> generate and store the registration VC and registration record
-> return the registration result and Root platform response to the caller
```

The registration VC here is issued by the Registrar node for the registration fact and cannot replace Root proof. When the Root platform rejects a request, the Registrar must not treat local acceptance as network-wide publication. The Registrar node can see public resource materials, public keys, signatures, and credentials, but the normal flow must not obtain the resource control private key. `oan-third-party-node-admission-test-kit/fixtures/v1-alpha/resources` provides fixed verification examples for DID, resource type, authorization domain, VC subject, hash, control challenge, proof integrity, and lifecycle; SDK and official website tests cover client-call shapes but do not replace server-side validation by the Registrar.

Maintenance priorities are to keep the fields of `/resources/register` consistent with the protocol types, make error codes and HTTP statuses explicit, ensure that upstream failures do not create false success, prevent duplicate submissions from producing conflicting records, and update configuration examples, public error documentation, the SDK, and the community skill package together. Registration issue diagnosis should follow request correlation: first confirm whether the request was rejected locally at the input-validation and error-conversion points of `register_resource`; then check whether `ResourceVerifyAndPublishRequest` for the Root platform was constructed correctly; finally compare the registration record, VC, and Root platform response to determine whether the result was only local acceptance. A reusable diagnostic record should retain at least the resource DID, request identifier, version, error code, upstream status, and database-write result; “the page displayed success” must not replace the actual result between nodes. The Registrar node’s implementation mapping should also state which layer each result belongs to: local request validation, registration-record write, submission to the Root platform, registration VC return, and Root platform publication result must not be collapsed into one “registration succeeded” outcome. For retries after network timeouts or incomplete responses, use the original request’s correlation information to check whether the server has already produced a registration fact; only after confirming idempotency conditions and final state can the client decide whether to submit again.

F.4 oan-discovery-node

This repository corresponds to Chapter 4, Chapter 7, Chapters 11 through 13, Chapters 15 through 16, and freshness and lifecycle requirements. `services/discovery-node/src/main.rs` provides interfaces for node DID, status, authorization domain, synchronization history, index statistics, query statistics, resource index, resource query, query suggestions, query explanations, rejection packages, and capability-label trees; `semantic_aliases.v1.json` contains Discovery-side semantic alias materials. `services/embedding-service` provides an optional text-vector service. Its configuration, tests, and real-model smoke files are located in that service directory; enabling a semantic model must not be written as a mandatory dependency for every deployment.

The Discovery node data path should be distinguished into four layers:

1. The synchronization layer receives materials from the Root platform or content distribution platform and records the source, cursor, and synchronization result.
2. The verification layer checks Root proof, `ResourcePackage` hashes, DID Document, version, authorization domain, and lifecycle.
3. The indexing layer writes verified materials into a structured index and builds retrieval indexes by capability labels, text fields, or optional vectors.
4. The query layer converts DID, structured conditions, and natural-language input into conditions and returns candidates with source, version, status, and explanation information.

Implementation point	Code/material	Boundary to preserve
Synchronization and rejection	<code>src/main.rs</code> , configuration, and synchronization-related tests	Unverified, out-of-domain, paused, revoked, or hash-inconsistent materials must not enter the visible index.
Exact and structured queries	<code>/discovery/resources/query</code> and query models	DID exact queries, resource type, protocol, labels, domain, and limit should have stable validation and error contracts.
Semantic retrieval	<code>services/embedding-service</code> , semantic aliases, and recommendation data	Relevance affects only ranking/explanation and cannot replace control, Root platform publication, or governance status.
Deduplication and versions	Resource index and version fields	Multiple copies of the same version of one DID must not be displayed as independent resources; current and historical versions must be distinguishable.
Freshness and degradation	Synchronization status, index statistics, and query responses	Old snapshots may support limited display, but must not create new publication, authorization, or call permissions.

Current fixed fixtures cover in-domain visibility, out-of-domain invisibility, empty results, arbitrary-label matching, and invalid queries. Real synchronization, vector-model quality, concurrent indexing, and cross-node convergence still require confirmation through service integration or deployment tests. Discovery node maintainers are responsible for index consistency, query error contracts, source retention, governance-status filtering, and semantic-configuration versioning. Maintenance diagnosis for discovery queries can trace backward from the entry point: `/discovery/resources/query` handles request parsing and query dispatch; index tables provide the read model for verified resources; semantic aliases and capability-label materials provide normalization and candidate assistance; `services/embedding-service/src/server.ts` and `src/embedding.ts` participate in text-vector generation only when vector mode is enabled. When investigating “duplicate results”, separately check same-DID version deduplication, index-copy deduplication, pagination cursors, and frontend display mapping; do not only adjust ranking or the frontend array. The Discovery node’s code mapping must also retain the conversion relationship “raw materials -> verification results -> index records -> query candidates”. An original ResourcePackage or Root platform publication notification must not be treated directly as a queryable result; it may enter the index only after source, hash, version, authorization-domain, and lifecycle checks are complete. Relevance scores, label hits, and text snippets in query responses are ranking or explanation evidence; the resource DID, version, source, and governance status are the factual entry points for callers to continue verification.

F.5 oan-trust-indexer

This repository corresponds to Chapter 14, Chapters 20 through 21, and authorization-status and recovery requirements. `src/event_source.rs` handles event sources, `src/sync.rs` handles cursor advancement and synchronization, `src/store.rs` handles projection storage, `src/domain.rs` handles domain state, `src/api.rs` and `src/grpc.rs` provide query interfaces, and `src/config.rs` handles runtime configuration. `configs/mainnet.example.toml`, `mainnet.current-events.toml`, and the single-checkpoint smoke configuration distinguish continuous synchronization, current-event observation, and diagnostic runtime modes.

The indexer’s trust path is “read on-chain events -> process them in order or by cursor -> validate event structure and state transitions -> write the off-chain read model -> provide state queries externally”. The read model must retain the event source, cursor, last successful processing time, projection time, errors, and necessary version information. The off-chain projection is a queryable cache of governance facts and does not create on-chain authorization; when using it, the Root platform, Registrar node, and Discovery node should account for delay, staleness, and fail-closed boundaries. The latest sequence returned by `trust/v1/status` or an equivalent interface can only serve as an observation of index progress and cannot alone prove that a particular on-chain transaction has reached finality.

Existing admission fixtures can verify consistency for active, suspended, and revoked states and governance snapshots, but they are not equivalent to proof of a real chain RPC, GraphQL endpoint, finality, checkpoint recovery, or continuously growing data. Maintenance responsibilities include event-source compatibility, non-regressing cursors, idempotent duplicate-event handling, exception retries, database backups, and state-freshness monitoring; when changing the chain endpoint, do not only change the address while overlooking the linked checks for package, objects, event types, and parsing logic. The interface response of the off-chain trust indexer can only describe the state of its read model. Maintainers should place the event source, last processed cursor, projection time, errors, and chain-endpoint configuration in the same runtime record, and record “on-chain event read”, “off-chain state projected”, and “Root platform adopted this state” separately; a successful health endpoint must not merge the three into one conclusion. An off-chain trust indexer version upgrade should perform at least one small-scope replay or checkpoint-recovery verification: use a known event sequence to check parsing results, cursors, duplicate-event handling, and state projections, then compare the subject, node, credential issuer, and event sequence returned by the API. If the package, object, or event type changes, determine separately whether the old read model can continue to be used; a restarted process with a successful `/health` response does not establish that historical governance state has been correctly recovered.

F.6 oan-sdk-ts

This repository corresponds to Chapters 5 through 9, Chapter 12, Chapters 16 through 17, and Appendices A through C. `packages/sdk-ts/src/index.ts` centrally provides resource DID Document drafts, ResourcePackage shape validation, lifecycle checks, registration, discovery, query explanations, capability-label assistance, and governance read interfaces; `identity.ts` handles Local Identity and resource identity records, key-pair generation, identity-package import/export, and generation

of registration materials from an identity; `did-oan.ts` handles `did:oan` parsing, normalization, mapping from subject codes to resource types, and semantic conflict checks. `packages/protocol-types/src/index.ts` provides cross-package protocol types. Tests are located in `tests/sdk-core-tests.ts`, `identity-store-node-tests.ts`, and `client-workflow-tests.ts`.

The SDK is a caller tool, not the authoritative implementation of the Registrar, Root, or Discovery. It can generate keys locally, construct DID Documents and registration materials, call public interfaces, and structurally validate `ResourcePackage`, but it cannot confirm on-chain finality for the caller and must not treat recommendation results as authorization conclusions. Endpoint stubs, fixed DIDs, and placeholder hashes in tests are used to verify interface behavior; publication examples must replace them with real endpoints, package hashes, authorization domains, and resource descriptions. Private keys and Local Identity packages are stored by the caller and are not sent to OpenAgenet (OAN) nodes.

SDK capability	Corresponding implementation	Verification evidence	Maintenance boundary
DID and identity	<code>identity.ts</code> , <code>did-oan.ts</code>	SDK core and identity-store tests	Preserve binding among DID, subject type, public key, and local private key; does not implement network governance.
Registration materials	<code>index.ts</code> , identity-record conversion functions	Client workflow tests	Materials are still revalidated by the Registrar and Root platform after generation.
Discovery and explanation	Discovery client in <code>index.ts</code>	Stub workflow and query-assistance tests	Result ranking/explanation cannot replace the caller's final verification.
Resource validation	<code>ResourcePackage</code> shape and lifecycle functions	SDK core tests	Structural validation is not equivalent to verification of real VC, governance, or endpoint availability.

SDK release maintenance must also check build output, export entry points, Node/browser runtime boundaries, and protocol-version compatibility, to avoid inconsistencies among the README, npm package contents, and source API. The SDK's public entry points are governed by `package.json exports`: the root entry, `./client`, `./governance`, `./protocol-types`, and `./identity-store-node` correspond to modules in the build output. After changing `packages/*/src`, check whether `npm run build` generates the corresponding JavaScript and declaration files, then run `tests/sdk-core-tests.ts`, `tests/client-workflow-tests.ts`, and `tests/identity-store-node-tests.ts`; even if only internal implementation changes but export paths, request fields, or error types change, it should still be treated as a client-compatibility change. SDK mapping should record the boundary between “local generation” and “remote confirmation”: identity, DID Document drafts, `ResourcePackage` shape, and signature materials can be generated or verified locally by the caller, whereas registration VC, Root proof, Discovery index status, and governance status come from nodes or subsequent caller

queries. The existence of an SDK method name does not mean that remote business processing has completed; examples and tests should make clear which responses are supplied by stubs and which require a real Registrar node or Discovery node.

F.7 oan-community-skill

This repository corresponds to Chapter 17 and the community-caller operation paths in Chapters 5 through 9, 11, 15, and 18. Its `SKILL.md` makes clear that the skill package is used for organizing resource descriptions, managing Local Identity, generating registration materials, submitting to the Registrar node, performing Discovery queries, explaining semantic queries, observing lifecycle, and assisting with read-only governance; it does not start or maintain the Root platform, Registrar node, Discovery node, content distribution platform, NATS, PostgreSQL, or off-chain trust indexer, and it does not perform on-chain governance writes. The implementation description requires continued checking of `src/index.ts`, `validation.ts`, `registration.ts`, `resource-description.ts`, `discovery.ts`, `lifecycle.ts`, `capability-assist.ts`, and `tests/oan-community-skill-tests.ts`.

Typical inputs to the community skill package are a resource README, skill-package description, or product description supplied by the user. It first generates candidate registration materials, then reports missing fields and quality issues, requires the user to supplement facts such as name, description, public access address, authorization domain, capability labels, use cases, inputs and outputs, license, and maintainer, and finally calls the SDK's normal registration path. It must not invent endpoints, maintainers, licenses, or authorization domains, and must not treat capability labels as an authorization scope. For batch resources, report success, skipped items, failures, missing inputs, and next steps item by item; do not silently discard other results because one item failed.

The skill's maintenance responsibility is to keep user-operation semantics, SDK calls, public endpoint defaults, error classification, and privacy boundaries consistent; a published package must include the scripts, documentation, and resources required for runtime, rather than only TypeScript compilation output. It may call public or explicitly user-provided Registrar, Discovery, and optional trust-query endpoints, but must not expose private node governance materials or read the Root platform's private indexer. Implementation tracing for the community skill package should start at the public call entry in `src/index.ts`, follow the inputs and outputs of `registration.ts`, `resource-description.ts`, `validation.ts`, `discovery.ts`, and `lifecycle.ts` separately, and then compare them with `tests/oan-community-skill-tests.ts` and the published-package file list. The skill package organizes and calls; it does not generate unconfirmed facts for the user. Therefore, when adding automatic completion, label suggestions, or retry logic, confirm that inferred content is not written directly into registration materials and that submissions are not repeated unconditionally when the result is unknown. Changes to the community skill package and SDK should also distinguish "helping generate materials" from "writing network facts". The skill package can organize descriptions, prompt for missing fields, and call public interfaces, but it cannot treat inferred endpoints, labels, authorization domains, or licenses as user-confirmed registration facts; when the request result is unknown, it also cannot create duplicate VCs, duplicate versions, or duplicate publication tasks through unconditional retries.

F.8 Official Website and Public Developer Entry Points

The official website is a public display and operation entry point, corresponding to parts of Chapters 15, 16, 19, 20, 22, and 24. The frontend uses public Registrar, Discovery, Root platform, content distribution platform, and off-chain trust indexer endpoints; the backend provides the aggregation, statistics, access analytics, or proxy capabilities required by the website through `/api/*` and `/health`. Production deployment instructions, build and smoke-test scripts, and Nginx and systemd configurations describe build, configuration, reverse-proxy, and verification boundaries. The website's Home, Register, Discover, Network, and Docs pages can call or display protocol data, but page copy, maps, statistics, cached snapshots, and recommendation ranking belong to the presentation layer.

The website's registration page can generate or import Local Identity in the browser, construct a DID Document and registration materials, and submit them to a specified Registrar node; private keys must not be sent to the website backend. The discovery page can submit a DID or natural-language task description and display candidate resources and DID Document previews returned by the Discovery node; users must still perform final verification based on the resource DID, VC, Root proof, status, and endpoints. When the Network page displays nodes and metrics, it should distinguish current real-time data, old snapshots, unknown status, and statistical aggregation, and must not treat UI state as governance authority.

Hard checks for website deployment maintenance include: production frontend build variables must not retain local-machine addresses; the backend should be exposed only through a reverse proxy; static assets and backend versions must match; public HTTPS, language switching, registration transmission, discovery queries, the Network summary, and browser routes must all pass smoke tests. SEO, access statistics, and page caching can only be loosely coupled enhancements and must not change the protocol behavior of registration, discovery, governance, or node services. Website issue diagnosis should first distinguish the browser layer, website backend aggregation layer, and node-service layer: frontend pages and `frontend/src/api.ts` initiate requests and present results; backend `backend/src/http/router.rs` and the corresponding handlers provide public aggregation interfaces; the Registrar node and Discovery node are the direct handlers of registration/discovery business logic. When access-statistics writes, SEO metadata, browser caching, or old-snapshot reads fail, the page may degrade or omit the record, but must not block registration submission, discovery queries, or node-status judgment; any cross-layer change must run page-route, public-interface, and node-business smoke tests separately. The change boundary between website components and protocol implementation can be checked by interface direction: `/api/*` from the browser to the website backend is a public aggregation or proxy path, requests from the website backend to nodes are service-to-service calls, and the node's own `/resources/register` and `/discovery/resources/query` are the protocol business entry points. When changing website caching, statistics, SEO, or page copy, verify that failures in these features do not change node request bodies, response semantics, registration/discovery results, or governance status; when changing node interfaces, do not perform only web regression testing while skipping protocol and client tests.

F.9 Specification Chapter and Implementation Module Mapping

The following table provides the primary index for maintenance and release work. Chapter numbers are the entry points for specification responsibility; specific implementations still need to be checked against the preceding repository sections, the current commit, and the target deployment configuration.

Specification scope	Main implementation repository/module	Main interfaces or data	Main tests/evidence	Implementation status and maintenance responsibility
Resource model, DID Document, ResourcePackage	oan-protocol-common, oan-sdk-ts	DidDocument , resource types, ResourcePackage, hash fields	Rust/TS unit tests, resource fixtures	Basic structure implemented; shared-library maintainers lead protocol-type changes.
Registration and control proofs	oan-registrar-node , oan-sdk-ts , oan-community-skill	/resources/register , registration materials, subject control proof, registration VC	registrar-resource .*, client workflow tests	Partially implemented; real idempotency, concurrency, and upstream failures require continuous-integration verification.
Root platform verification and trusted publication	oan-root-services , oan-protocol-common	Root resource verification/publication interface, Root proof, version	Root platform unit tests, Root/CDN harness, publication logs	Partially implemented; on-chain governance and production finality cannot be inferred from offline tests.
Content distribution and synchronization	oan-root-services/services/cdn-publisher , cdn-node , Discovery node synchronization logic	Publication queue, ResourcePackage, cursor, hash	Root/CDN workflow tests, synchronization logs	Partially implemented; checkpointing, out-of-order processing, and real fault recovery require deployment-level verification.
Discovery and semantic governance	oan-discovery-node , oan-protocol-common/oan-semantic-recommender	Query requests, capability-label tree, semantic aliases, index records	Discovery fixtures, query-explanation and semantic-service tests	Structured/semantic paths exist; model quality and cross-node convergence require separate records.

Specification scope	Main implementation repository/module	Main interfaces or data	Main tests/evidence	Implementation status and maintenance responsibility
Node authorization and governance status	oan-trust-indexer , Root platform, admission test kit	Governance events, authorization domain, lifecycle, cursor	governance/node/domain/lifecycle fixtures	Implemented; indexer delay, chain endpoint, and finality are runtime-verification concerns.
Public APIs and error contracts	oan-protocol-common , each node's main.rs , oan-sdk-ts	HTTP paths, status codes, error codes, pagination	docs/error-codes.md , request fixtures, SDK tests	Main interfaces implemented; new fields/error codes must be synchronized across all clients.
Community developer experience	oan-community-skill , oan-sdk-ts	Local Identity, resource descriptions, registration/discovery assistance	Skill tests, SDK tests, npm build output	User path implemented; package completeness and version compatibility are the package maintainer's responsibility.
Official website and operations entry point	Website frontend, website backend, and deployment workflow	/api/* , static frontend, health and smoke tests	Production deployment instructions, build/smoke-test scripts, browser checks	Presentation/aggregation layer; not protocol authority, deployment maintained by operations maintainers.
Third-party node conformance	oan-third-party-node-admission-test-kit	Fixed manifest, fixtures, expected results, reports	49 requirement checks, case matrix, traceability matrix	Evaluation support; current V1-alpha does not cover a real Root platform, chain, CDN, or indexer.

Cross-index maintenance follows four rules: first, when adding or modifying a shared field, update the specification and **oan-protocol-common** first, then check all production services and clients; second, when adding an interface, update the error contract, SDK, community skill package, and fixed request examples at the same time; third, governance status, Root proof, and semantic matching must retain their respective sources of fact and must not be merged in an aggregation layer into an unexplained “trusted” boolean; fourth, passing deployment scripts and smoke tests only proves that the target environment met the conditions checked at that time and must not turn environment variables, domain names, models, node counts, or database selections into fixed protocol requirements.

When implementation and specification diverge, maintainers should state in the change record

the affected chapters, input/output differences, compatibility strategy, test evidence, whether data migration is created, and rollback method. Only when code, tests, configuration, and runtime verification jointly support it may “Reference / to be verified” be promoted to “Implemented”; any aggregate result that lacks a source of fact, verification subject, or status boundary can only be observation information and cannot serve as cross-node trust proof.

The minimum tracking record for a cross-repository change can be established in the following order:

Chapter or field change

```
-> oan-protocol-common types/validation/errors
-> Root, Registrar, Discovery, or indexer call paths
-> SDK and community Skill inputs and outputs
-> fixtures, unit tests, and integration tests
-> configuration, systemd/reverse proxy, and deployment smoke tests
-> compatibility conclusion, release commit, and rollback basis
```

If a change concerns only website copy, SEO, or access statistics, explicitly record it as a presentation-layer website change and do not list it as a protocol-version change; if a change concerns protocol fields, DID Document, ResourcePackage, authorization domain, error codes, or life-cycle status, it must return to the shared types and all consumers for item-by-item verification. This tracking chain is for maintenance and review and does not change the original responsibility boundaries of the repositories.

“Implemented” in the cross-index should be based on the minimum evidence set: the corresponding code entry point is locatable, there is at least one targeted test or repeatable check, the runtime configuration can resolve the required dependencies, and there is no conflict with current protocol fields or error contracts. If a directory only contains a file but has no call path, test result, or runtime configuration, it should remain “Reference / to be verified”; if only test fixtures pass without service integration, it should be marked “Evaluation support” or “Partially implemented” and must not be promoted directly to production capability.

F.10 References

Source	Type	Link
<code>oan-protocol-common</code>	Code repository: shared protocol foundations	https://github.com/wolfbrother/oan-protocol-common
<code>oan-root-services</code>	Code repository: Root platform and content distribution coordination	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	Code repository: Registrar node	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	Code repository: Discovery node	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-trust-indexer</code>	Code repository: off-chain trust indexer	https://github.com/OpenAgenet/oan-trust-indexer

Source	Type	Link
oan-sdk-ts	Code repository: TypeScript SDK	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	Code repository: community Skill	https://github.com/OpenAgenet/oan-community-skill